CSC10004: Data Structure and Algorithms

Lecture 9: Graphs

Lecturer: Bùi Văn Thạch
TA: Ngô Đình Hy/Lê Thị Thu Hiền
{bvthach,ndhy}@fit.hcmus.edu.vn, lththien@hcmus.edu.vn

Course topics

0. Introduction
1. Algorithm complexity analysis
2. Recurrences
3. Search
4. Sorting
5. Linked list
6. Stack, and Queue, Priority queue
7. Hashing
8. Trees
    1. Binary search trees (BST)
    2. AVL trees
8. Graphs
    1. Graph representation
    2. Graph search
9. Algorithm designs
    1. Greedy algorithm
    2. Divide-and-Conquer
    3. Dynamic programming

Goals

1. Be fluent in the language of graphs so you can analyze and model problems effectively.
   - Understand foundational terms: vertex, edge, degree, path, cycle, connectedness, adjacency.
   - Distinguish between types of graphs: directed, undirected, weighted, unweighted, cyclic, acyclic.
2. Choose the right data structure for a given graph problem to optimize performance.
   - Adjacency matrix, Adjacency list, Edge list.
3. Explore, analyze, and manipulate graph structures.
   - Depth-First Search (DFS),
   - Breadth-First Search (BFS).
4. Find efficient ways to connect all parts of a graph with minimal cost.
   - Understand what a spanning tree is.
   - Learn algorithms to construct minimum spanning trees (MSTs).
5. Compute optimal paths through graphs under constraints like weights or directions.
   - Understand shortest path problems in weighted graphs.

Outline

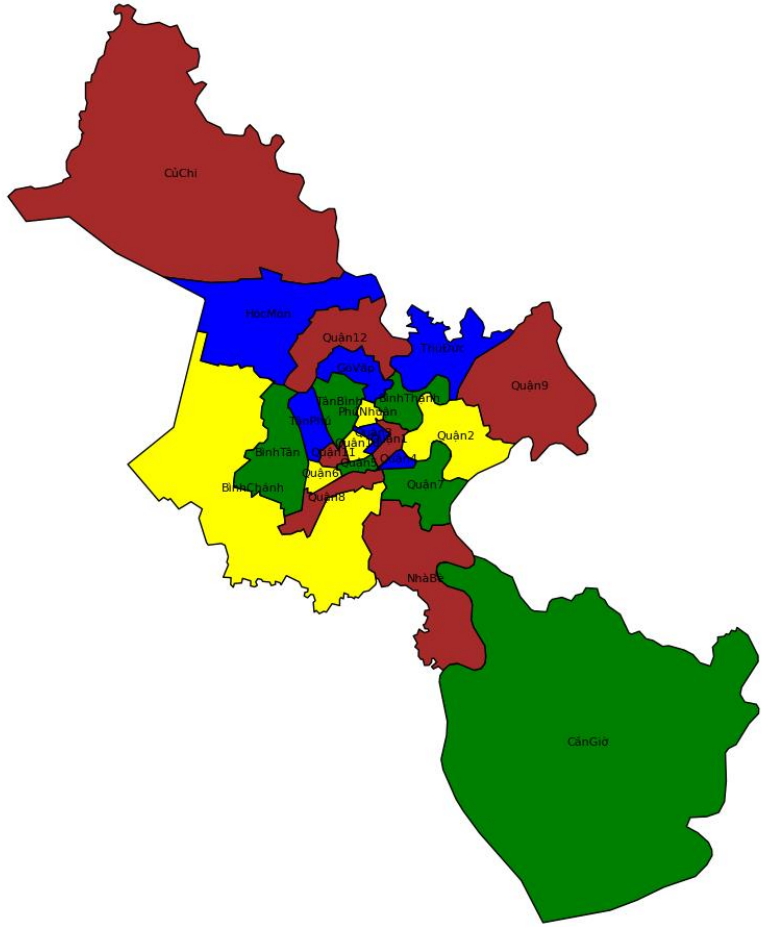
1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

22 districts in Ho Chi Minh city

Shall we add some colors to this map of Ho Chi Minh City?



The Four-Color Theorem (1/)

- First proposed by Francis Guthrie in 1852 [1], the conjecture stated that:
  - Any map drawn on a plane can be colored using NO MORE THAN FOUR COLORS, in such a way that NO TWO adjacent regions SHARE the same color.
- Over the years, many incorrect proofs were attempted.
- In 1977, the theorem was **first rigorously proven by Appel and Haken** [2], with the aid of computer verification.
- A subsequent proof was later provided by Robertson and collaborators in 1996 [3].

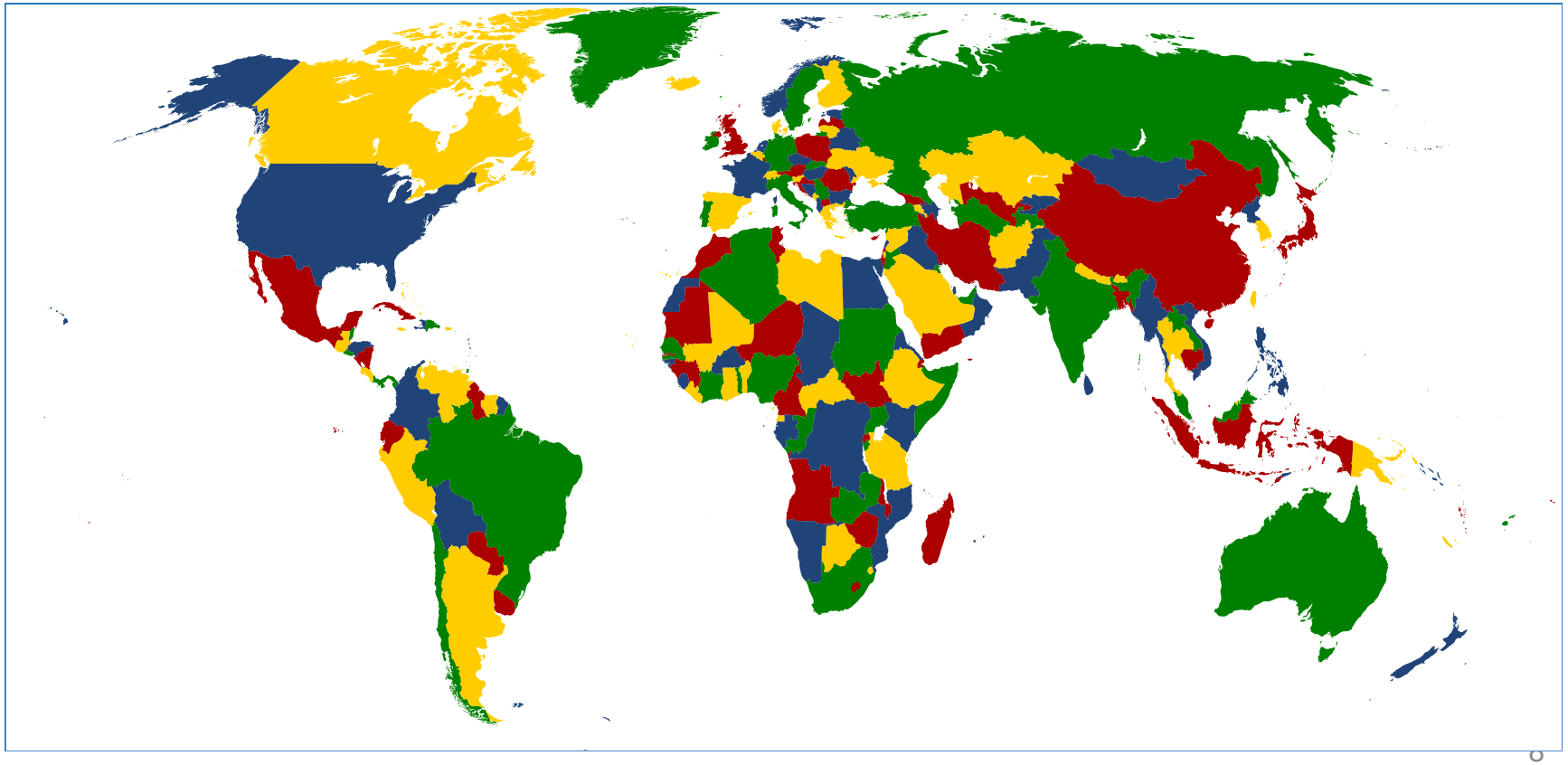
[1] May, Kenneth O. "The origin of the four-color conjecture." *Isis* 56.3 (1965): 346-348.
[2] Appel, Kenneth, and Wolfgang Haken. "The solution of the four-color-map problem." Scientific American 237.4 (1977): 108-121.
[3] Robertson, Neil, et al. "Efficiently four-coloring planar graphs." Proceedings of the 28th annual ACM STOC. 1996.

The Four-Color Theorem (2/)

Can we color the world map by countries with up to 4 colors such that

NO TWO adjacent countries SHARE the same color?

- But this is a map, not a graph!
- However, we can model it as a graph.
- But what is a graph?

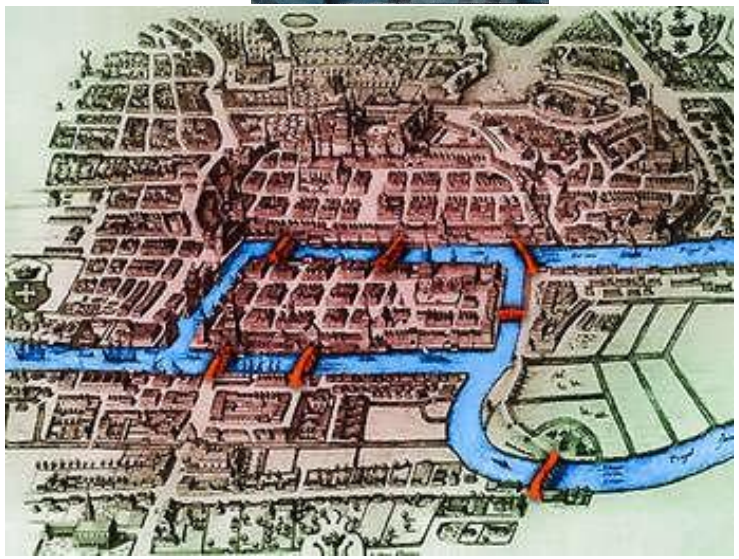


History: Königsberg bridges (1/2)

- The subject of graph theory began in the year 1736 when the great mathematician Leonhard Euler published a paper giving the solution to the following puzzle:

The town of Königsberg in Prussia (now Kaliningrad in Russia) was built at a point where two branches of the Pregel River came together. It consisted of an island and some land along the river banks.

These were connected by 7 bridges.



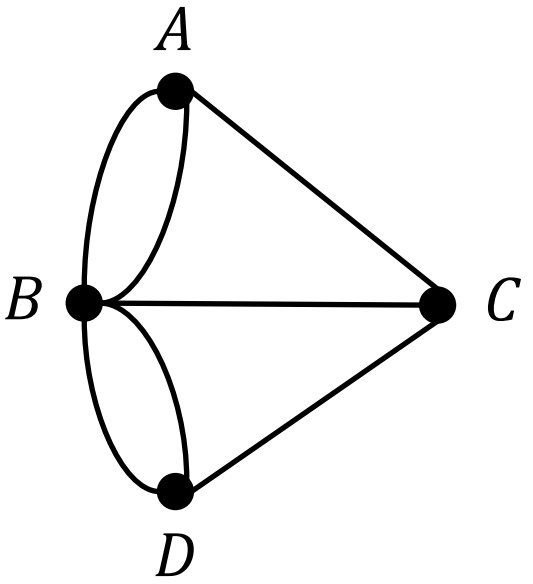
Euler, Leonhard. "Solutio problematis ad geometriam situs pertinentis." *Commentarii academiae scientiarum Petropolitanae* (1741): 128-140.

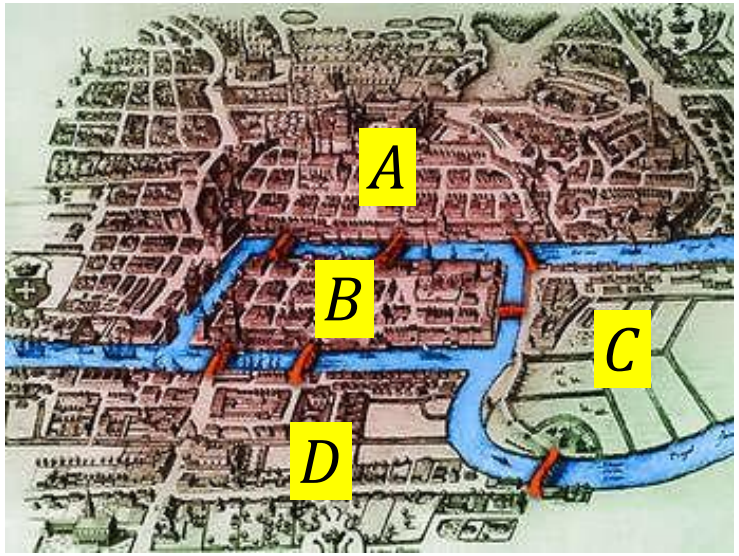
History: Königsberg bridges (2/2)

- Question: Is it possible to take a walk around town, starting and ending at the same location and crossing each of the 7 bridges exactly once?

In terms of this graph, the question is:

Is it possible to find a route through the graph that starts and ends at some vertex, one of *A*, *B*, *C*, or *D*, and traverses each edge exactly once?





Euler, Leonhard. "Solutio problematis ad geometriam situs pertinentis." *Commentarii academiae scientiarum Petropolitanae* (1741): 128-140.

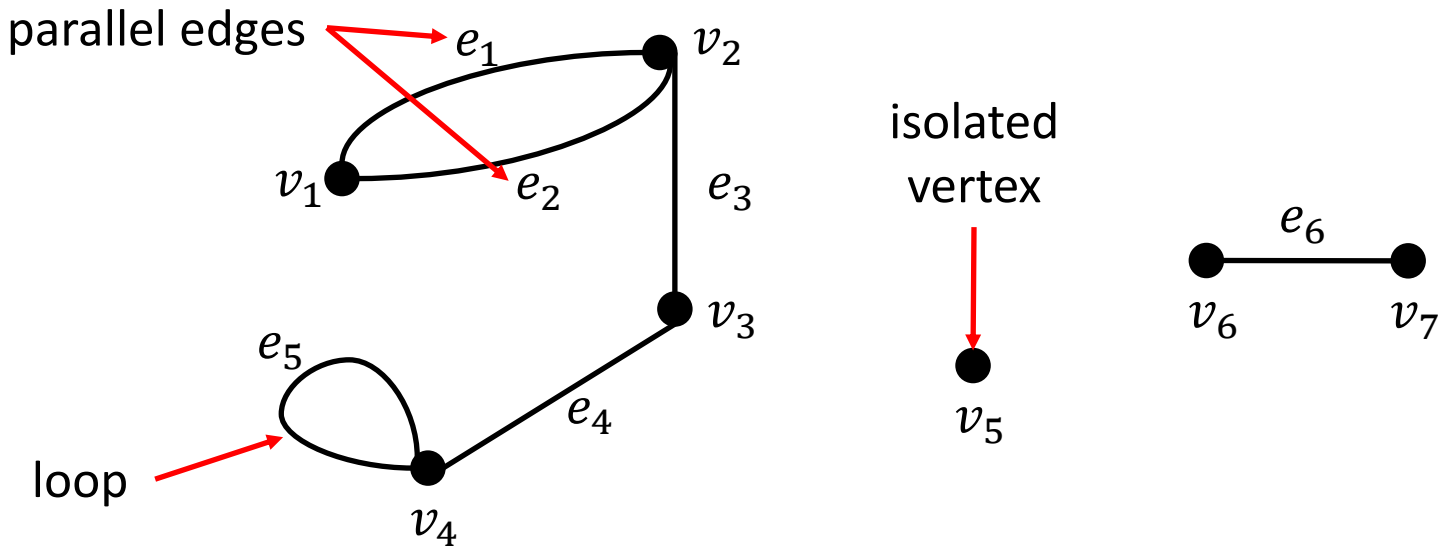
Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

Definitions and basic properties

- A graph G consists of
  - a set of vertices $V(G)$, and
  - a set of edges $E(G)$.
- We often write $G = (V, E)$.

An edge connects one vertex to another, or a vertex to itself.



Graph definition

Definition 1. A graph G consists of 2 finite sets: a nonempty set V(G) of vertices and a set E(G) of edges, where each edge is associated with a set consisting of either one or two vertices called its endpoints.

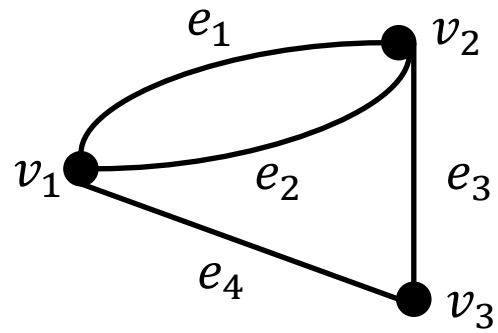
An edge is said to connect its endpoints; two vertices that are connected by an edge are called adjacent vertices; and a vertex that is an endpoint of a loop is said to be adjacent to itself.

An edge is said to be incident on each of its endpoints, and two edges incident on the same endpoint are called adjacent edges.

We write $e = (v, w)$ for an edge e incident on vertices v and w.

Example 1

- Consider the following graph:



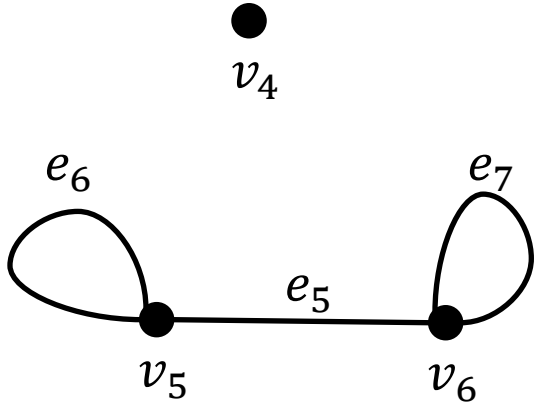
List the vertex set V, the edge set E, and specify each edge in E by naming its two incident vertices.

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$$

$$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$$

$e_1 = (v_1, v_2)$
$e_2 = (v_1, v_2)$
$e_3 = (v_2, v_3)$

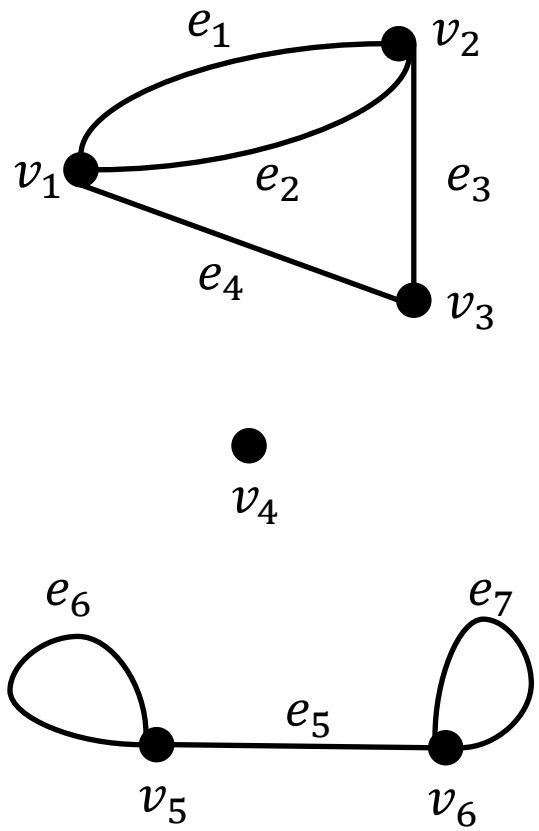
$e_4 = (v_1, v_3)$
$e_5 = (v_5, v_6)$
$e_6 = (v_5)$
$e_7 = (v_6)$



Example 2

- Consider the following graph:

Find all edges that are incident on v_1, all vertices that are adjacent to v_1, all edges that are adjacent to e_1, all loops, all parallel edges, all vertices that are adjacent to themselves, and all isolated vertices.



Edges incident on v_1: e_1, e_2 and e_4.
Vertices adjacent to v_1: v_2 and v_3.
Edges adjacent to e_1: e_2, e_3 and e_4.
Loops: e_6 and e_7.
e_1 and e_2 are parallel.

v_5 and v_6 are adjacent to themselves.
Isolated vertex: v_4.

Directed graphs

Definition 1. A directed graph, or digraph, G, consists of 2 finite sets: a nonempty set V(G) of vertices and a set D(G) of directed edges, where each edge is associated with an ordered pair of vertices called its endpoints.

If edge e is associated with the pair (v, w) of vertices, then e is said to be the (directed) edge from v to w. We write e = (v, w).

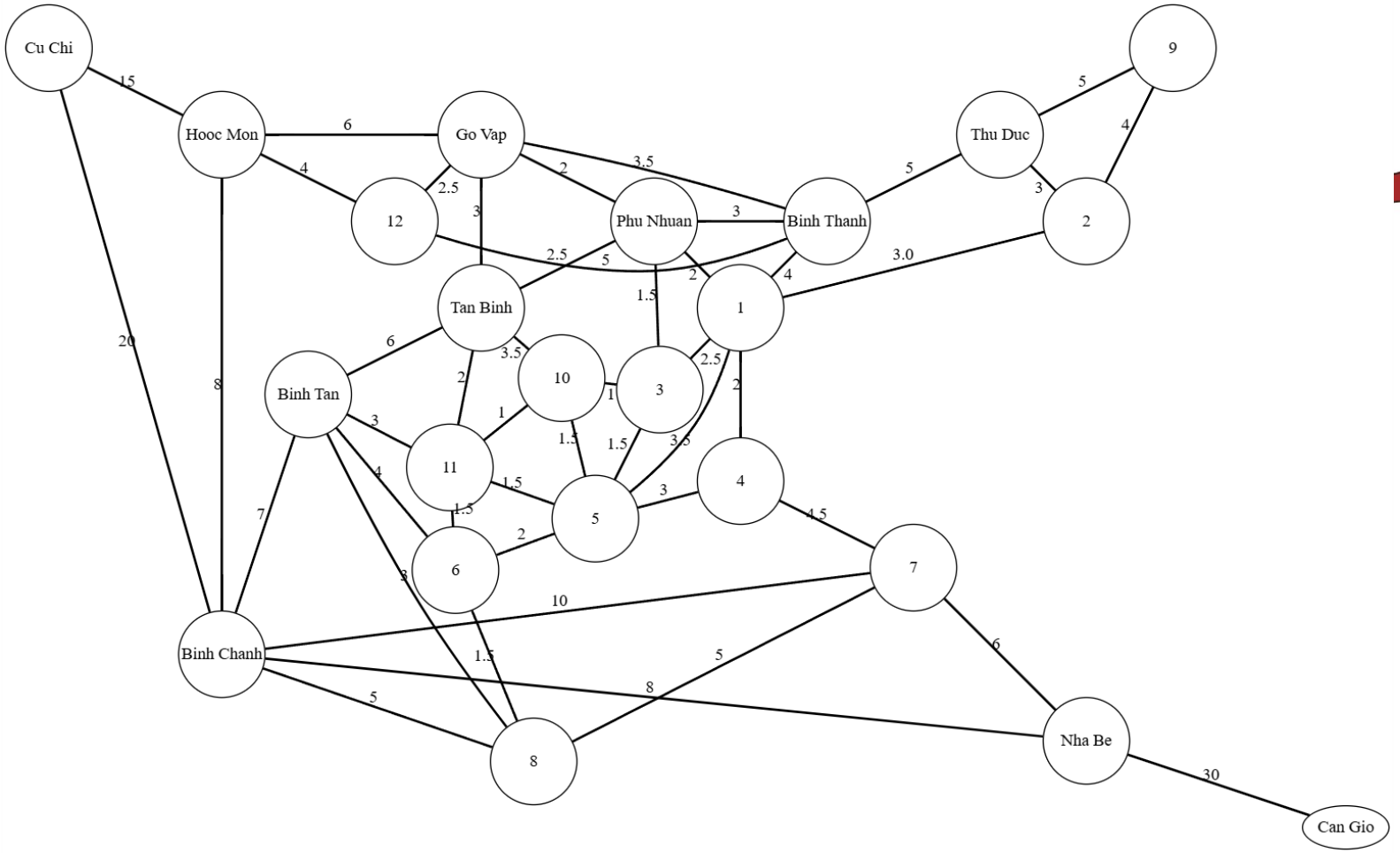
Modelling Graph Problems: map colouring problem (1/2)

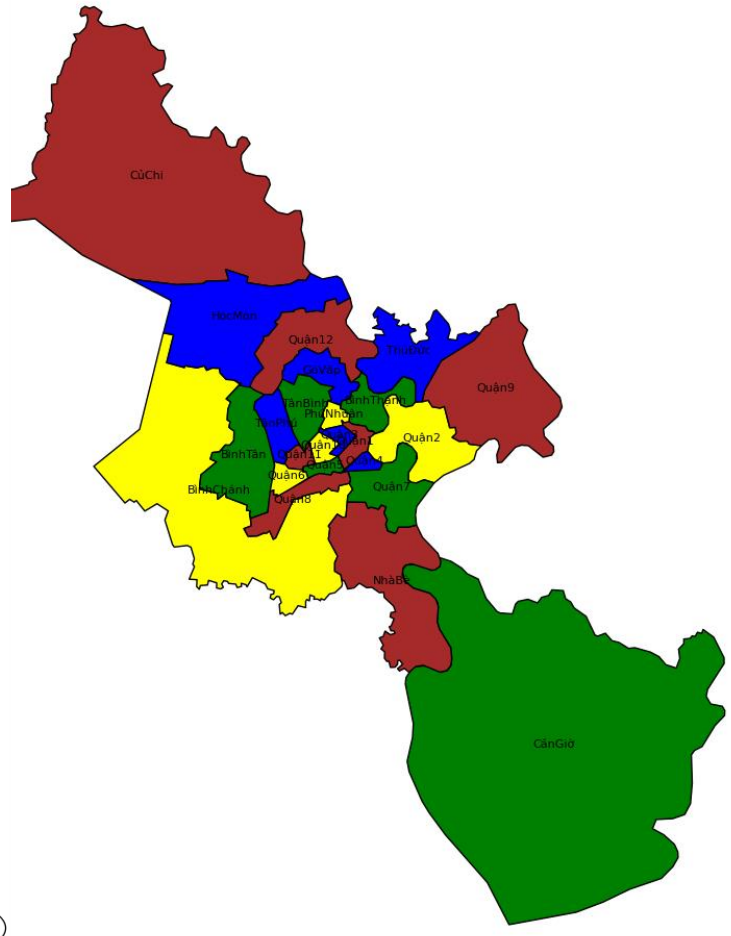
Map Colouring Problem

- Solve it as a graph problem.
- Draw a graph in which the vertices represent the districts with every edge joining two vertices represents the state sharing a common border.
- Such two vertices cannot be coloured with the same colour.
- A vertex colouring of a graph is an assignment of colours to vertices so that no two adjacent vertices have the same colour.

Modelling Graph Problems: map colouring problem (2/2)

Shall we add some colors to this map of Ho Chi Minh City?

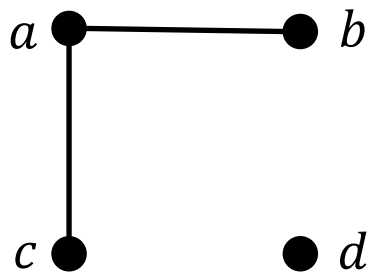


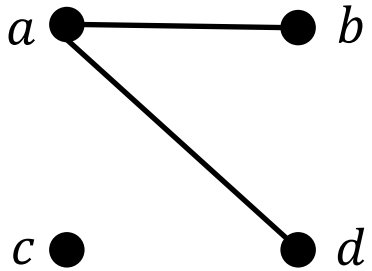


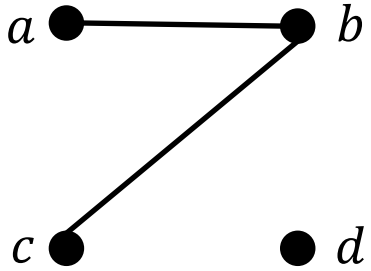
Simple graph

Definition 1. A simple graph is an undirected graph that does not have any loops or parallel edges.

- Example: Draw all simple graphs with the 4 vertices {a, b, c, d} and two edges, one of which is {a,b}.
- Each possible edge of a simple graph corresponds to a subset of two vertices. Hence there are $\binom{4}{2}$ = 6 such subsets. One is given.





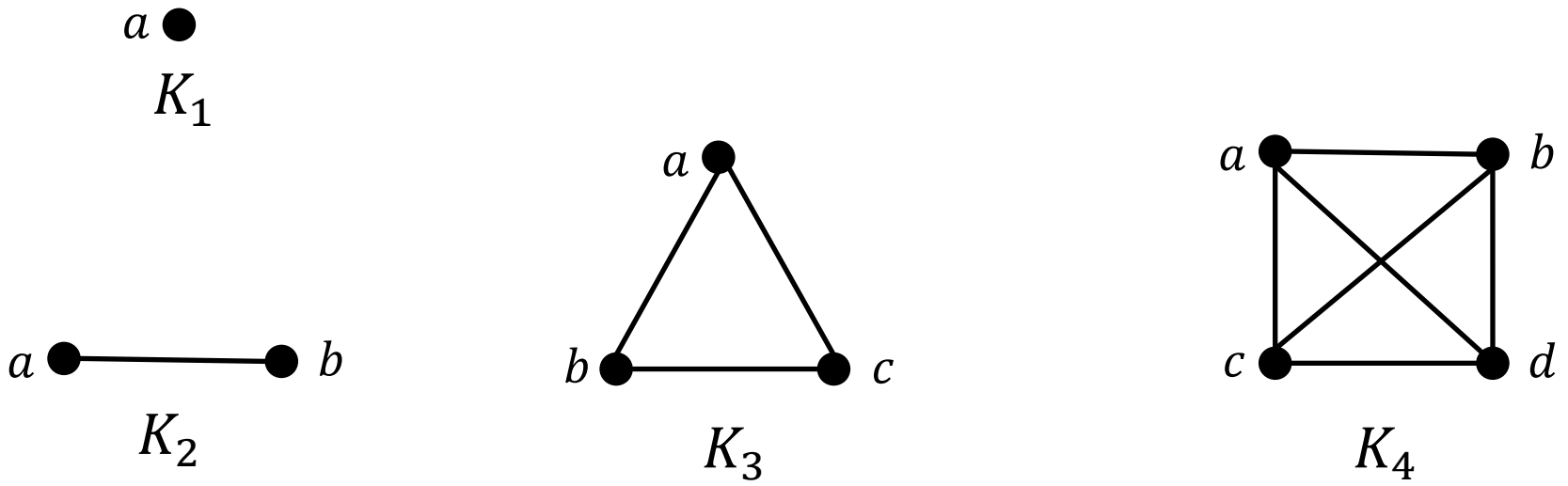


Draw the other two.

Complete Graph

Definition 1. A complete graph on n vertices, $n > 0$, denoted K_n, is a simple graph with n vertices and exactly one edge connecting each pair of distinct vertices

- Example: The complete graphs K_1, K_2, K_3 and K_4.



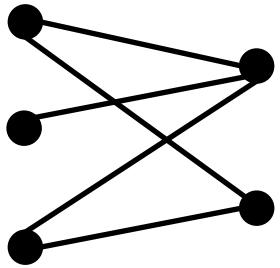
Complete Bipartite Graph

Definition 1. A complete bipartite graph on (m, n) vertices, where $m, n > 0$, denoted $K_{m,n}$, is a simple graph with distinct vertices $v_1, v_2, \dots, v_m$, and $w_1, w_2, \dots, w_n$ that satisfies the following properties:

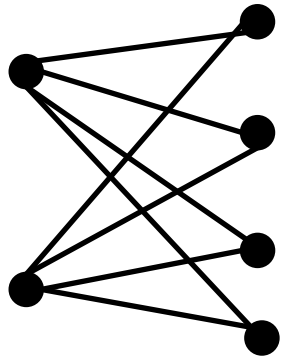
For all $i, k = 1, 2, \dots, m$ and for all $j, l = 1, 2, \dots, n$,

1. There is an edge from each vertex v_i to each vertex w_j.
2. There is no edge from any vertex v_i to any other vertex v_k.
3. There is no edge from any vertex w_j to any other vertex w_l.

$K_{3,2}$



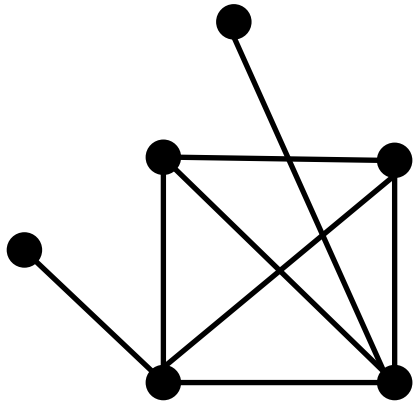
$K_{2,4}$

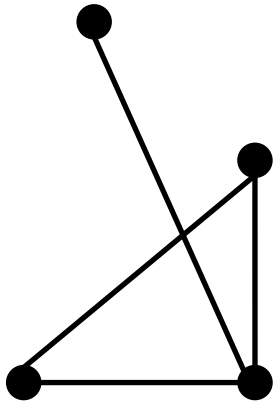


Subgraph of a Graph

Definition 1. A graph H is said to be a subgraph of graph G if, and only if, every vertex in H is also a vertex in H, every edge in H is also an edge in G, and every edge in H has the same endpoints as it has in G.

A graph *G*





A subgraph of *G*

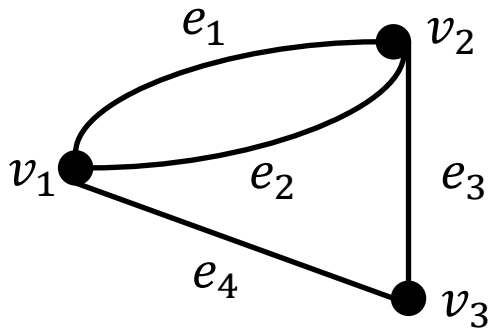
Degree of a Vertex and Total Degree of a Graph

Definition 1. Let G be a graph and v be a vertex of G. The degree of v, denoted deg(v), equals the number of edges that are incident on v, with an edge that is a loop counted twice.

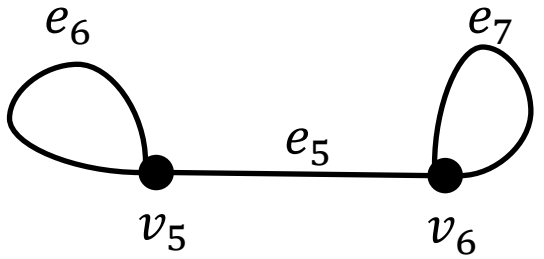
The total degree of G is the sum of the degrees of all the vertices of G.

The degree of a vertex can be obtained from the drawing of a graph by counting how many end segments of edges are incident on the vertex.

$\deg(v_1) = 3$

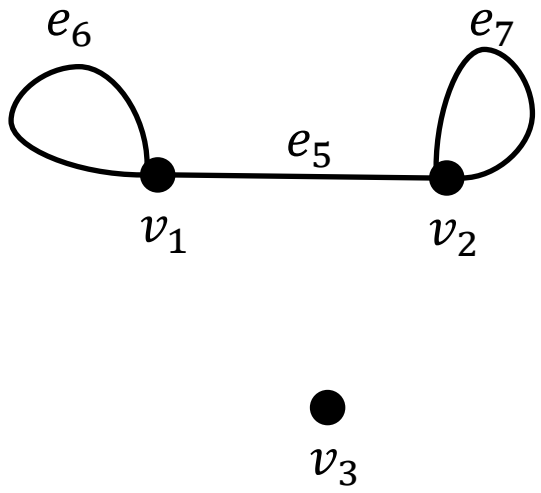


$\deg(v_5) = 3$



Example 1

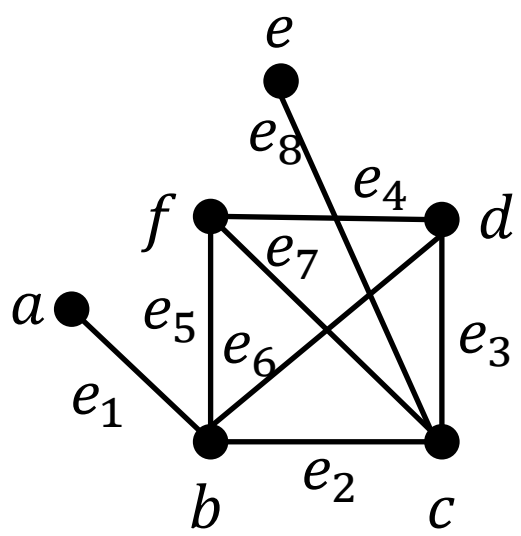
- Find the degree of each vertex of the graph *G* shown behind. Then find the total degree of *G*.



$\deg(v_3) = 0$
$\deg(v_1) = 3$
$\deg(v_2) = 3$
Total degree of $G = 6$

Traversal in graph

Travel in a graph is accomplished by moving from one vertex to another along a sequence of adjacent edges. In the graph below, for instance, you can go from a to d by taking e_1 to b and then e_6 to d. This is represented by writing ae_1be_6d.



Or, you could take a roundabout route:

$$ae_1be_5fe_7ce_8ee_8ce_2be_6d$$

Walk, Trail, Path

Definition 1. Let G be a graph, and let v and w be vertices of G.

A walk from v to w is a finite alternating sequence of adjacent vertices and edges of G. Thus a walk has the form

$$v_0 e_1 v_1 e_2 \dots v_{n-1} e_n v_n,$$

where the v's represent vertices, the e's represent edges, $v_0 = v, v_n = w$, and for all $i \in \{1, 2, \dots, n\}$, v_{i-1} and v_i are the endpoints of e_i.

The trivial walk from v to v consist of the single vertex v.

A trail from v to w is a walk from v to w that does not contain a repeated edge.

A path from v to w is a trail that does not contain a repeated vertex.

Closed Walk, Circuit, Simple Circuit

Definition 1. A closed walk is a walk that starts and ends at the same vertex.

A circuit (or cycle) is a closed walk that contains at least one edge and does not contain a repeated edge.

A simple circuit (or simple cycle) is a circuit that does not have any other repeated vertex except the first and last.

Quiz

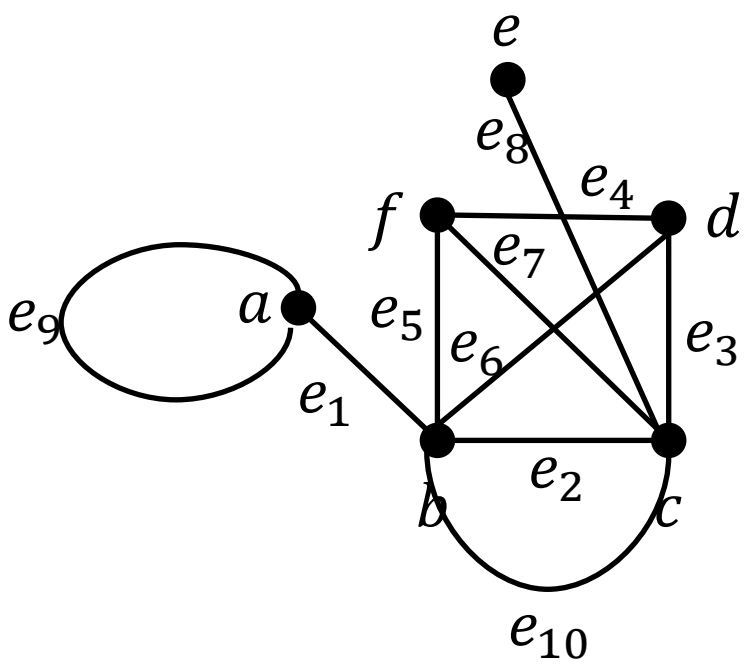
In this graph, determine which of the following walks are trails or paths:

a) $ae_1be_2ce_3de_4f$.

b) $abcbdf$.

c) $be_{10}ce_3de_4fe_7ce_2b$.

d) a.



Walk, Trail, Path, Closed Walk, Circuit, Simple Circuit

	Repeated edge?	Repeated vertex?	Starts and Ends at the same point?	Must contain at least one edge?
Walk	ALLOWED	ALLOWED	ALLOWED	NO
Trail	NO	ALLOWED	ALLOWED	NO
Path	NO	NO	NO	NO
Closed walk	ALLOWED	ALLOWED	YES	NO
Circuit (Cycle)	NO	ALLOWED	YES	YES
Simple circuit	NO	First and last only	YES	YES

Connectedness

Definition 1. **Two vertices** *v* and *w* of a graph *G* are **connected** if, and only if, there is a walk from *v* to *w*.

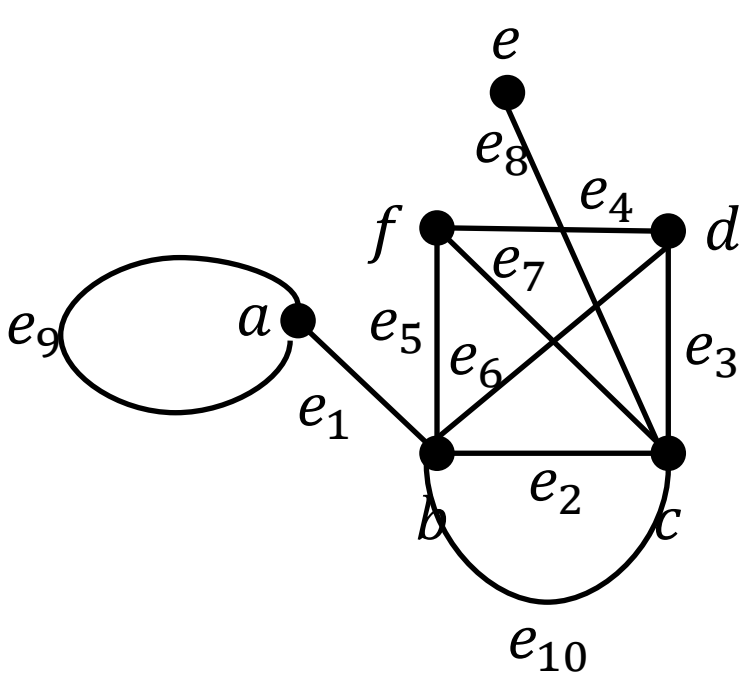
The graph *G* is connected if, and only if, given *any* two vertices *v* and *w* in *G*, there is a walk from *v* to *w*. Symbolically,

G is connected iff ∀vertices $v, w \in V(G)$, ∃ a walk from *v* to *w*.

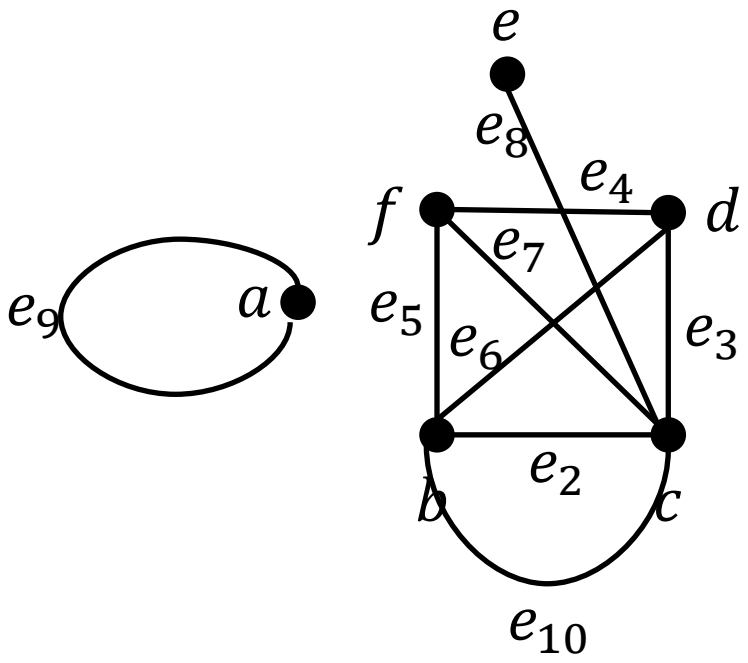
- A graph is connected if it is possible to travel from any vertex to any other vertex along a sequence of adjacent edges of the graph.

Example

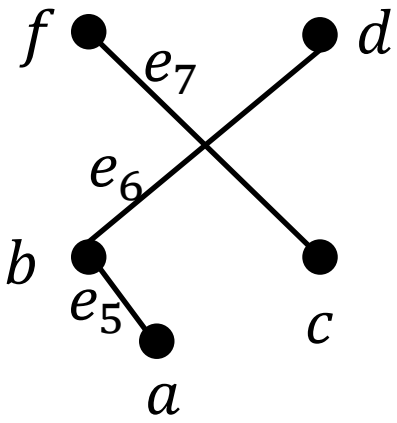
- Which of the following graphs are connected?



(a)



(b)



(c)

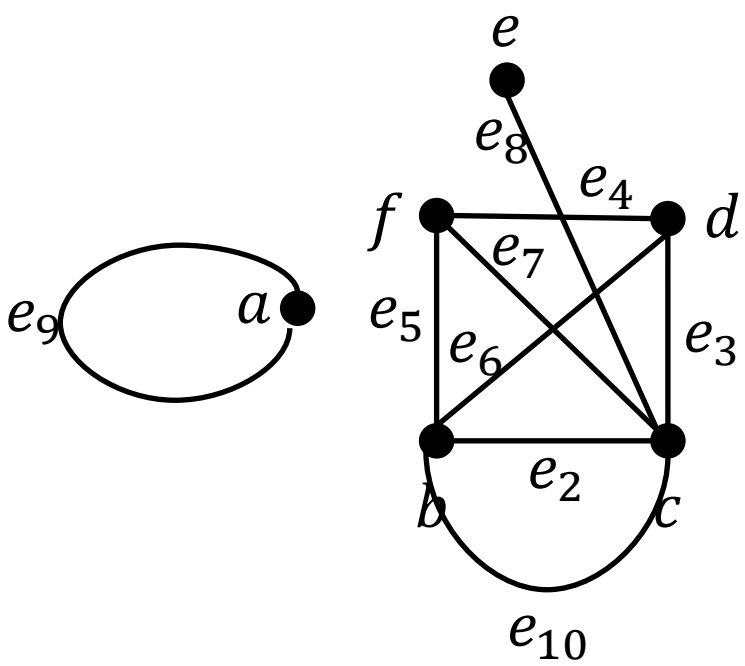
Useful facts

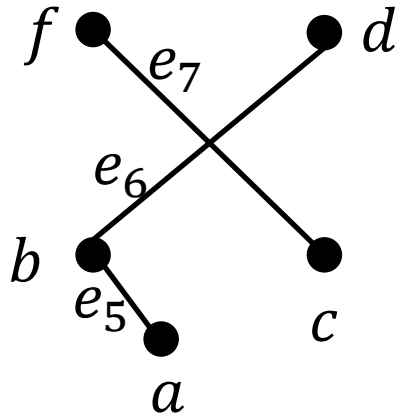
- Some useful facts relating circuits and connectedness are collected as follows:

- Let G be a graph.
- If G is connected, then any two distinct vertices of G can be connected by a path.
- If vertices v and w are part of a circuit in G and one edge is removed from the circuit, then there still exists a trail from v to w in G.
- If G is connected and G contains a circuit, then an edge of the circuit can be removed without disconnecting G.

Connected Component

- The graphs in (b) and (c) are both made up of three pieces, each of which is itself a connected graph.
- A connected component of a graph is a connected subgraph of largest possible size.





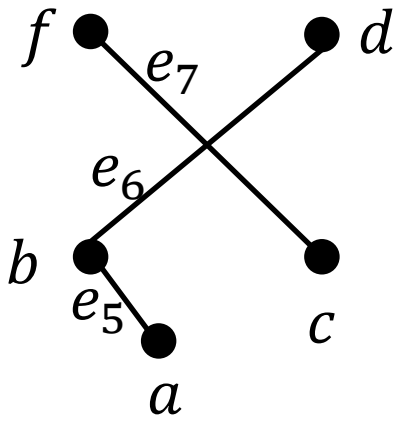
Connected Component

Definition 1. A graph H is a connected component of a graph G if, and only if,

1. The graph H is a subgraph of G;
2. The graph H is connected; and
3. No connected subgraph of G has H has a subgraph and contains vertices or edges that are not in H.

Example

- Find all connected components of the following graph G.



G has 2 connected components H_1 and H_2 with vertex sets V_1 and V_2 and edge sets E_1 and E_2 where

$$V_1 = \{a, b, d\}, E_1 = \{e_5, e_6\}$$
$$V_2 = \{f, c\}, E_2 = e_7$$

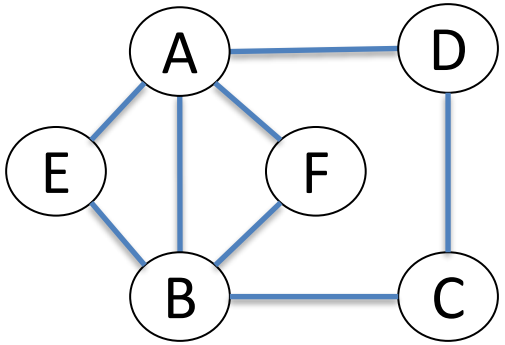
Euler Circuit

Definition 1. Let G be a graph. An Euler circuit for G is a circuit that contains every vertex and every edge of G.

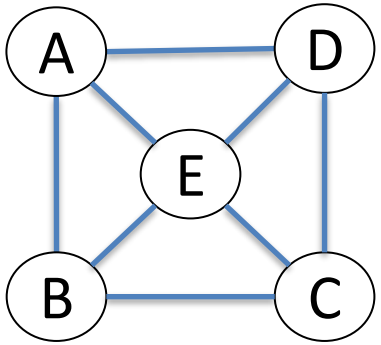
That is, an Euler circuit for G is a sequence of adjacent vertices and edges in G that has at least one edge, starts and ends at the same vertex, uses every vertex of G at least once, and uses every edge of G exactly once.

Theorem 1. If a graph has an Euler circuit, then every vertex of the graph has positive even degree. Conversely, if some vertex of a graph has odd degree, then the graph does not have an Euler circuit.

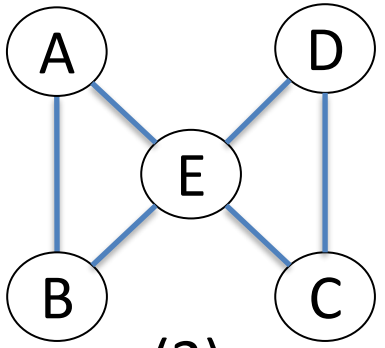
- Does each of the following graphs have an Euler circuit?



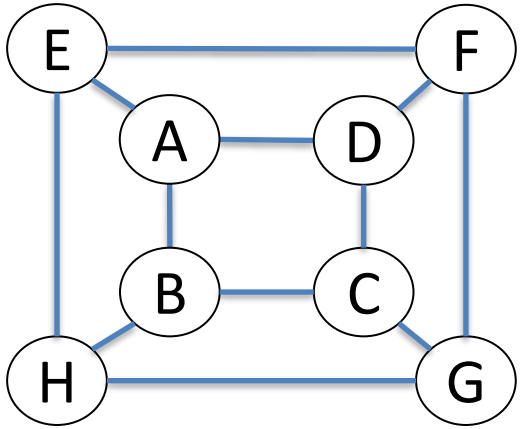
(1)



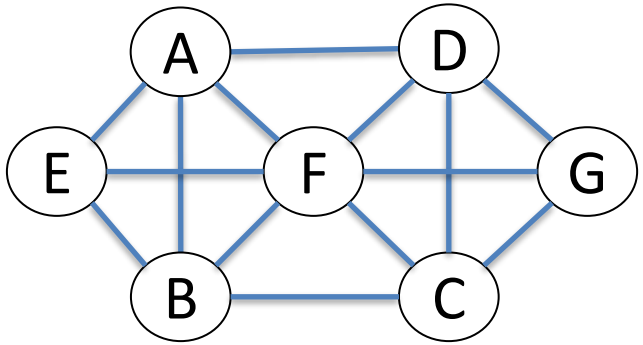
(2)



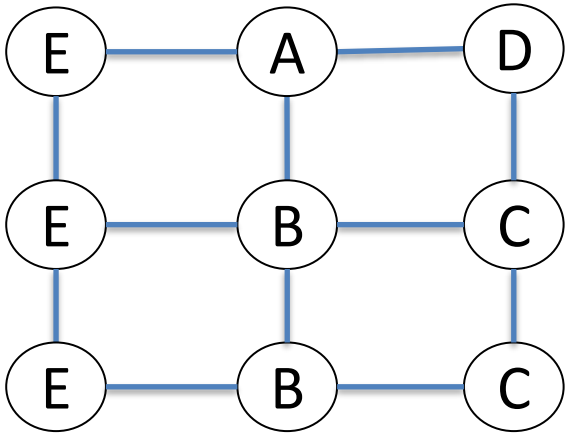
(3)



(4)



(5)



(6)

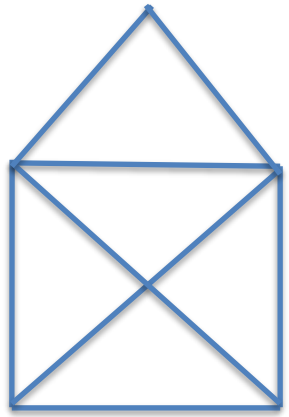
Euler Trail

Definition 1. Let G be a graph, and let v and w be two distinct vertices of G. An Euler trail/path from v to w is a sequence of adjacent edges and vertices that starts at v, ends at w, passes through every vertex of G at least once, and traverses every edge of G exactly once.

Corollary 1. Let G be a graph, and let v and w be two distinct vertices of G. There is an Euler trail from v to w if, and only if, G is connected, v and w have odd degree, and all other vertices of G have positive even degree.

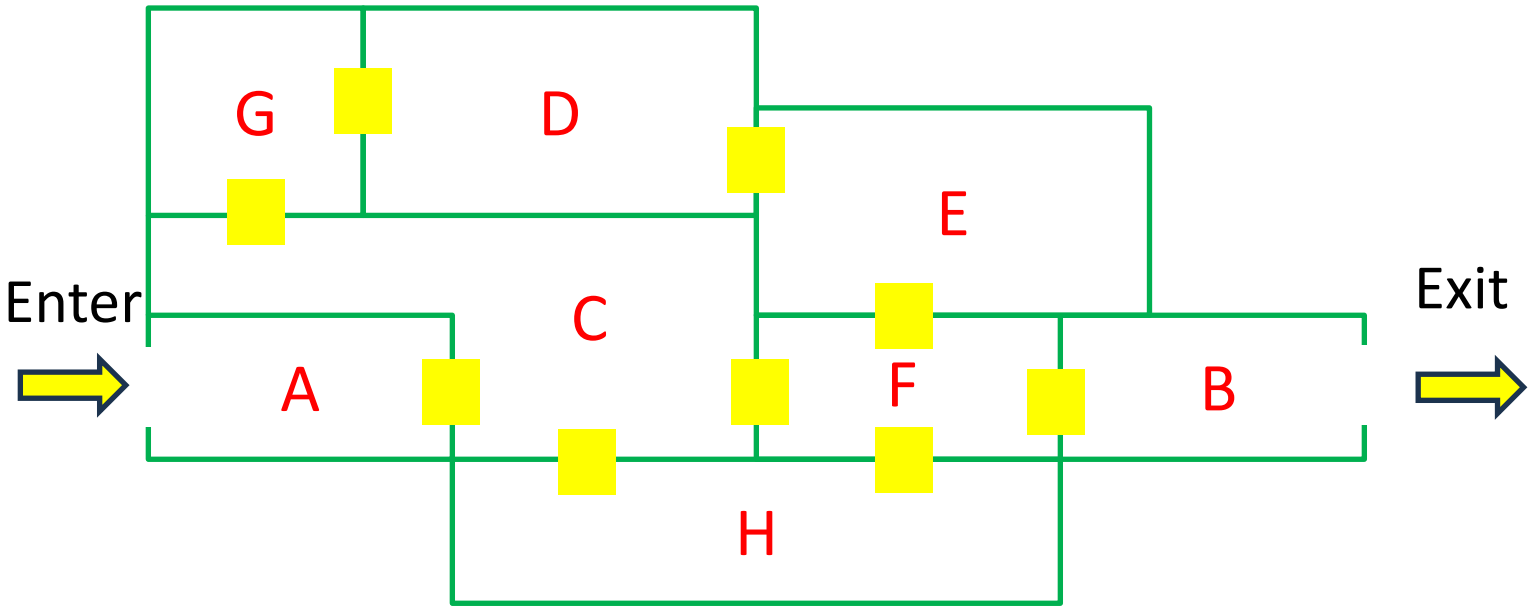
Exercise 1

- The following graphs do not have an Euler circuit. Do they have an Euler trail/path?

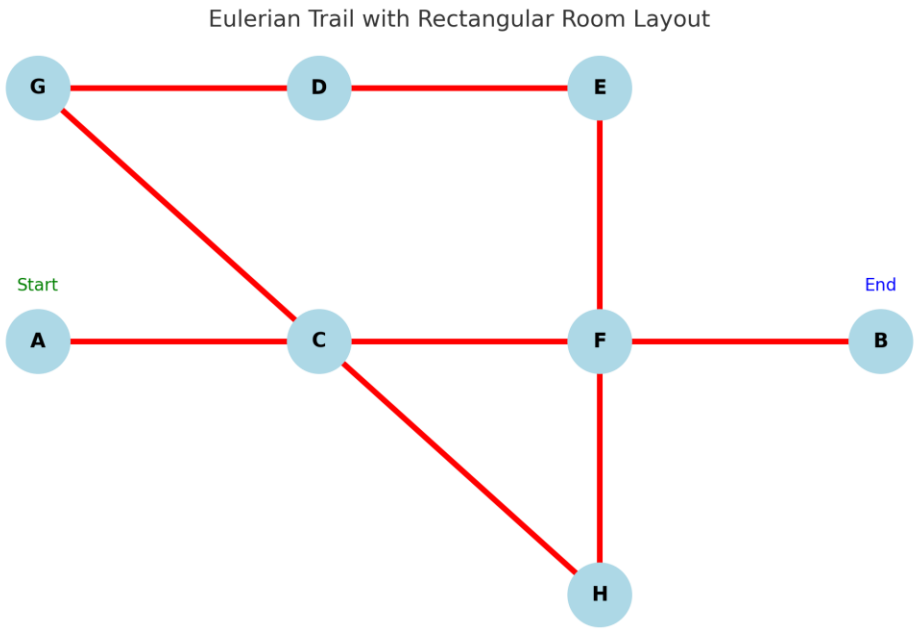


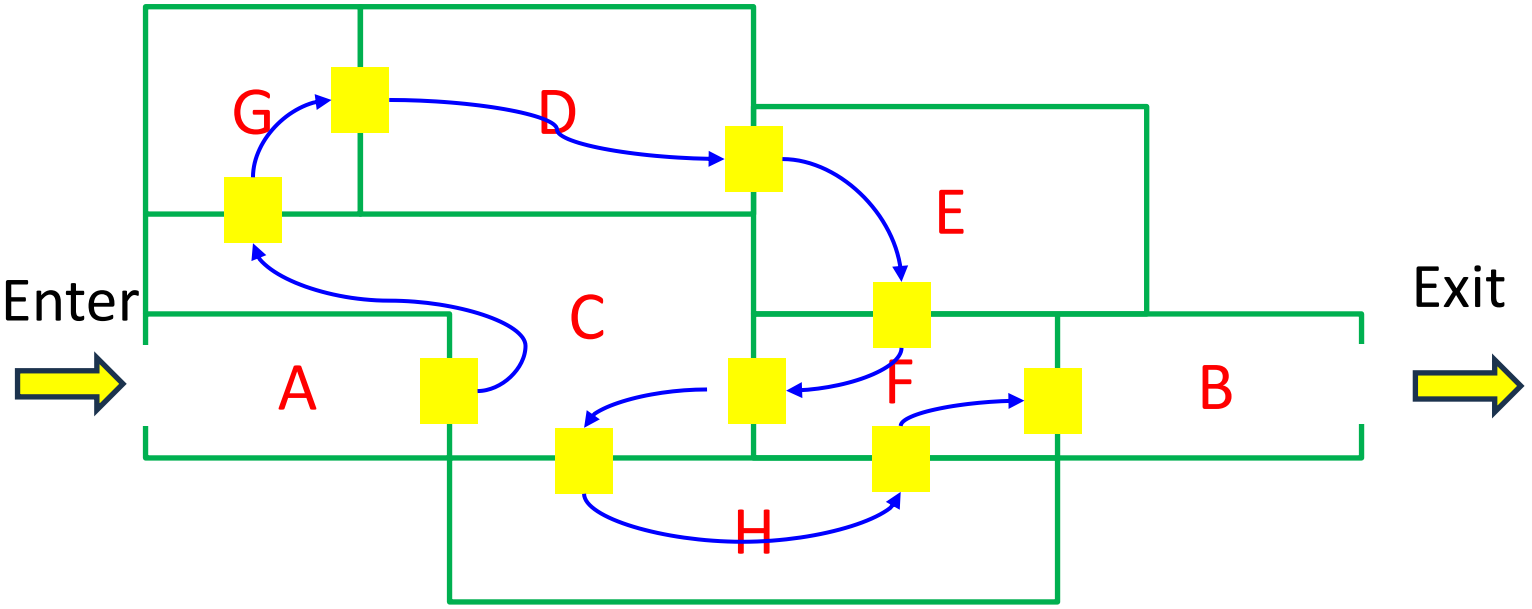
Exercise 2

- The diagram below shows the floor plan of a house that is open for public viewing. Is it possible to find a path that starts in Room A and ends in Room B, passing through every interior doorway of the house exactly once? If such a path exists, determine it.



Exercise 2: Solution



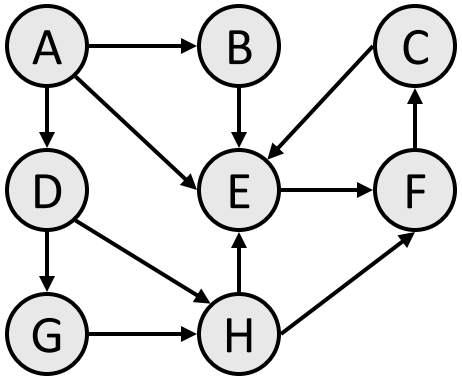


Each vertex of this graph has even degree except for *A* and *B*, each of which has degree 1. Hence by Corollary 1, there is an Euler path from *A* to *B*. One such trail is *ACGDEFCHB*.

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

- **Breadth-First Search** (BFS): Take one step down all paths and immediately backtrack.
  - A queue of vertices to visit.
  - Always returns the shortest path (one with the fewest edges):
    - to B: {A, B}
    - to C: **{A, E, F, C}**
    - to D: {A, D}
    - to E: **{A, E}**
    - to F: **{A, E, F}**
    - to G: {A, D, G}
    - to H: **{A, D, H}**



- **Depth-First Search** (DFS): Explore each possible path as far as possible before backtracking.
  - Often implemented recursively.
  - DFS paths from A to all vertices (assuming ABC edge order):
    - to B: {A, B}
    - to C: {A, B, E, F, C}
    - to D: {A, D}
    - to E: {A, B, E}
    - to F: {A, B, E, F}
    - to G: {A, D, G}
    - to H: {A, D, G, H}

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

BFS algorithm

BFS(G, start)

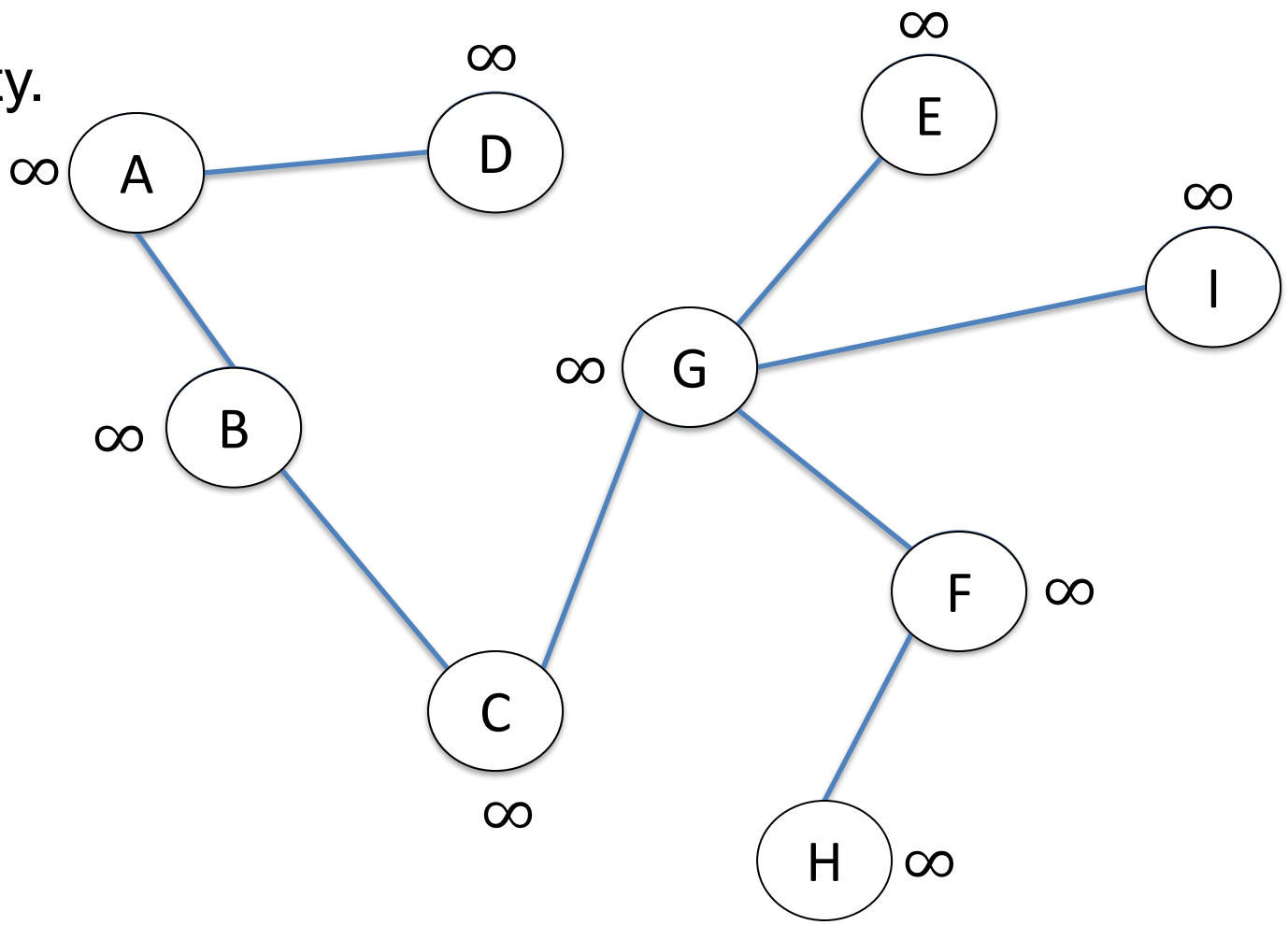
```
Create a new queue Q
Q.push(start)
dist[1..n] = {∞, …, ∞}
dist[start] = 0

while Q is not empty
    u = Q.pop()
    for each node v adjacent to u
        if dist[v] = ∞ then
            dist[v] = dist[u] + 1
            Q.push(v)
```

Suppose we have a graph $G = (V, E)$ containing $|V| = n$ nodes and $|E| = m$ edges.

BFS takes $O(n + m)$ time and space.

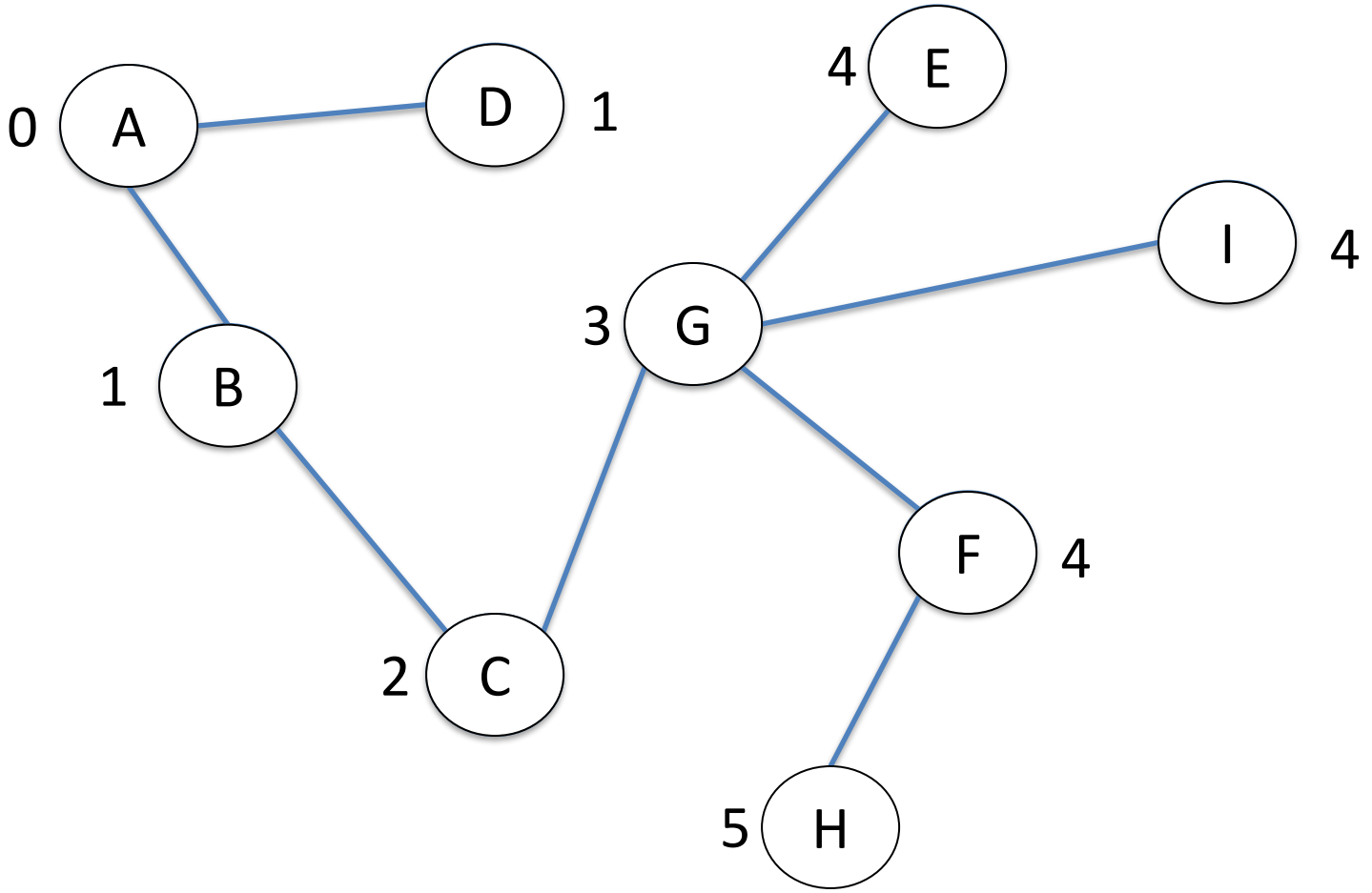
Example

- BFS starting at node A.
- All weights initially set to infinity.



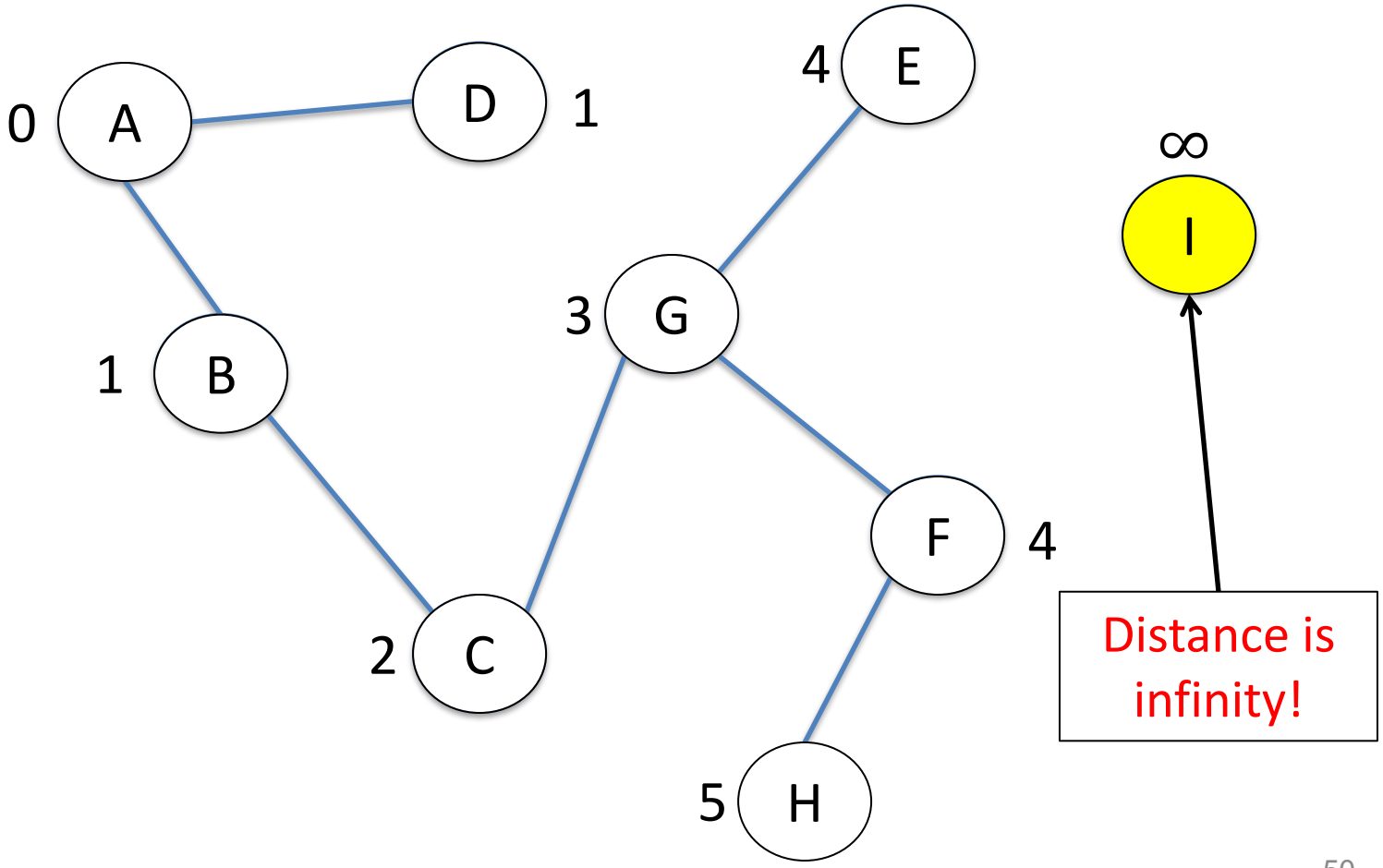
Application 1: checking connectedness (1/2)

- Run BFS on any node
- If no node has distance infinity, it's **connected**!



Application 1: checking connectedness (2/2)

- Run BFS on any node
- If no node has distance infinity, it's **connected**!



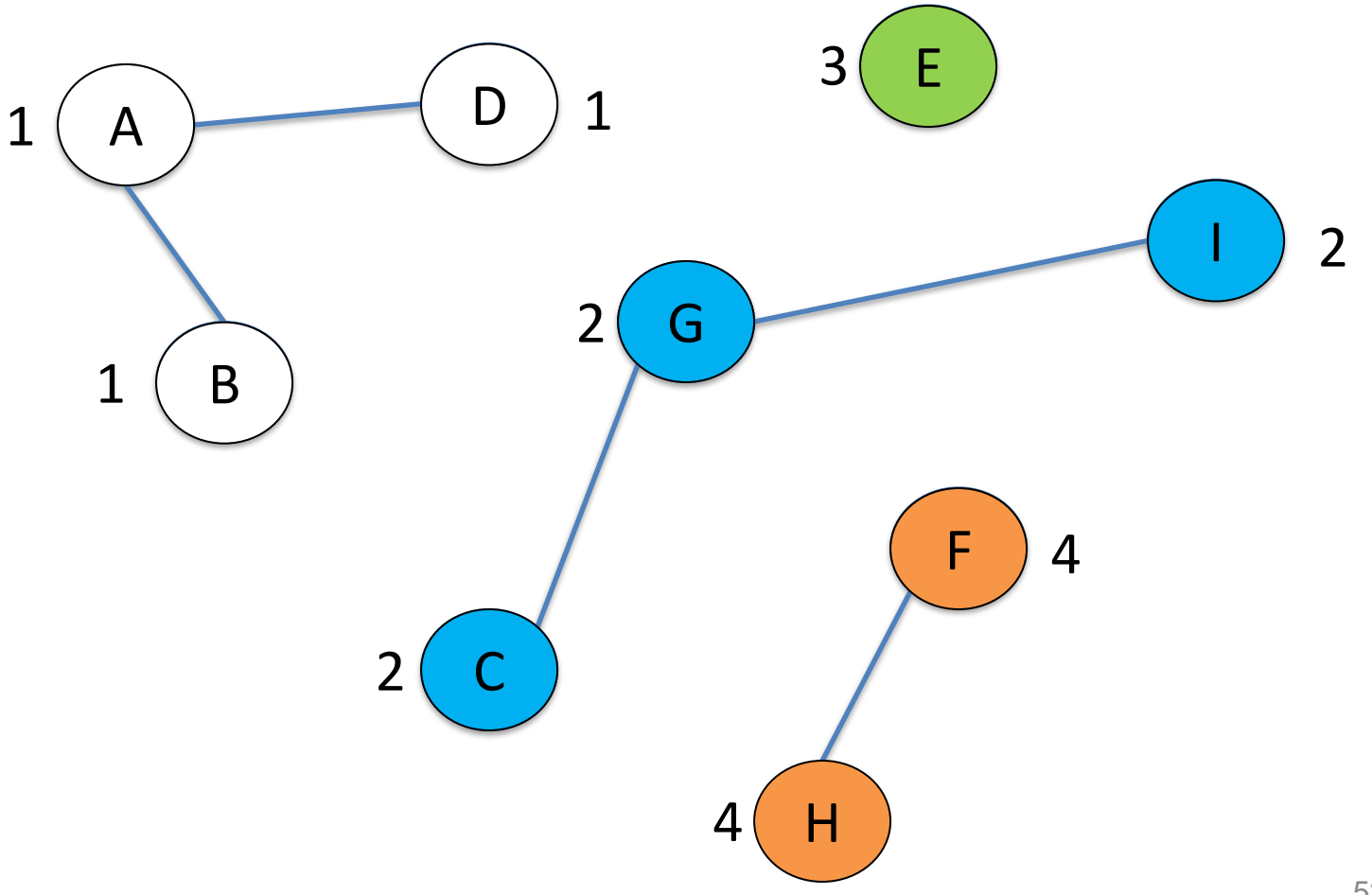
Application 2: finding connected components (1/2)

Algorithm:

```
label[1..n] ← 0                 # Initialize all node labels to 0 (unvisited)
id ← 1                          # Component ID counter

for u ← 1 to n:                 # Loop over all nodes
   if label[u] = 0:             # If node u is unvisited
      Q ← [u]                   # Start BFS from u
      label[u] ← id             # Assign current component ID

      while Q ≠ ∅:
         v ← Q.pop()            # Dequeue node v
         for w in neighbors(v):# Check neighbors of v
            if label[w] = 0:    # If neighbor is unvisited
               label[w] ← id    # Label it with current component ID
               Q.push(w)        # Enqueue neighbor

      id ← id + 1               # Move to next component ID
```

Application 2: finding connected components (1/2)

- Initially, each node has label 0



BFS algorithm

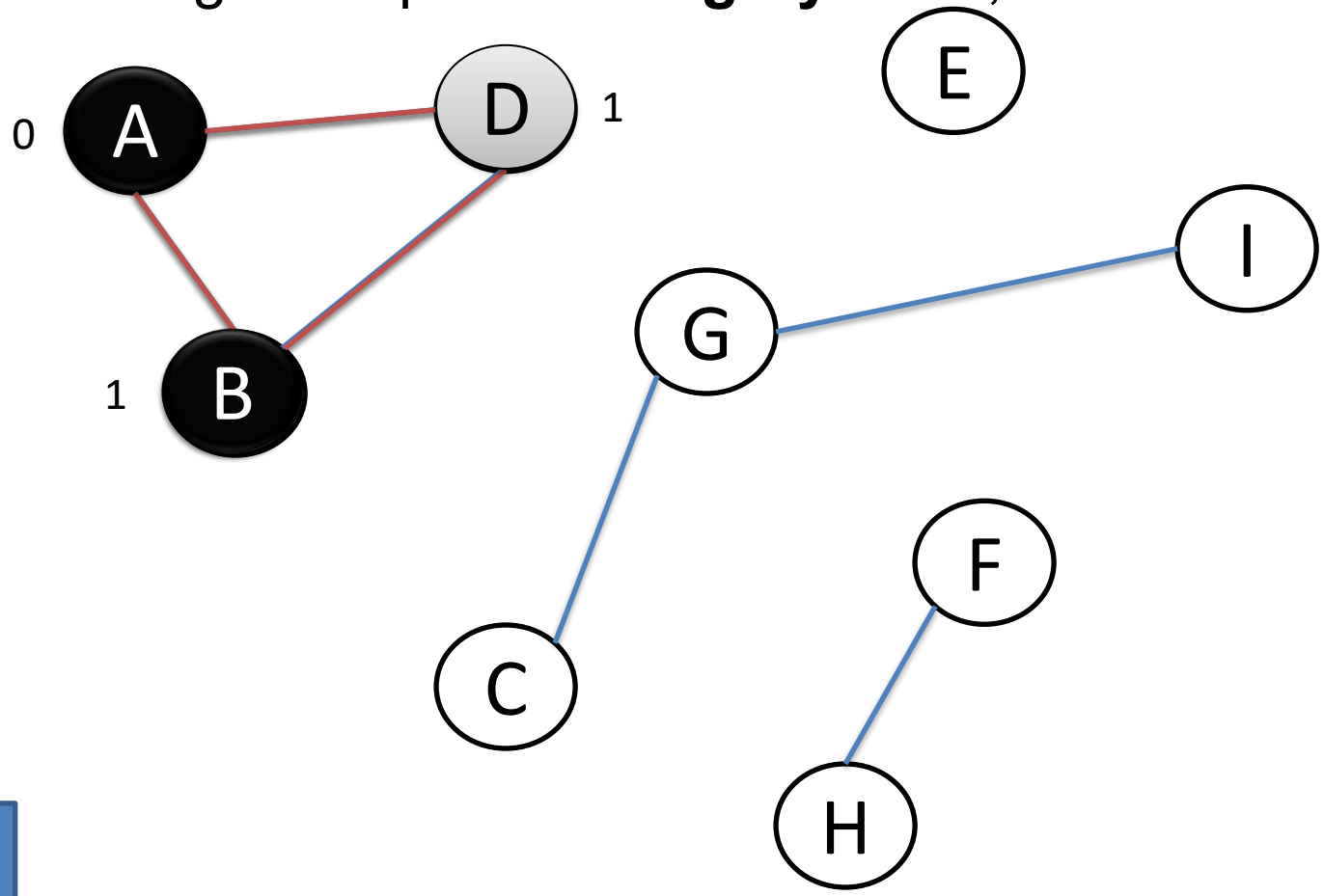
```
BFS(G, start):
    for u in G:                     # Initialize all nodes
        dist[u] ← ∞                 # Distance from start is infinite
        colour[u] ← white           # Mark all nodes unvisited

    dist[start] ← 0
    colour[start] ← gray            # Start node discovered
    Q ← [start]                     # Initialize queue with start

    while Q ≠ ∅:
        u ← Q.pop()                 # Dequeue next node
        for v in neighbors(u):      # Explore neighbors
            if colour[v] = white:   # If unvisited
                dist[v] ← dist[u] + 1   # Update distance
                colour[v] ← gray        # Mark as discovered
                Q.push(v)               # Enqueue neighbor
        colour[u] ← black           # Mark node as fully explored
```

Suppose we have a graph $G = (V, E)$ containing $|V| = n$ nodes and $|E| = m$ edges.

BFS takes $O(n + m)$ time and space.

Application 3: finding cycles

- Repeatedly do BFS from any unvisited node, until all nodes are visited.
- In any of these BFSs, if we see an edge that points to a **gray** node, then **there is a cycle**!



Enqueue node A

Dequeue node A

Enqueue node B

Enqueue node D

Dequeue node B

Try to enqueue A; black, so do nothing

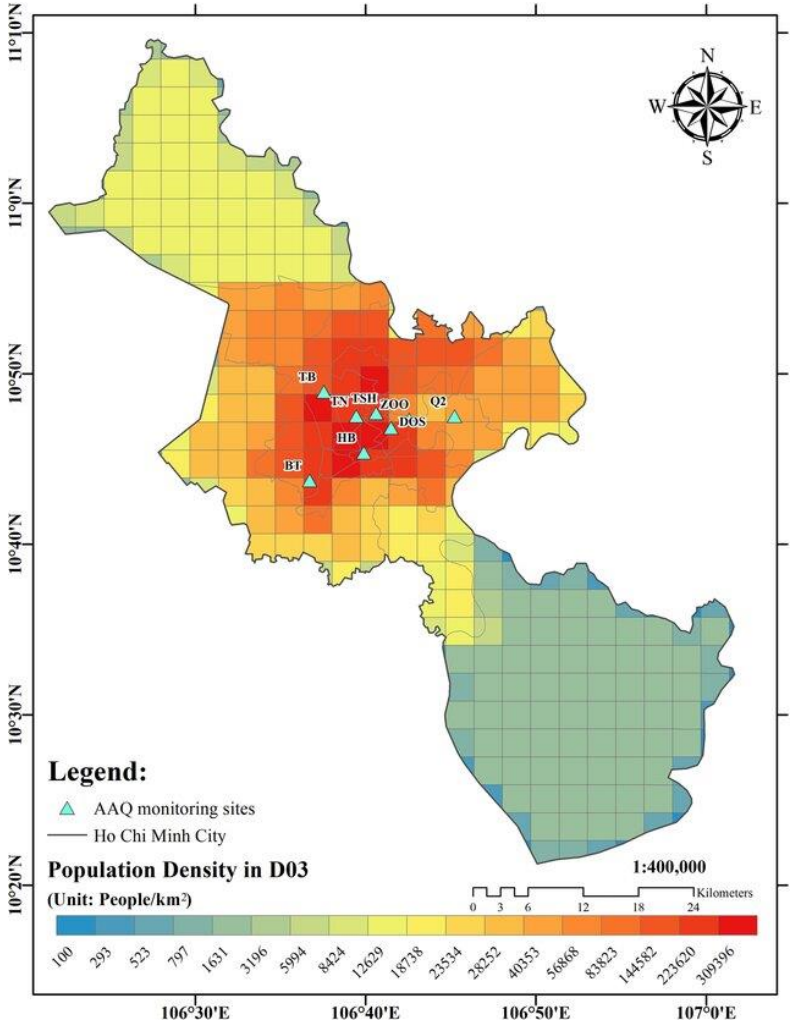
Try to enqueue D; gray node, so **cycle!**

Application 4: computing distance

- Suppose we have an $m \times n$ grid of areas.
- Initially, there is one area affected by an infectious disease!
- Each day, the virus spreads from each infected area to its nearest area (going up, down, left and right).
- How long before all area are infected?

- How can we represent this problem as a graph problem?
  - Nodes are areas.
  - There is an edge between two areas if the areas are adjacent (next to one another).

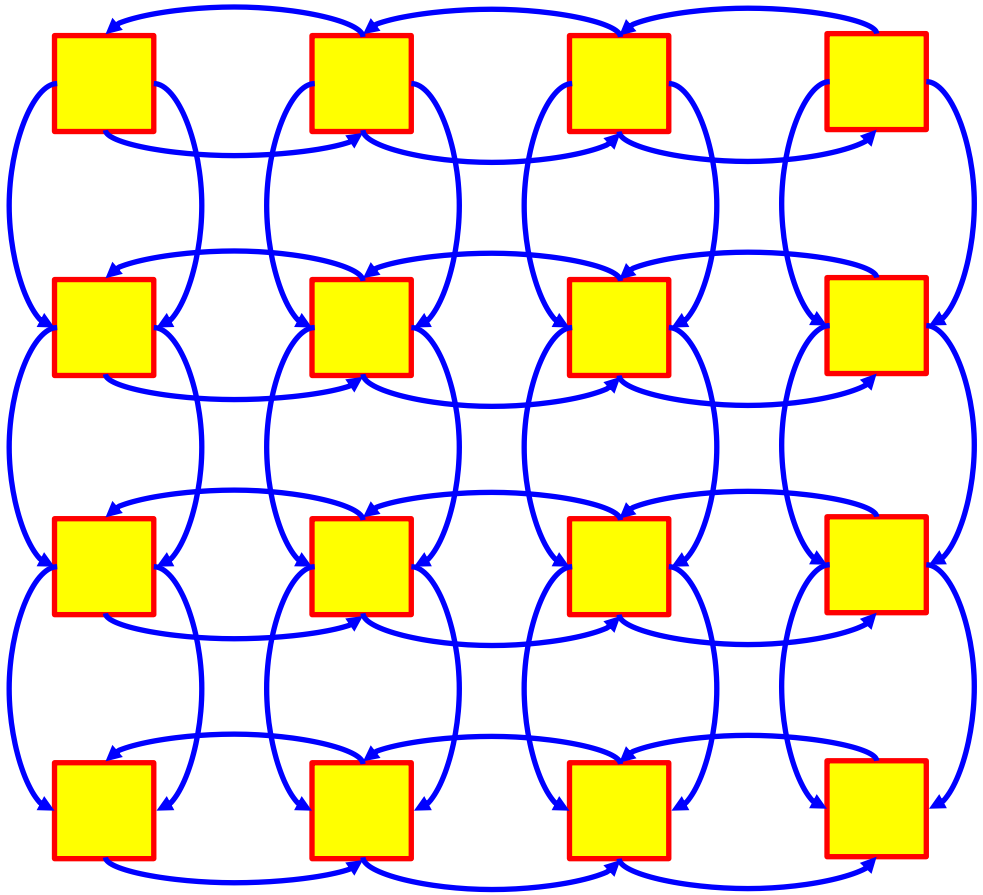
Application 4: computing distance (1/22)



Bui, Long Ta, and Phong Hoang Nguyen. "Evaluation of the annual economic costs associated with PM2. 5-based health damage—a case study in Ho Chi Minh City, Vietnam." *Air Quality, Atmosphere & Health* 16.3 (2023): 415-435.

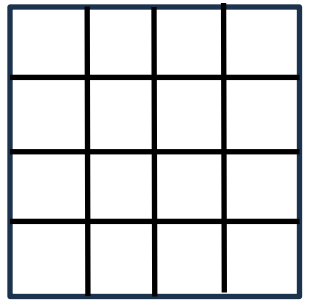
Application 4: computing distance (2/22)

- How should we store this graph in memory?
- One possibility (for a 4x4 grid of areas):



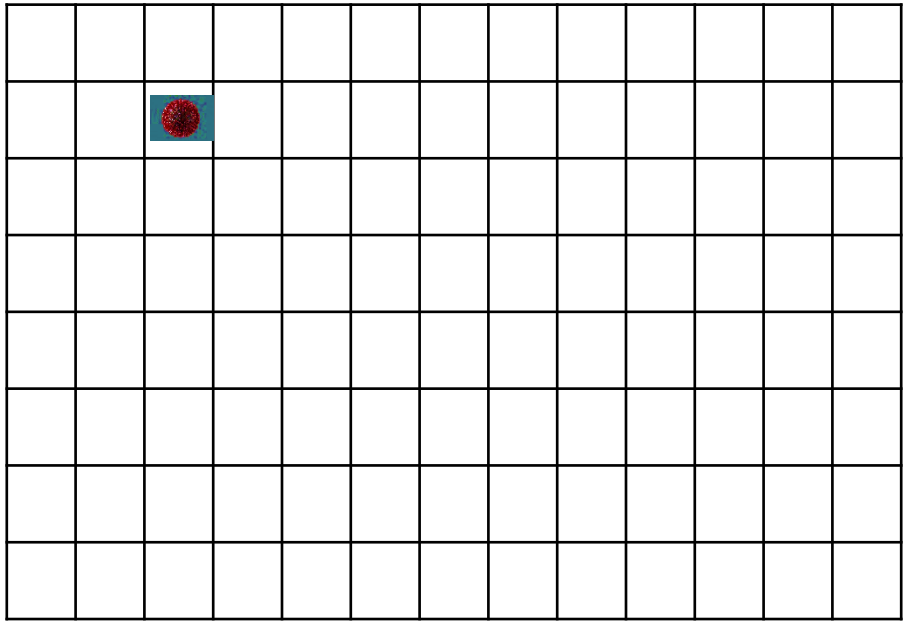
Wastes lots of memory!

- A more memory-efficient approach is to use a 4×4 array, where each cell corresponds to a specific area.



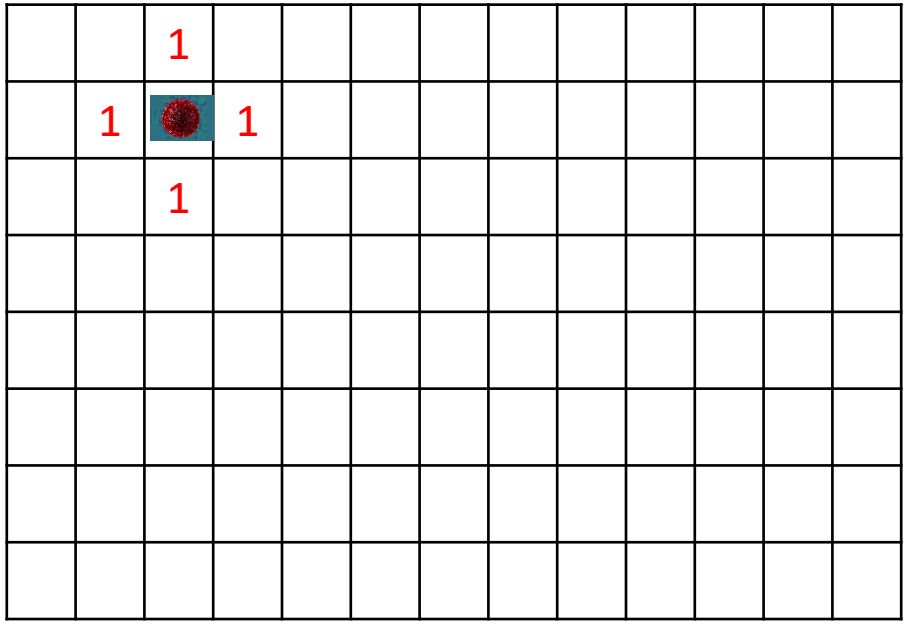
- To determine adjacency between areas, check whether their corresponding elements are adjacent (horizontally or vertically) in the array.

Application 4: computing distance (3/



Run BFS starting from the infected origin to compute the “distance” (actually time) to each area

Application 4: computing distance (4/



Application 4: computing distance (5/

	2	1	2									
2	1		1	2								
	2	1	2									
		2										

Application 4: computing distance (6/

3	2	1	2	3								
2	1		1	2	3							
3	2	1	2	3								
	3	2	3									
		3										

Application 4: computing distance (7/

3	2	1	2	3	4							
2	1		1	2	3	4						
3	2	1	2	3	4							
4	3	2	3	4								
	4	3	4									
		4										

Application 4: computing distance (8/

3	2	1	2	3	4	5						
2	1		1	2	3	4	5					
3	2	1	2	3	4	5						
4	3	2	3	4	5							
5	4	3	4	5								
	5	4	5									
		5										

Application 4: computing distance (9/

3	2	1	2	3	4	5	6					
2	1		1	2	3	4	5	6				
3	2	1	2	3	4	5	6					
4	3	2	3	4	5	6						
5	4	3	4	5	6							
6	5	4	5	6								
	6	5	6									
		6										

Application 4: computing distance (10/

3	2	1	2	3	4	5	6	7				
2	1		1	2	3	4	5	6	7			
3	2	1	2	3	4	5	6	7				
4	3	2	3	4	5	6	7					
5	4	3	4	5	6	7						
6	5	4	5	6	7							
7	6	5	6	7								
	7	6	7									

Application 4: computing distance (11/

3	2	1	2	3	4	5	6	7	8			
2	1		1	2	3	4	5	6	7	8		
3	2	1	2	3	4	5	6	7	8			
4	3	2	3	4	5	6	7	8				
5	4	3	4	5	6	7	8					
6	5	4	5	6	7	8						
7	6	5	6	7	8							
8	7	6	7	8								

Application 4: computing distance (12/

3	2	1	2	3	4	5	6	7	8	9		
2	1		1	2	3	4	5	6	7	8	9	
3	2	1	2	3	4	5	6	7	8	9		
4	3	2	3	4	5	6	7	8	9			
5	4	3	4	5	6	7	8	9				
6	5	4	5	6	7	8	9					
7	6	5	6	7	8	9						
8	7	6	7	8	9							

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	
4	3	2	3	4	5	6	7	8	9	10		
5	4	3	4	5	6	7	8	9	10			
6	5	4	5	6	7	8	9	10				
7	6	5	6	7	8	9	10					
8	7	6	7	8	9	10						

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	
5	4	3	4	5	6	7	8	9	10	11		
6	5	4	5	6	7	8	9	10	11			
7	6	5	6	7	8	9	10	11				
8	7	6	7	8	9	10	11					

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	
6	5	4	5	6	7	8	9	10	11	12		
7	6	5	6	7	8	9	10	11	12			
8	7	6	7	8	9	10	11	12				

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	
7	6	5	6	7	8	9	10	11	12	13		
8	7	6	7	8	9	10	11	12	13			

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	14
7	6	5	6	7	8	9	10	11	12	13	14	
8	7	6	7	8	9	10	11	12	13	14		

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	14
7	6	5	6	7	8	9	10	11	12	13	14	15
8	7	6	7	8	9	10	11	12	13	14	15	

Application 4: computing distance (13/

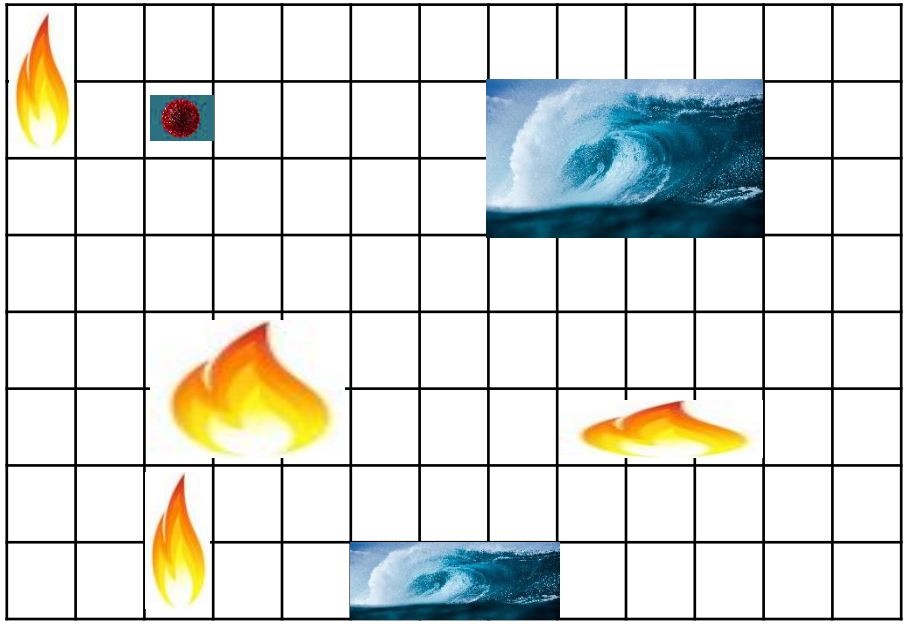
3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	14
7	6	5	6	7	8	9	10	11	12	13	14	15
8	7	6	7	8	9	10	11	12	13	14	15	16

16 hours until all areas are infected!

average time = 7,28 hours

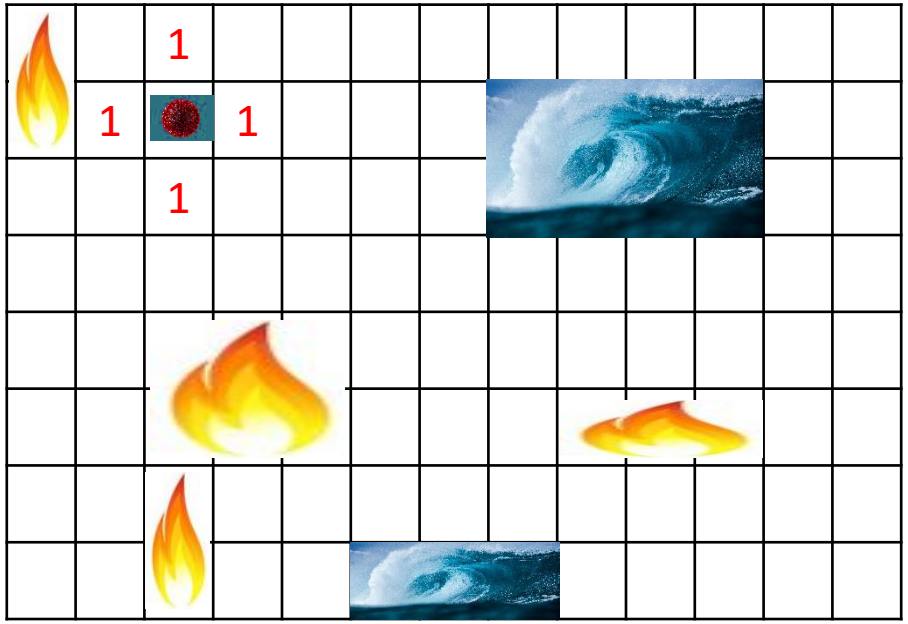
Application 4 (modified): computing distance with fires and oceans (1/17)

What if there are fires and oceans in the grid, where viruses cannot pass (bc of being killed)?

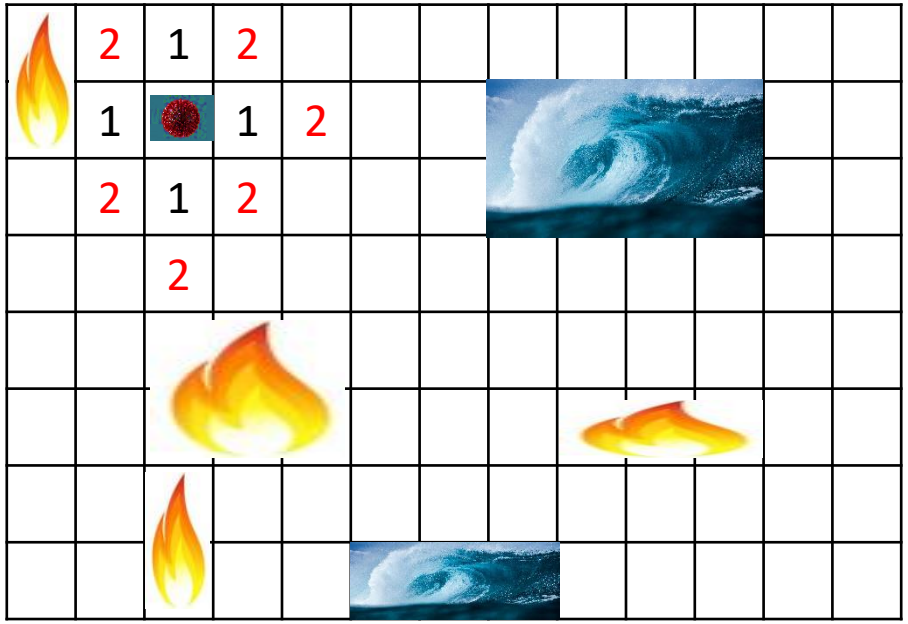


Just don’t let BFS visit the fires and oceans!

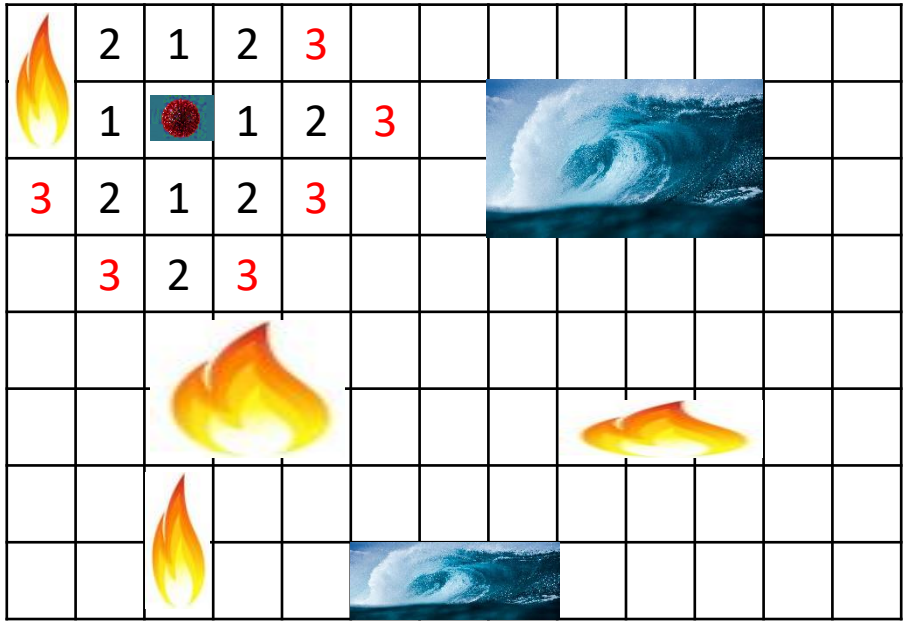
Application 4 (modified): computing distance with fires and oceans (2/17)



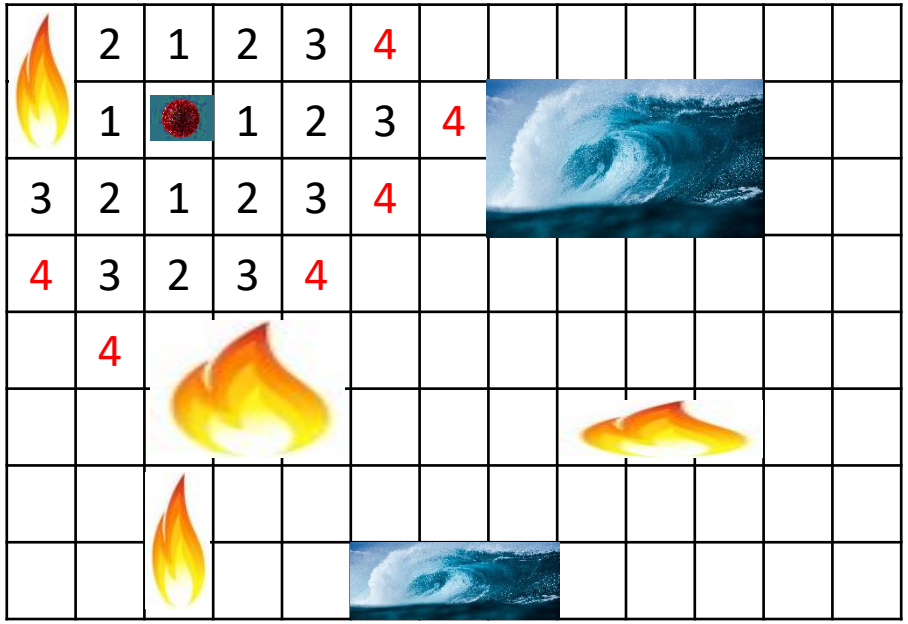
Application 4 (modified): computing distance with fires and oceans (3/17)



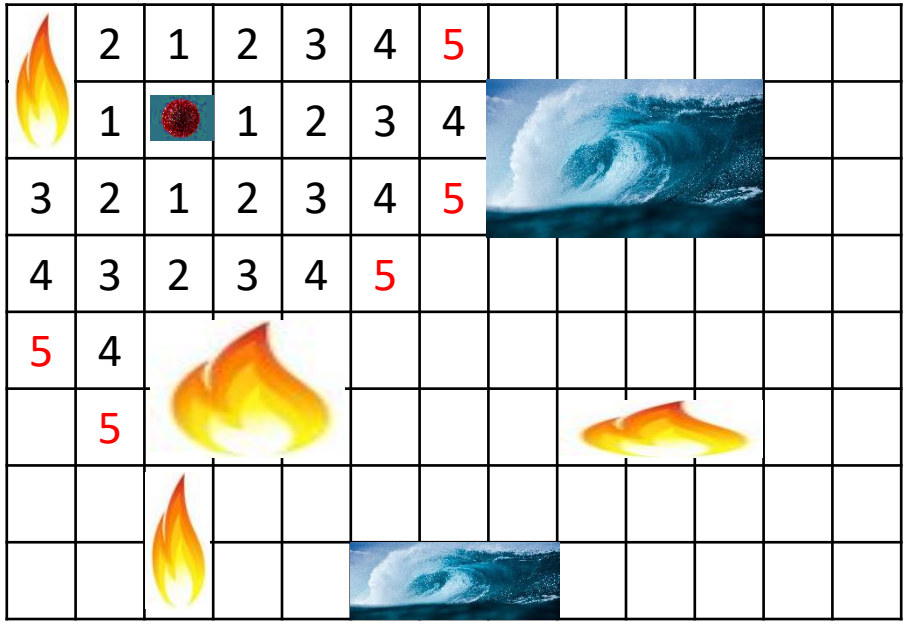
Application 4 (modified): computing distance with fires and oceans (4/17)



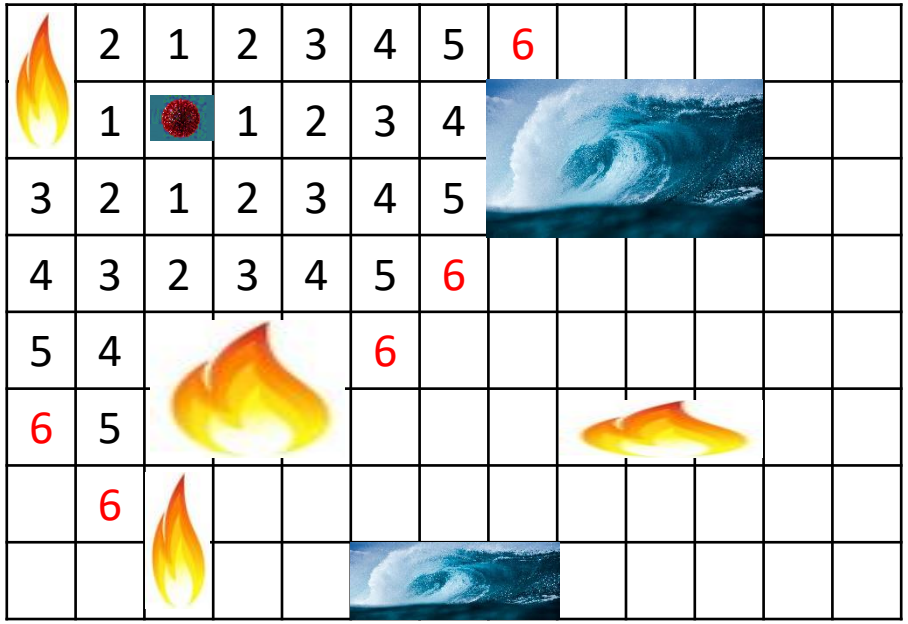
Application 4 (modified): computing distance with fires and oceans (5/17)



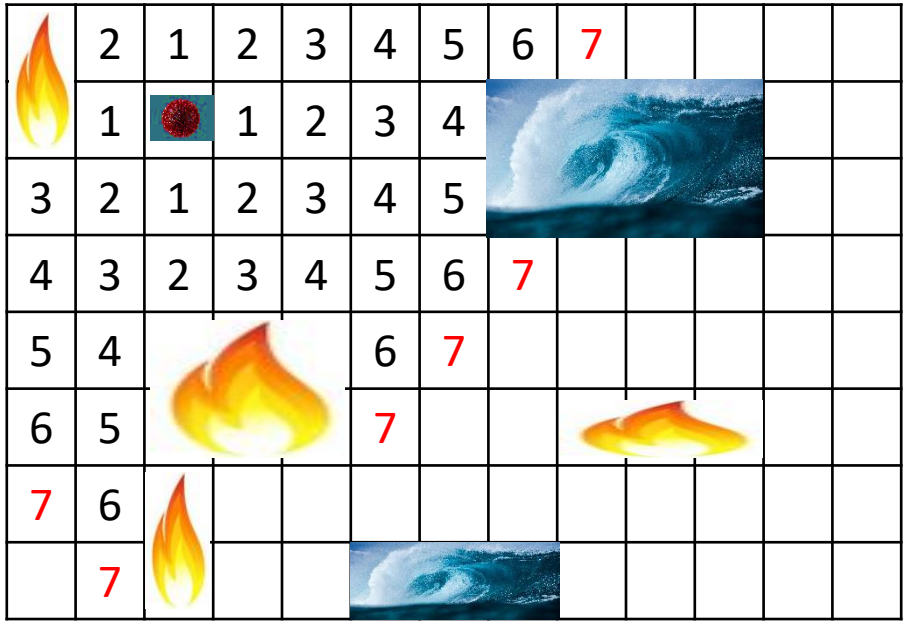
Application 4 (modified): computing distance with fires and oceans (6/17)



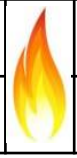
Application 4 (modified): computing distance with fires and oceans (7/17)



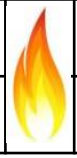
Application 4 (modified): computing distance with fires and oceans (8/17)



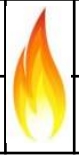
Application 4 (modified): computing distance with fires and oceans (9/17)

	2	1	2	3	4	5	6	7	8			
	1		1	2	3	4						
3	2	1	2	3	4	5						
4	3	2	3	4	5	6	7	8				
5	4				6	7	8					
6	5				7	8						
7	6				8							
8	7											

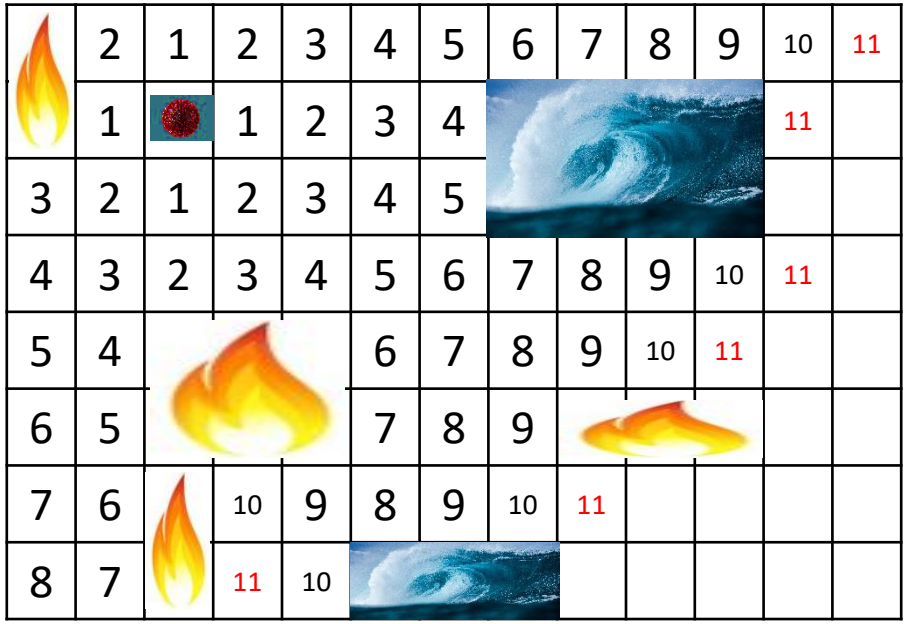
Application 4 (modified): computing distance with fires and oceans (10/17)

	2	1	2	3	4	5	6	7	8	9		
	1		1	2	3	4						
3	2	1	2	3	4	5						
4	3	2	3	4	5	6	7	8	9			
5	4				6	7	8	9				
6	5				7	8	9					
7	6			9	8	9						
8	7											

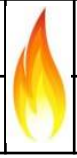
Application 4 (modified): computing distance with fires and oceans (11/17)

	2	1	2	3	4	5	6	7	8	9	10	
	1		1	2	3	4						
3	2	1	2	3	4	5						
4	3	2	3	4	5	6	7	8	9	10		
5	4				6	7	8	9	10			
6	5				7	8	9					
7	6		10	9	8	9	10					
8	7			10								

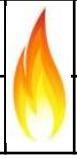
Application 4 (modified): computing distance with fires and oceans (12/17)



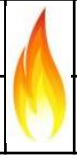
Application 4 (modified): computing distance with fires and oceans (13/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	
6	5				7	8	9					
7	6		10	9	8	9	10	11	12			
8	7		11	10				12				

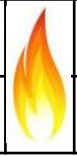
Application 4 (modified): computing distance with fires and oceans (14/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	13
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	13
6	5				7	8	9				13	
7	6		10	9	8	9	10	11	12	13		
8	7		11	10			12	13				

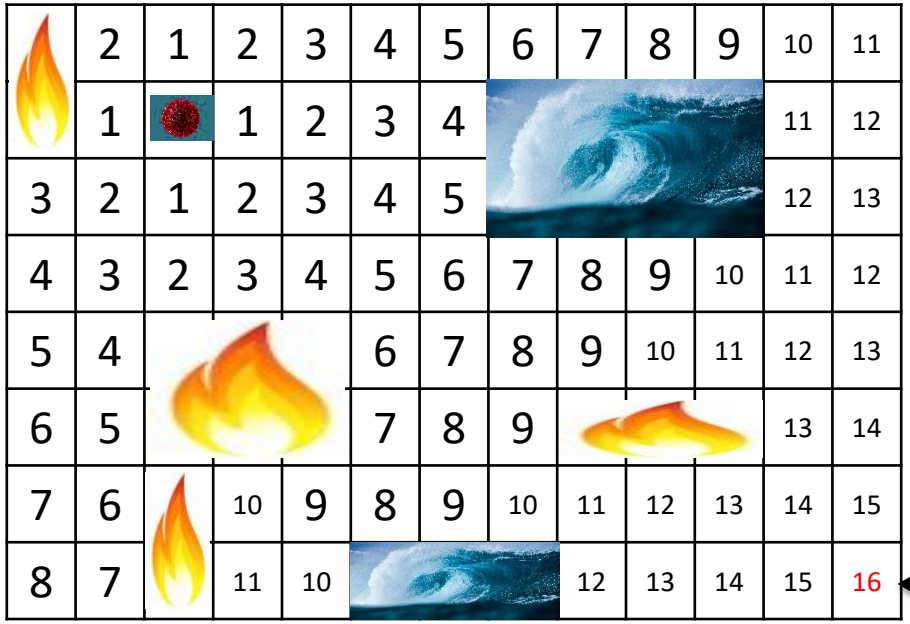
Application 4 (modified): computing distance with fires and oceans (15/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	13
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	13
6	5				7	8	9				13	14
7	6		10	9	8	9	10	11	12	13	14	
8	7		11	10			12	13	14			

Application 4 (modified): computing distance with fires and oceans (16/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	13
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	13
6	5				7	8	9				13	14
7	6		10	9	8	9	10	11	12	13	14	15
8	7		11	10			12	13	14	15		

Application 4 (modified): computing distance with fires and oceans (17/17)

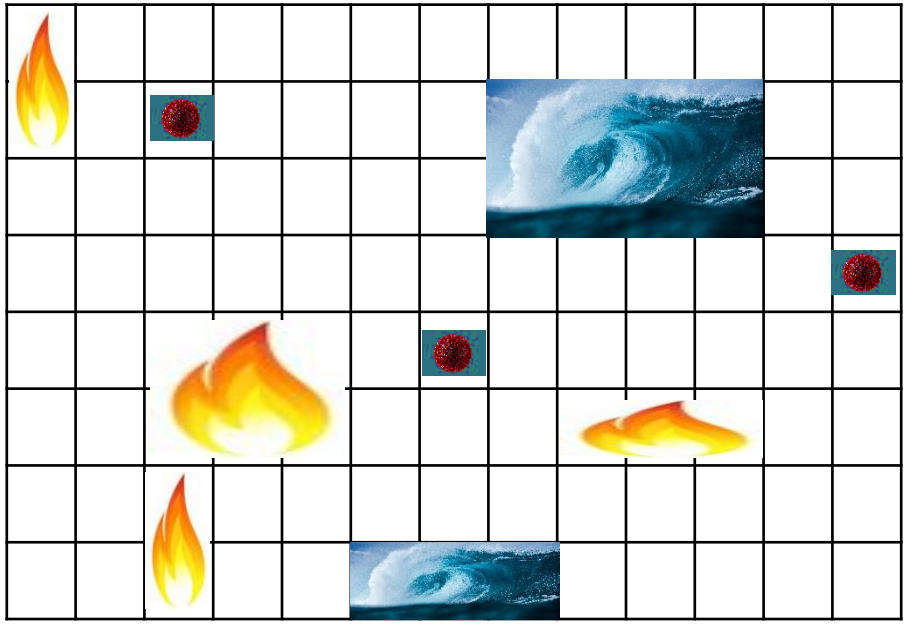


average time = 7,55 hours

> 7,28 hours when no obstacles encountered

Application 4 (modified 2): computing distance with multiple viruses (1/2)

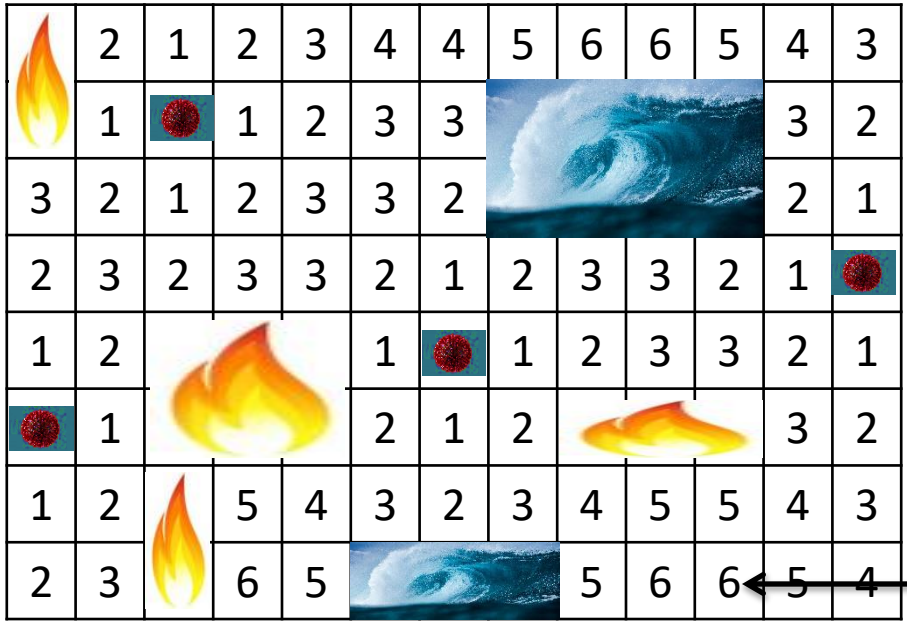
What if there are many sources of viruses?



The time that an area is reached is the smallest among ALL VIRUSES

Application 4 (modified 2): computing distance with multiple viruses (2/2)

What if there are many sources of viruses?



average time = 2,8 hours

Application 4 (modified 3): other models

- What if viruses start at different times?
- What if some viruses stop spreading?
- What if viruses spread at different speeds?

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

Depth-First Search (DFS): Recursive algorithm

- Goes as far as possible from a vertex before backing up.

```
DFS(v: vertex)
{
        Mark v as visited
        for (each unvisited vertex u adjacent to v)
                DFS(u)
}
```

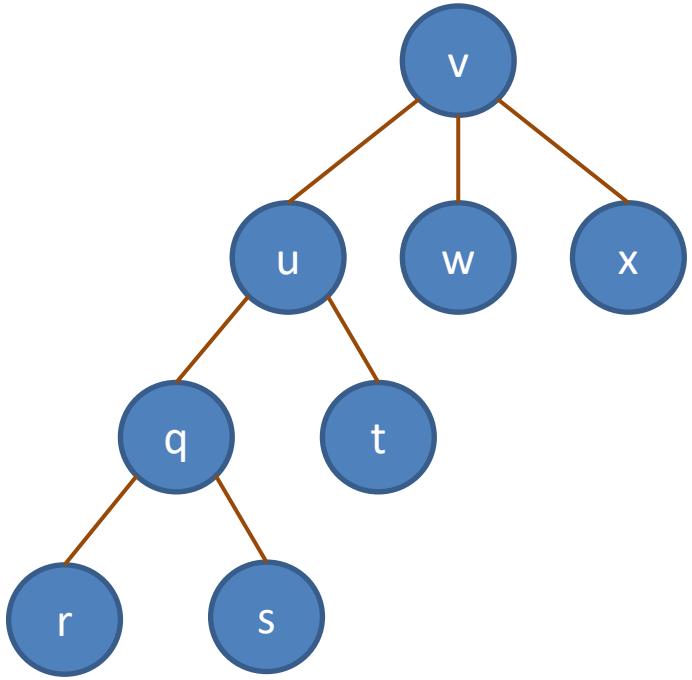
Depth-First Search (DFS): Stack-based algorithm

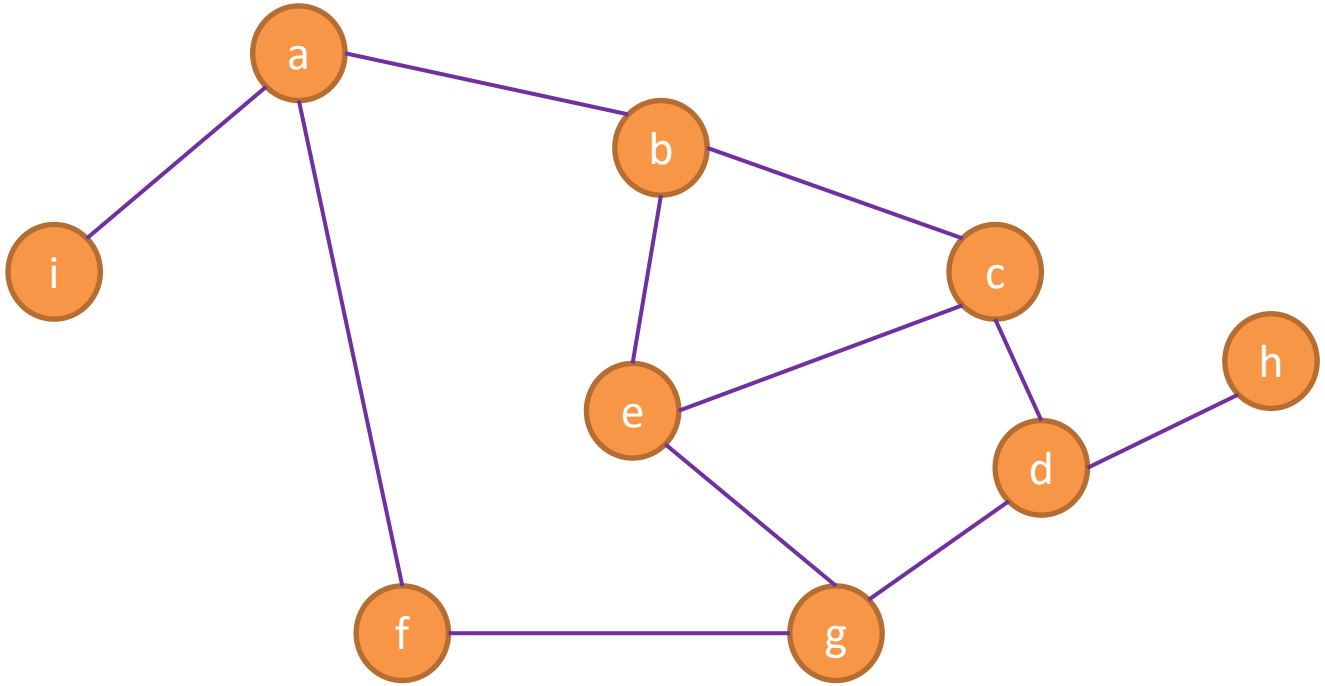
```
DFS(v: vertex)
        s = new empty stack
        s.push(v)
        Mark v as visited
        while (s is not empty)  {
                if (no unvisited vertices are adjacent to the vertex on the top of the stack)
                        s.pop()
                else {
                        s.push(u)
                        Marked u as visited
                }
        }
```

Depth-First Search (DFS): Stack-based algorithm

```
DFS(v: vertex)
        s = new empty stack
        s.push(v)
        Mark v as visited
        while (s is not empty)  {
                if (no unvisited vertices are adjacent to the vertex on the top of the stack)
                        s.pop()
                else {
                        s.push(u)
                        Marked u as visited
                }
        }
```

Depth-First Search (DFS): Example





v - u - q - r - s - t - w - x

DFS starts at ***a***:

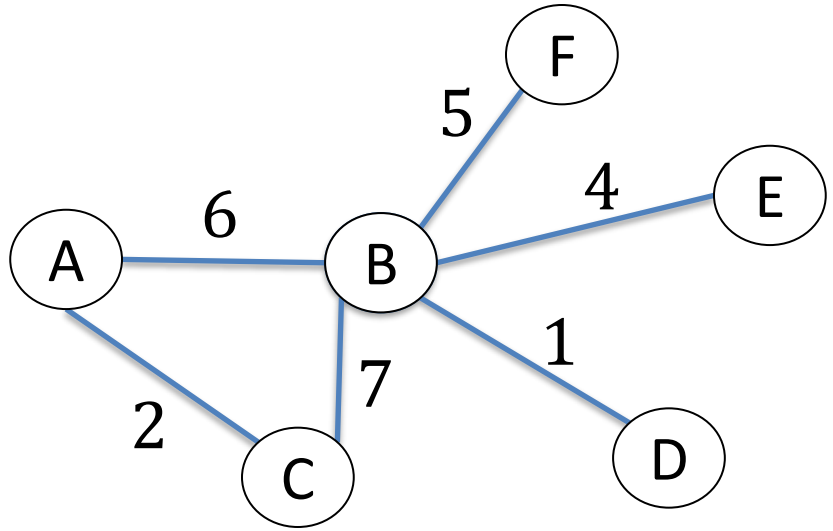
DFS starts at ***e***:

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
   1. Adjacency matrix
   2. Adjacency list
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

Matrix representation

Use a square matrix $A[n][n]$ with n is the number of vertices



$$A[i][j] = \begin{cases} w, \text{where } w \text{ is the weight of edge } (i,j) \\ 0, \text{if there is no edge } (i,j) \end{cases}$$

$$A[i][j] = \begin{cases} w, \text{where } w \text{ is the weight of edge } (i,j) \\ \infty, \text{if there is no edge } (i,j) \end{cases}$$

	A	B	C	D	E	F
A	0	6	2	0	0	0
B	6	0	7	1	4	5
C	2	7	0	0	0	0
D	0	1	0	0	0	0
E	0	4	0	0	0	0
F	0	5	0	0	0	0

Space costs $|V|^2$

	A	B	C	D	E	F
A	0	6	2	0	0	0
B	6	0	7	1	4	5
C	2	7	0	0	0	0
D	0	1	0	0	0	0
E	0	4	0	0	0	0
F	0	5	0	0	0	0

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
   1. Adjacency matrix
   2. Adjacency list
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

List representation

Use $n = 6$ lists representing for connected vertices to a vertex

A→C,2→B,6

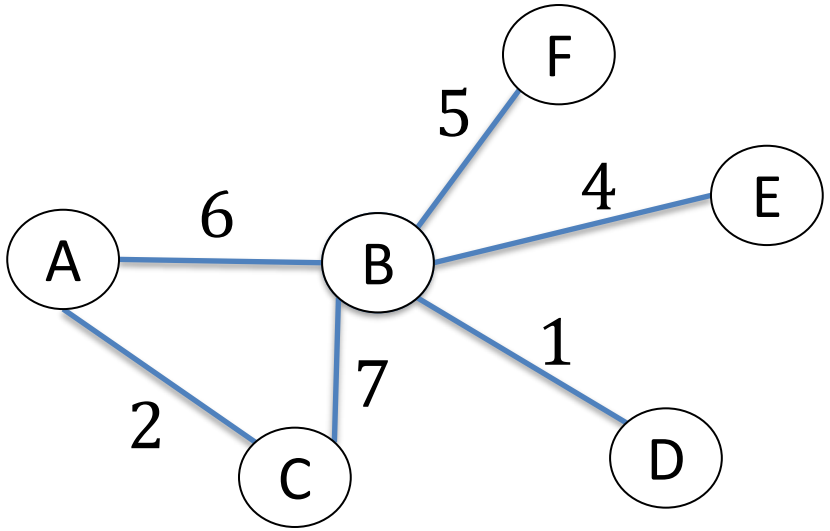
B→A,6→C,7→D,1→E,4→F,5

C→A,2→B,7

D→B,1

E→B,4

F→B,5



Space costs $|V| + |E|$

Each row contains an edge and its weight

A C 2

A B 6

B C 7

B D 1

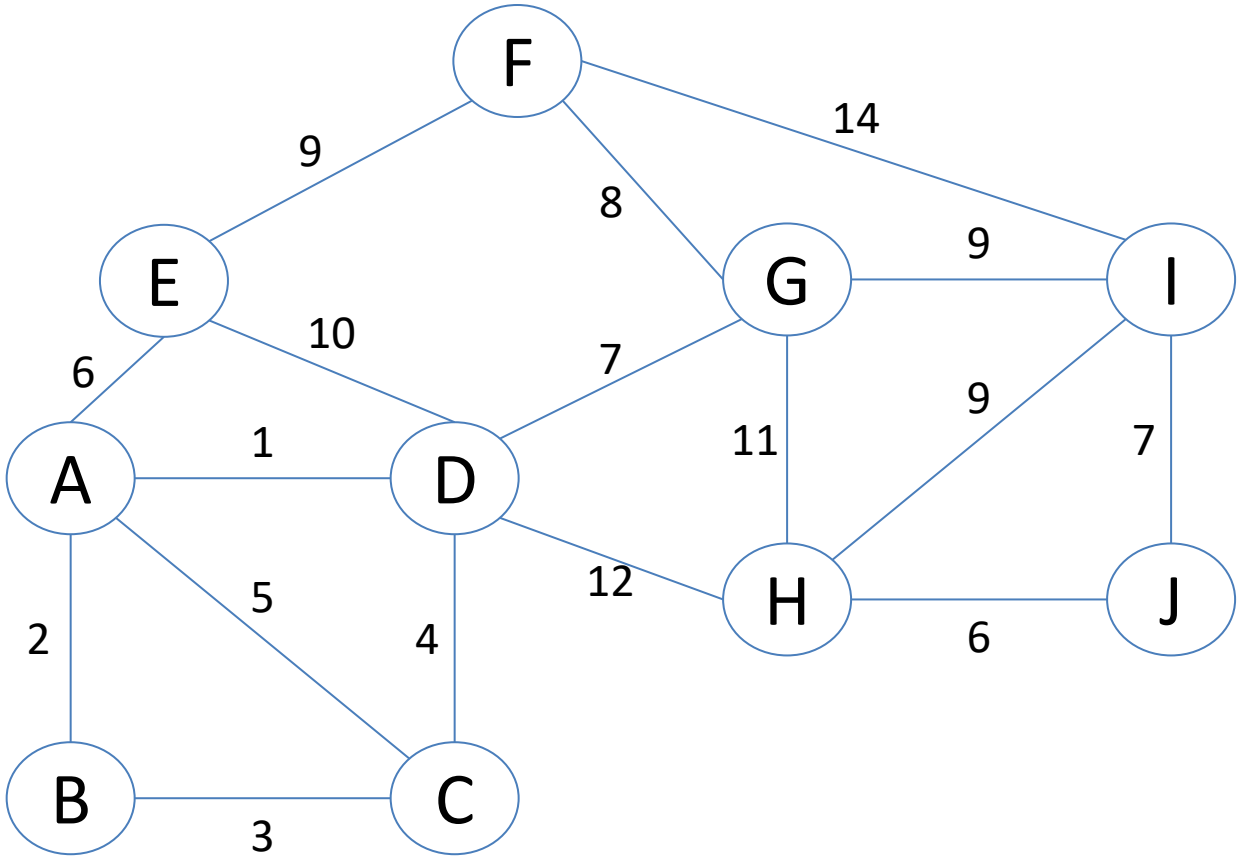
B E 4

B F 5

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

A Networking Problem



find a minimum (cost) spanning tree

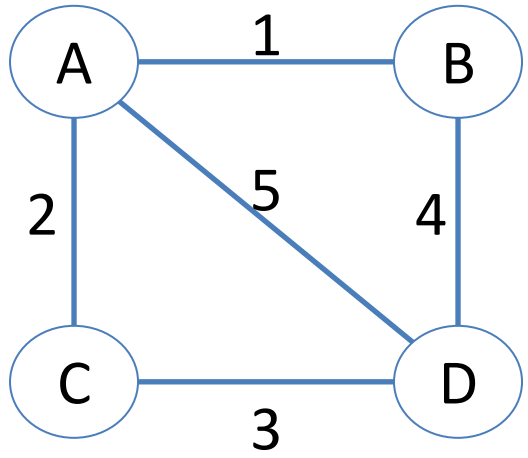
The vertices represent 10 regional data centers that need to be connected with high-speed data lines. Feasibility studies show that the links illustrated above are possible, and the cost in millions of USD is displayed next to the link. Which links should be constructed to enable full communication (with relays allowed) and keep the total cost minimal?

What is a Minimum (Cost) Spanning Tree (MST)? (1/4)

- Tree:
  - A structure with no cycles; meaning there is exactly one unique path between any two nodes.
- Spanning:
  - The tree includes all nodes in the graph.
- Minimum Cost:
  - Among all possible spanning trees, this one has the lowest total edge weight.

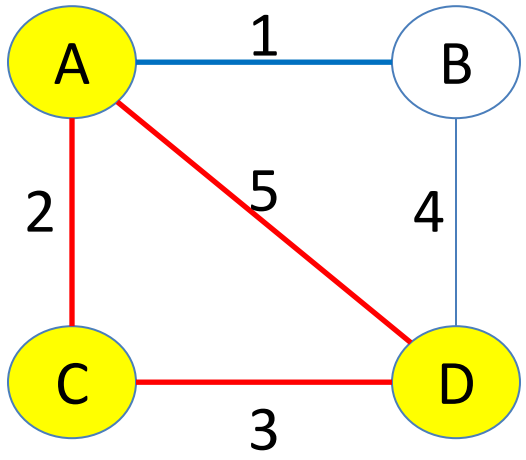
What is a Minimum (Cost) Spanning Tree (MST)? (2/4)

- Let's think about simple MSTs on this graph:

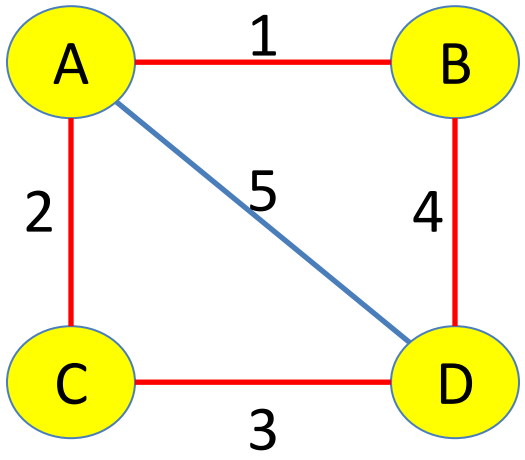


What is a Minimum (Cost) Spanning Tree (MST)? (3/4)

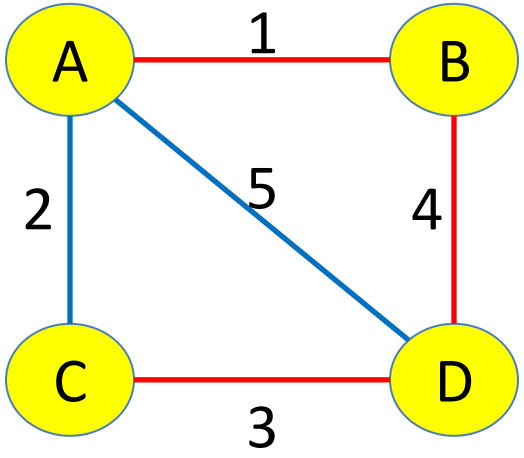
- Red edges and nodes are in T
- Is T a minimum cost spanning tree?



Not spanning; B is not in T.



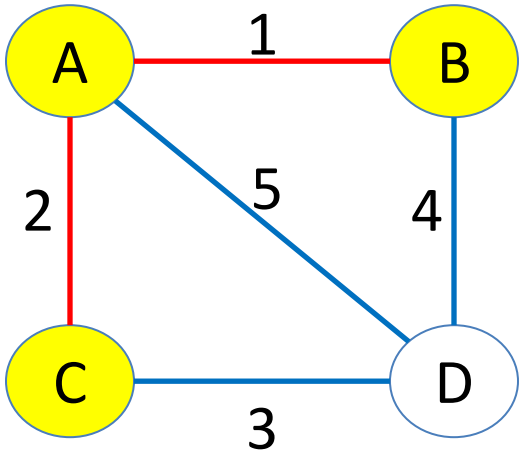
Not a tree; has a cycle.

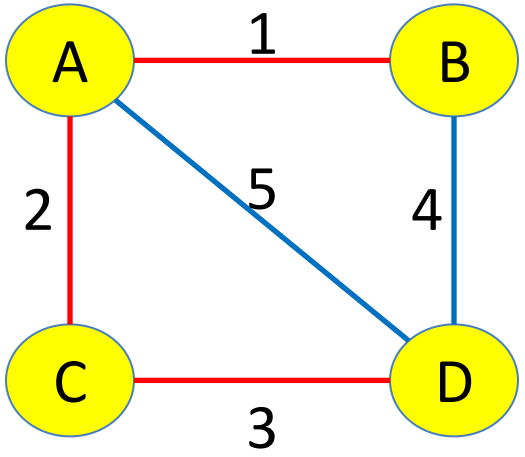


Not minimum cost; can swap edges weighted 4 and 2.

What is a Minimum (Cost) Spanning Tree (MST)? (4/4)

- Which edges form an MST?





Quiz

- If we build an MST from a graph G = (V, E), how many edges does the MST have?

- When can we find an MST for a graph?

An application of MSTs

- Electronic circuit designs [1]
  - Circuits often need to wire together the pins of several components to make them electrically equivalent.
  - To connect n pins, we can use $n - 1$ wires, each connecting two pins.
  - Want to use the minimum amount of wire.
  - Model a problem with a graph where each pin is a node, and every possible wire between a pair of pins is an edge.

[1] Cormen, Thomas H., et al. *Introduction to algorithms*. MIT press, 2022.

A few other applications of MSTs

- Clustering Algorithms:
  - MSTs can be used as a basis for some clustering algorithms. By removing the longest edges in an MST, the tree can be broken into disconnected components, which can represent clusters.
- Network Design (Telecommunications, Computer Networks, Utilities):
  - Designing power grids or water distribution networks with the minimum cost of laying down the infrastructure.
  - Optimizing the path data as it travels across routers to its destination on the internet.
- Generating a 2D maze design such as the Pacman game.

Prim's vs. Kruskal's algorithms

- Two famous algorithms for finding an MST: Prim's and Kruskal's algorithms.

Feature	Kruskal's	Prim's
Approach	Edge-based	Vertex-based
Best for	Sparse graphs	Dense graphs
Data structure needed	Union-Find	Priority Queue (Heap)
Time Complexity	$O(\|E\| \log \|E\|)$	$O\big((\|V\| + \|E\|) \log \|V\|\big)$ with Binary Heap
Works well with	Edge list	Adjacency list or matrix

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

History

Vojtěch Jarník
(1897–1970)
(dml.cz/jarnik-en)

Robert Clay Prim
(1921–2021)
do.ithistory.org/honor-roll/dr-robert-clay-prim

Edsger Wybe Dijkstra
(1930–2002)
amturing.acm.org/award_winners/dijkstra_1053701.cfm

- The algorithm was developed in 1930 by the Czech mathematician Vojtěch Jarník [1].
- Later rediscovered and republished by computer scientists Robert C. Prim in 1957 [2] and Edsger W. Dijkstra in 1959 [3].

[1] Boruvka, O. "O jistém problému minimálním (about a certain minimal problem). Práce moravské prírodovedecké spolecnosti v Brne 3: 37–58." (1926).
[2] Prim, Robert Clay. "Shortest connection networks and some generalizations." *The Bell System Technical Journal* 36.6 (1957): 1389-1401.
[3] Dijkstra, Edsger W. "A note on two problems in connexion with graphs." *Edsger Wybe Dijkstra: his life, work, and legacy*. 2022. 287-290.

Building an MST

- Prim's algorithm takes a graph $G = (V, E)$ and builds an MST with a list of vertices T and its list of edges L

PrimMST(V, E)

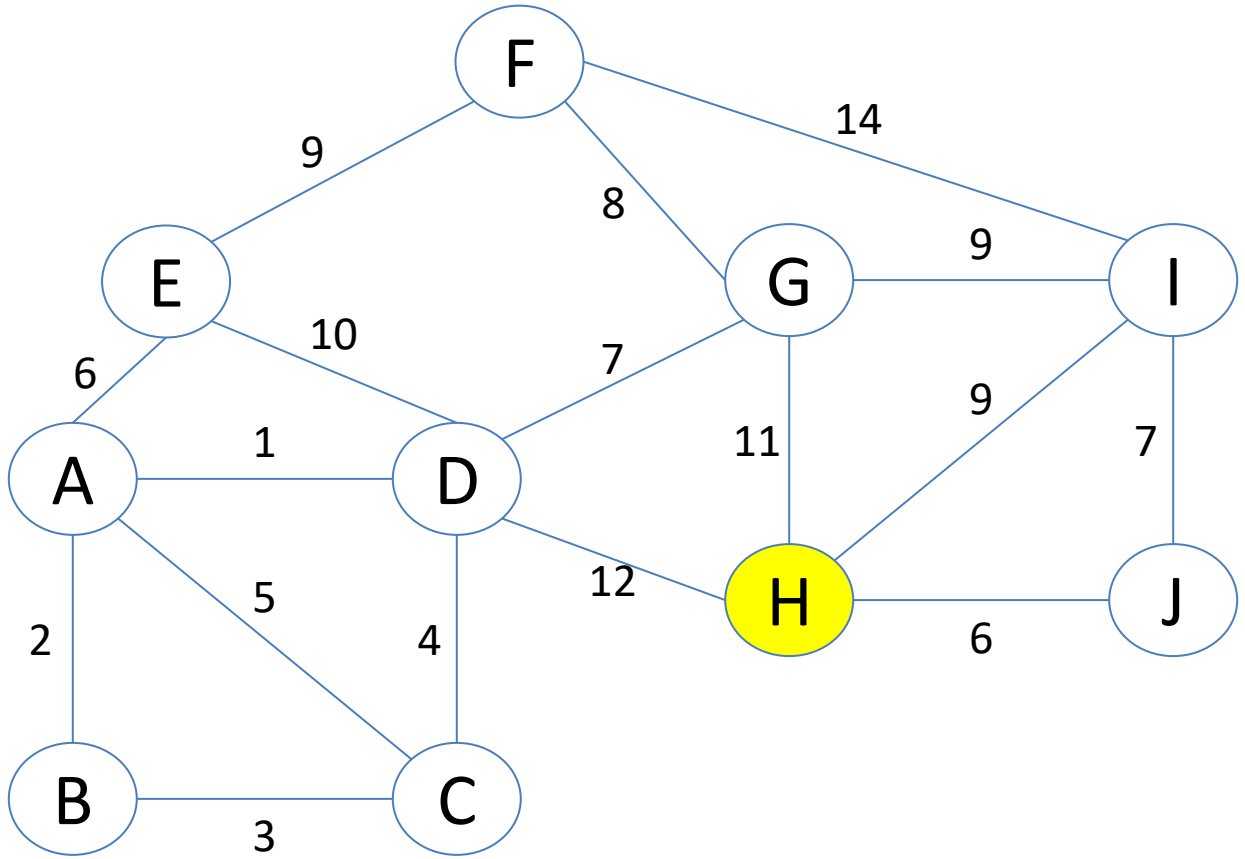
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L

$v \in \bar{T} = V \setminus T$

Example (1/11)

- Find an MST starting at vertex h by using Prim's algorithm

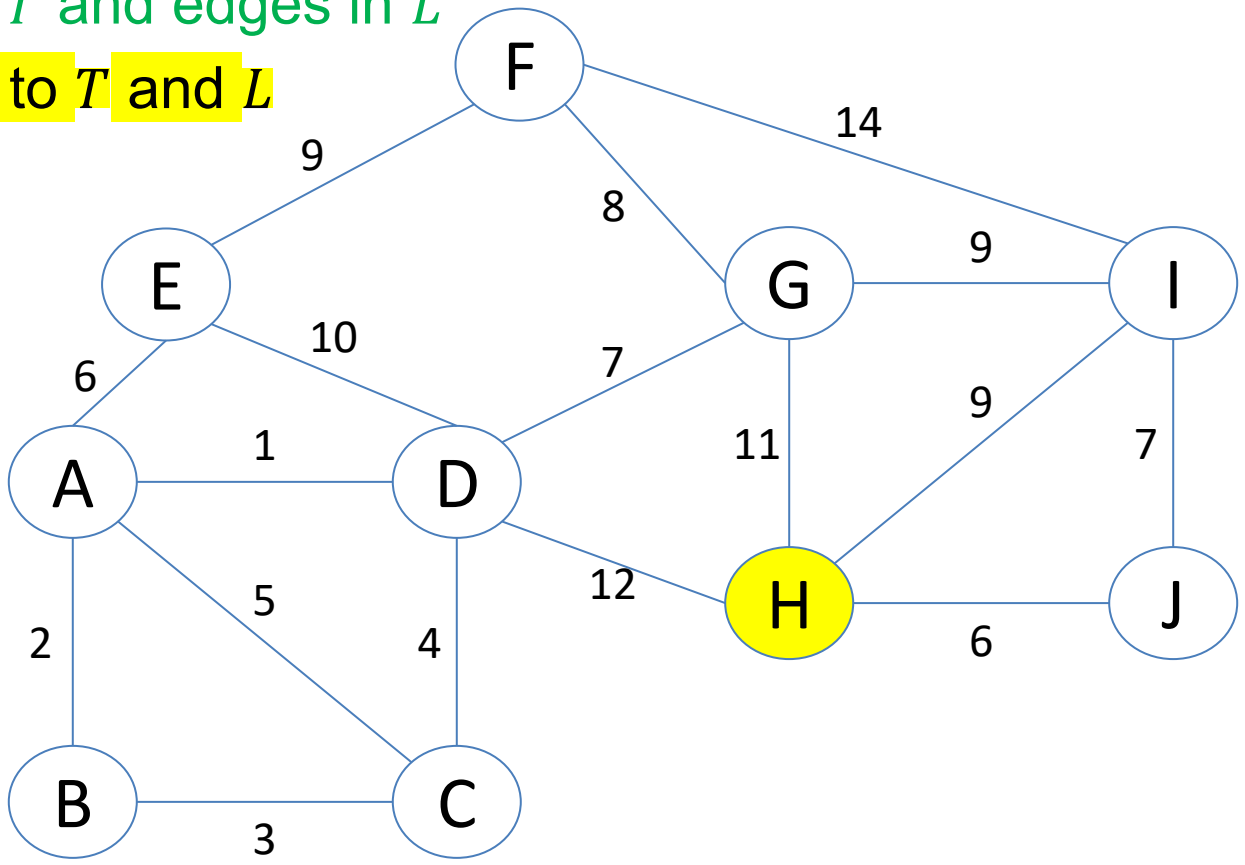
PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



Example (1/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \emptyset$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, i, j, h\}$$



$$L = \emptyset$$

$$\text{cost}(L) = 0$$

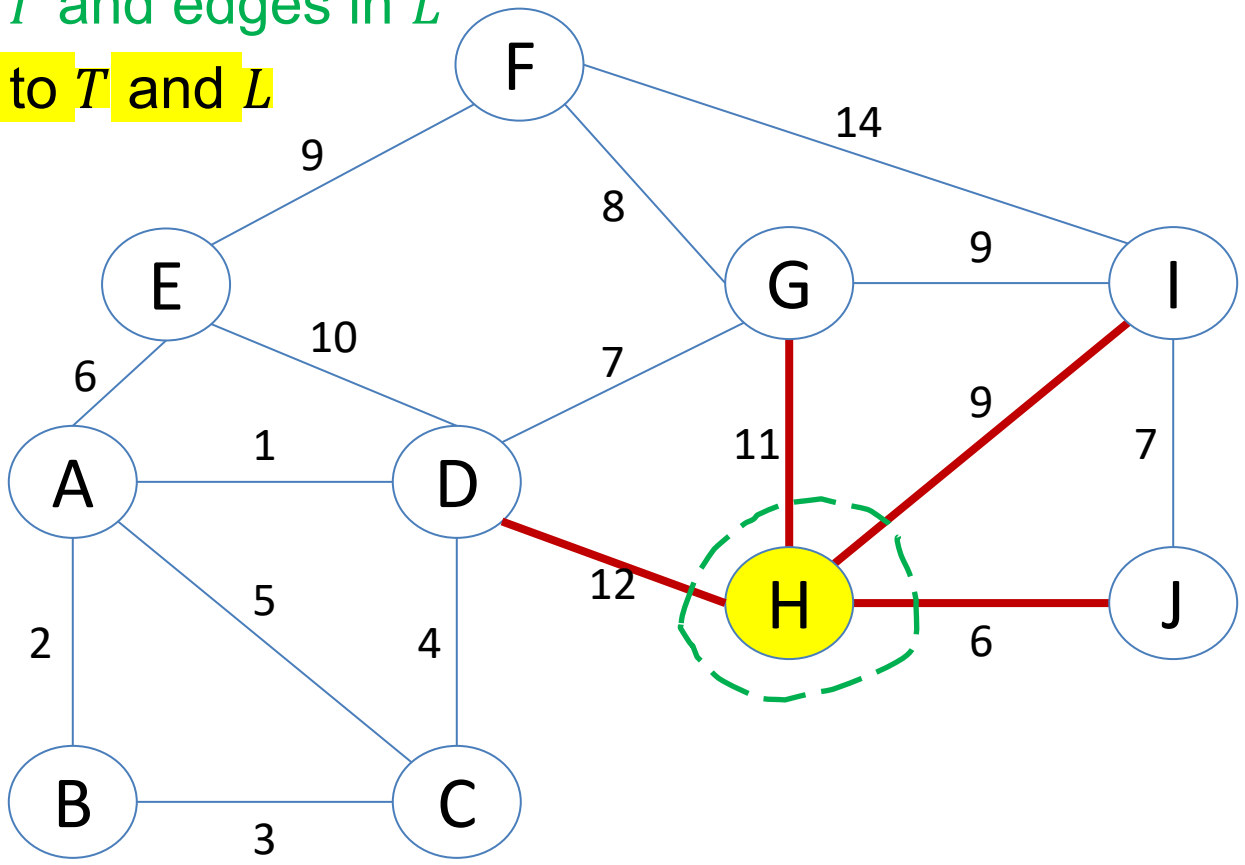
PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L

Example (2/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h\}$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, i, j, \sout{h}\}$$

$$L = \emptyset$$



$$\text{cost}(L) = 0$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L

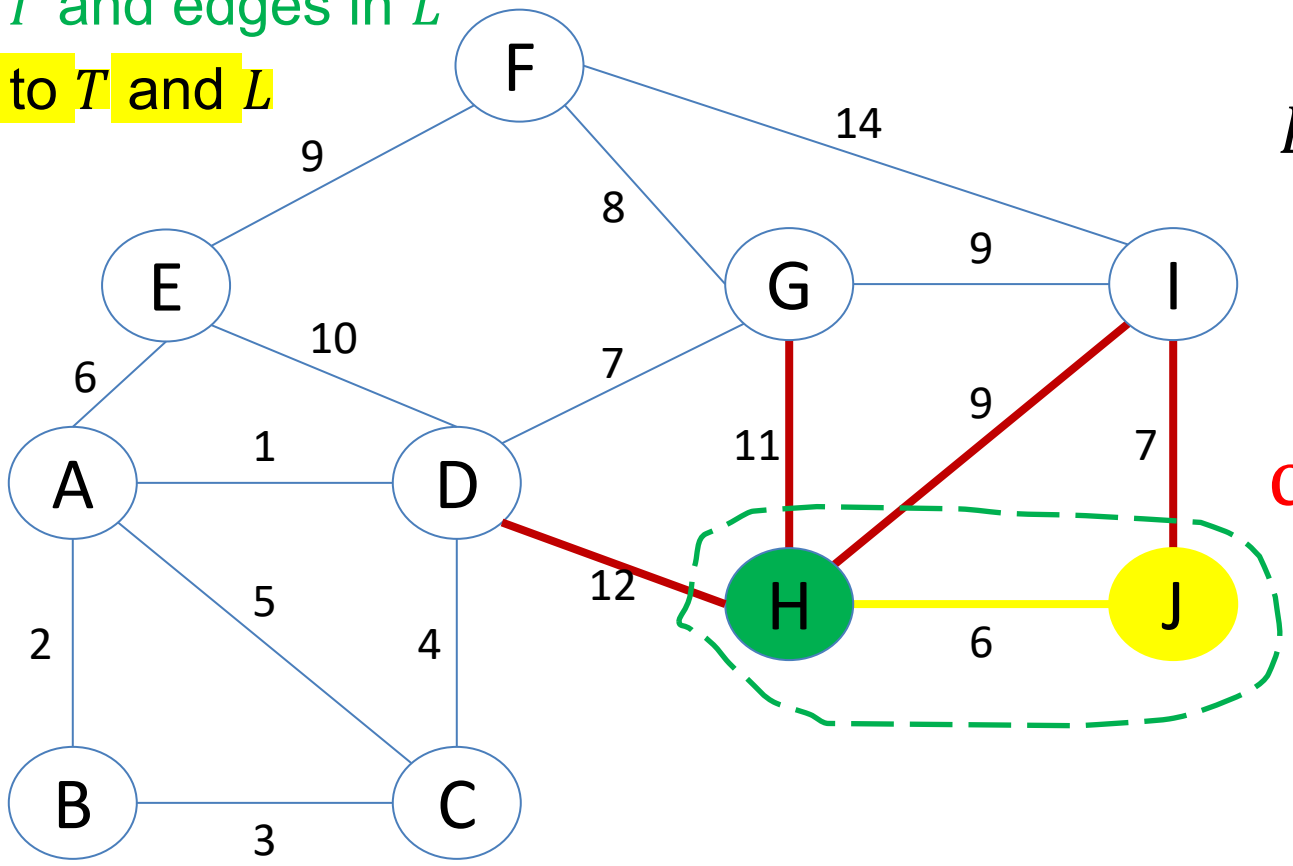
Example (3/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j\}$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, i, \cancel{j}, \cancel{h}\}$$

$$L = \{(h, j)\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$$\text{cost}(L) = 6$$

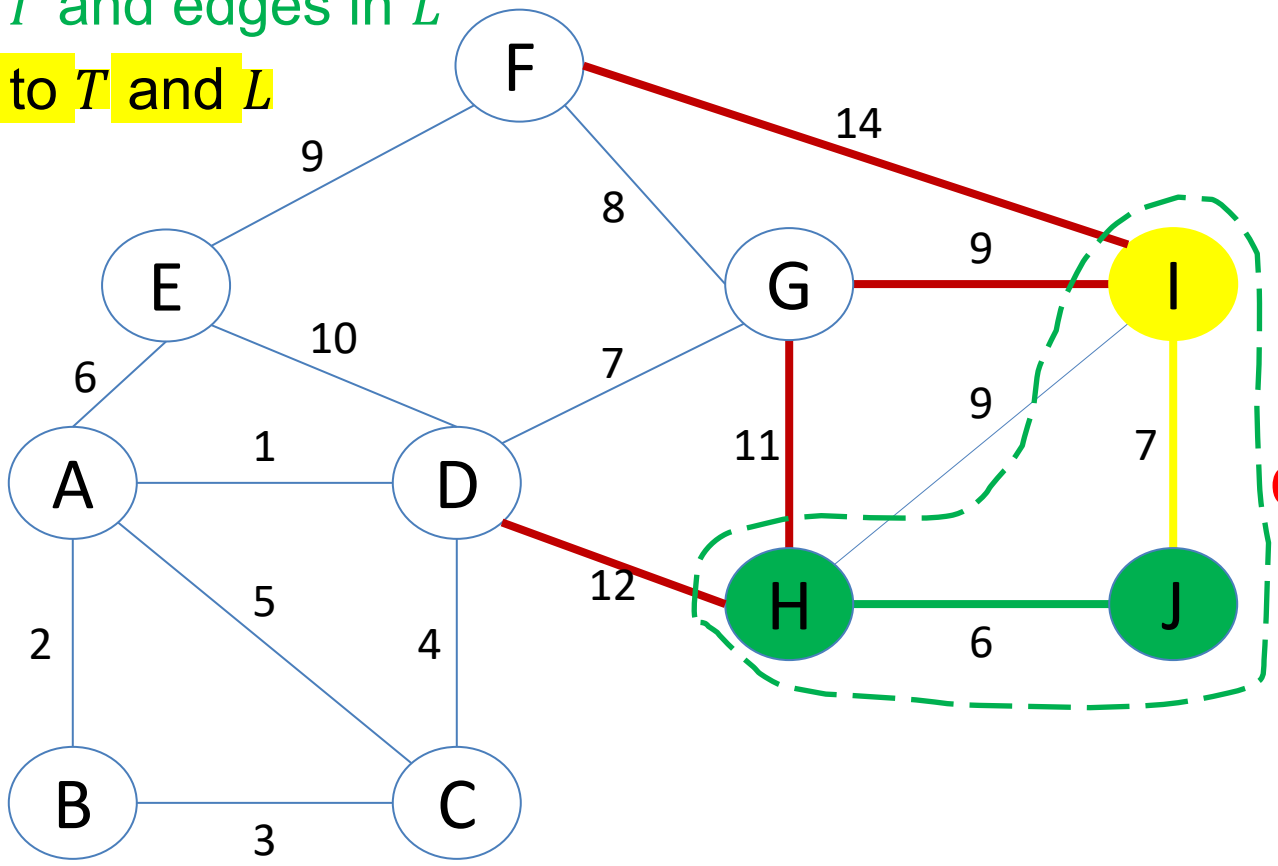
Example (4/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i\}$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, \cancel{i}, \cancel{j}, \cancel{h}\}$$

$$L = \{(h, j), (j, i)\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L

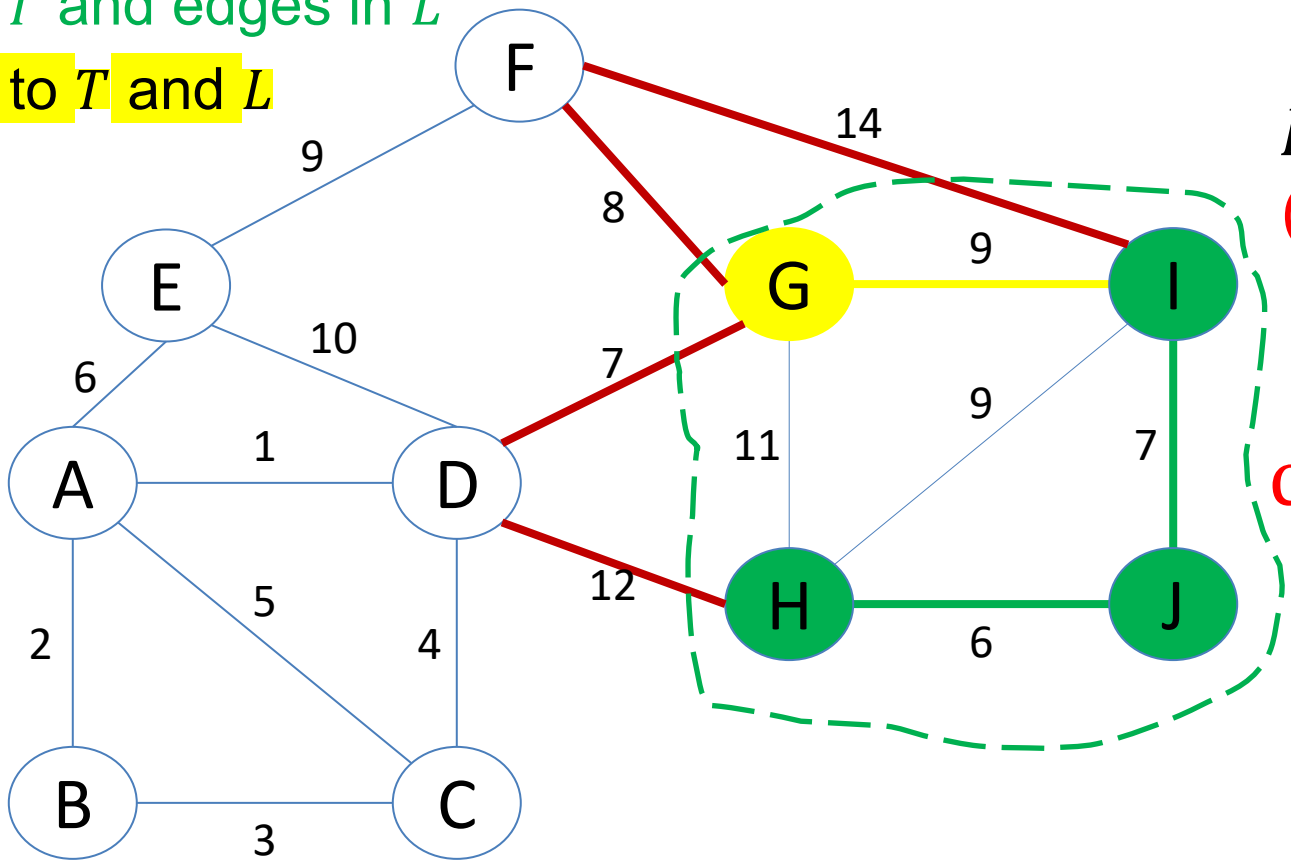


$$\text{cost}(L) = 13$$

Example (5/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$T = \{h, j, i, g\}$

$\bar{T} = V \setminus T = \{a, b, c, d, e, f, \sout{g}, \sout{i}, \sout{j}, \sout{h}\}$

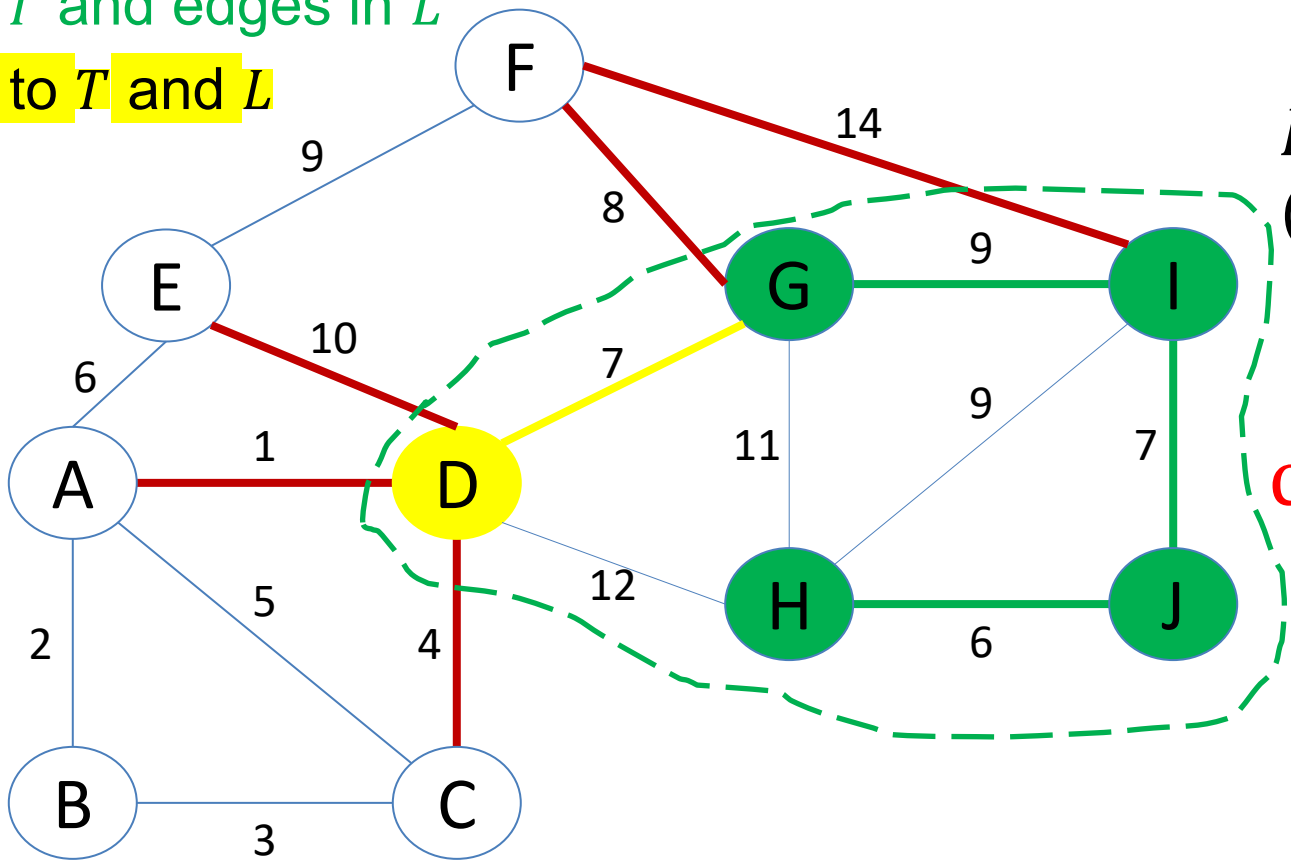
$L = \{(h, j), (j, i)$
$(i, g)\}$

$\text{cost}(L) = 22$

Example (6/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



$T = \{h, j, i, g, d\}$

$\bar{T} = V \setminus T = \{a, b, c, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$

$L = \{(h, j), (j, i)$
$(i, g), (g, d)\}$

$\text{cost}(L) = 29$

Example (7/11)

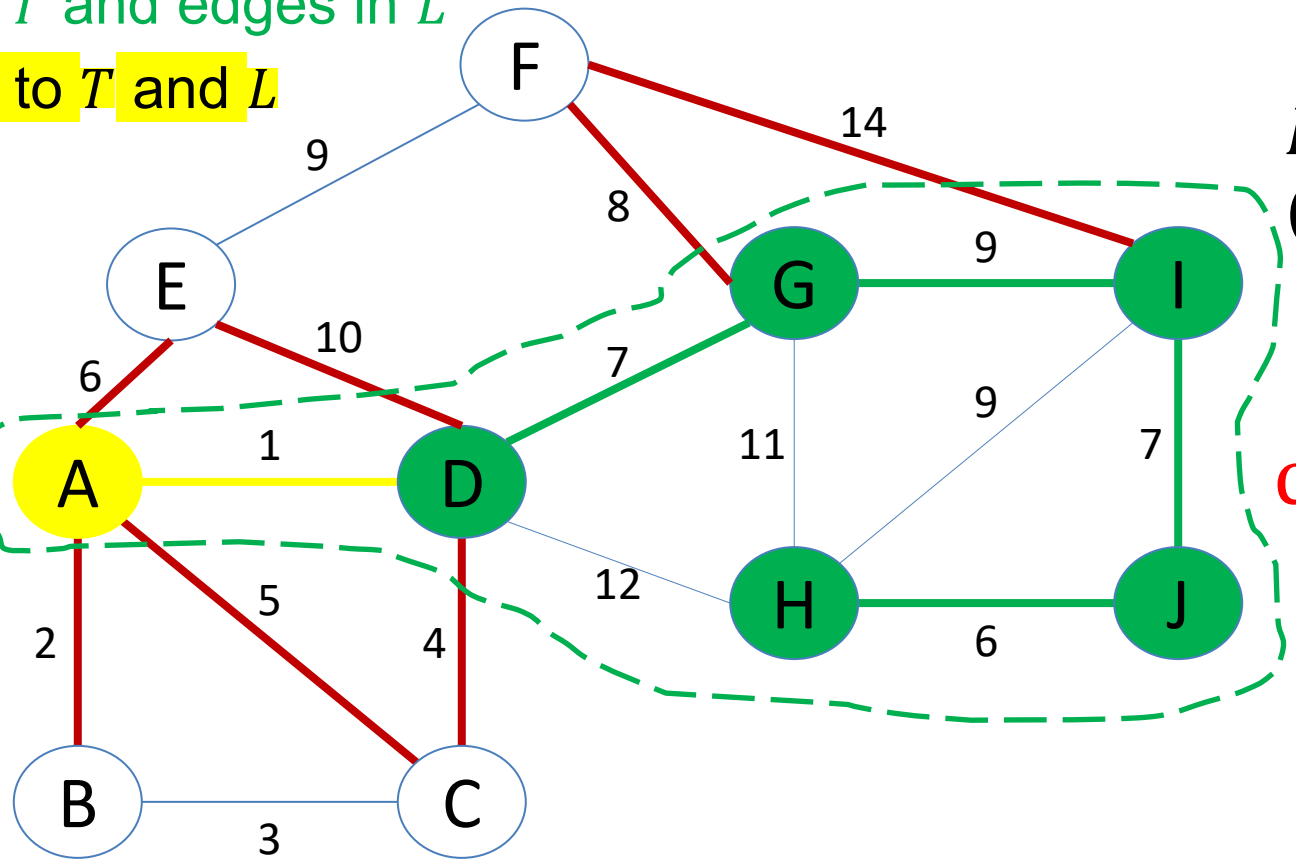
- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, b, c, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

$$L = \{(h, j), (j, i) (i, g), (g, d), (d, a)\}$$

$$\text{cost}(L) = 30$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices ✓
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L

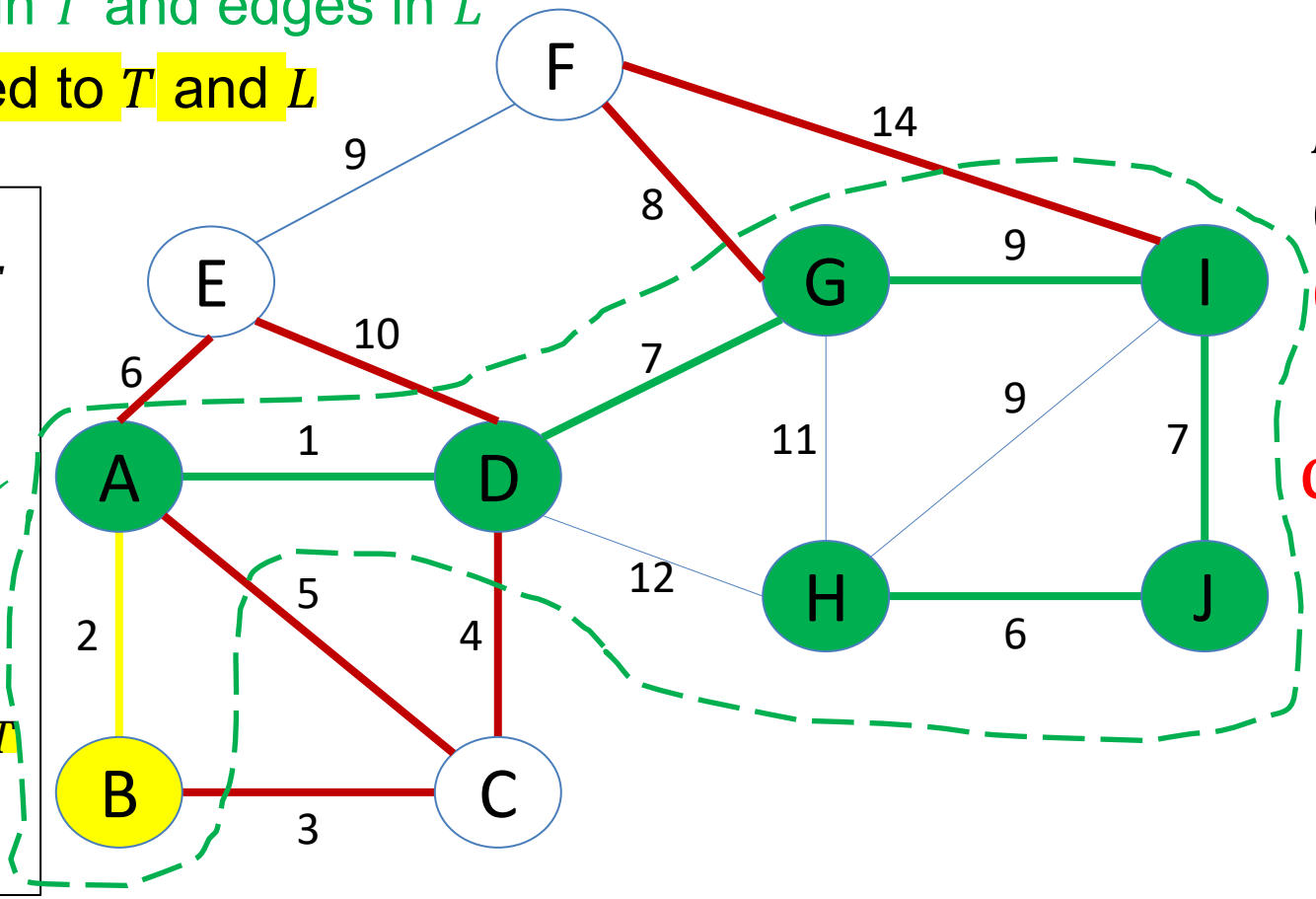


Example (8/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, c, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b)\}$$

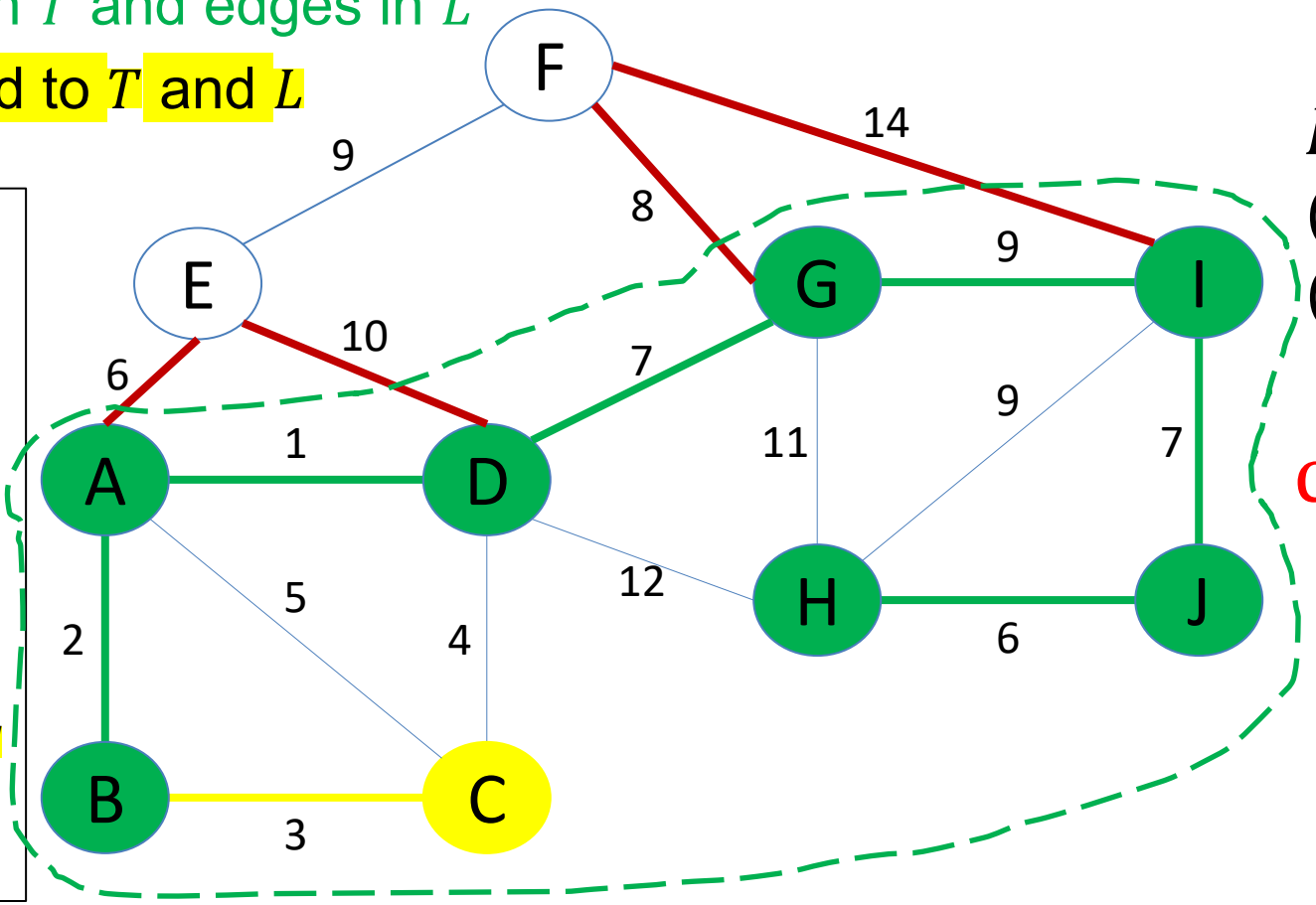
$$\text{cost}(L) = 32$$

Example (9/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices ✓
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b), (b, c)\}$$

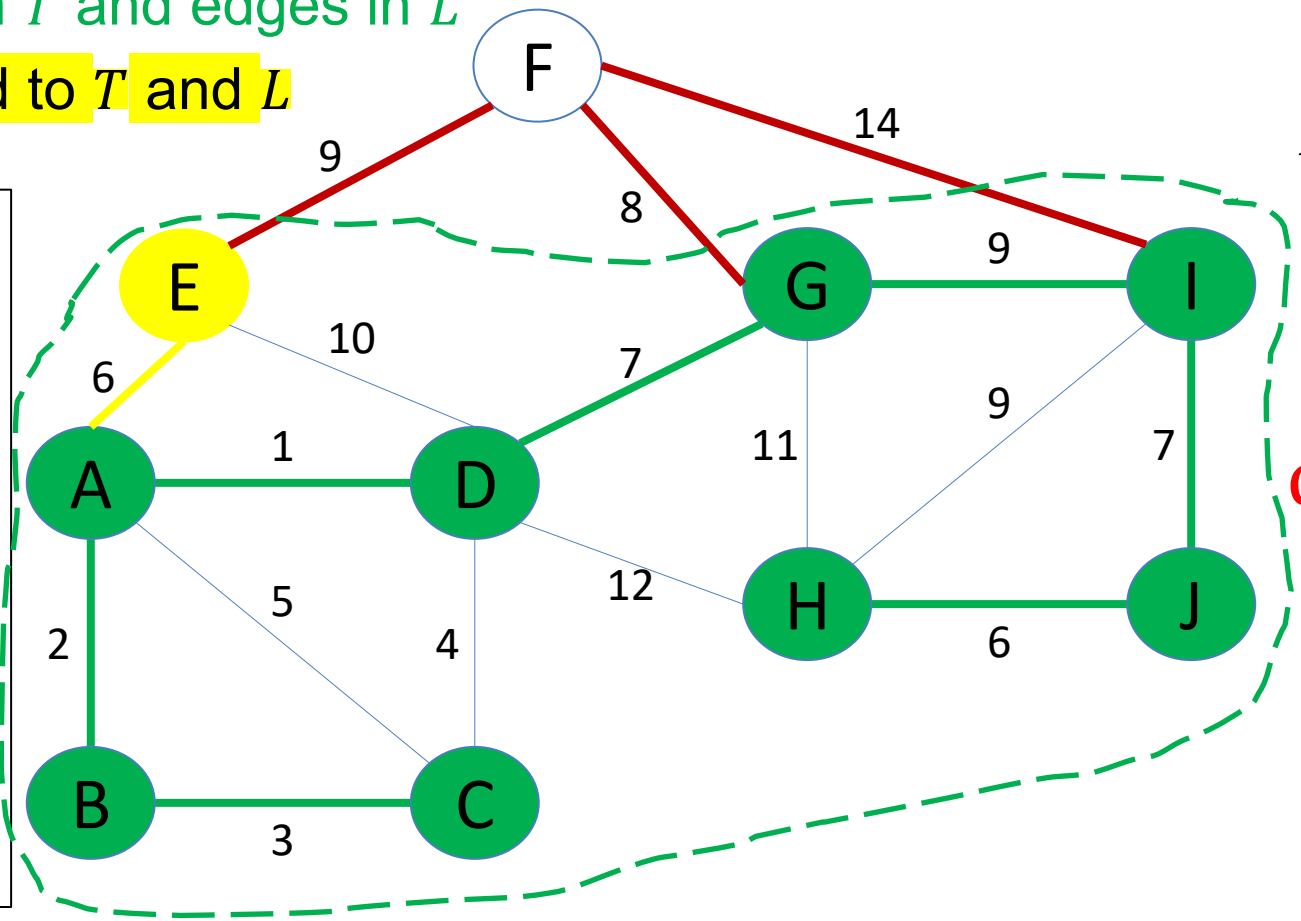
$$\text{cost}(L) = 35$$

Example (10/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c, e\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, \cancel{e}, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



$$L = \{(h, j), (j, i)$$
$$(i, g), (g, d), (d, a),$$
$$(a, b), (b, c), (a, e)\}$$

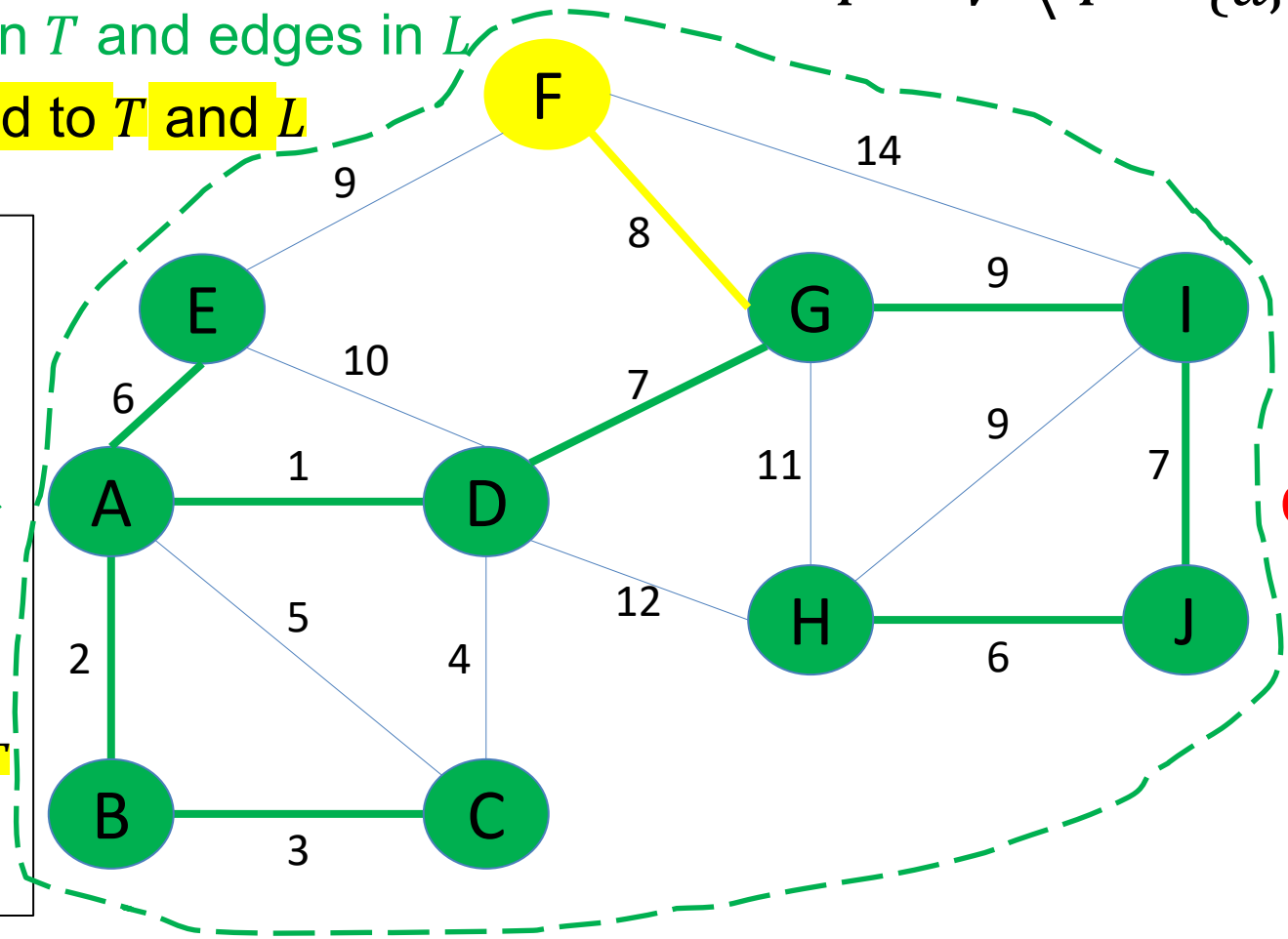
$$\text{cost}(L) = 41$$

Example (11/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c, e, f\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, \cancel{e}, \cancel{f}, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



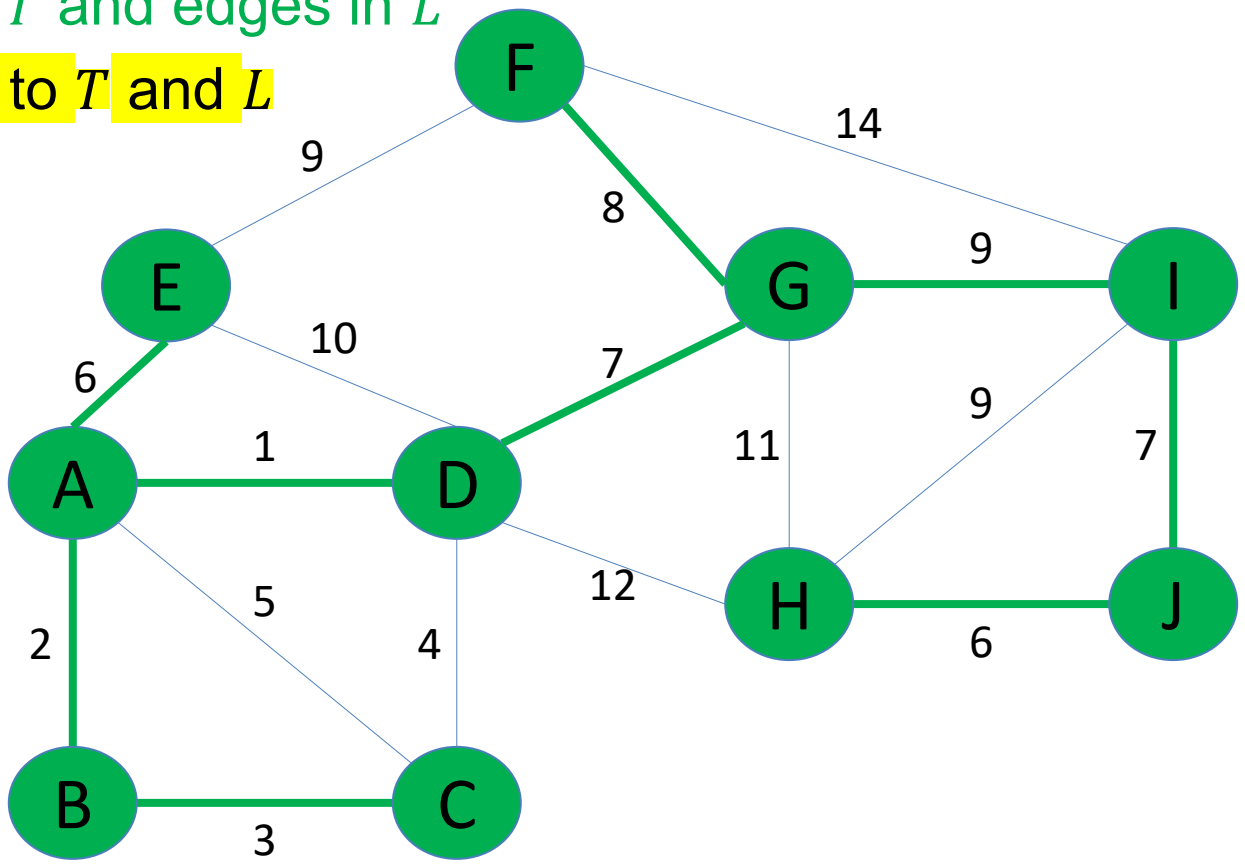
$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b), (b, c), (a, e), (g, f)\}$$
$$\text{cost}(L) = 49$$

Example (11/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c, e, f\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, \cancel{e}, \cancel{f}, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices ✗
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b), (b, c), (a, e), (g, f)\}$$
$$\text{cost}(L) = 49$$

Minimum Cost:

49

Implementation (2/3): adding a priority queue

- What should we store in the priority queue?
  - Edges
  - From nodes $\in T$ to nodes $\notin T$

- What should we use as the key of an edge?
  - Weight of the edge

PrimMST(V, E)
1. Pick an arbitrary node r from V
2. Add r to *T*
3. While *T* contains < |V| nodes
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add node v to *T*

Finding lots of minimums?
Use a priority queue!

Implementation (3/3): adding a priority queue

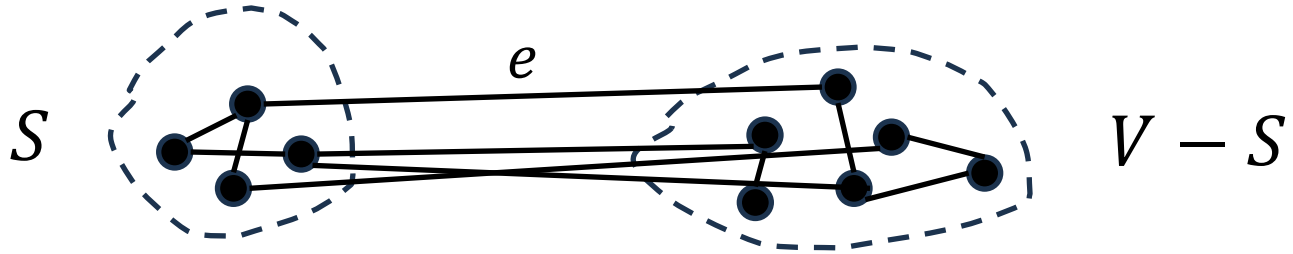
where r is any arbitrary starting node

```
PrimMST(G = (V, E), start r):
    inTree[r] := true
    parent[r] := r
    Q := min-heap priority queue of edges from r

    while Q not empty:
        (w, u, v) := Q.extractMin()
        if not inTree[v]:
            inTree[v] := true
            parent[v] := u
            for each edge (v, z):
                if not inTree[z]:
                    Q.insert((weight(v, z), v, z))

    return parent
```

Correctness (1/4): Cut Property (MST Lemma)

Lemma 1. Given a connected, weighted, undirected graph $G = (V, E)$ and a cut $(S, V - S)$ (a partition of the vertex set V into two non-empty sets S and $V - S$). Let $e = (u, v)$ be an edge crossing the cut (i.e., $u \in S$ and $v \in V - S$). Then

1. If e has a strictly minimum weight among all crossing edges, then e must be part of every Minimum Spanning Tree (MST) of G.
2. If there are multiple edges with the same minimum weight crossing the cut, then at least one of these minimum-weight crossing edges must be in some MST of G.



Part 1 (1/2)

- Assume, for the sake of contradiction, that there exists an MST, let's call it M^*, that **does not** contain the edge $e = (u, v)$, where e is the unique*minimum-weight edge crossing the cut $(S, V - S)$.
- Since M^* is an MST and $e \notin M^*$, but G is connected, M^* must still connect the two sets S and V-S. This implies that there must be at least one path in M^* from u to v.
- Consider the path in M^* connecting u to v. Since $u \in S$ and $v \in V - S$, this path must cross the cut $(S, V - S)$ at least once. Let (x, y) be any edge on this path that crosses the cut (i.e., $x \in S$ and $y \in V - S$, or vice-versa).
- By our assumption, $e = (u, v)$ is the unique minimum-weight edge crossing the cut. Therefore, the weight of e must be strictly less than the weight of (x, y), i.e., $w(e) < w(x, y)$.

Part 1 (2/2)

- Now, let's form a new spanning tree, M', from M^*. We do this by adding e to M^* and removing (x, y) from M^*.
- Adding e to M^* creates a cycle (because M^* already contains a path between u and v).
- Since (x, y) is an edge on this cycle and also crosses the cut, removing (x, y) from this cycle breaks it and maintains connectivity (it simply reroutes the connection between S and V-S through e instead of (x, y)). Thus, M' is a spanning tree.
- Let's compare the total weight of M' with M^*:

$$w(M') = w(M^*) - w(x, y) + w(e).$$

- Since we established that $w(e) < w(x, y)$, it follows that:

$$w(M') < w(M^*).$$

- This is a contradiction! We assumed M^* was an MST, meaning it had the minimum possible total weight. But we found a spanning tree M' with a strictly smaller total weight.
- Therefore, our initial assumption that M^* does not contain e $must\ be\ false$. Hence, e must be part of every MST.

Part 2 (1/2)

- Assume, for the sake of contradiction, that NO minimum-weight edge crossing the cut $(S, V - S)$ is part of an MST, let's call it M^*.
- Let $e_0 = (u_0, v_0)$ be any minimum-weight edge crossing the cut $(S, V - S)$ (i.e., $u_0 \in S, v_0 \in V - S$).
- By our assumption, $e_0 \notin M^*$.
- Similar to Part 1, since M^* is an MST and $e_0 \notin M^*$, there must be a path in M^* connecting u_0 to v_0. This path must cross the cut $(S, V - S)$ at least once. Let (x, y) be any edge on this path that crosses the cut (i.e., $x \in S, y \in V - S$, or vice-versa).
- Since e_0 is a minimum-weight edge crossing the cut, the weight of e_0 must be less than or equal to the weight of any other edge crossing the cut, including (x, y):

$$w(e_0) \leq w(x, y).$$

Part 2 (2/2)

- Let's form a new spanning tree, M', from M^*. We add e_0 to M^* and remove (x, y) from M^*.
- As before, adding e_0 to M^* creates a cycle, and removing (x, y) (which is on this cycle and crosses the cut) results in a valid spanning tree M'.
- Compare the total weights:

$$w(M') = w(M^*) - w(x, y) + w(e_0).$$

- Since $w(e_0) \leq w(x, y)$, it follows that $w(M') \leq w(M^*)$.
- As M^* is an MST, it has the minimum possible weight. Therefore, $w(M')$ cannot be strictly less than $w(M^*)$. This implies:

$$w(M') = w(M^*).$$

- This means M' is also an MST. Crucially, M' contains the edge e_0, which is a minimum-weight crossing edge. This contradicts our initial assumption that no minimum-weight edge crossing the cut is part of M^*.

Correctness (2/4)

- First, we have to realize that Prim's algorithm is greedy.
- This is because at each step, it always tries to select the next valid edge e with MINIMUM weight (greedy!)
- The greedy algorithm is usually simple to implement.

Correctness (3/4)

We prove the correctness of the Prim's algorithm by induction on the vertex set V.

- Base case: Prim's algorithm is correct for $|V| = 2,3$.
- Inductive Hypothesis: Assume that Prim's algorithm is correct for $|V| = n$.
- Inductive Step: We now prove that Prim's algorithm is also correct for $|V| = n + 1$.
- Indeed, consider a cut $(S, V - S)$ where $|S| = n$, i.e., $|V - S| = 1$. Let $u \in V - S$ and $F = \{e_1, \dots, e_k\}$ be the set of edges that has one endpoint as u.
- Let $e \in F$ be an edge such that $w(e) \leq w(e_i), \forall i \in [k] = \{1, \dots, k\}$.

Correctness (4/4)

- By using the Cut Property, the edge e must belong to some MST of $G = (V, E)$.
- For any spanning tree T^* of G, we can decompose T^* into two trees, which are T_1^* and $T_2^* = \{e^*\}$, where T_1^* is a spanning tree of G_S and $e^* \in F$.
- Let T be an MST of $G_S = \big(S, E(S)\big)$ be an induced graph of G.
- Then we have

$$w(T) + e \leq w(T_1^*) + w(e^*).$$

- Moreover, since one of the endpoint of the edge e belongs to T, the tree $T \cup \{e\}$ is an MST of G.

Complexity: analysis of running time

```
PrimMST(G = (V, E), start r):
    inTree[r] := true
    parent[r] := r
    Q := min-heap priority queue of edges from r

    while Q not empty:
        (w, u, v) := Q.extractMin()
        if not inTree[v]:
            inTree[v] := true
            parent[v] := u
            for each edge (v, z):
                if not inTree[z]:
                    Q.insert((weight(v, z), v, z))
    return parent
```

$O(|V|\log|V|)$

$O(1)$

$O(|E|)$

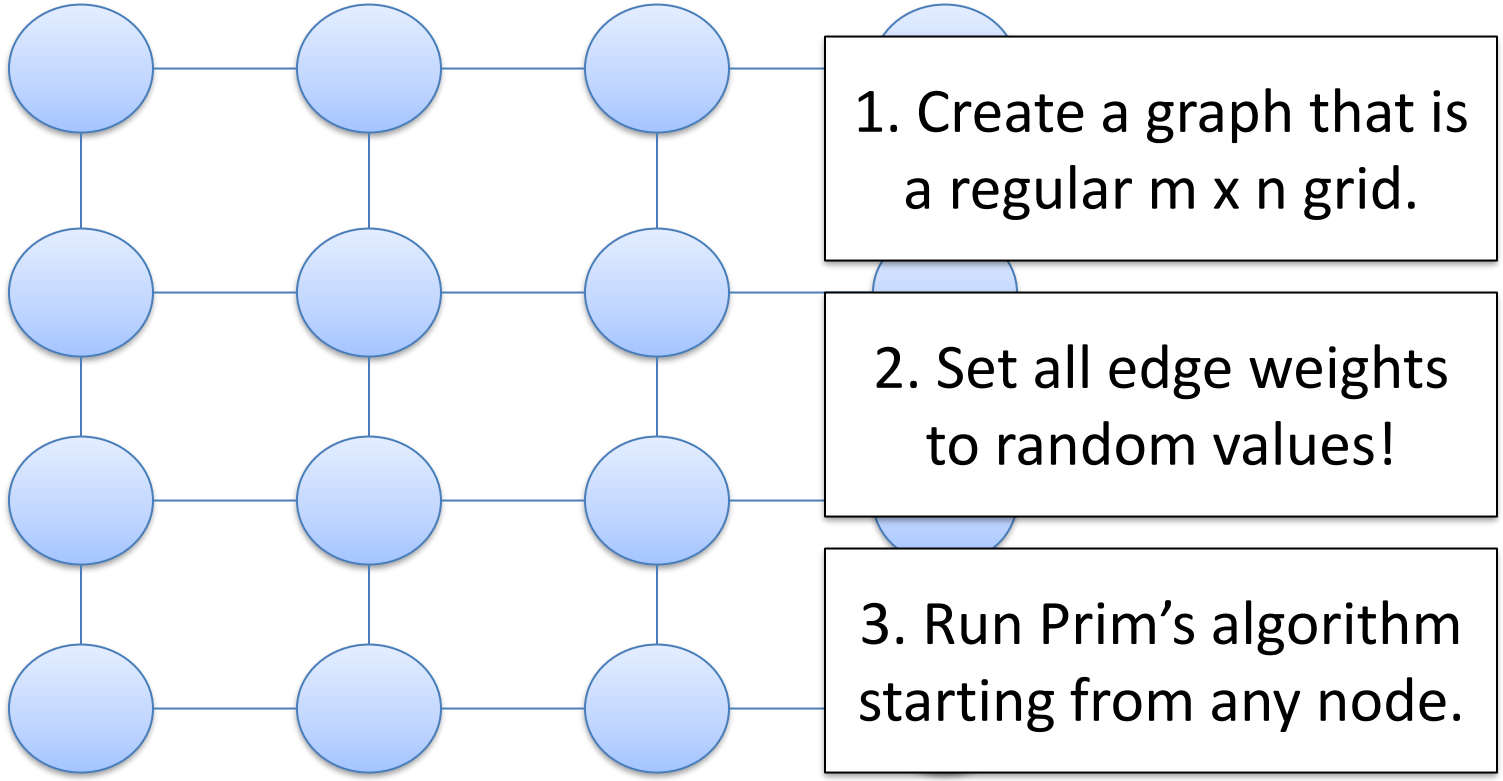
$O(|\text{adj}(v)|\log|V|)$

$O(|E|\log|V|)$

$$O\big((|V| + |E|)\log|V|\big)$$

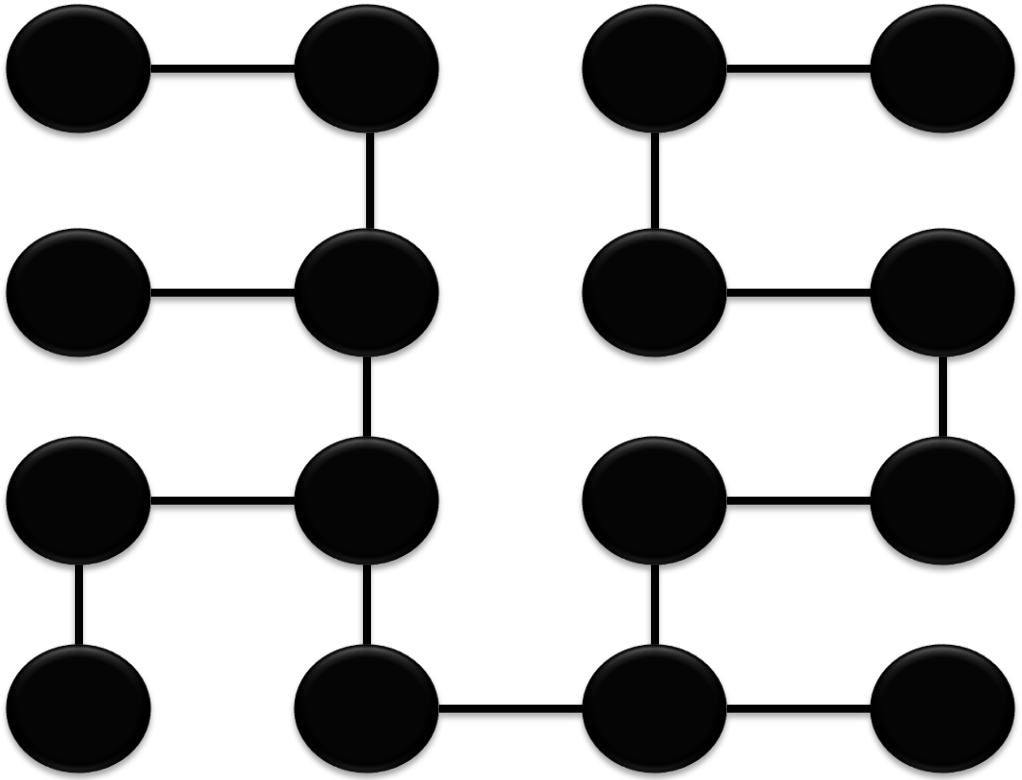
More applications: generating 2D maze (1/2)

- Prim's algorithm maze building video
- How can we use Prim's algorithm to do this?



More applications: generating 2D maze (2/2)

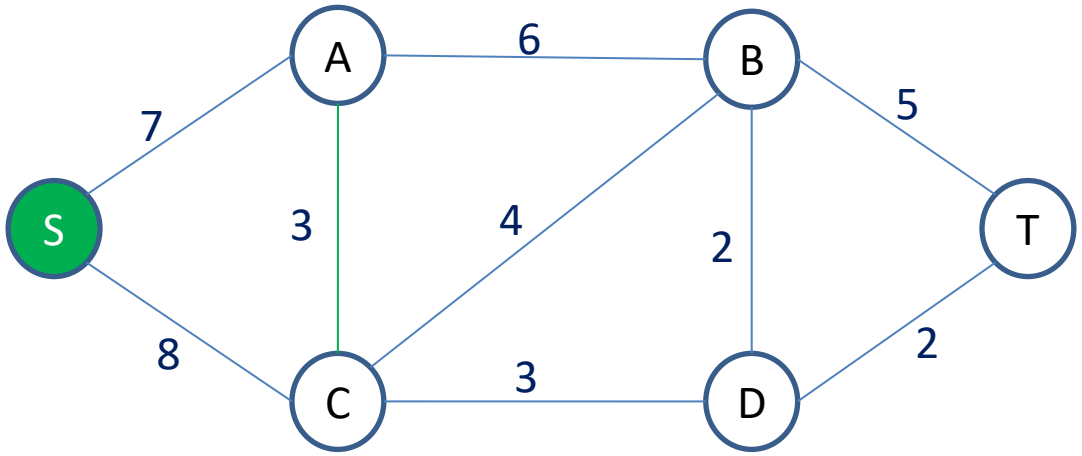
- After Prim's, we end up with something like:



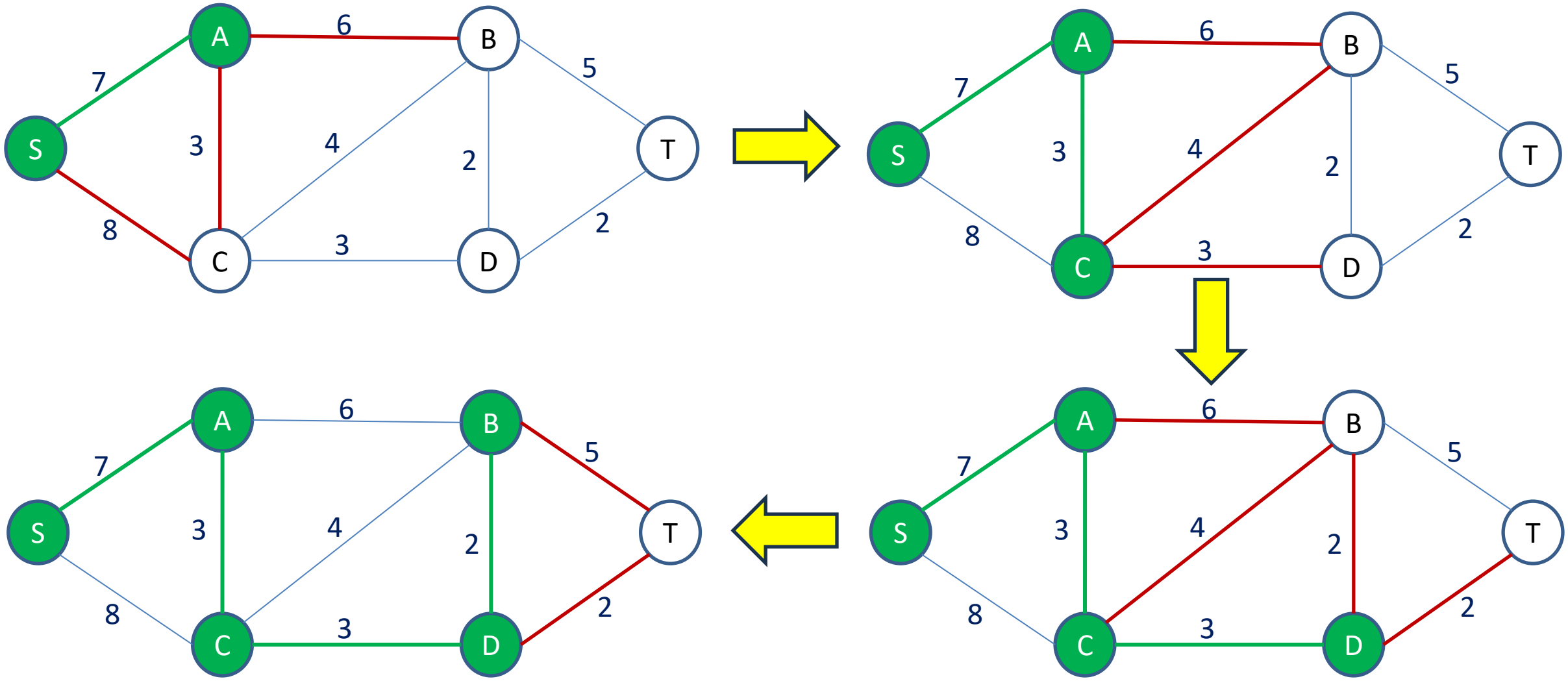
- For any two adjacent vertices that are disconnected, build a wall.

Exercise 1

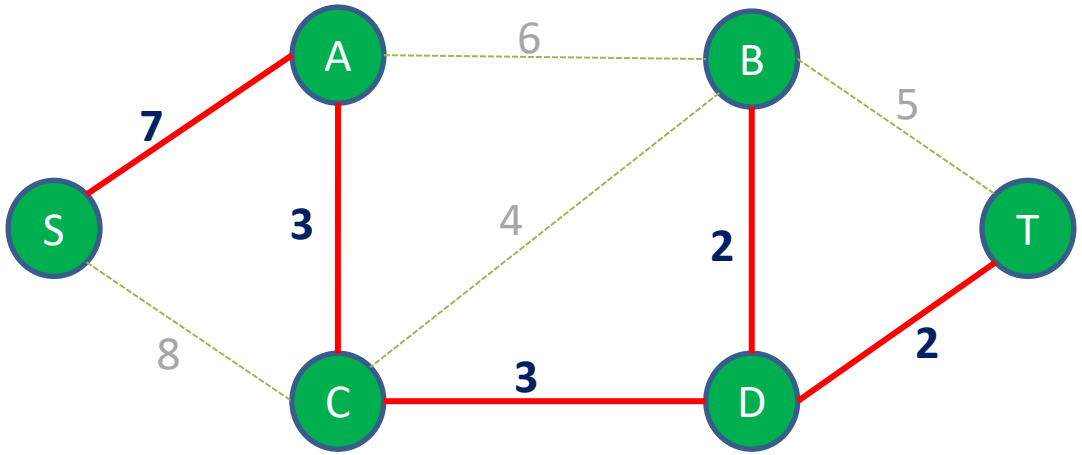
- Find an MST using Prim's algorithm



Exercise 1: Solution

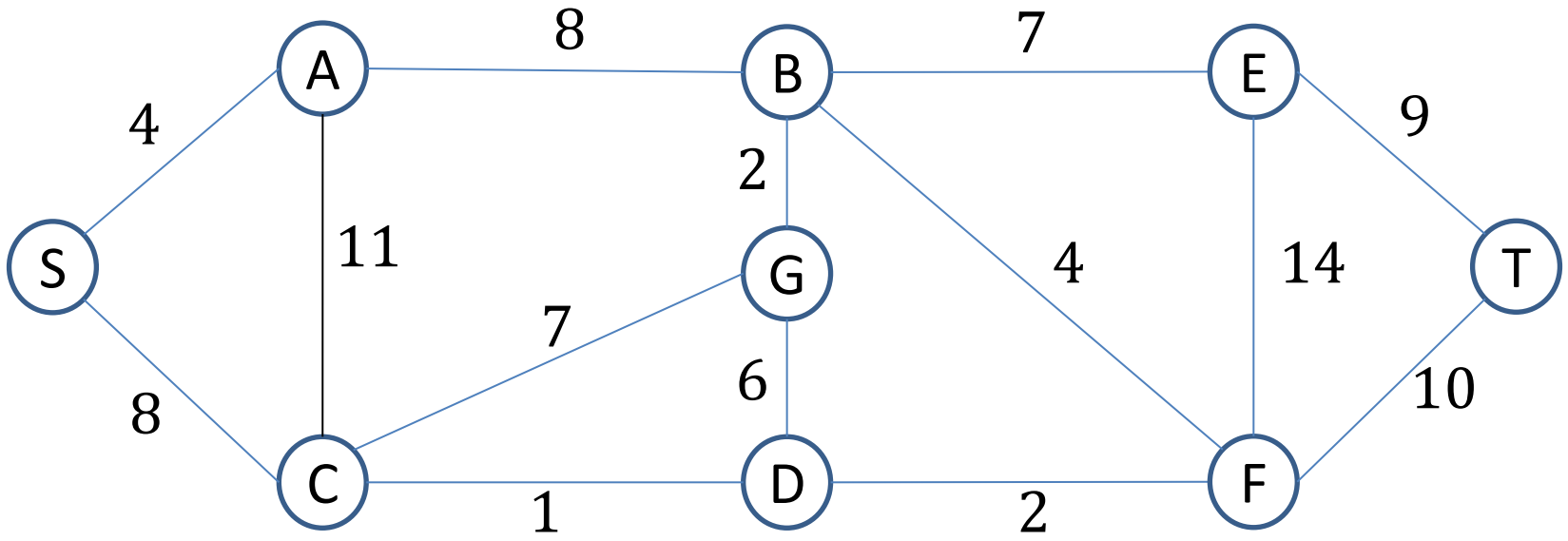


Exercise 1: Solution

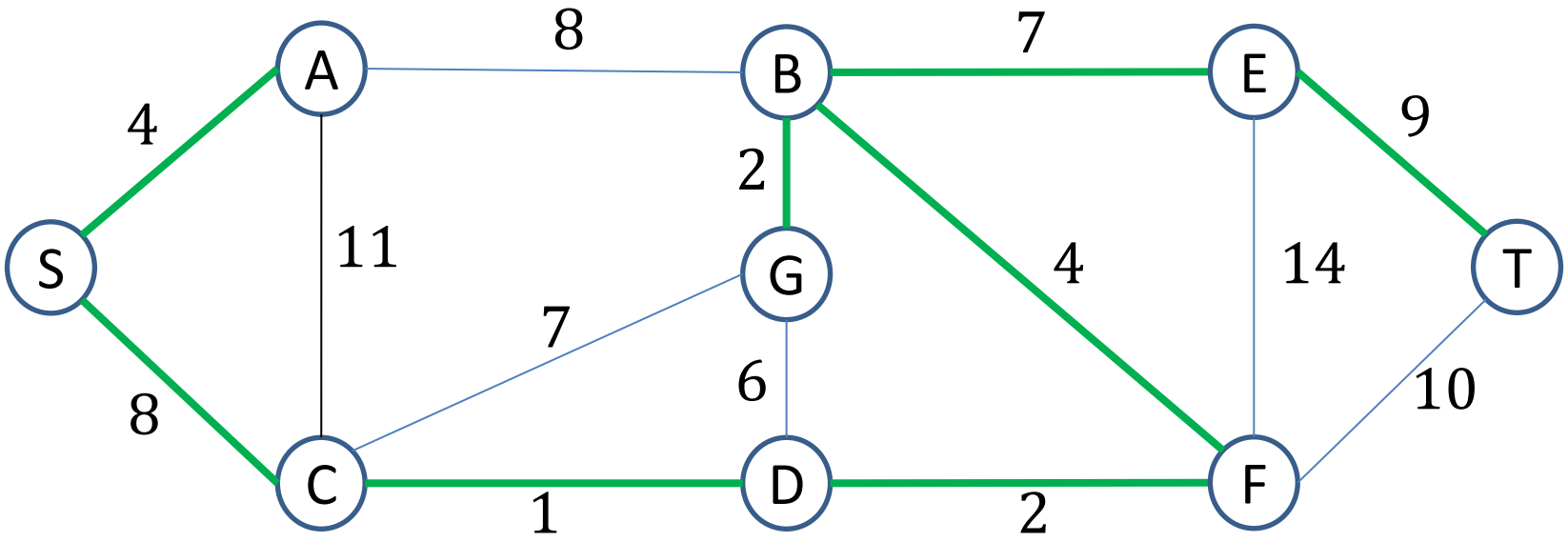


Exercise 2

- Find an MST using Prim's algorithm



Exercise 2: Solution



Minimum Cost:

37

Outline

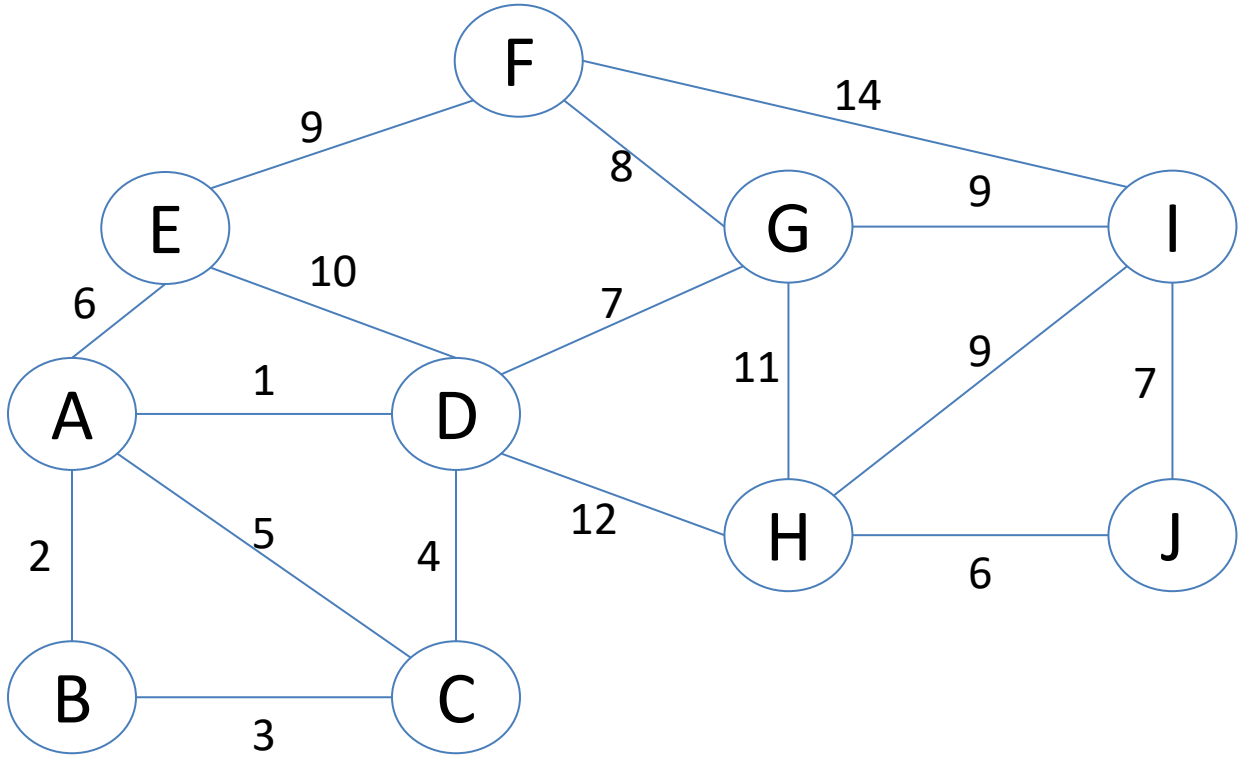
1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

Kruskal's algorithm

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST as an empty set.
3. For each edge in the sorted list:

   If the edge doesn't form a cycle in the MST, add it to the MST.
4. Stop when the MST contains exactly $(|V| - 1)$ edges.

A Networking Problem



Example (1/16)

Weight	2	5	1	6	3	4	10	7	12	9	8	14	11	9	9	6	7
Edge	(A, B)	(A, C)	(A, D)	(A, E)	(B, C)	(C, D)	(D, E)	(D, G)	(D, H)	(E, F)	(F, G)	(F, I)	(G, H)	(G, I)	(H, I)	(H, J)	(I, J)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

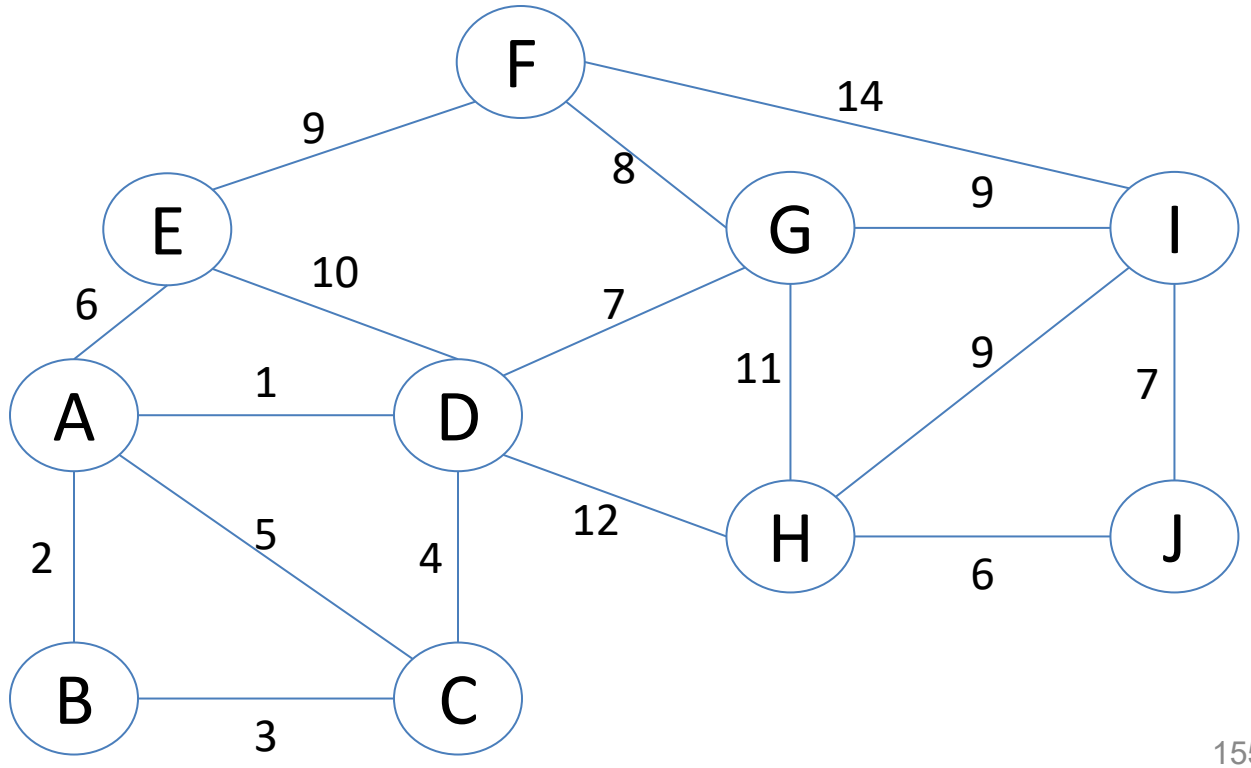
1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

Example (2/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$T = \emptyset$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

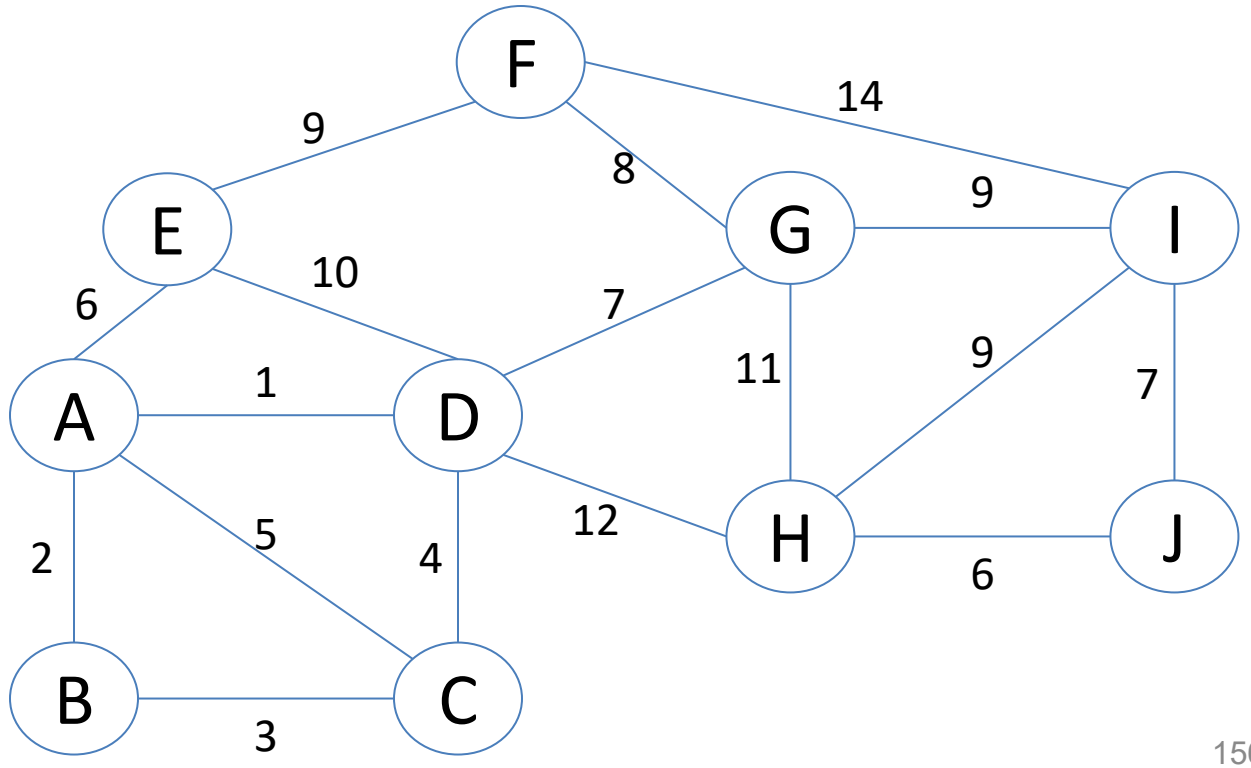


Example (3/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \emptyset, \text{cost}(T) = 0$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

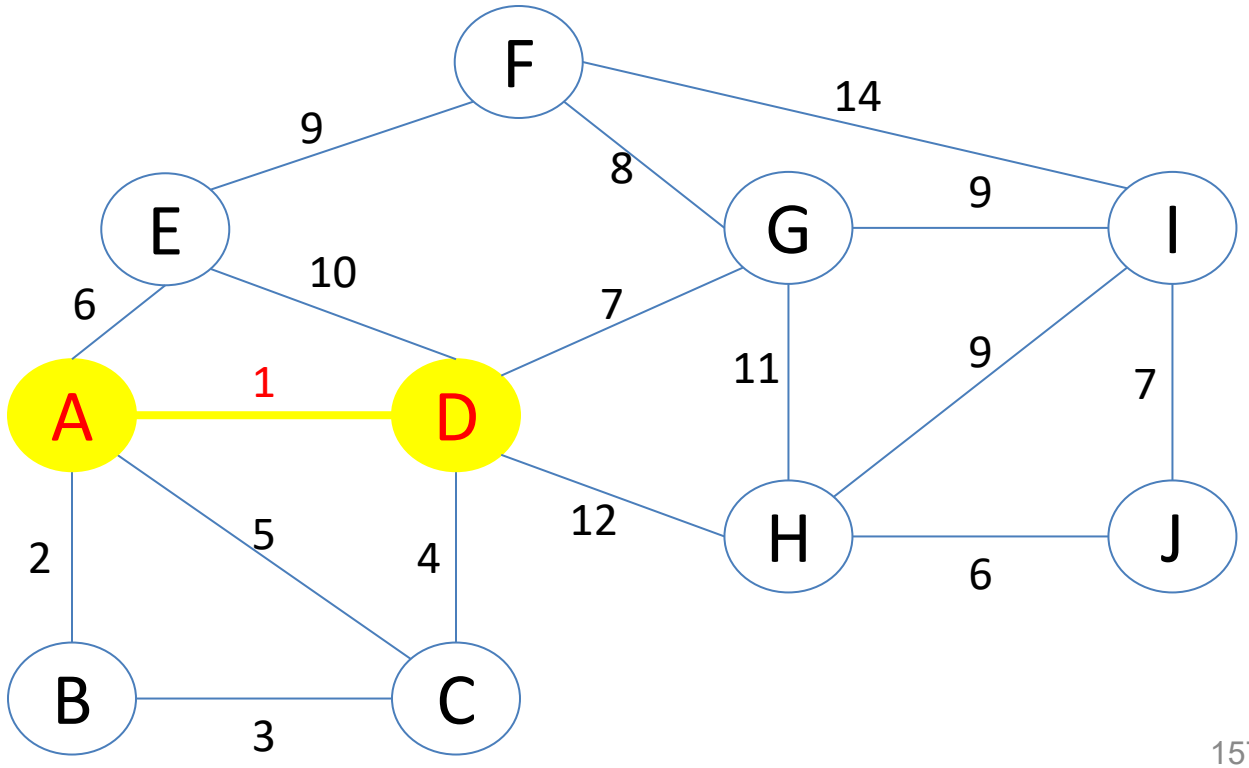


Example (4/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D)\}, \text{cost}(T) = 1$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

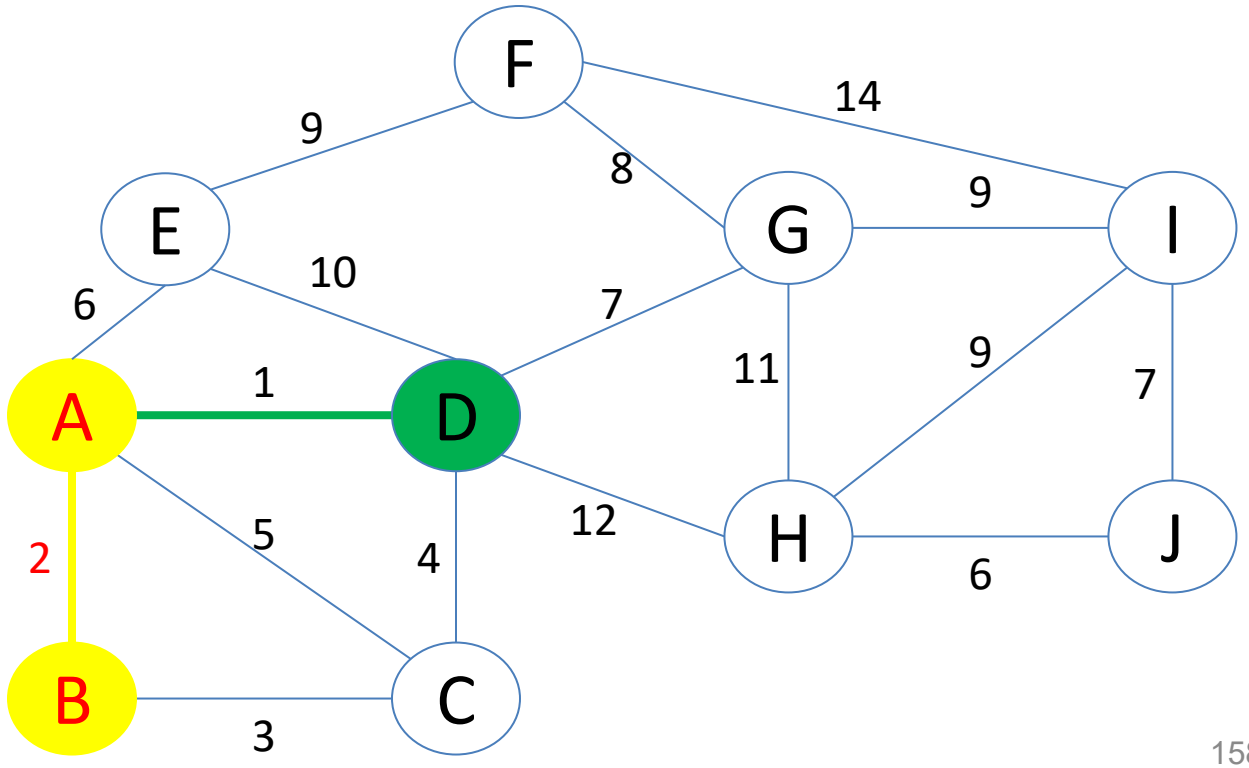


Example (5/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B)\}, \text{cost}(T) = 3$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

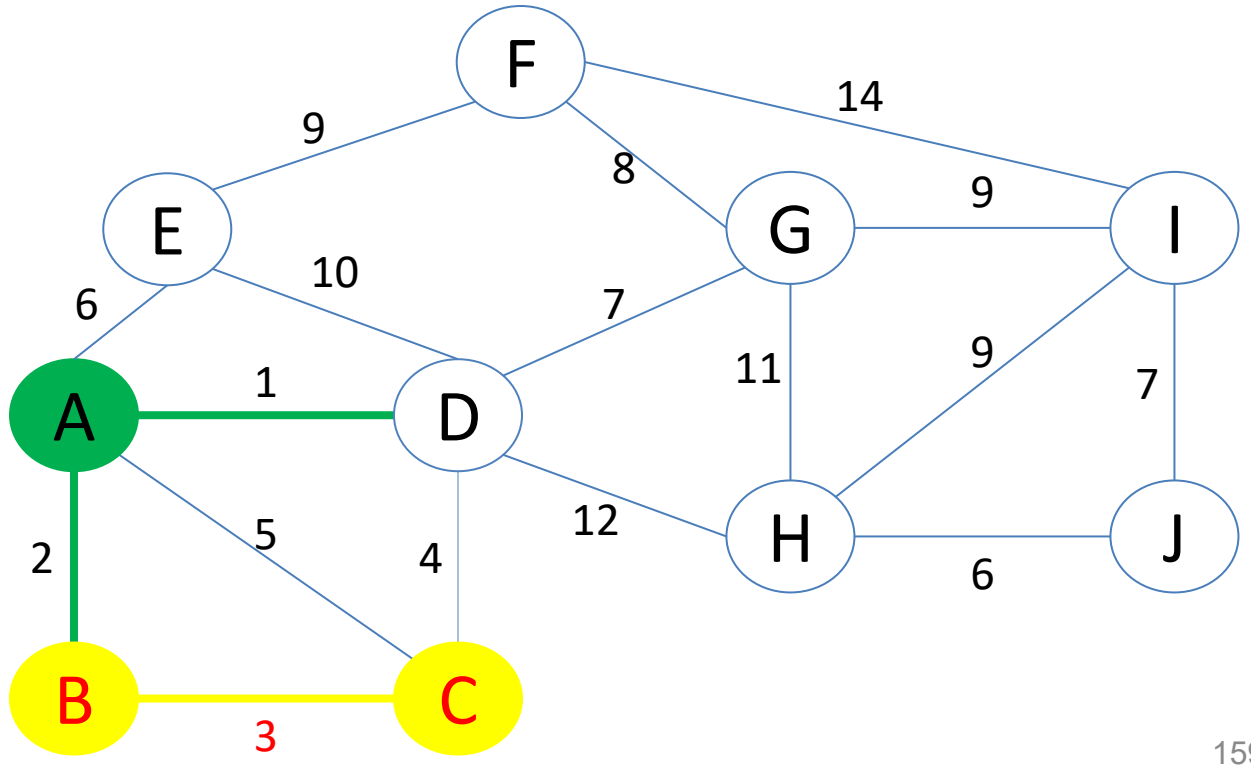


Example (6/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

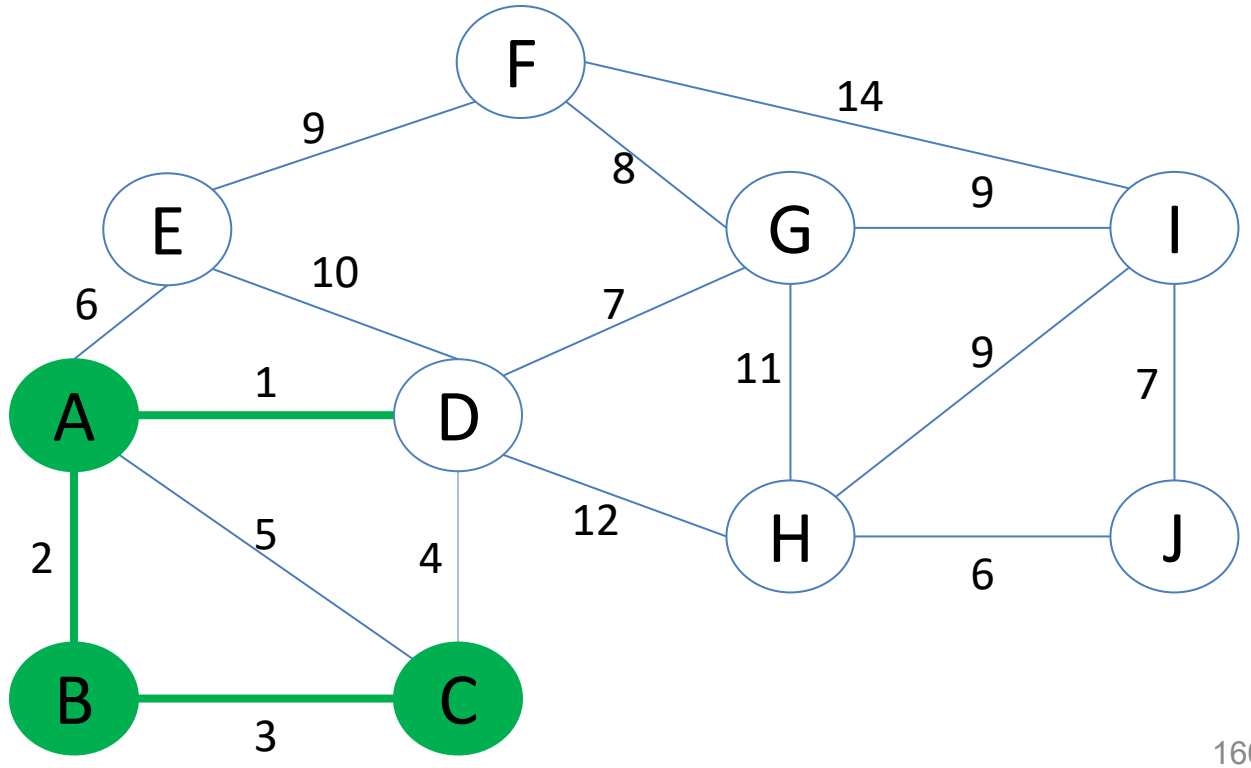


Example (7/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST. ✗
5. Stop when the MST contains exactly $(|V| - 1)$ edges. ✗

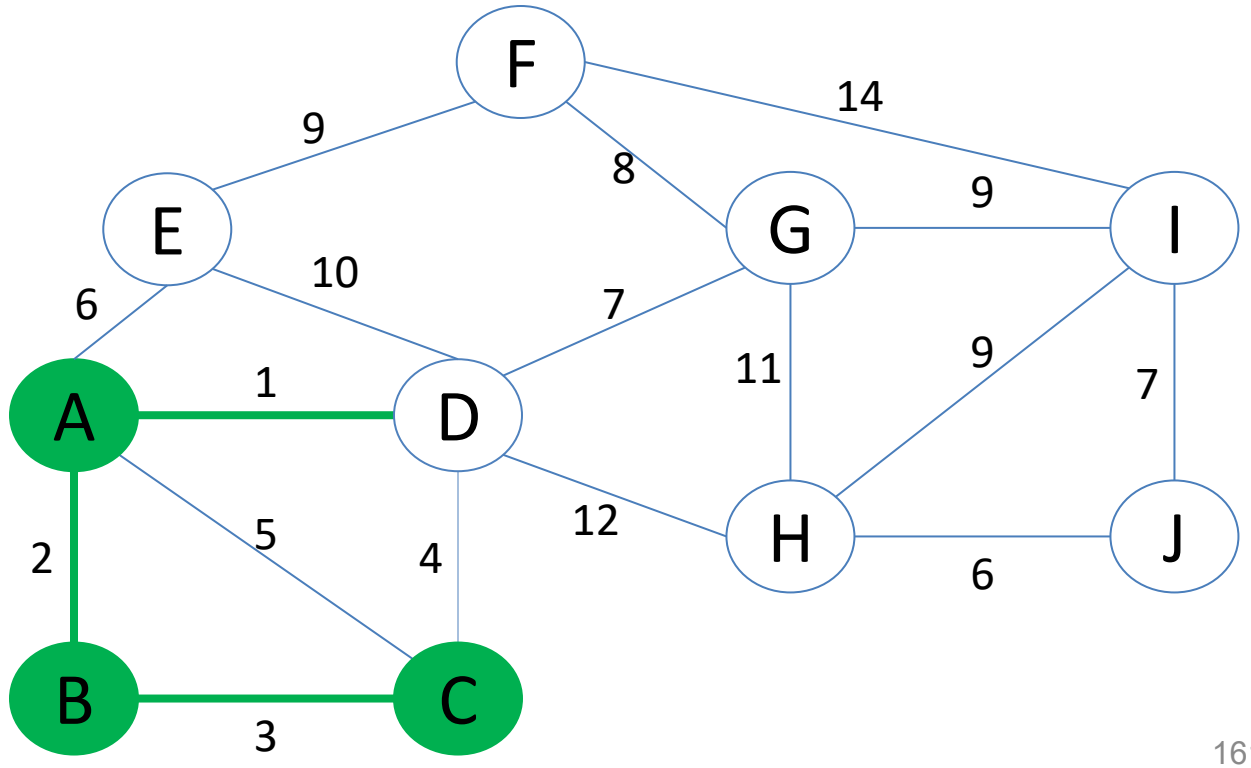


Example (8/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST. ✗
5. Stop when the MST contains exactly $(|V| - 1)$ edges. ✗

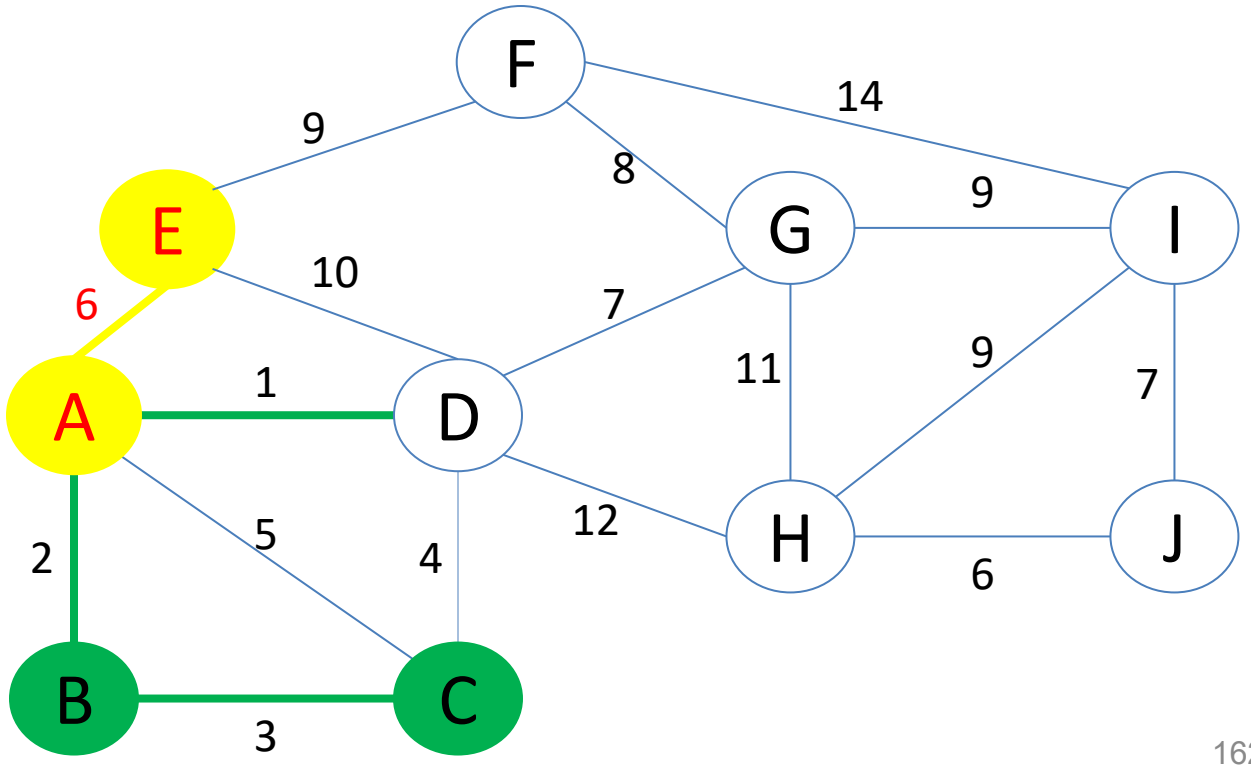


Example (9/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E)\}, \text{cost}(T) = 12$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

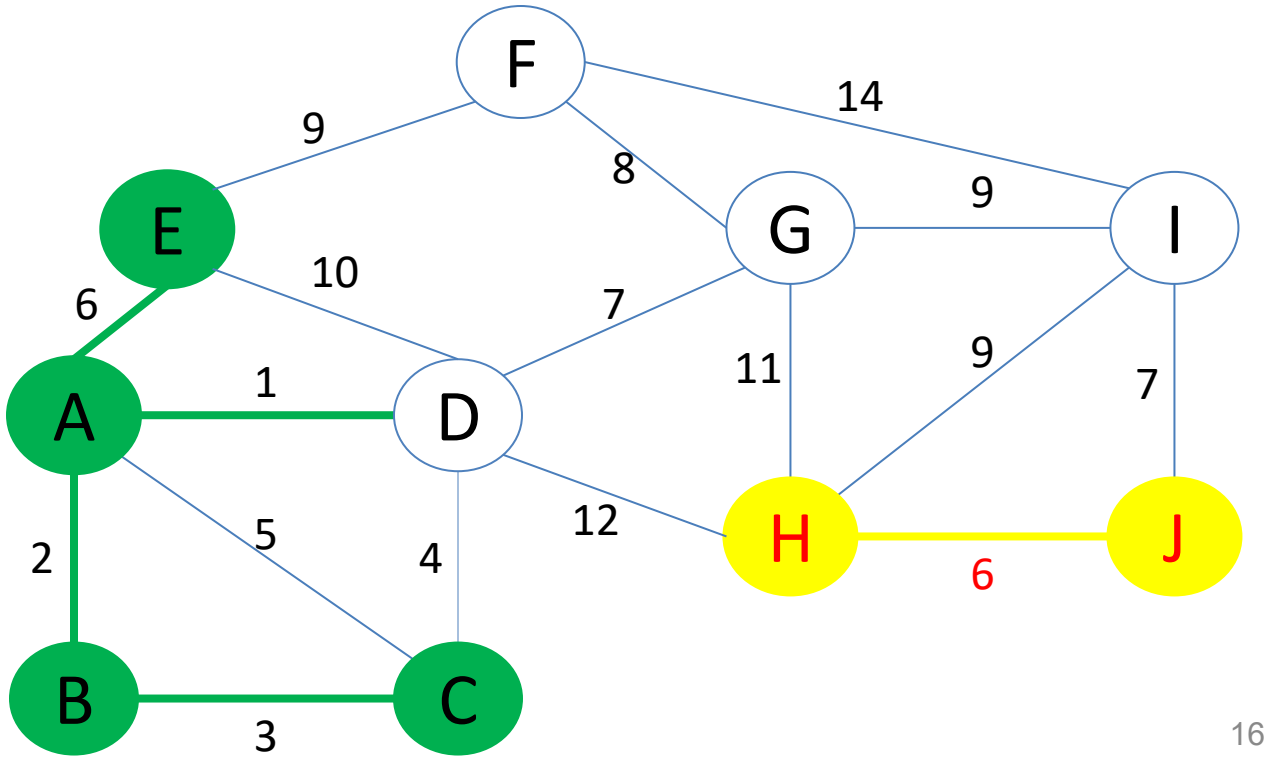


Example (10/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J)\}, \text{cost}(T) = 18$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

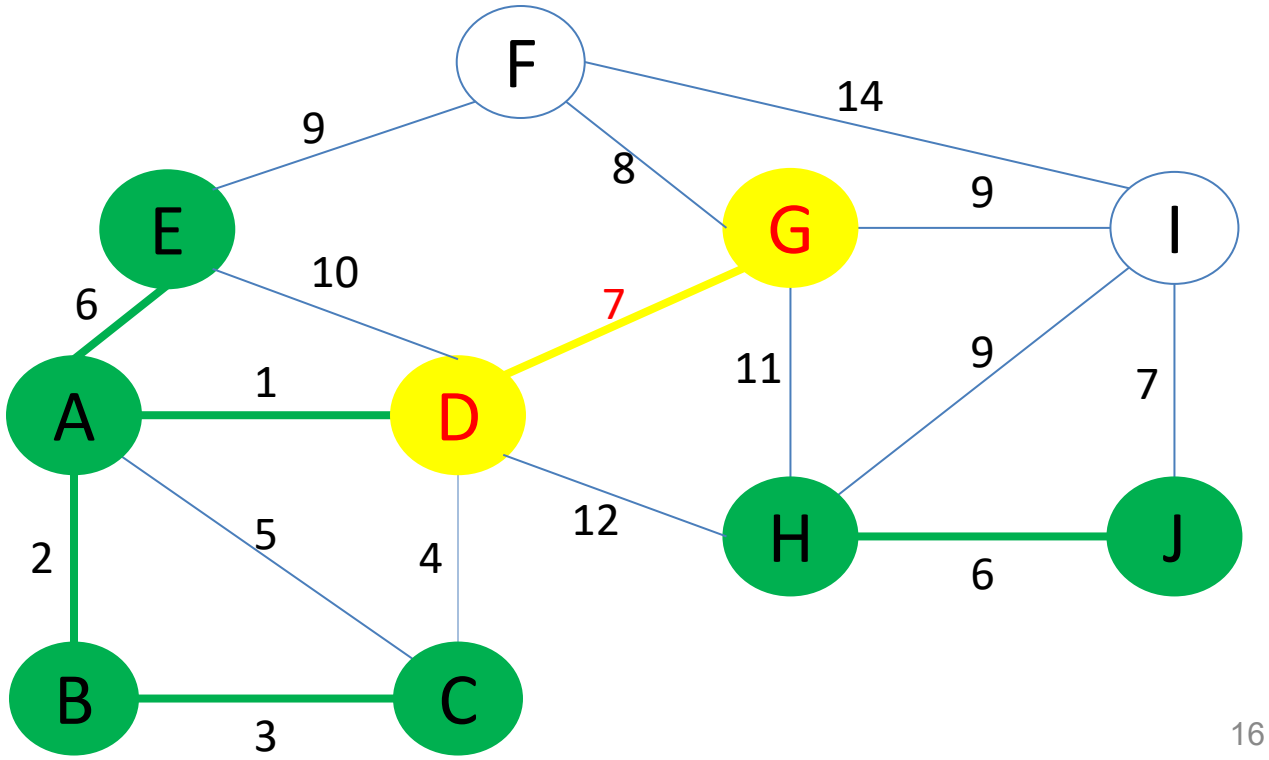


Example (11/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G)\}, \text{cost}(T) = 25$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

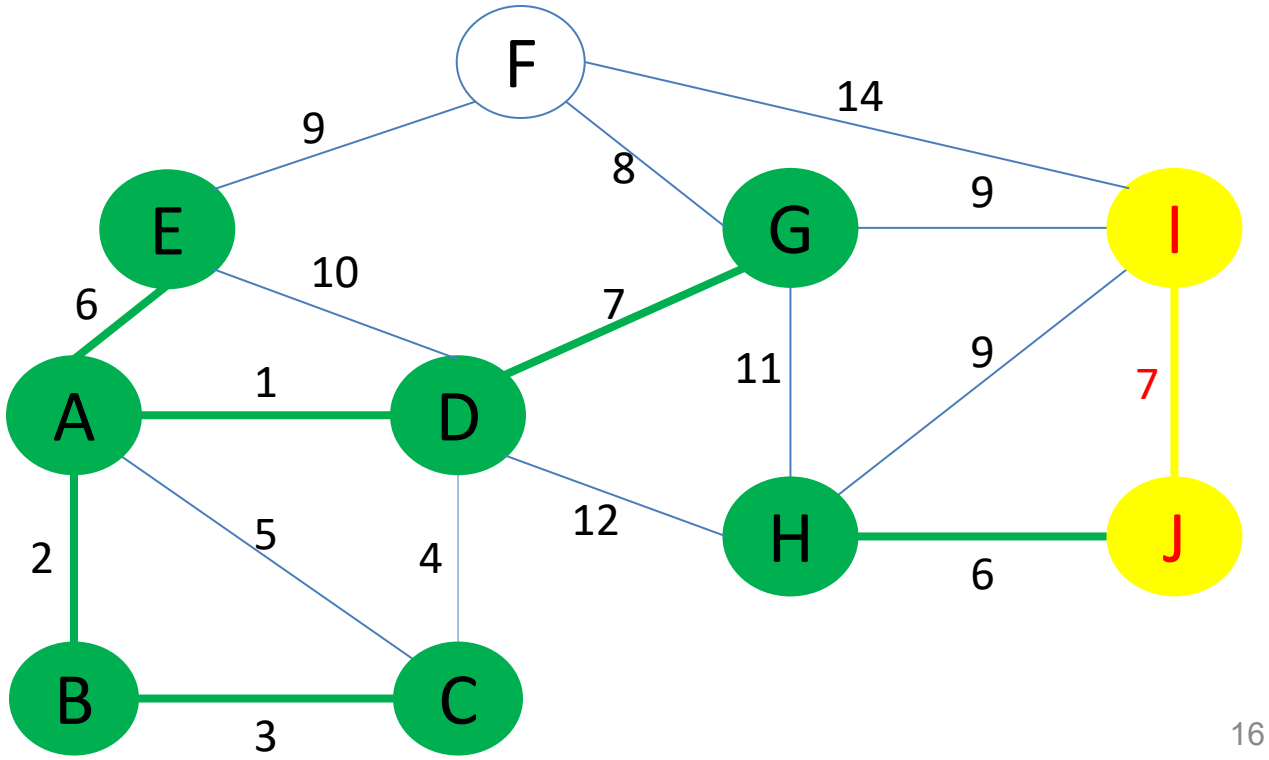


Example (12/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J)\}, \text{cost}(T) = 32$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

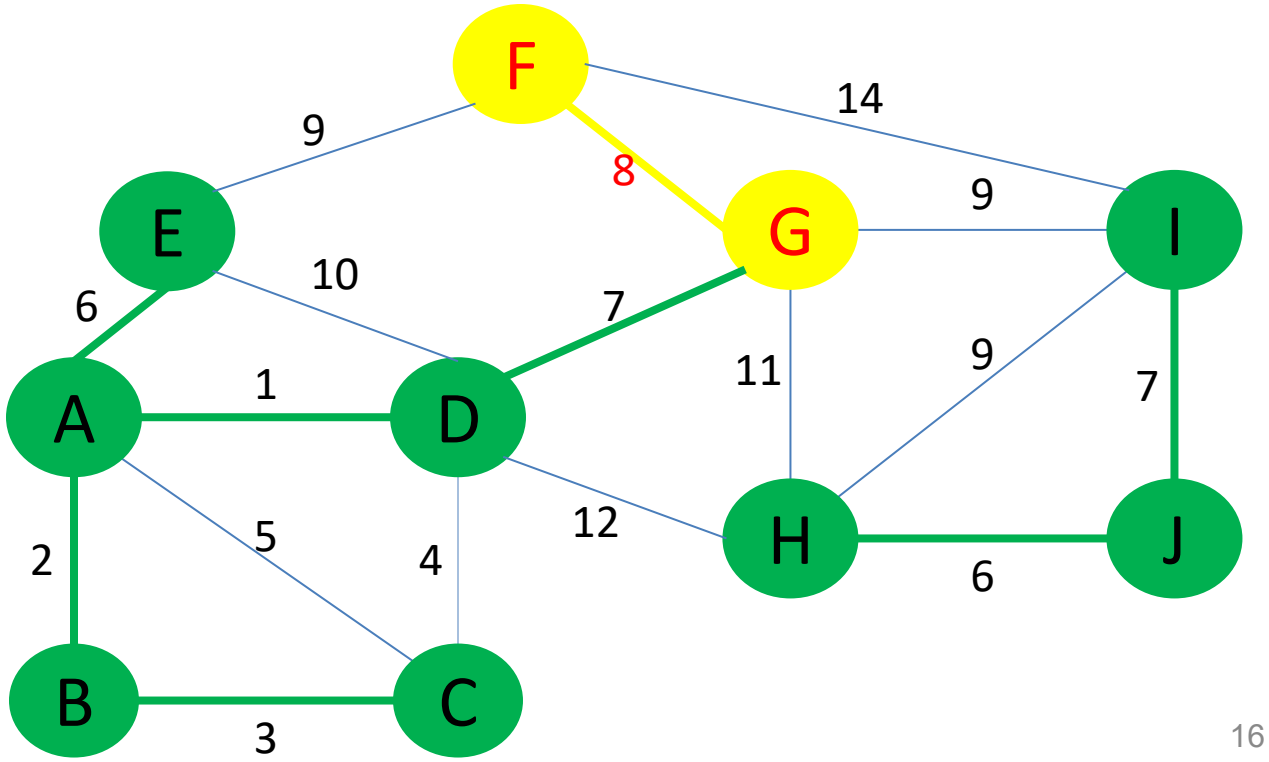


Example (13/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 40$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

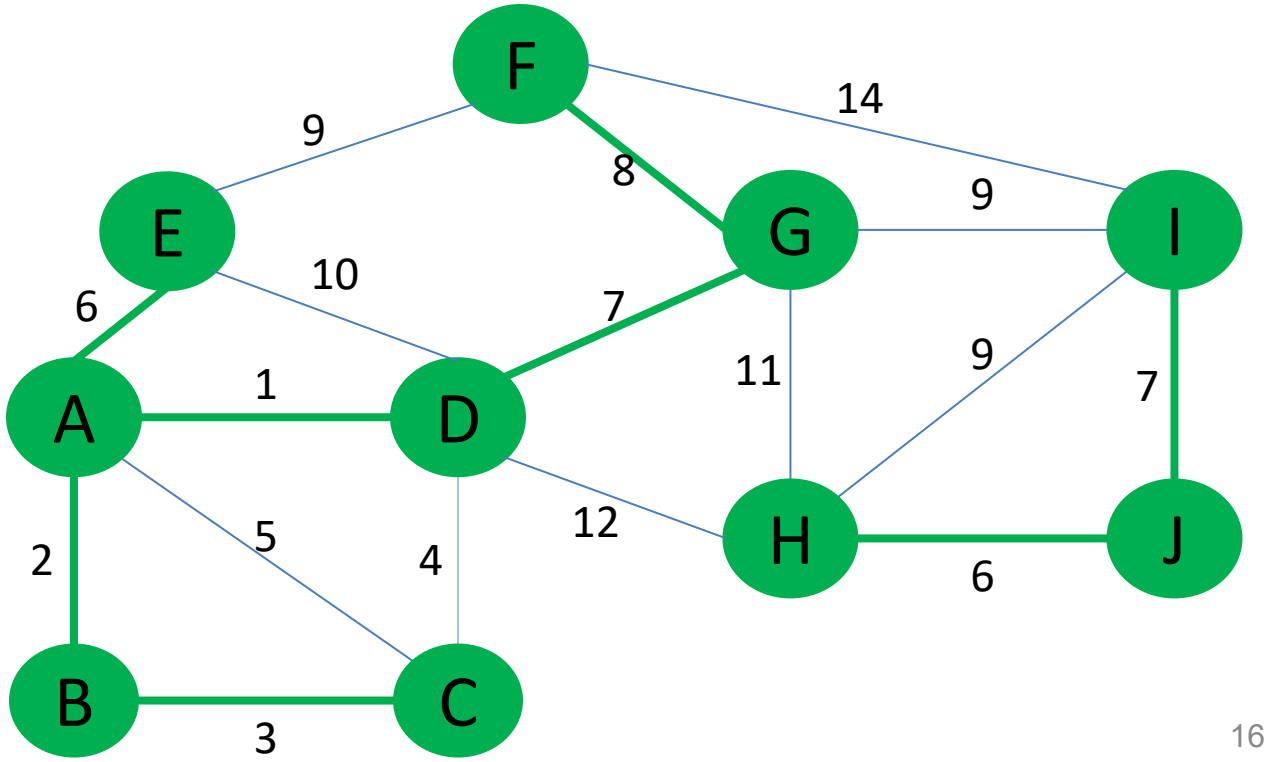


Example (14/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	~~9~~	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	~~(E, F)~~	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 40$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST. ✗
5. Stop when the MST contains exactly $(|V| - 1)$ edges. ✗

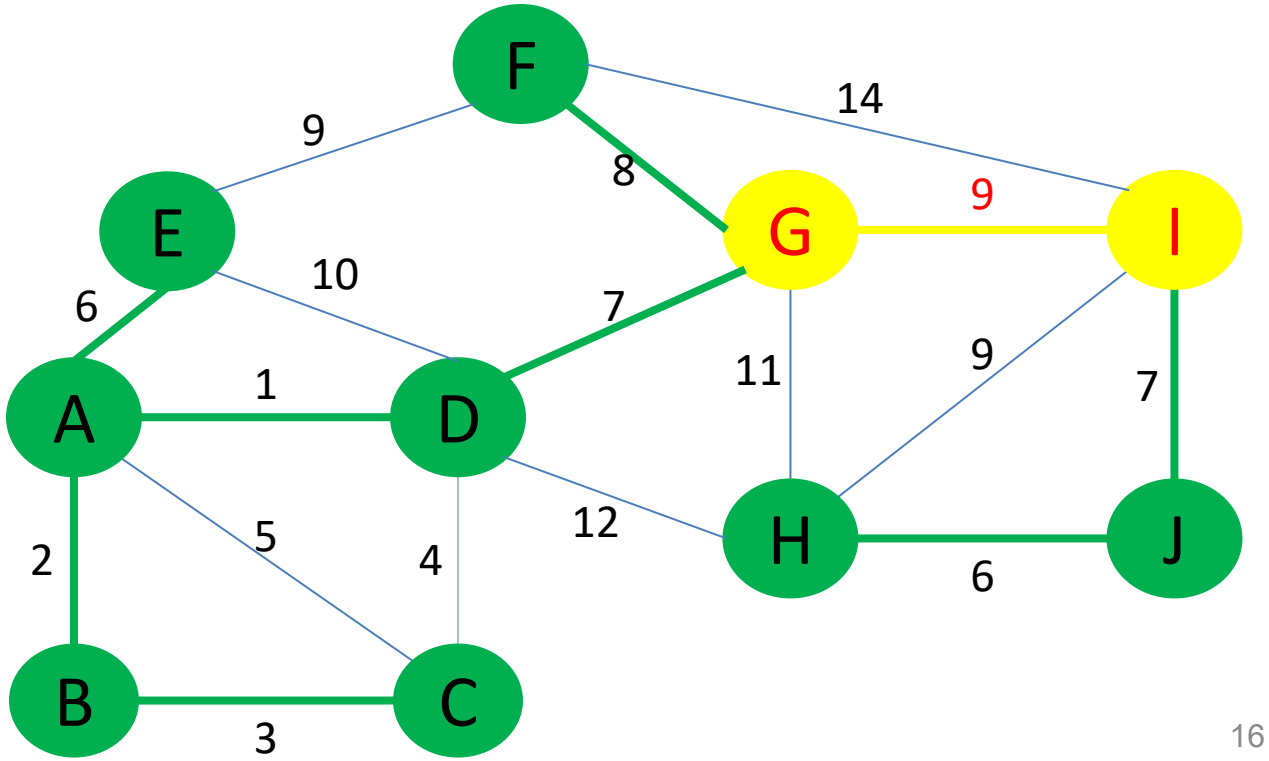


Example (15/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	~~9~~	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	~~(E, F)~~	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 49$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

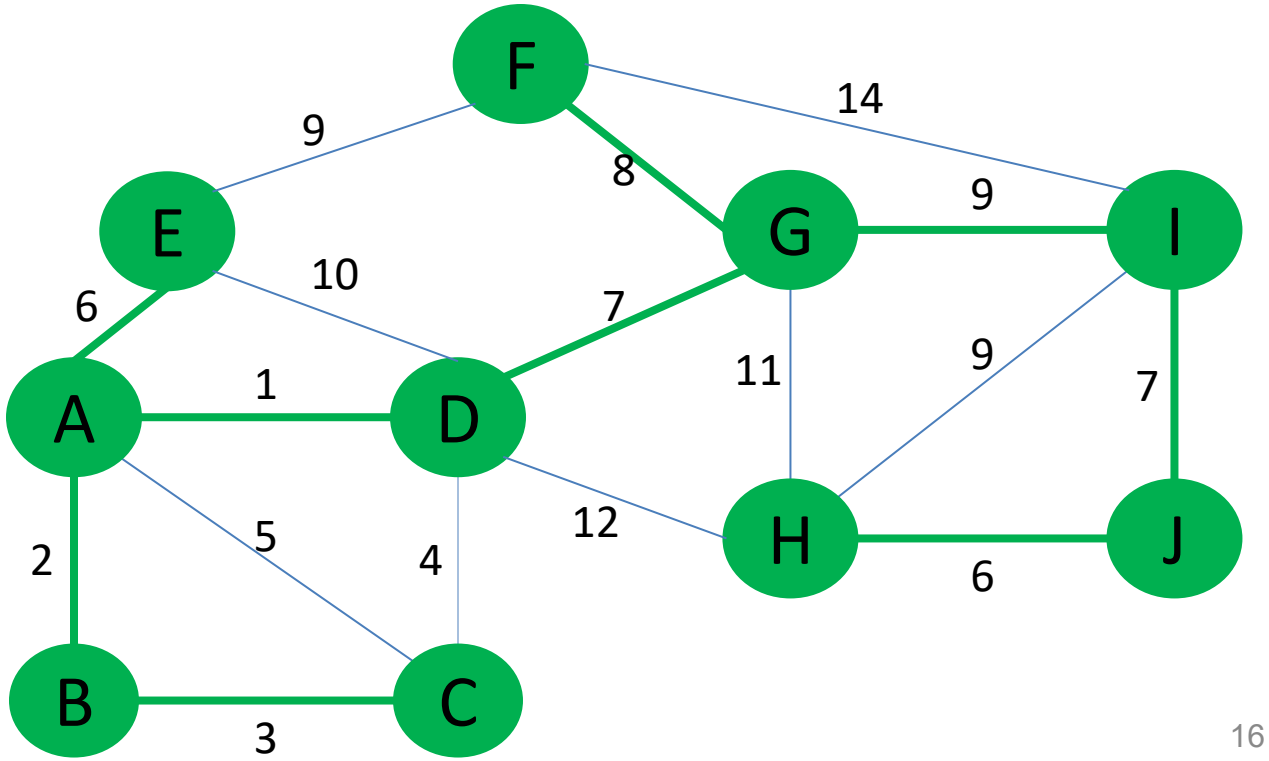


Example (16/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	~~9~~	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	~~(E, F)~~	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 49$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.



Correctness (1/2)

- Kruskal's algorithm is also a greedy algorithm
- Because at each step, it always tries to select the next unprocessed edge e with minimum weight (greedy!).
- Simple proof on how this greedy strategy works
- Loop invariant: Every edge e that is added to T by Kruskal's algorithm is part of the MST.

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

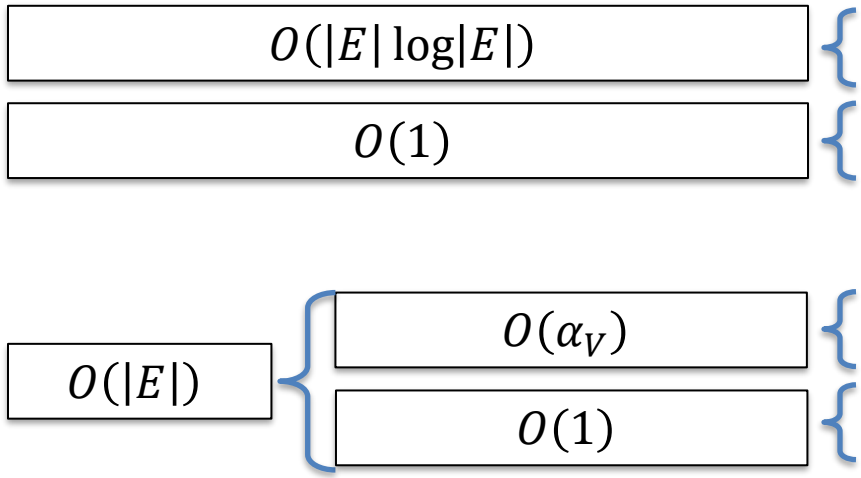
Correctness (2/2)

- Kruskal's algorithm has a special **cycle check** before adding an edge e into T. Edge e will never form a cycle.
- At the start of every loop, T is always part of MST.
- At the end of the loop, we have selected $|V| - 1$ edges from a connected weighted graph G without having any cycles. This implies that we have a Spanning Tree.
- By keep adding the next unprocessed edge e with min cost,

  $$\text{cost}(T \cup \{e\}) \leq \text{cost}(T \cup \{\text{any other unprocessed edge}\} \text{ that does not form a cycle}).$$

- At the start of every loop, T is always part of **MST**.
- At the end of the loop, the Spanning Tree T must have minimum weight $\text{cost}(T)$, so T is the final **MST**.

Complexity: analysis of running time

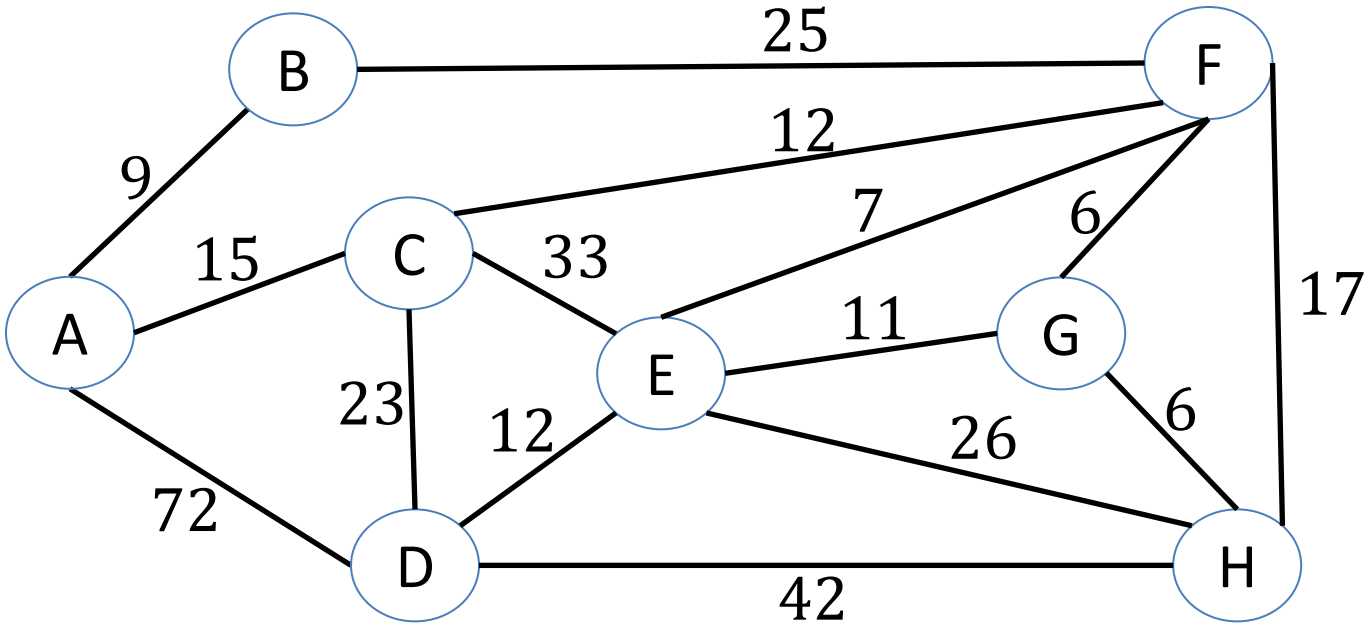


1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

$$O(|E|\log|E|) = O(|E|\log|V|^2) = O(|E|\log|V|)$$

Exercise: A Networking Problem

The vertices represent 8 regional data centers that need to be connected with high-speed data lines. Feasibility studies show that the links illustrated above are possible, and the cost in millions of dollars is displayed next to the link. Which links should be constructed to enable full communication (with relays allowed) and keep the total cost minimal?



find a minimum (cost) spanning tree

Example (1/12)

Weight	9	15	72	25	33	23	12	12	42	7	11	26	6	17	6
Edge	(A, B)	(A, C)	(A, D)	(B, F)	(C, E)	(C,D)	(C, F)	(D, E)	(D, H)	(E, F)	(E, G)	(E, H)	(F, G)	(F, H)	(G, H)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

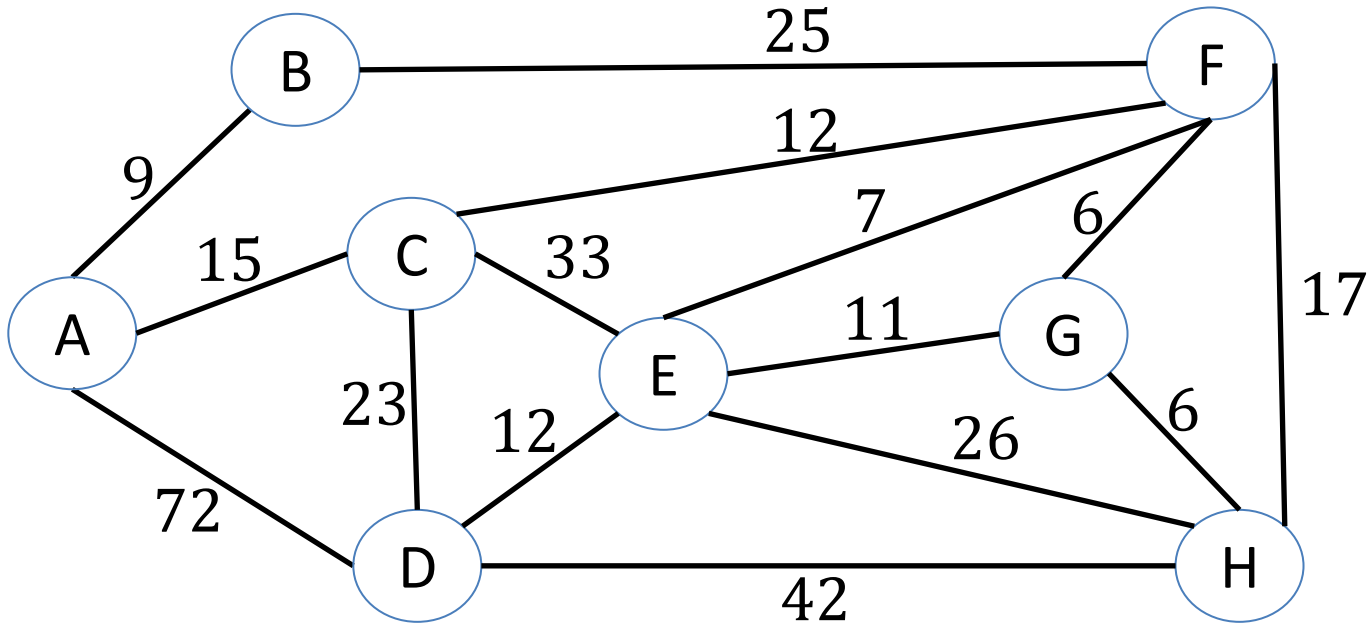
1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

Example (2/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$T = \emptyset$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

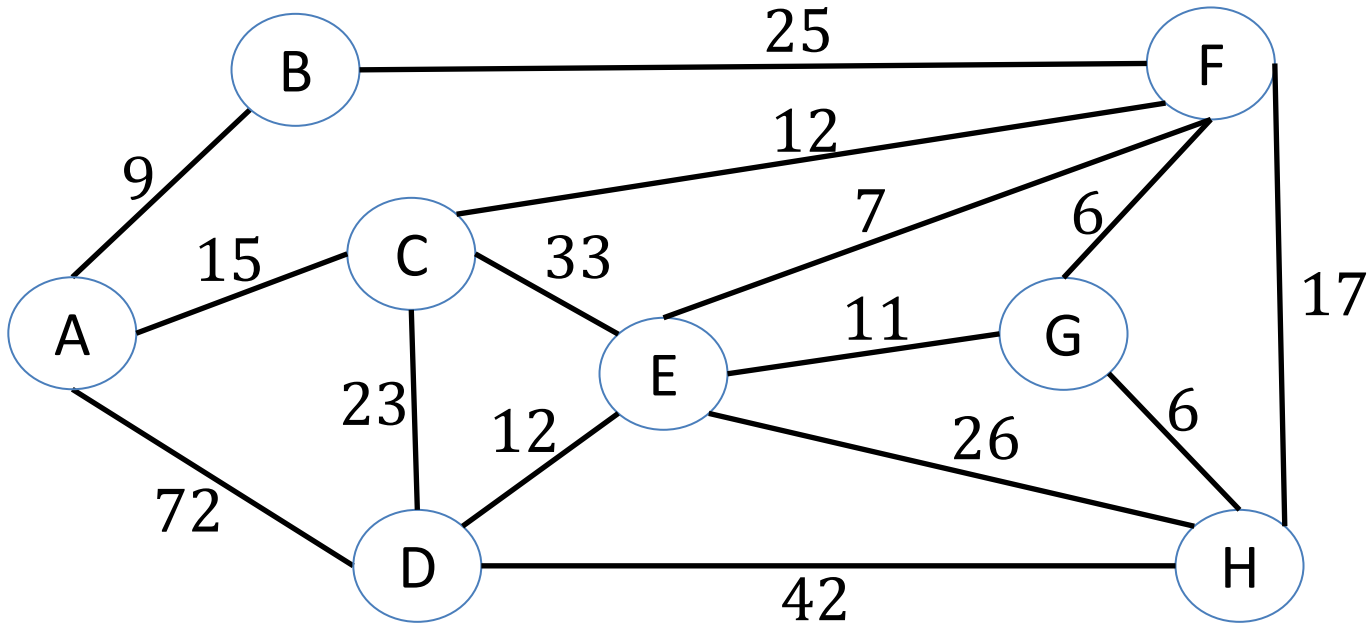


Example (3/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \emptyset, \text{cost}(T) = 0$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

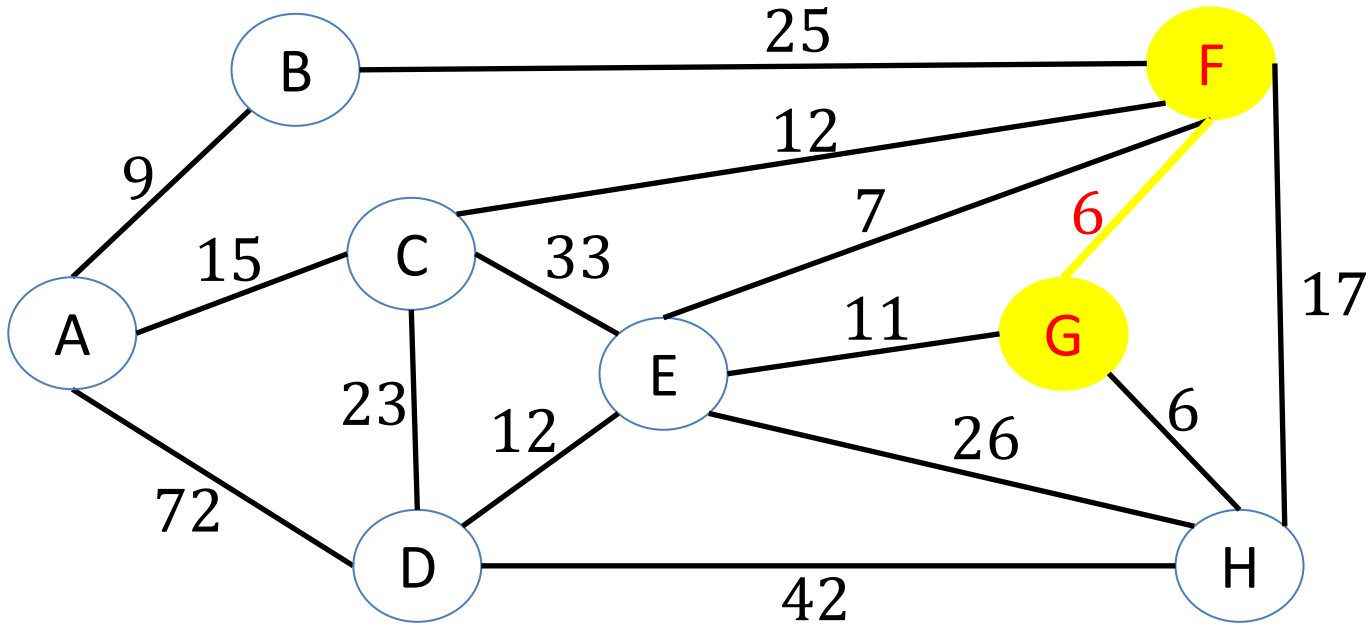


Example (4/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

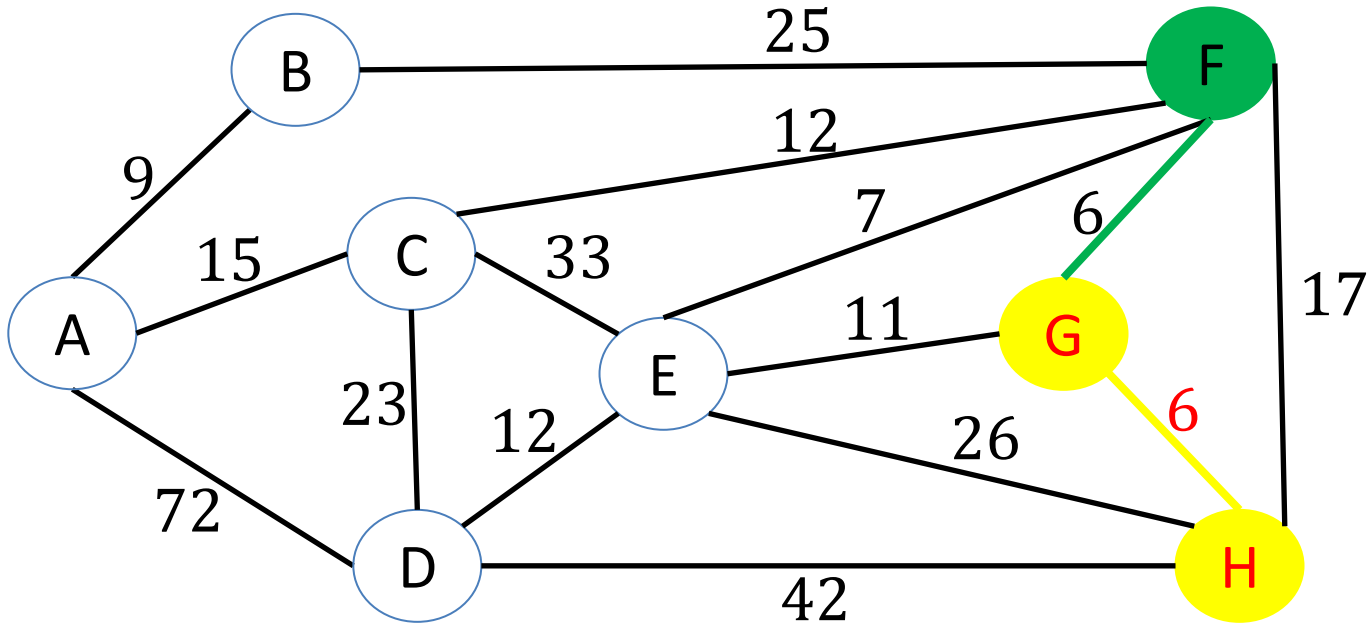


Example (5/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H)\}, \text{cost}(T) = 12$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

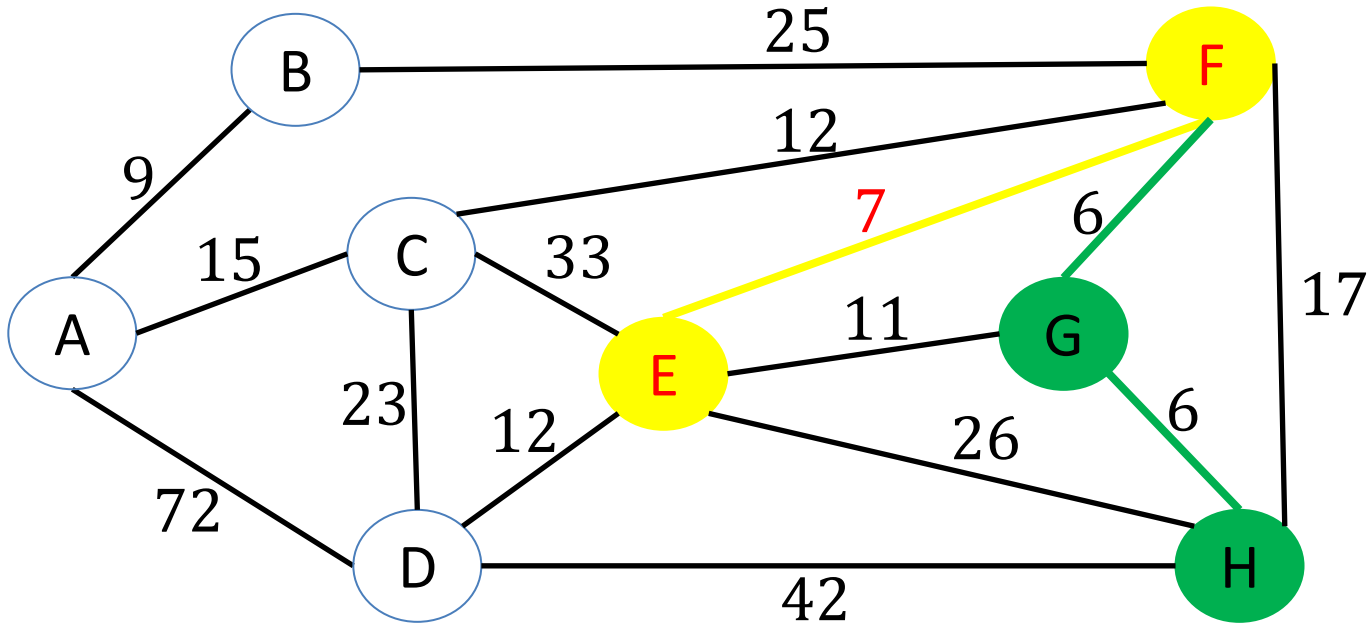


Example (6/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F)\}, \text{cost}(T) = 19$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

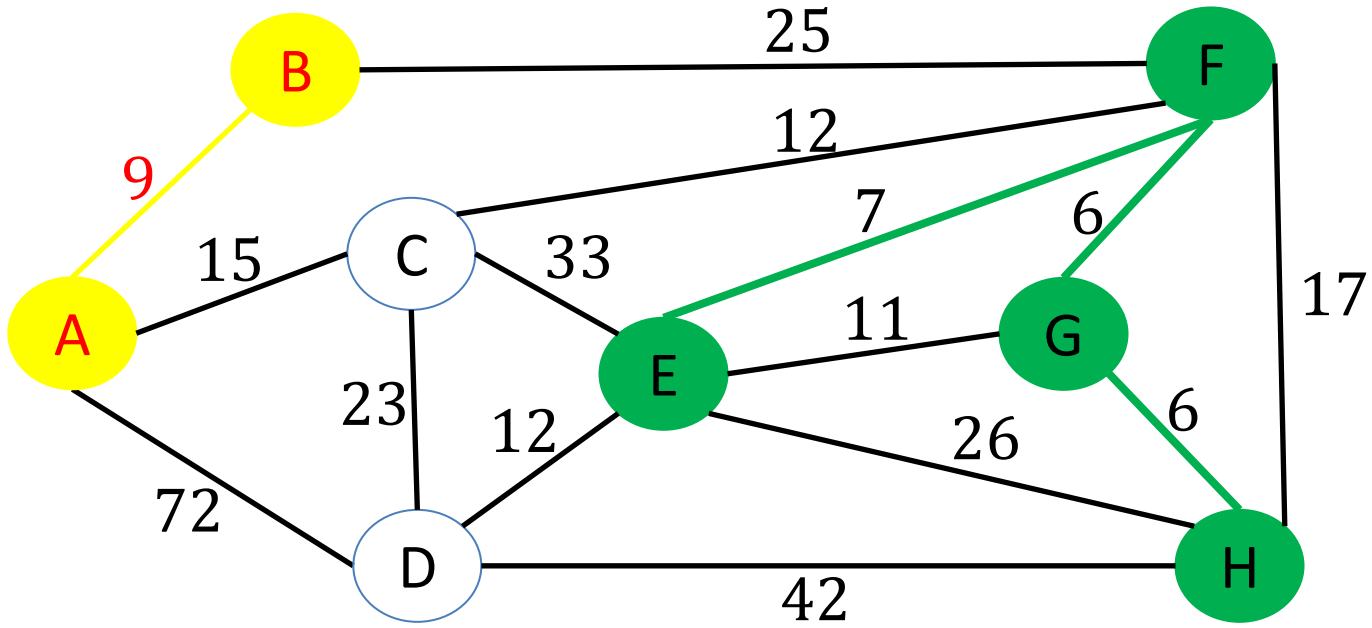


Example (7/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B)\}, \text{cost}(T) = 28$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

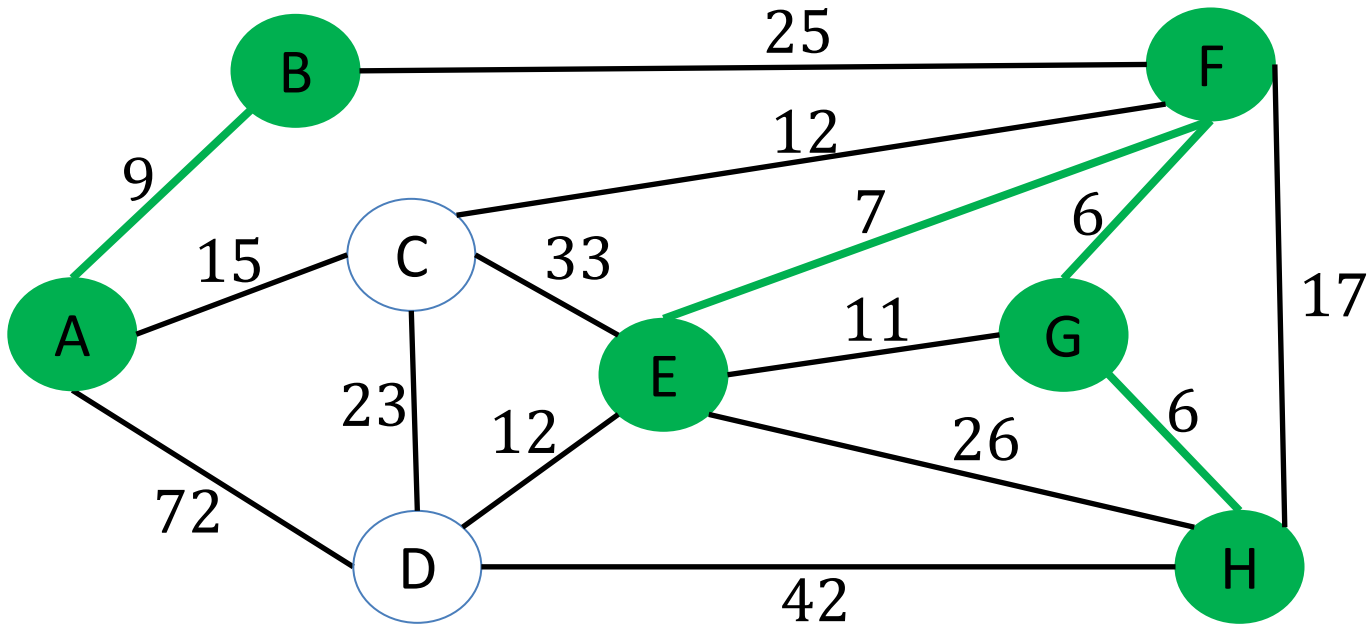


Example (8/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B)\}, \text{cost}(T) = 28$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

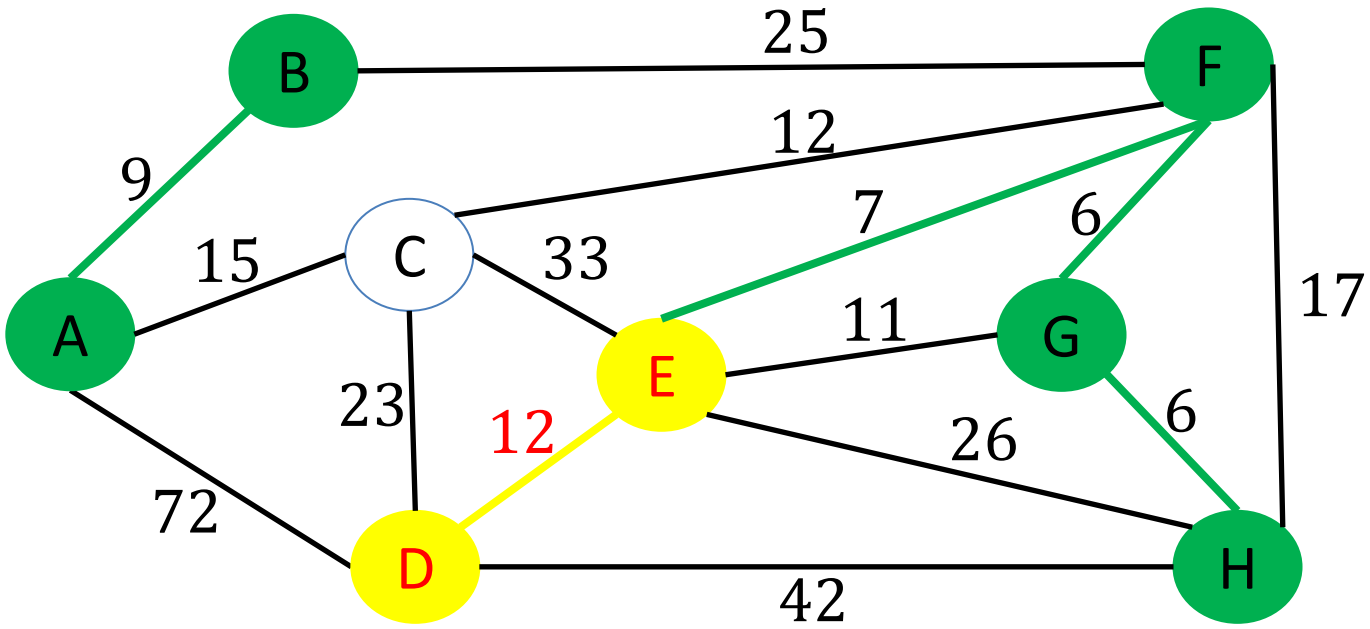


Example (9/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B), (D, E)\}, \text{cost}(T) = 40$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

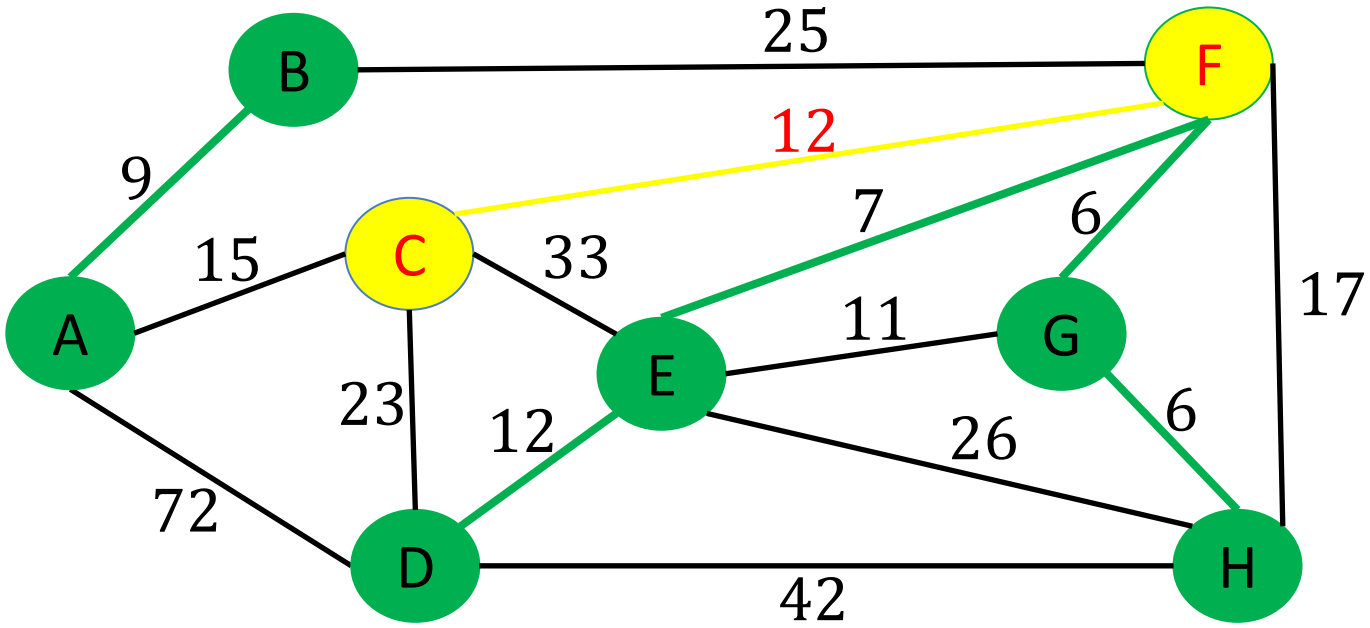


Example (10/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F,G),(G,H),(E,F),(A,B),(D,E),(C,F)\}, \text{cost}(T) = 52$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

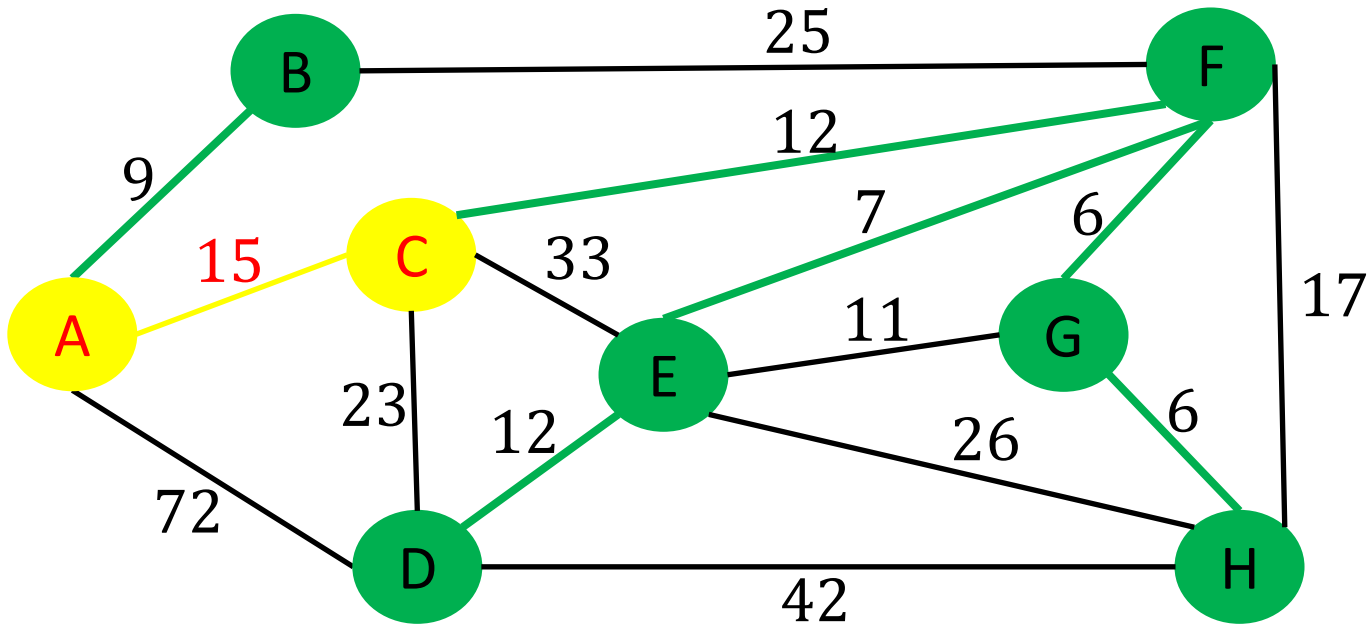


Example (11/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F,G),(G,H),(E,F),(A,B),(D,E),(C,F),(A,C)\}, \text{cost}(T) = 67$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

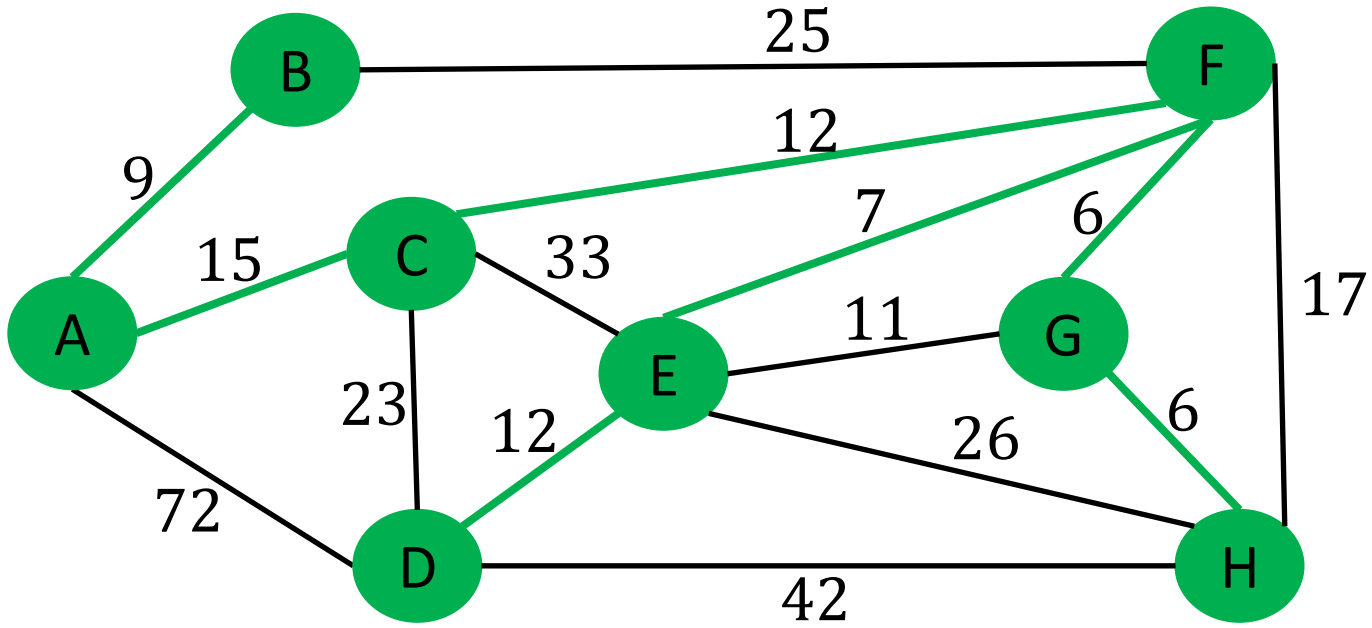


Example (12/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B), (D, E), (C, F), (A, C)\}, \text{cost}(T) = 67$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.



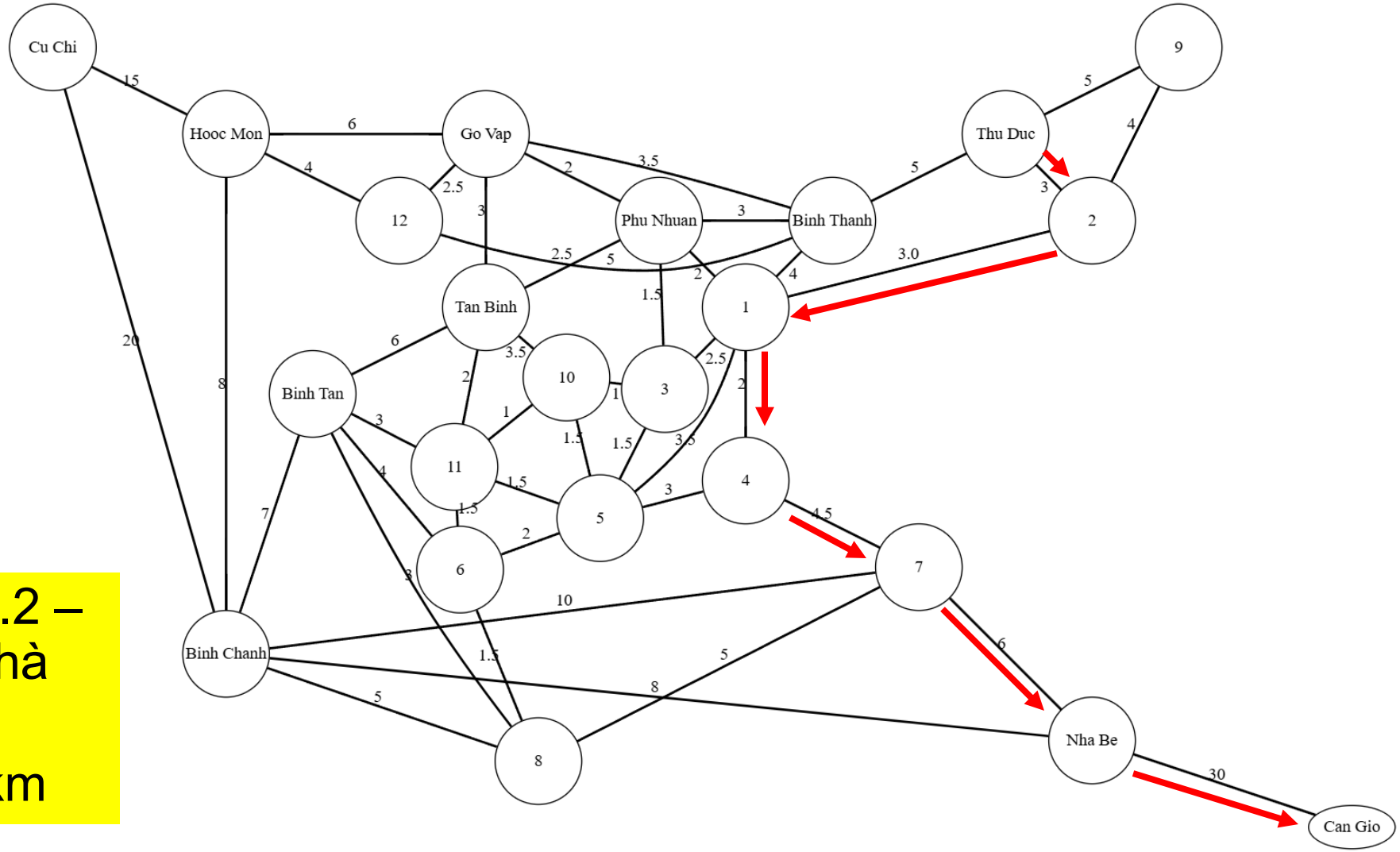
Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

Problem

- We plan to visit Cần Giờ for food, sea, and sightseeing.
- We are at Thủ Đức.

How to find the shortest path to reach Cần Giờ?



- Route: Thủ Đức – Q.2 – Q.1 – Q.4 – Q.7 – Nhà Bè - Cần Giờ
- Total distance: 48.8km

Outline

1. Introduction
2. Terminologies
3. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Graph representation
5. (Minimum) Spanning Trees (MSTs)
   - Prim's algorithm
   - Kruskal's algorithm
6. Shortest paths
   - Dijkstra's algorithm

History

Edsger Wybe Dijkstra
(1930–2002)
amturing.acm.org/award_winners/dijkstra_1053701.cfm

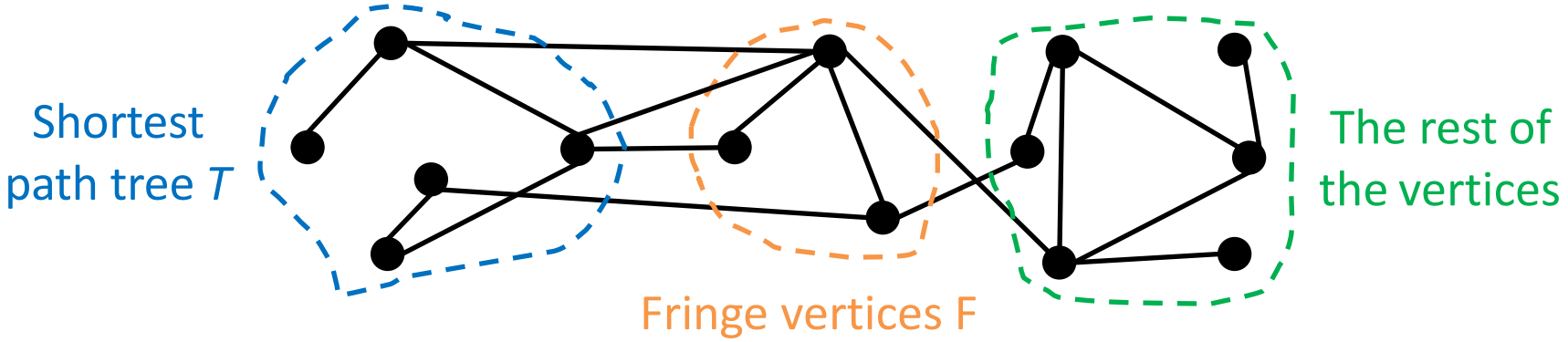
- The goal of the algorithm is to find the shortest paths between nodes in a weighted graph.
- It was developed in 1956 by the Dutch scientist Edsger W. Dijkstra [1].
- He received a Turing award in 1972.
  - "For fundamental contributions to programming as a high, intellectual challenge; for eloquent insistence and practical demonstration that programs should be composed correctly, not just debugged into correctness; for illuminating perception of problems at the foundations of program design."

[1] Dijkstra, Edsger W. "A note on two problems in connexion with graphs." *Edsger Wybe Dijkstra: his life, work, and legacy*. 2022. 287-290.

Idea

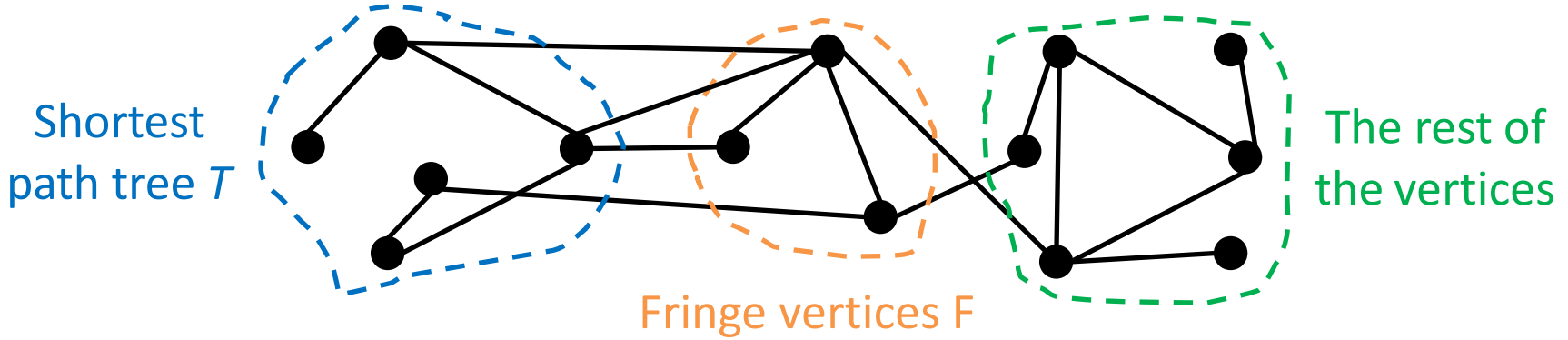
- Like Prim's algorithm, it grows a tree T one vertex and edge at a time, beginning at the source a.
- List the vertices in ascending order of their distances from the source.
- Build the shortest-path tree incrementally: at each iteration, add a single edge that establishes the newly discovered shortest path to the current vertex being incorporated.

Dijkstra's Algorithm (1/2): initialization



- Initially, the cost from the starting vertex a to vertex u is set to be infinity, i.e., $c(u) = \infty$.
- The costs are continuously updated until they represent the actual minimum cost paths from the starting vertex a to each vertex u.
  - We build the shortest path tree T starting from vertex a and expanding outward.
- At every step, only those vertices connected to at least one vertex already in T are eligible to be added next. These are referred to as the fringe vertices.
- Thus, the graph G can be conceptually divided into three regions:
  1. The tree T currently under construction,
  2. The fringe vertices adjacent to T, and
  3. The remaining vertices not yet connected.

Dijkstra's Algorithm (2/2): The Inductive Step



If $c(v) + w(v,u) < c(u)$, then the value of $c(u)$ is changed to $c(v) + w(v,u)$.

- The one that is chosen is the one for which the length of the shortest cost path to it from a through T is a minimum among all the vertices in the fringe.
- After each addition of a vertex v to T, each fringe vertex u adjacent to v is examined and two numbers are compared: the current value of $c(u)$ and the value of $c(v) + w(v,u)$, where $c(v)$ is the length of the shortest path to v (in T) and $w(v,u)$ is the weight of the edge joining v and u.

Algorithm

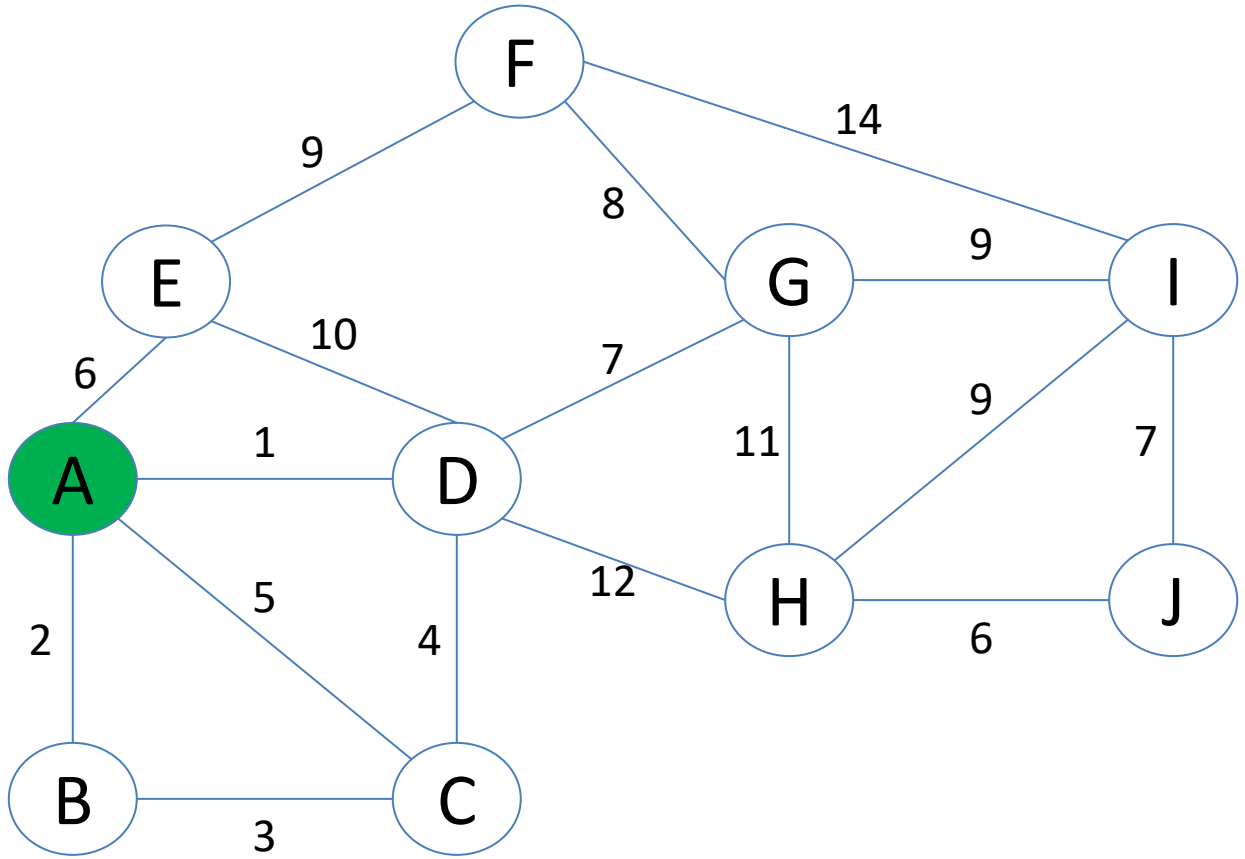
1. DIJKSTRA(G, s, t):
2. $T \coloneqq (\{s\}, \emptyset)$ ▸ tree starts with the source only
3. for each vertex u in $V - \{s\}$, set $c(u) \coloneqq \infty$
4. $c(s) \coloneqq 0$
5. $v \coloneqq s$ ▸ most recently added vertex
6. $F \coloneqq \{s\}$ ▸ fringe initially contains the source
7. while $t \notin V(T)$: ▸ repeat until destination is in the tree
    1. $F \coloneqq \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
    2. for each u in Adj(v) \ V(T):
    3. if c(v) + w(v, u) < c(u): ▸ find a better co
    4. c(u) := c(v) + w(v, u) ▸ update the cost
    5. previous(u) := v ▸ keep track of shortest path to u
    6. $x \coloneqq \underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
    7. V(T) := V(T) ∪ {x}
    8. E(T) := E(T) ∪ { {previous(x), x} } ▸ add the chosen edge
    9. v := x ▸ x becomes the new "current" vertex
8. return c(t) ▸ length of shortest s→t path

adjacent vertices of V(T) except those in V(T)

only UPDATE the cost of the vertices adjacent to v but not in V(T)

Example (1/68)

Given a connected undirected graph G with non-negative weights on the edges and a root vertex A, find for each vertex X, a directed path P(X) from A to X so that the sum of the weights on the edges in P(X) is as small as possible.



Example (2/68)

A	B	C	D	E	F	G	H	I	J
0	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

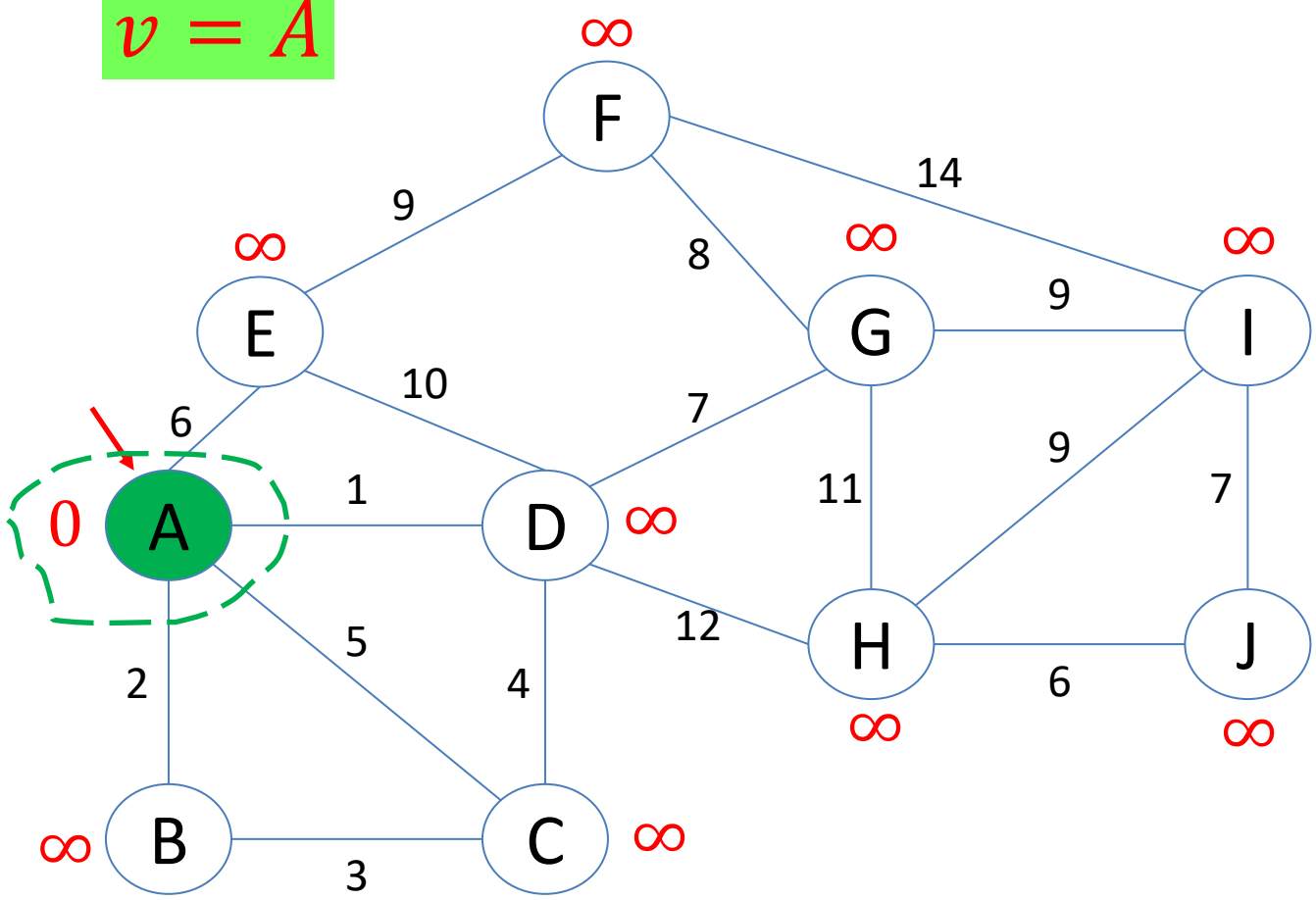
cost

previous vertex that the shortest path to u must go through

$V(T) = \{A\}$
$F = \{A\}$
$E(T) = \emptyset$

$v = A$

1. $T := (\{s\}, \emptyset)$ ▸ tree starts with the source only
2. $c(s) := 0$
3. for each vertex u in V − {s}:
4. $c(u) := \infty$ ▸ initialise zero cost
5. $v := s$ ▸ most recently added vertex
6. $F := \{s\}$ ▸ fringe initially contains the source



Example (3/68)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A\}$

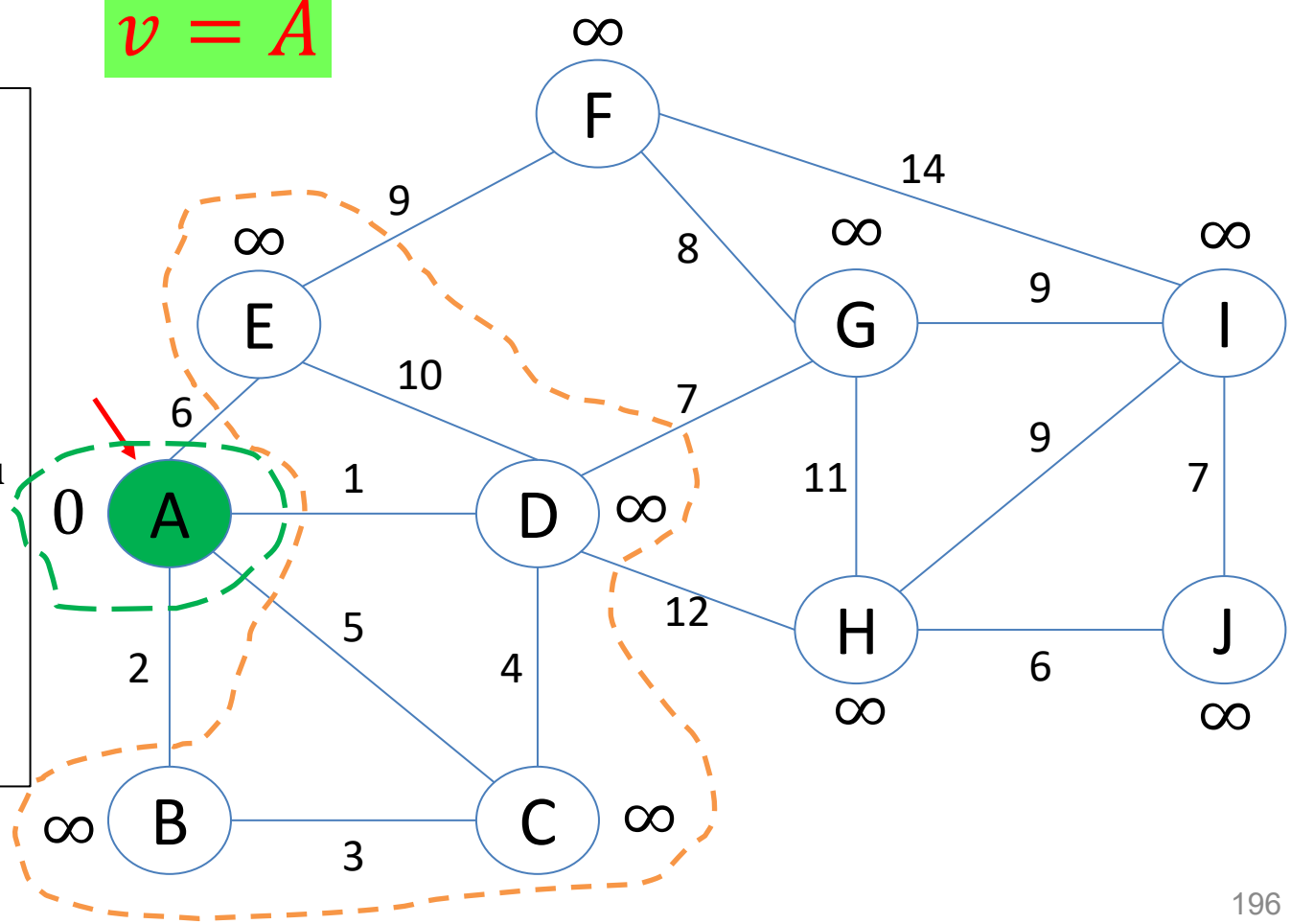
$F = \{B, C, D, E\}$

$E(T) = \emptyset$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u): ▸ find a better cost
4. c(u) := c(v) + w(v, u) ▸ update the cost
5. previous(u) := v ▸ keep track of shortest path to u
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (4/68)

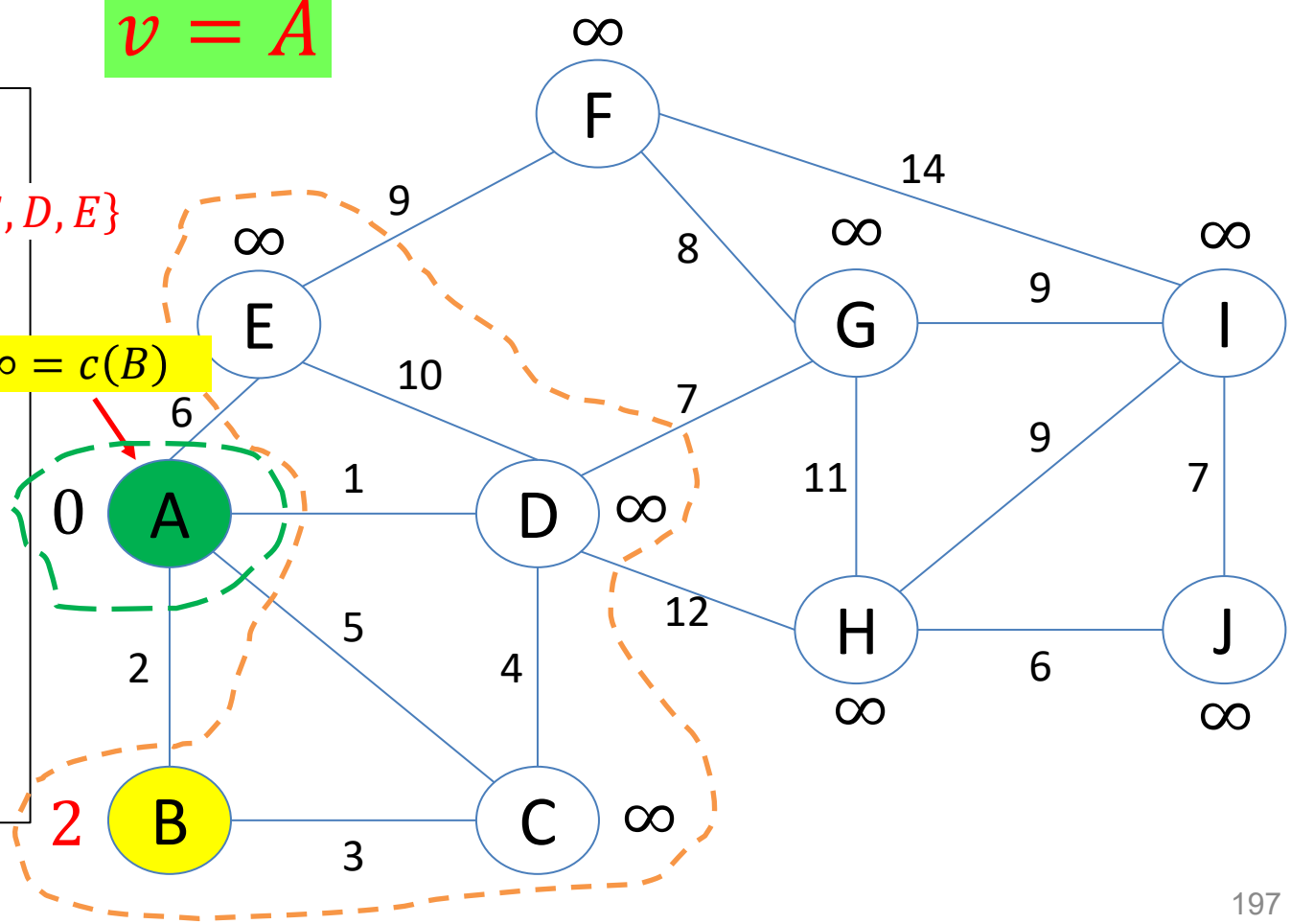
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A\}$
$F = \{B, C, D, E\}$
$E(T) = \emptyset$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{B, C, D, E\}$
2. for each u in Adj(v) \ V(T): $u = B$
3. if c(v) + w(v, u) < c(u): $c(A) + w(B, A) = 2 < \infty = c(B)$
4. c(u) := c(v) + w(v, u) $c(B) = 2$
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (5/68)

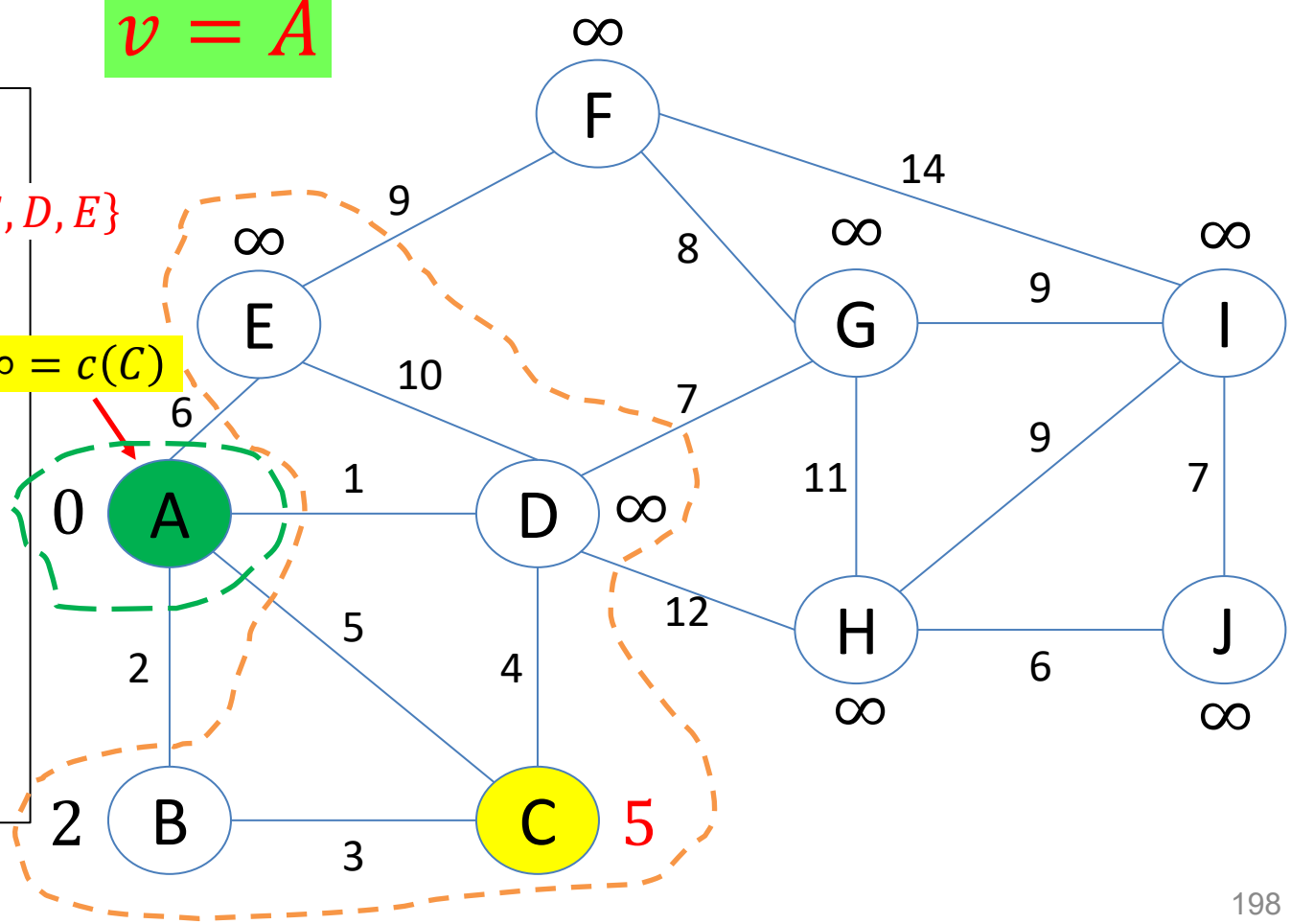
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A\}$
$F = \{B, C, D, E\}$
$E(T) = \emptyset$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{B, C, D, E\}$
2. for each u in Adj(v) \ V(T): $u = C$
3. if c(v) + w(v, u) < c(u): $c(A) + w(C, A) = 5 < \infty = c(C)$
4. c(u) := c(v) + w(v, u) $c(C) = 5$
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (6/68)

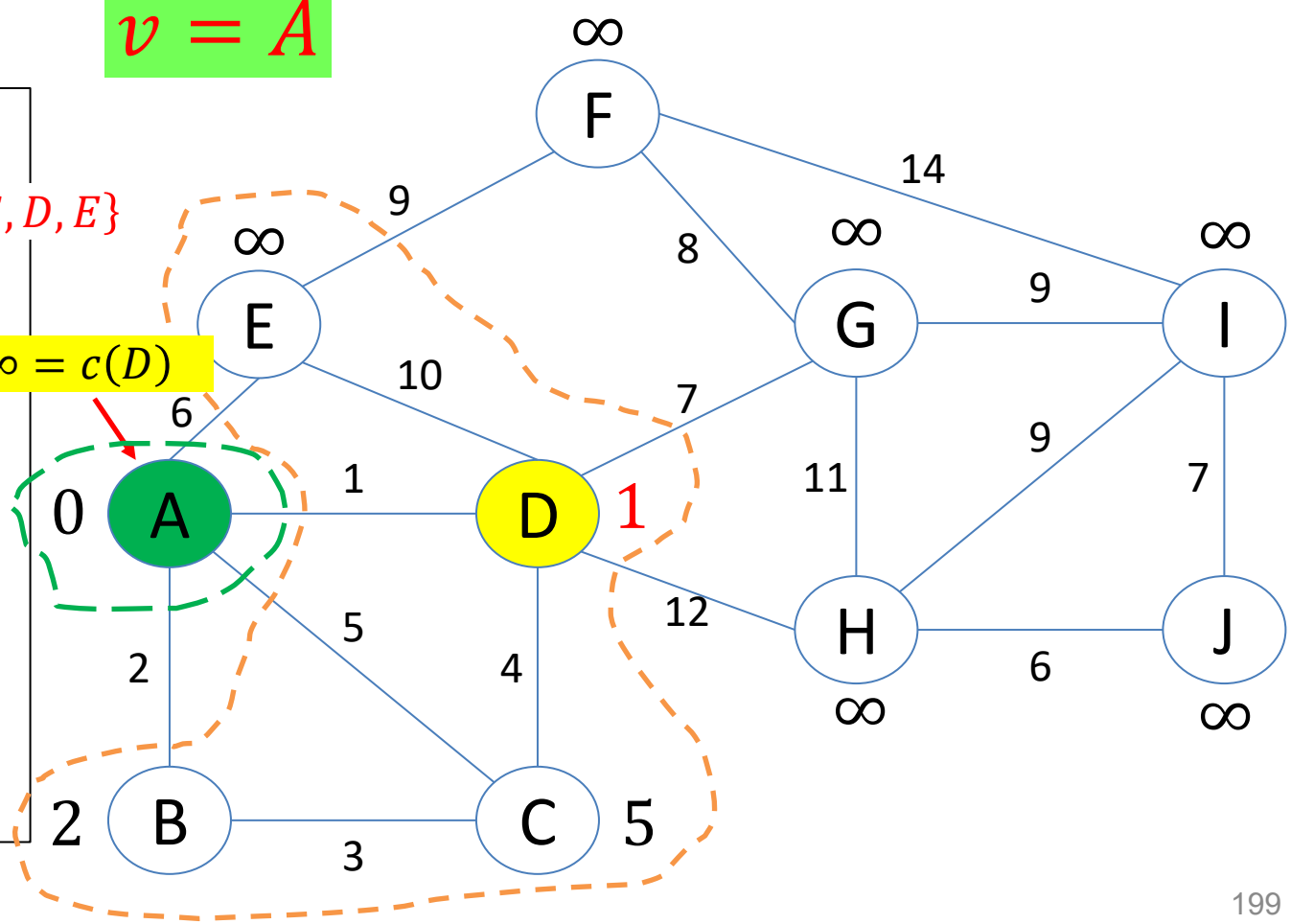
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A\}$
$F = \{\cancel{B}, \cancel{C}, D, E\}$
$E(T) = \emptyset$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{B, C, D, E\}$
2. for each u in Adj(v) \ V(T): $u = D$
3. if c(v) + w(v, u) < c(u): $c(A) + w(D, A) = 1 < \infty = c(D)$
4. c(u) := c(v) + w(v, u) $c(D) = 1$
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (7/68)

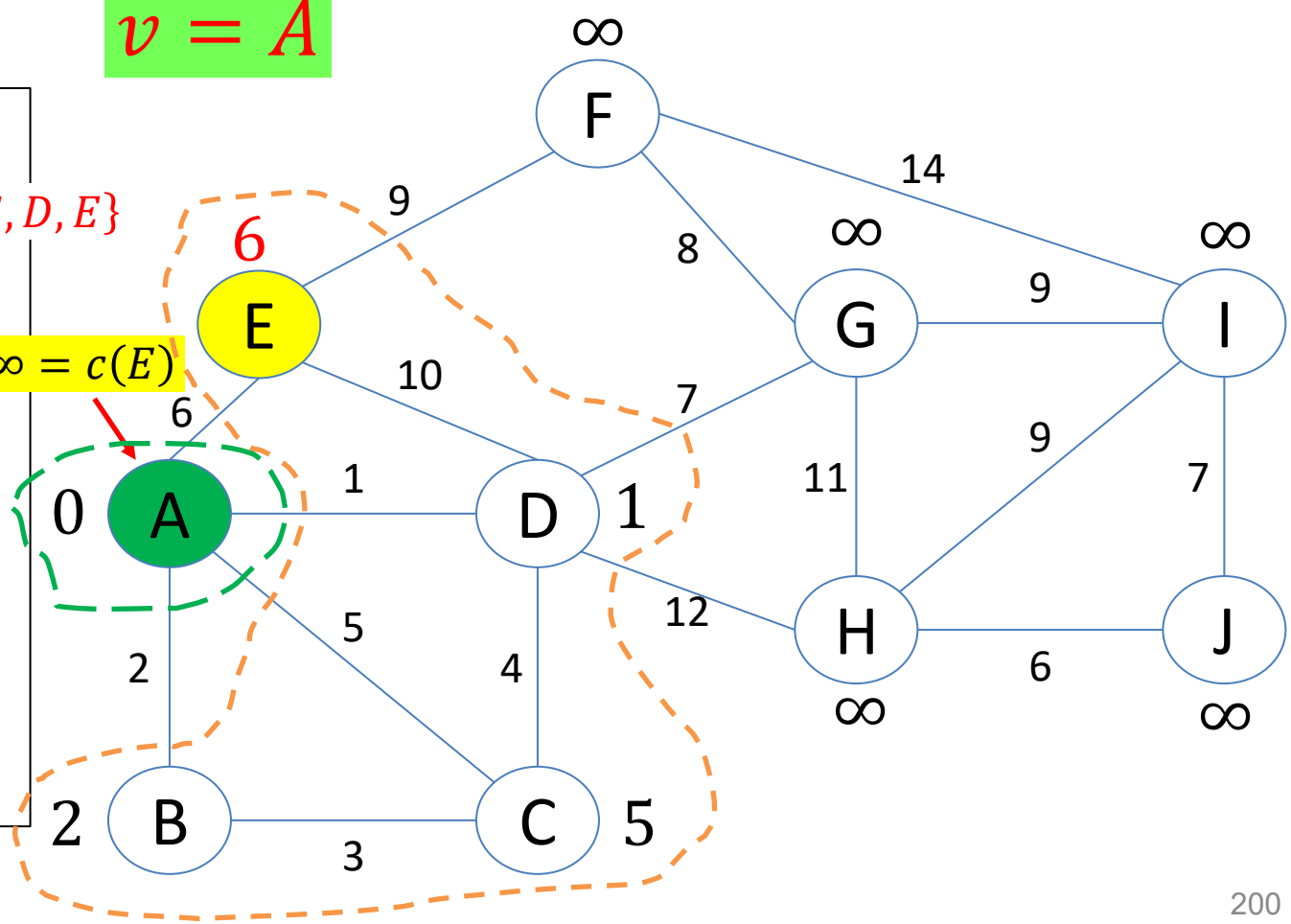
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A\}$
$F = \{\cancel{B}, \cancel{C}, \cancel{D}, E\}$
$E(T) = \emptyset$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{B, C, D, E\}$
2. for each u in Adj(v) \ V(T): $u = E$
3. if c(v) + w(v, u) < c(u): $c(A) + w(E, A) = 6 < \infty = c(E)$
4. c(u) := c(v) + w(v, u) $c(E) = 6$
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (8/68)

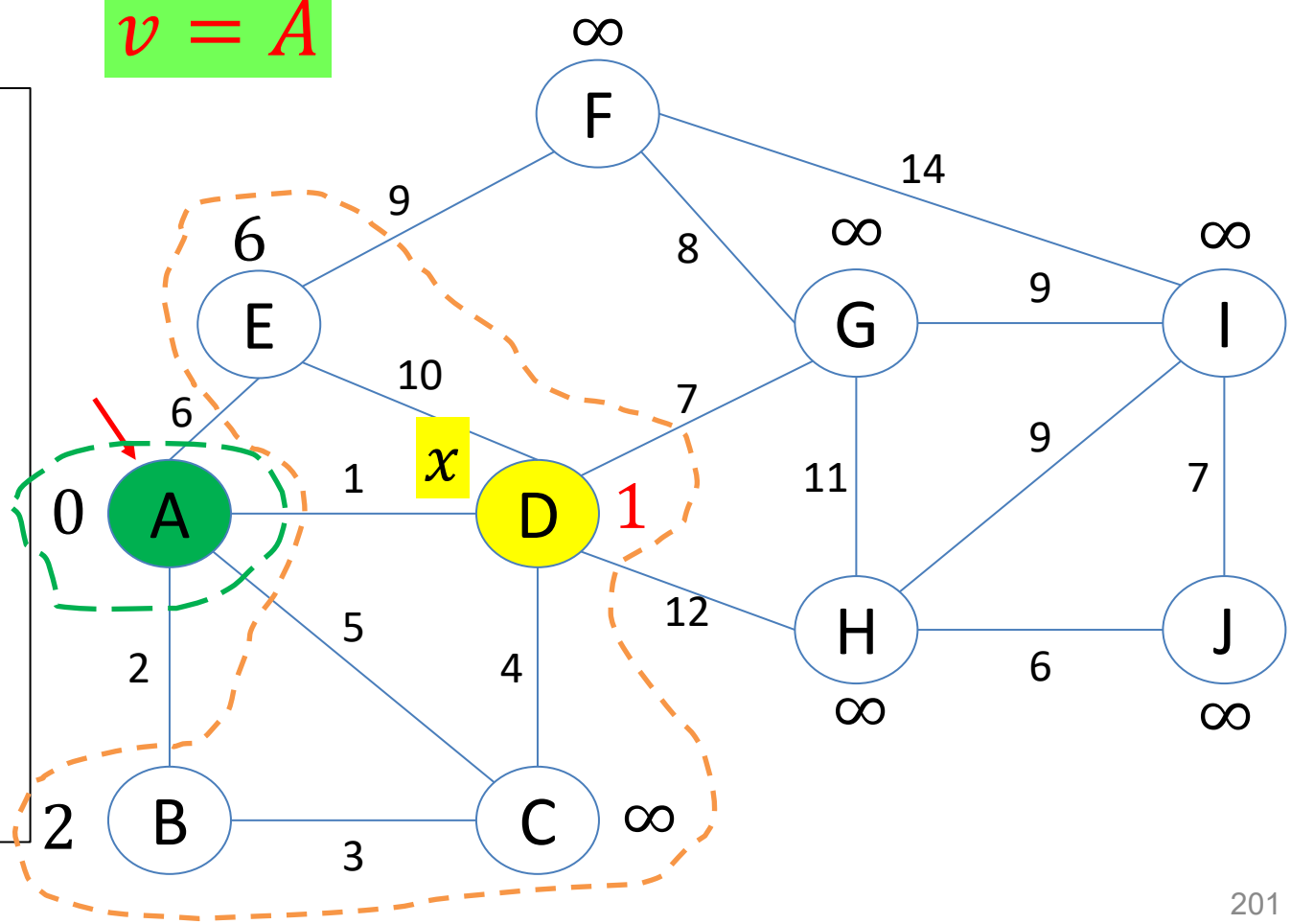
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A\}$
$F = \{B, C, D, E\}$
$E(T) = \emptyset$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (9/68)

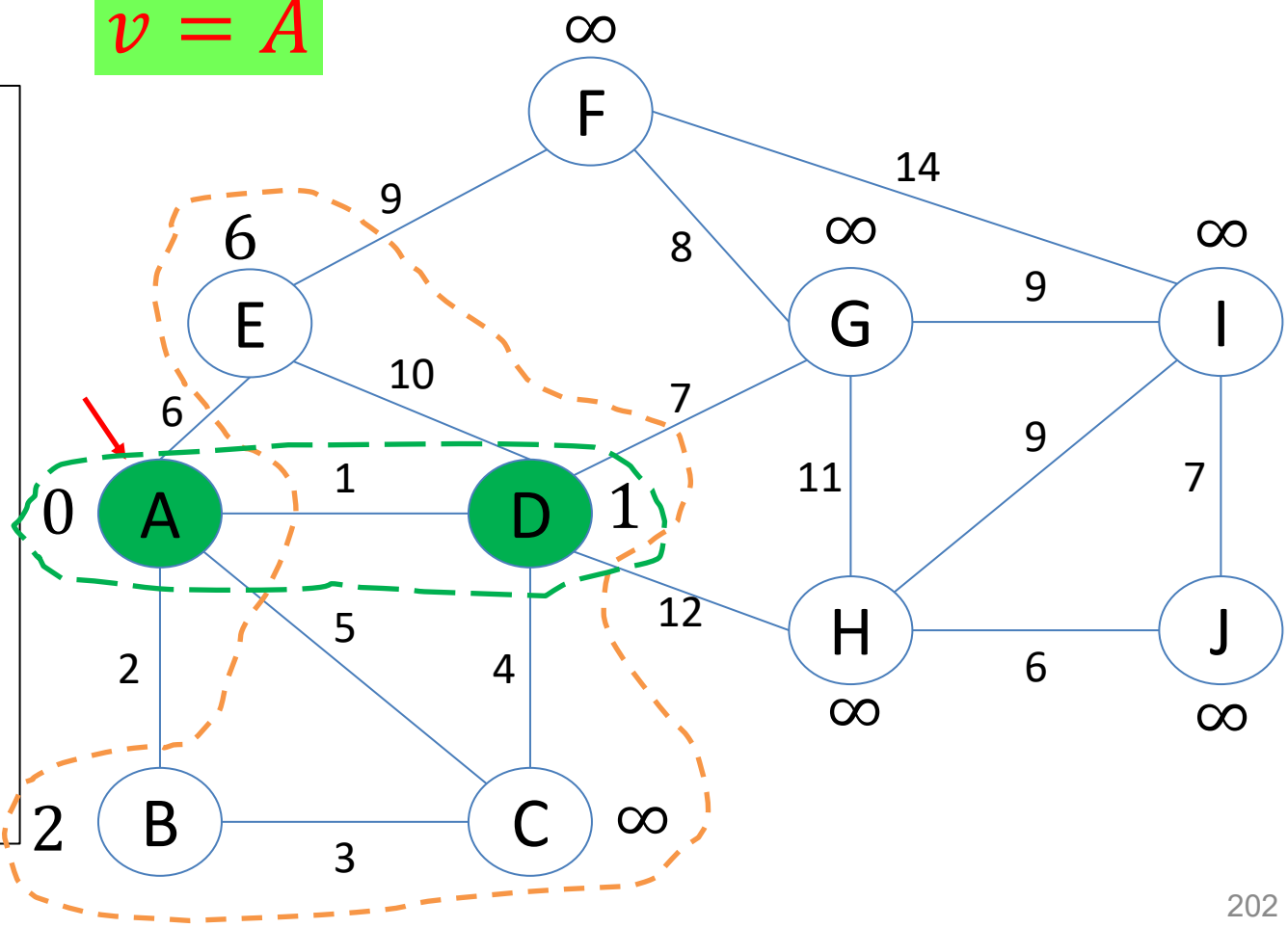
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$
$F = \{B, C, D, E\}$
$E(T) = \emptyset$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

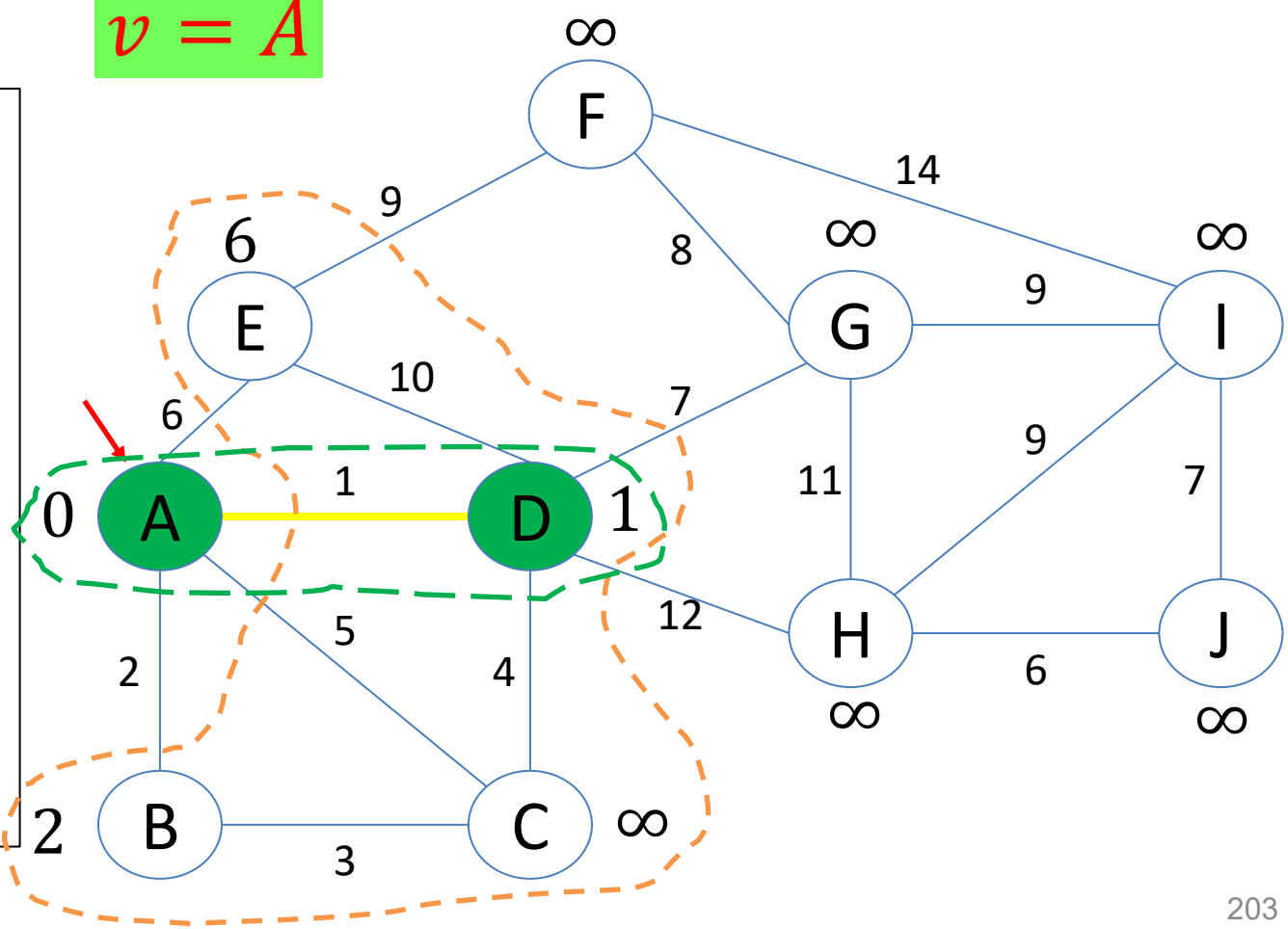
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$
$F = \{B, C, D, E\}$
$E(T) = \{(A, D)\}$

$v = A$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

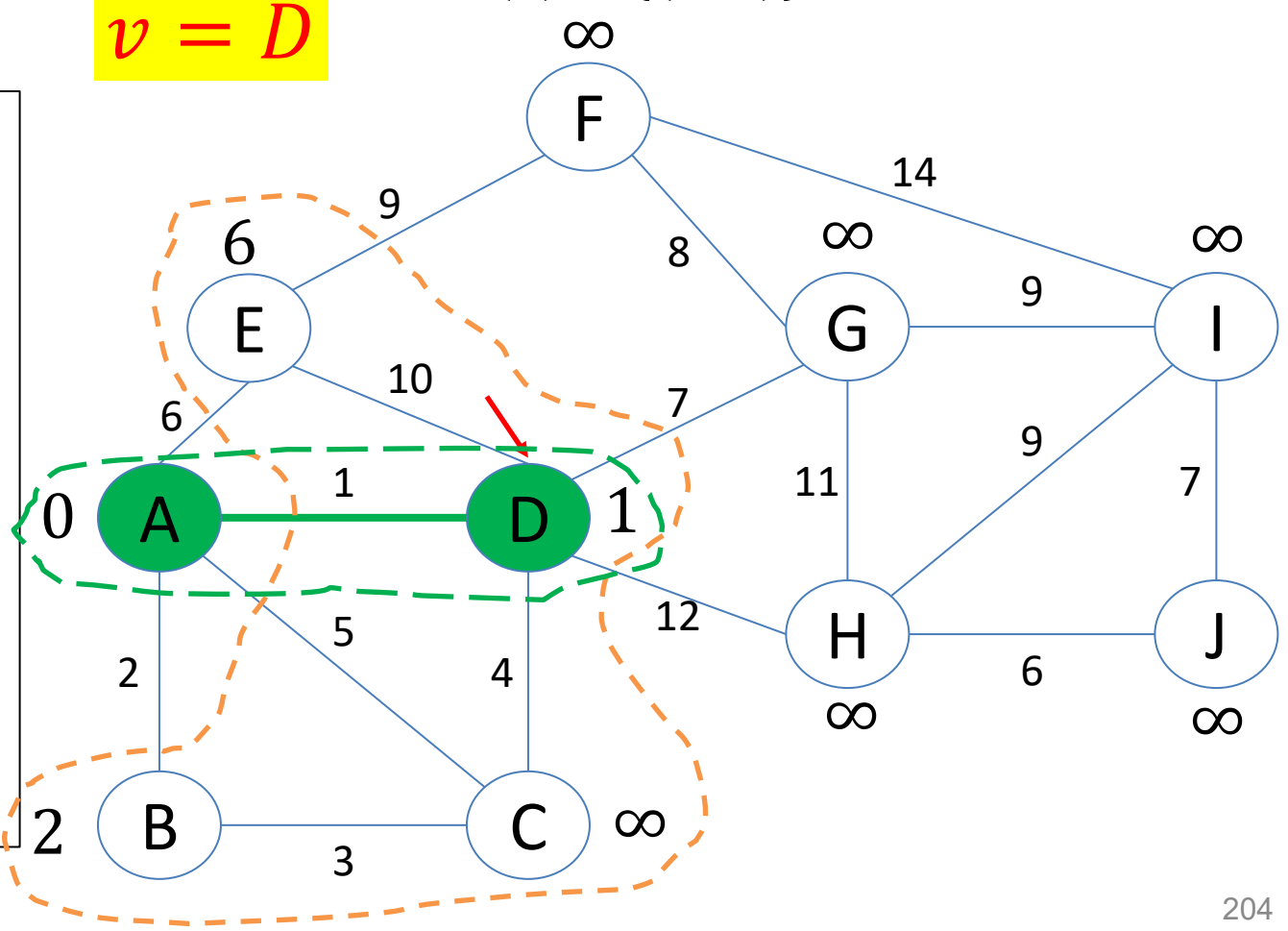
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$
$F = \{B, C, D, E\}$
$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$

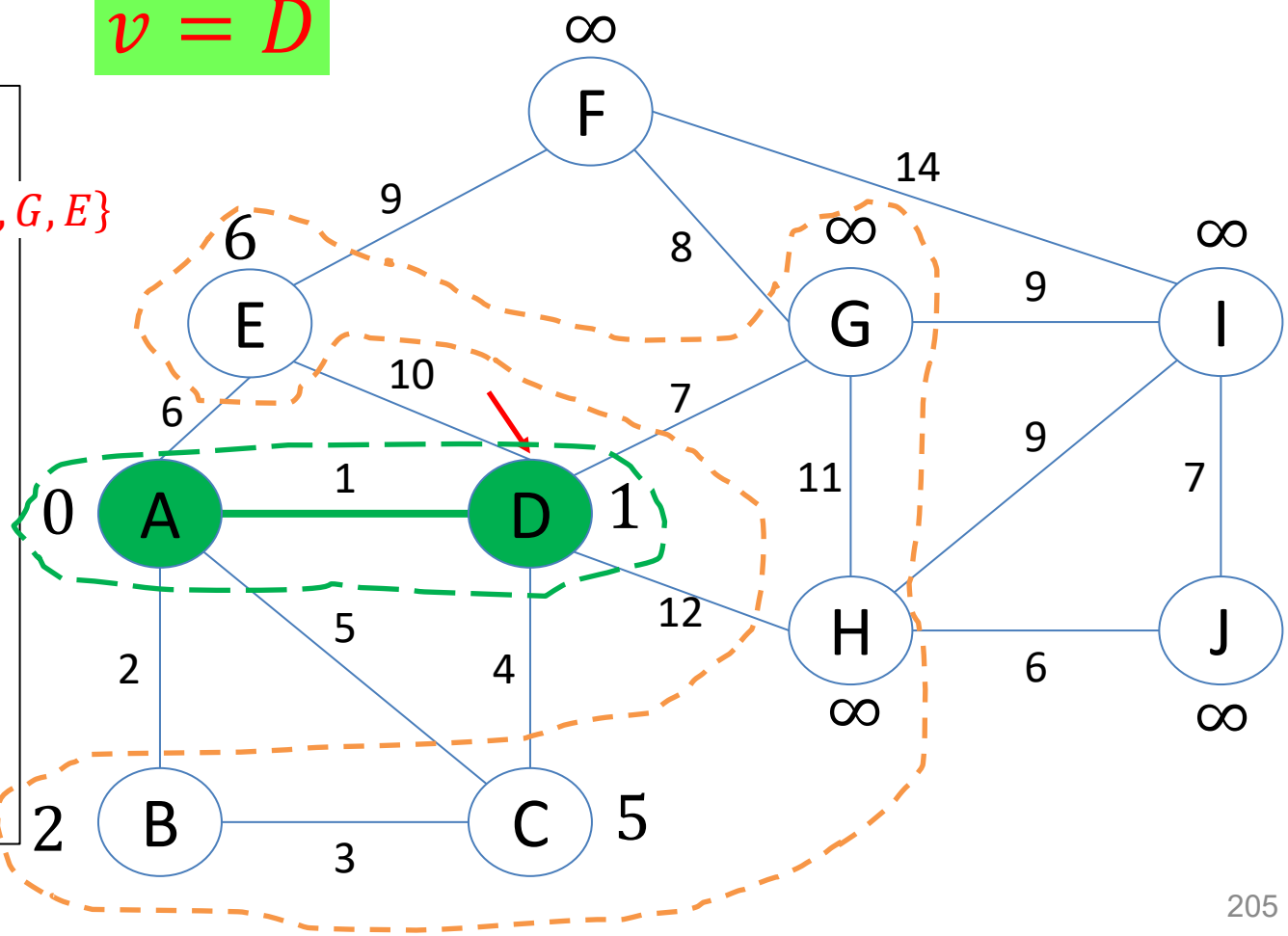
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{C, H, G, E\}$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$

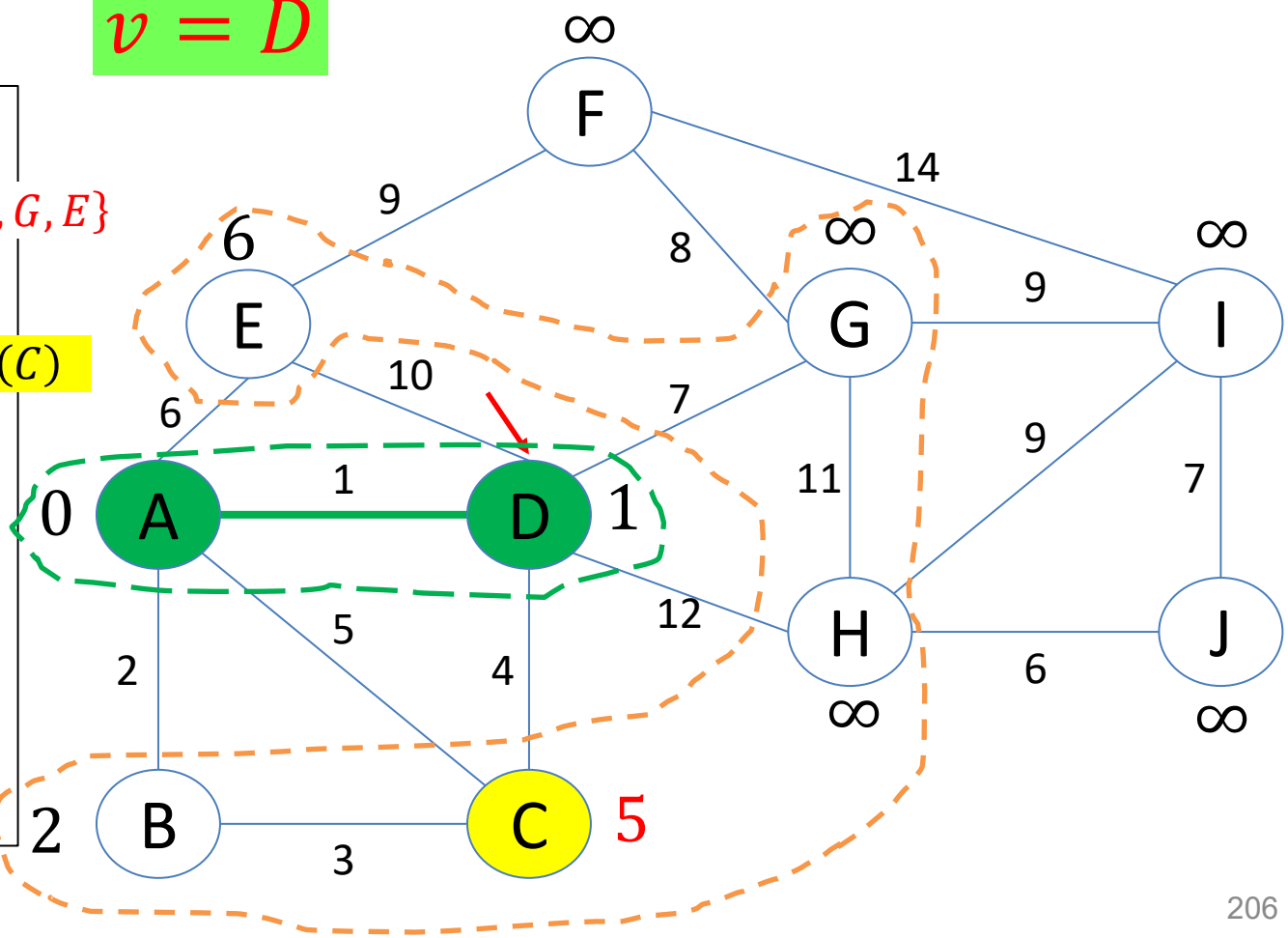
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{C, H, G, E\}$
2. for each u in Adj(v) \ V(T): $u = C$
3. if c(v) + w(v, u) < c(u): $c(D) + w(C, D) = 5 = c(C)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$

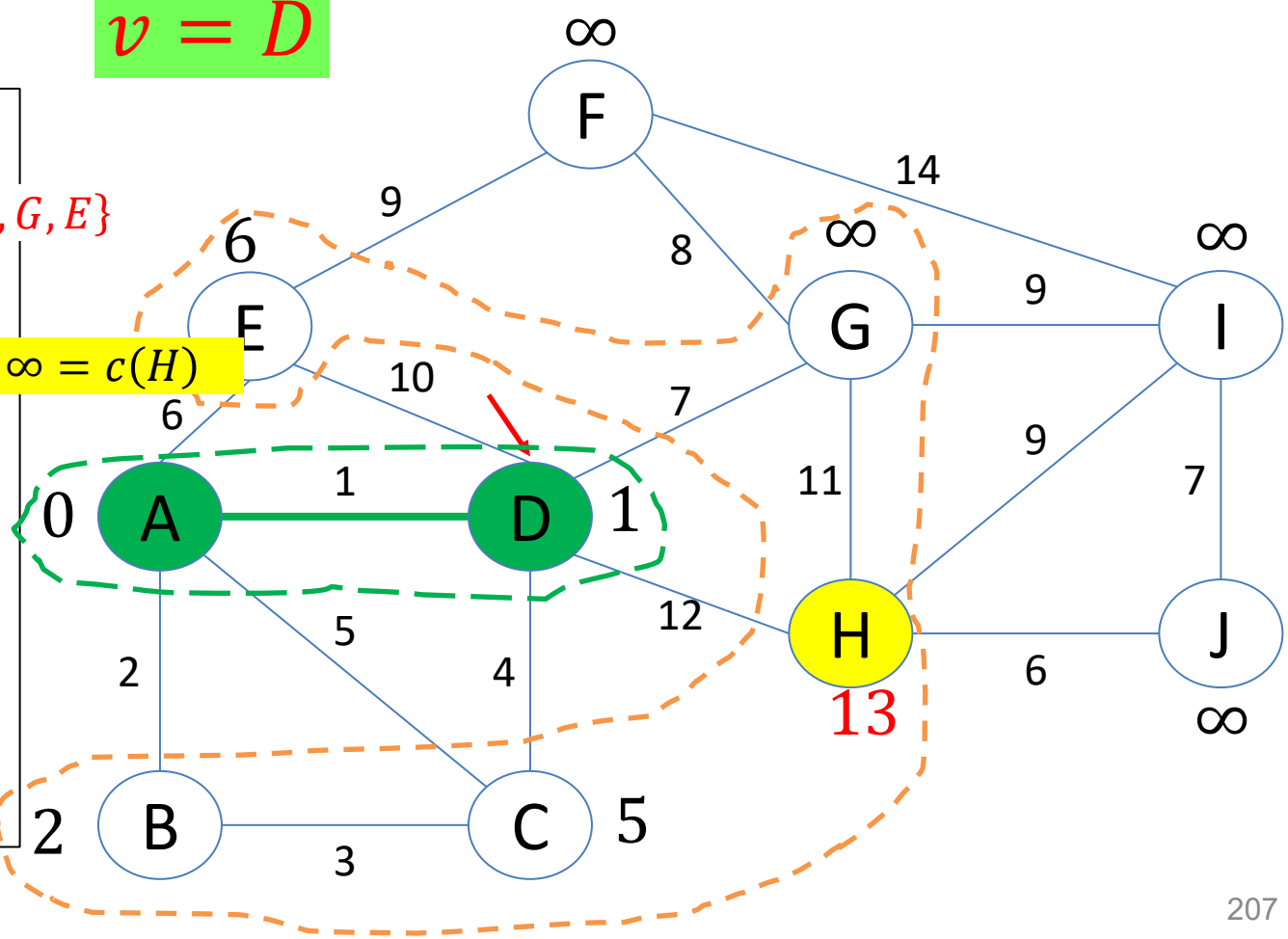
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{C, H, G, E\}$
2. for each u in Adj(v) \ V(T): $u = H$
3. if c(v) + w(v, u) < c(u): $c(D) + w(H, D) = 13 < \infty = c(H)$
4. c(u) := c(v) + w(v, u) $c(H) = 13$
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, D)$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$

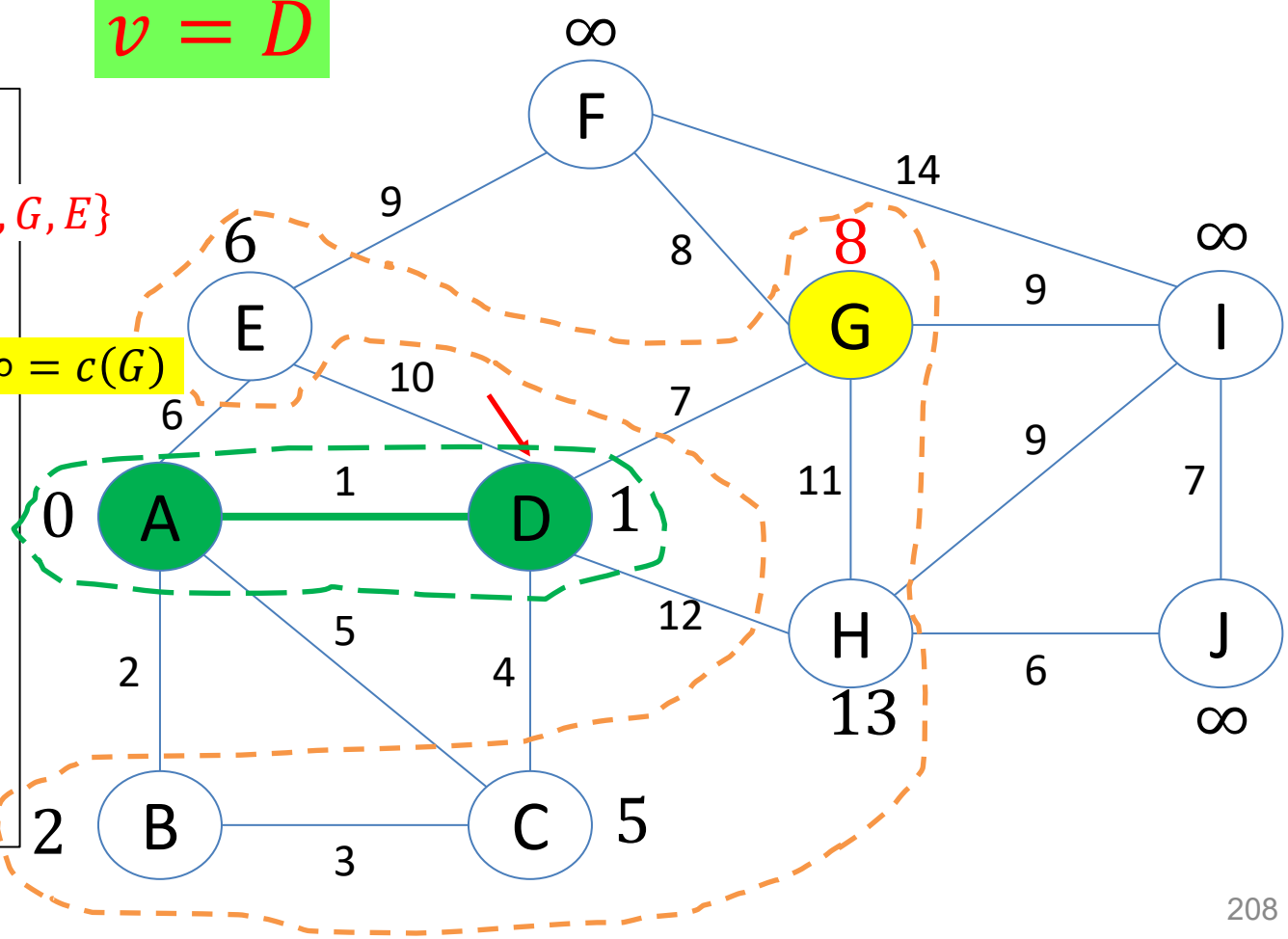
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{C, H, G, E\}$
2. for each u in Adj(v) \ V(T): $u = G$
3. if c(v) + w(v, u) < c(u): $c(D) + w(G, D) = 8 < \infty = c(G)$
4. c(u) := c(v) + w(v, u) $c(G) = 8$
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$

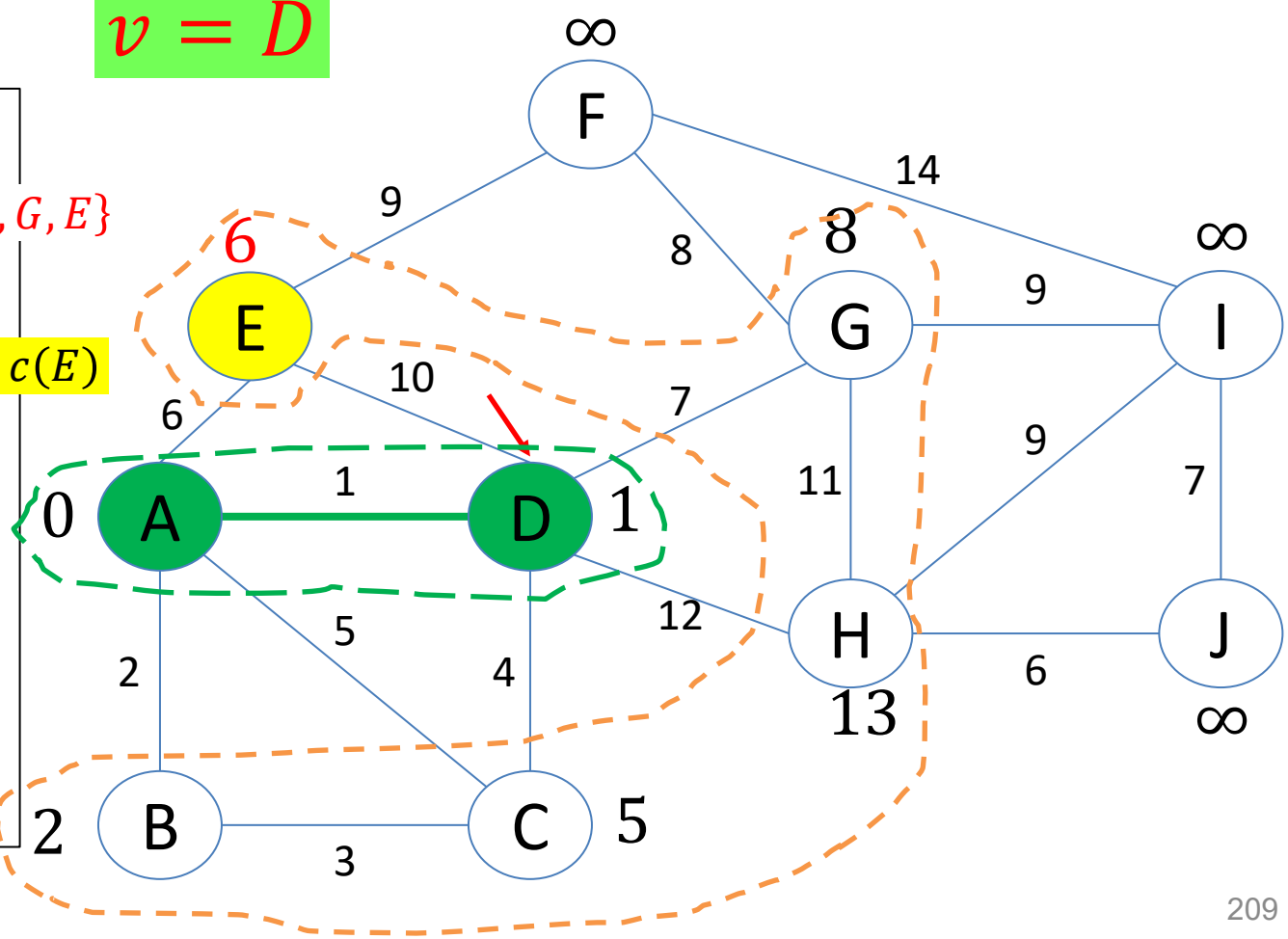
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{C, H, G, E\}$
2. for each u in Adj(v) \ V(T): $u = E$
3. if c(v) + w(v, u) < c(u): $c(D) + w(E, D) = 11 > c(E)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D\}$

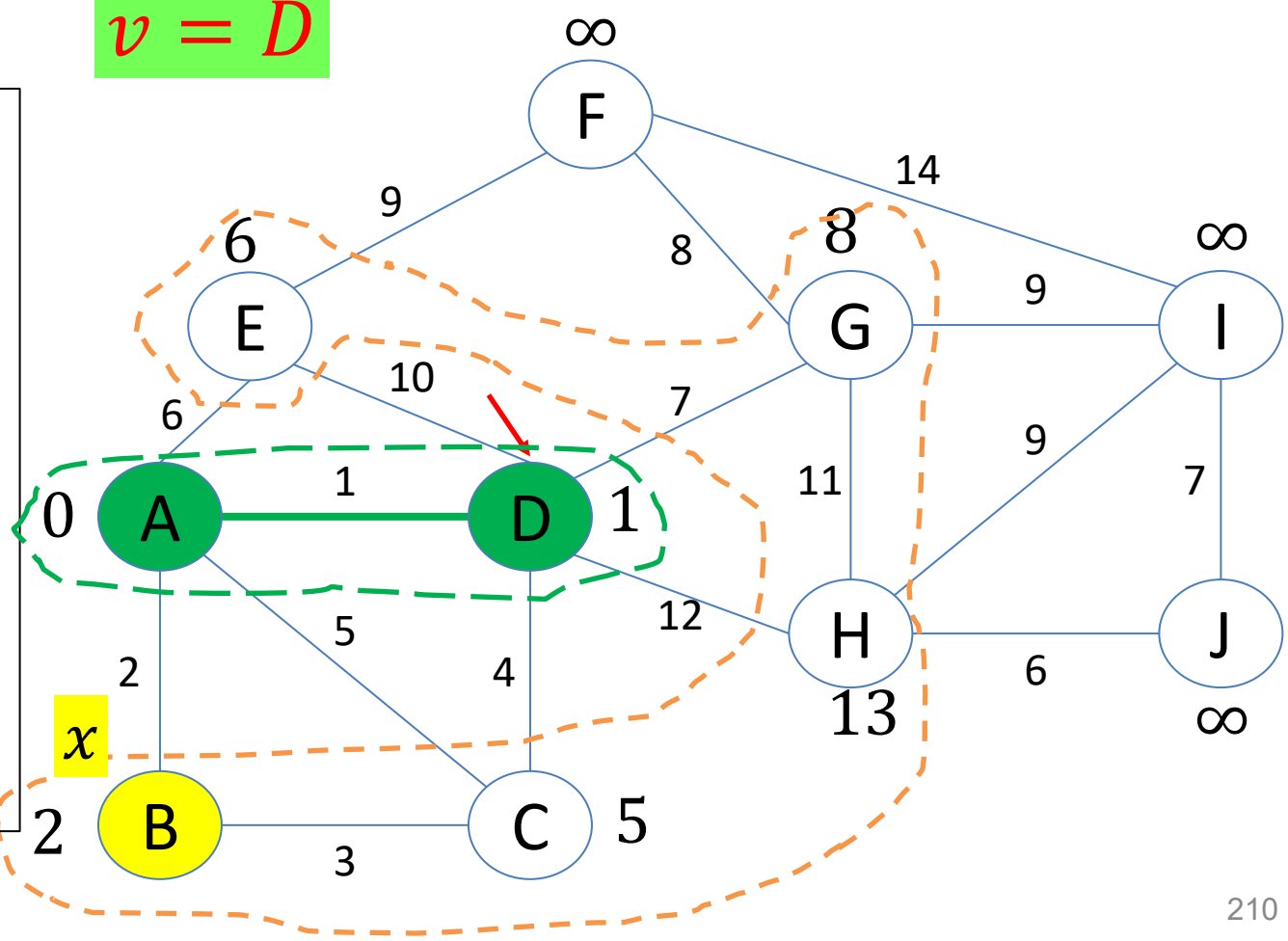
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

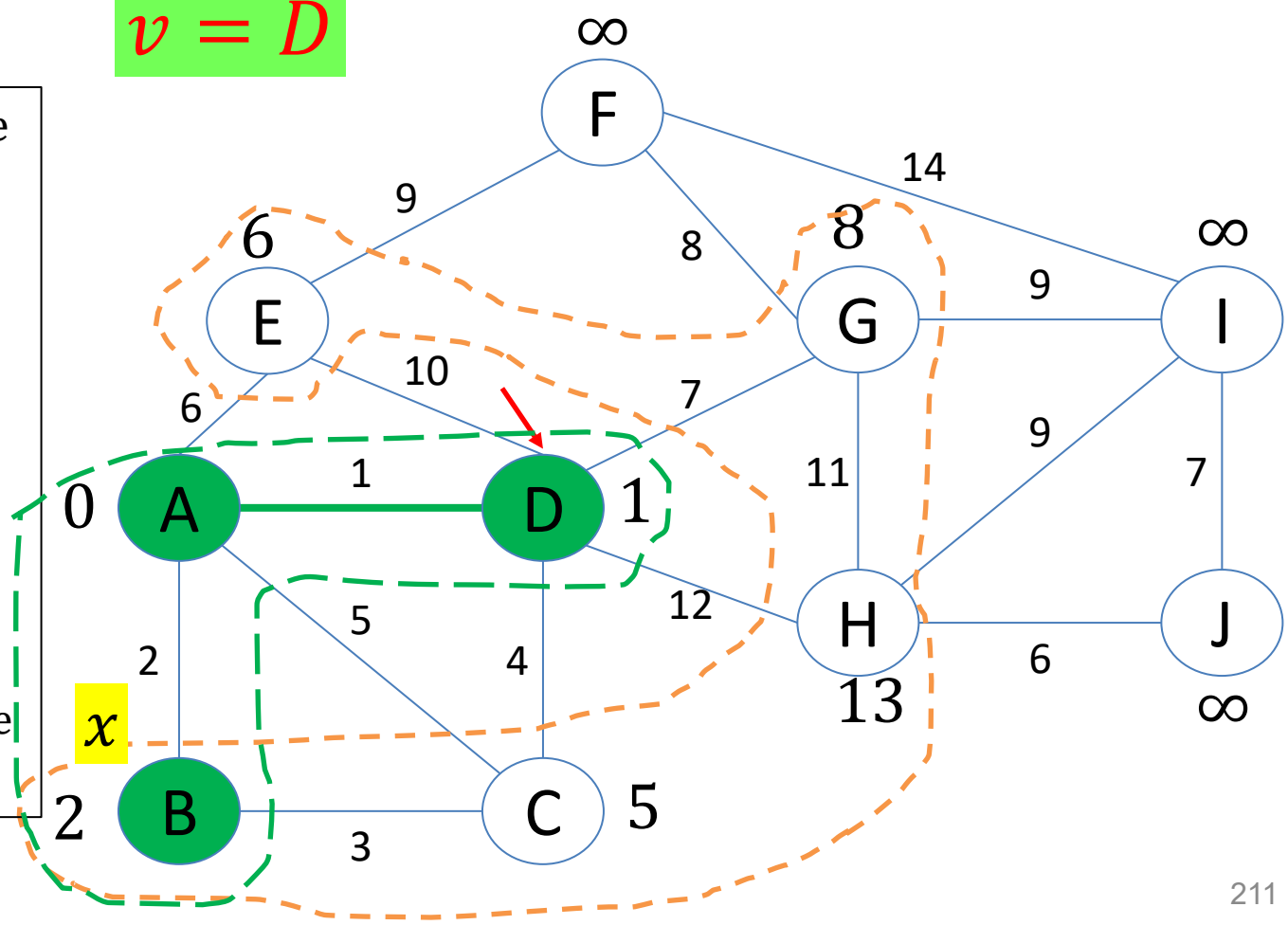
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$
$F = \{B, C, H, G, E\}$
$E(T) = \{(A, D)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

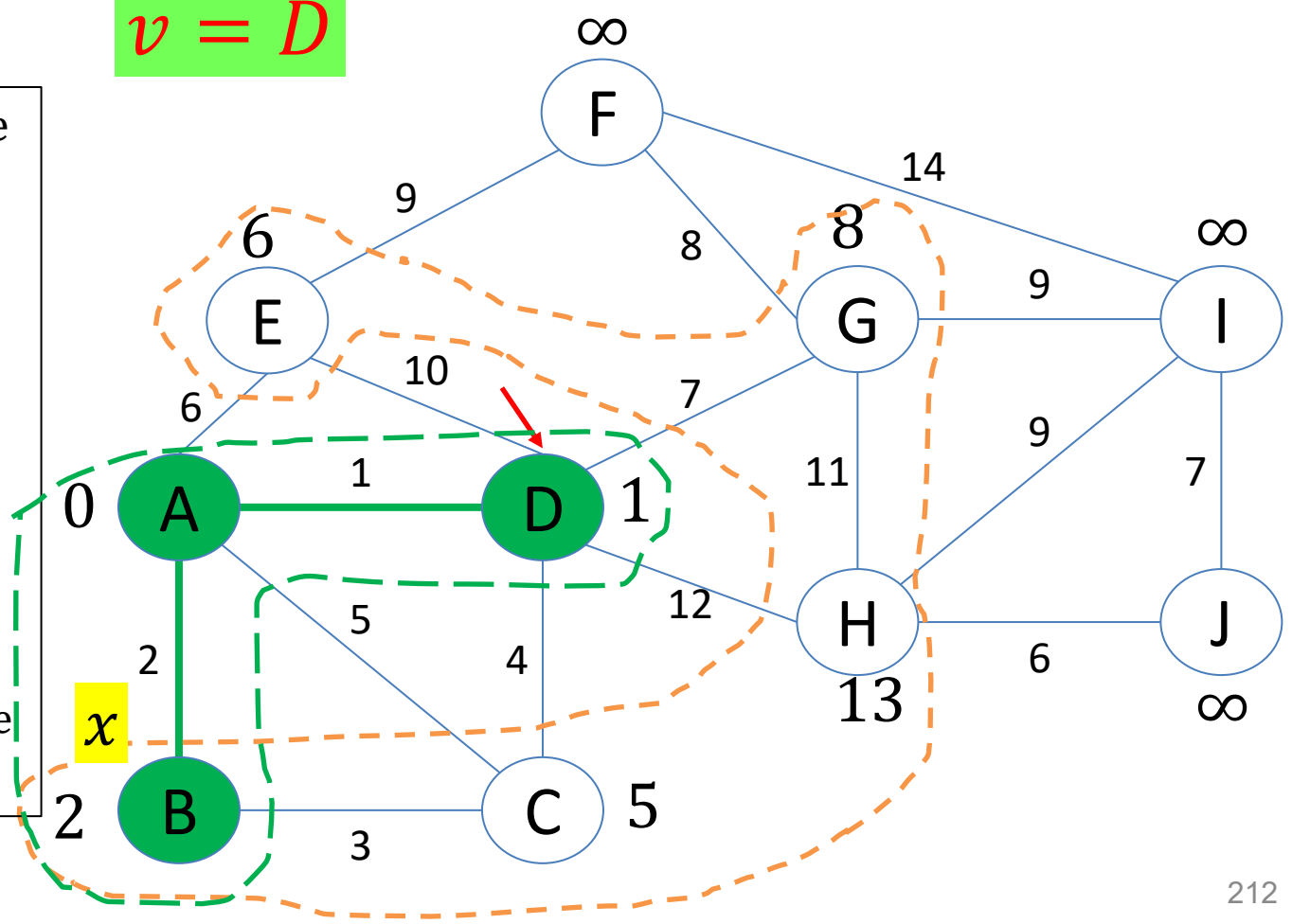
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D), (A, B)\}$

$v = D$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

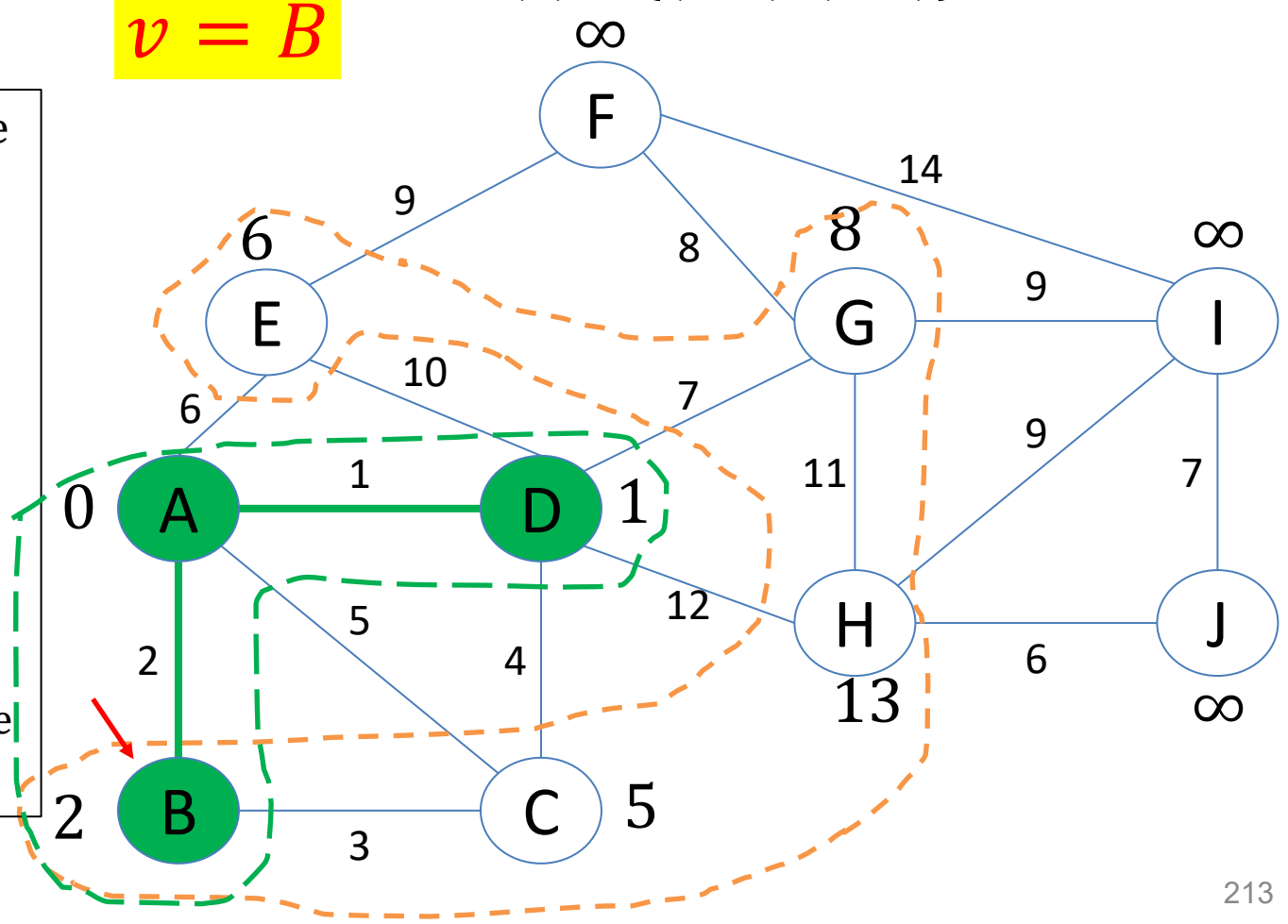
$F = \{B, C, H, G, E\}$

$E(T) = \{(A, D), (A, B)\}$

$v = B$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

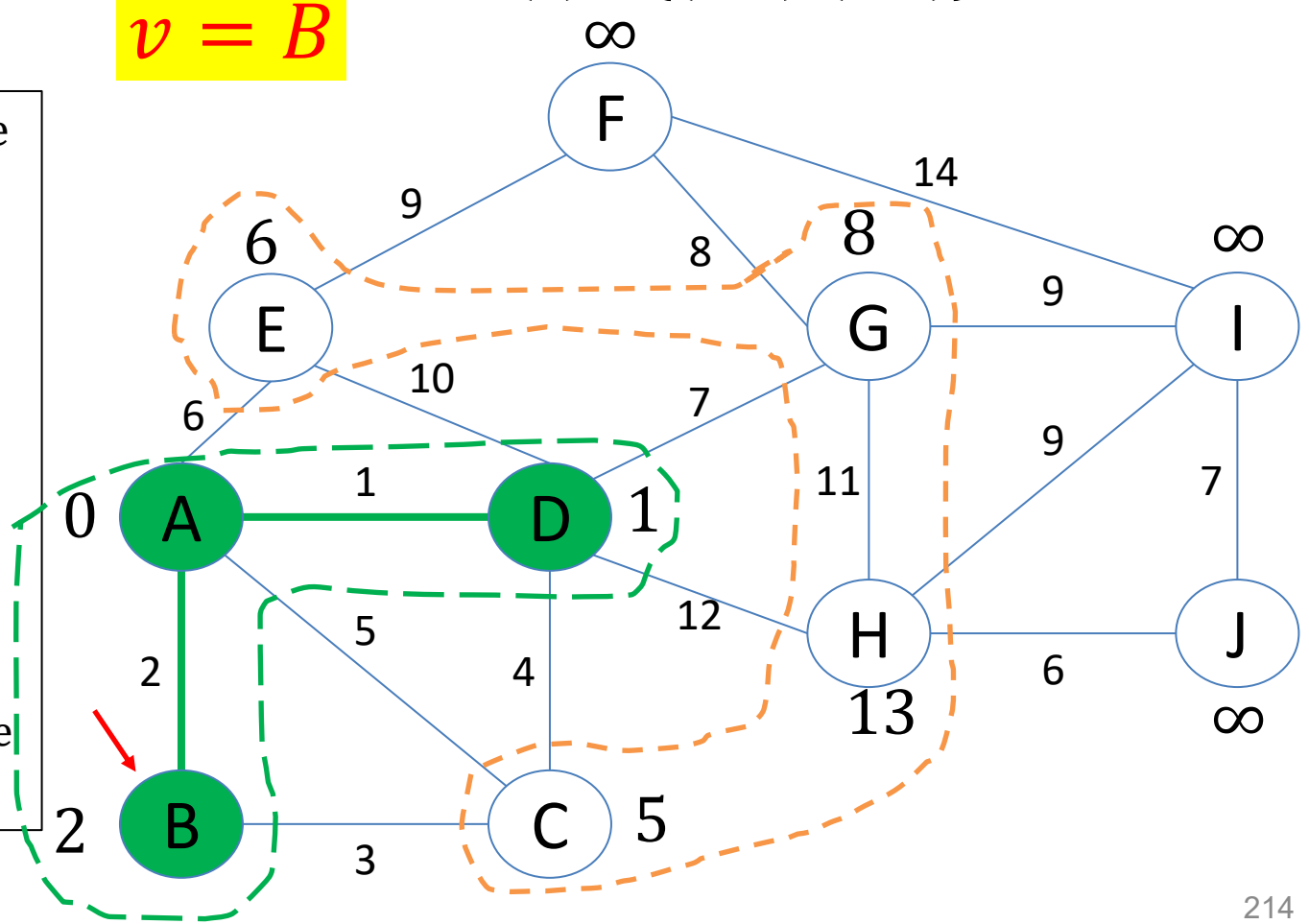
$F = \{C, H, G, E\}$

$E(T) = \{(A, D), (A, B)\}$

$v = B$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

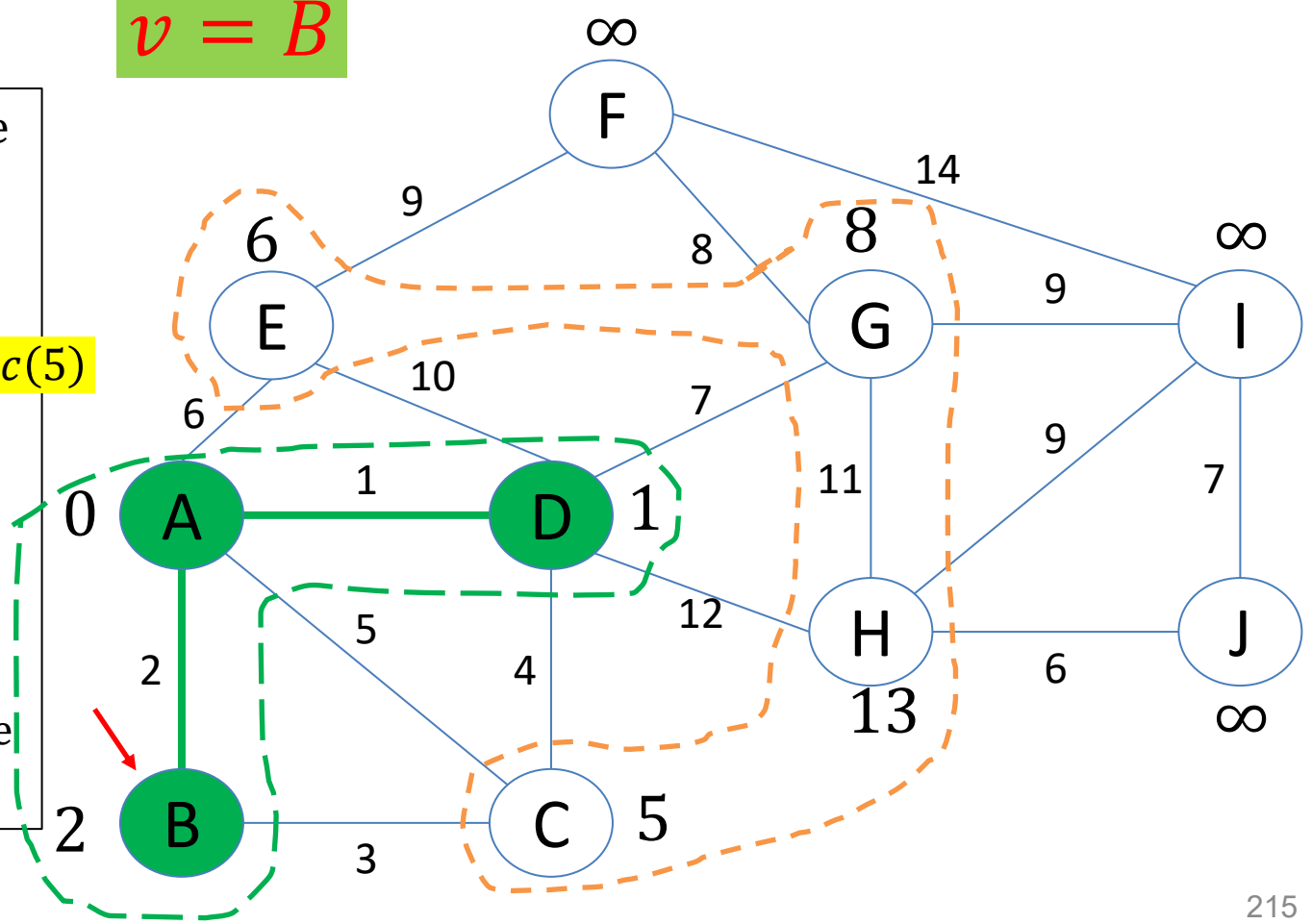
$F = \{C, H, G, E\}$

$E(T) = \{(A, D), (A, B)\}$

$v = B$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{C\}$
2. for each u in Adj(v) \ V(T): $u = C$
3. if c(v) + w(v, u) < c(u): $c(B) + w(C, B) = 5 = c(5)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

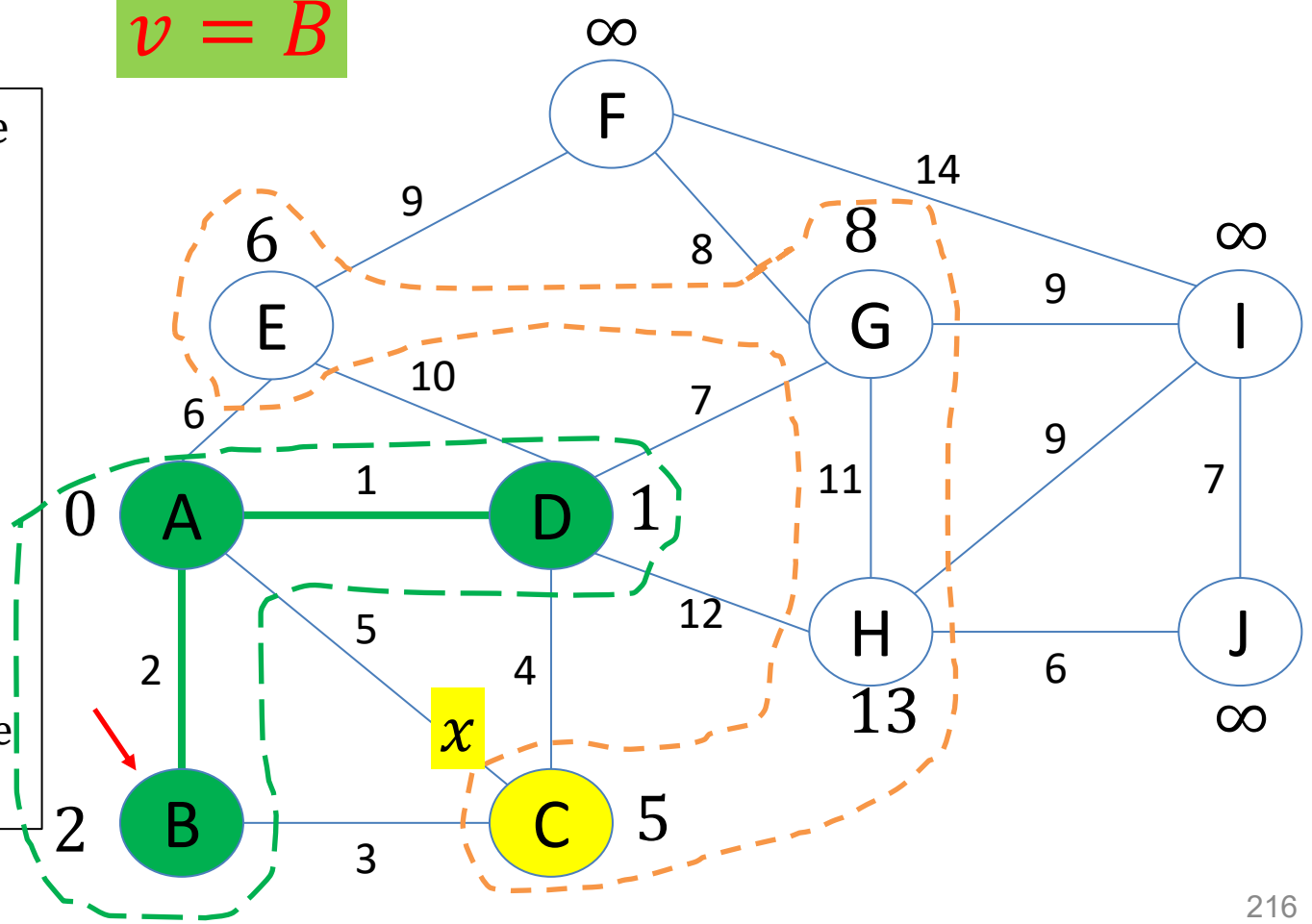
$F = \{C, H, G, E\}$

$E(T) = \{(A, D), (A, B)\}$

$v = B$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

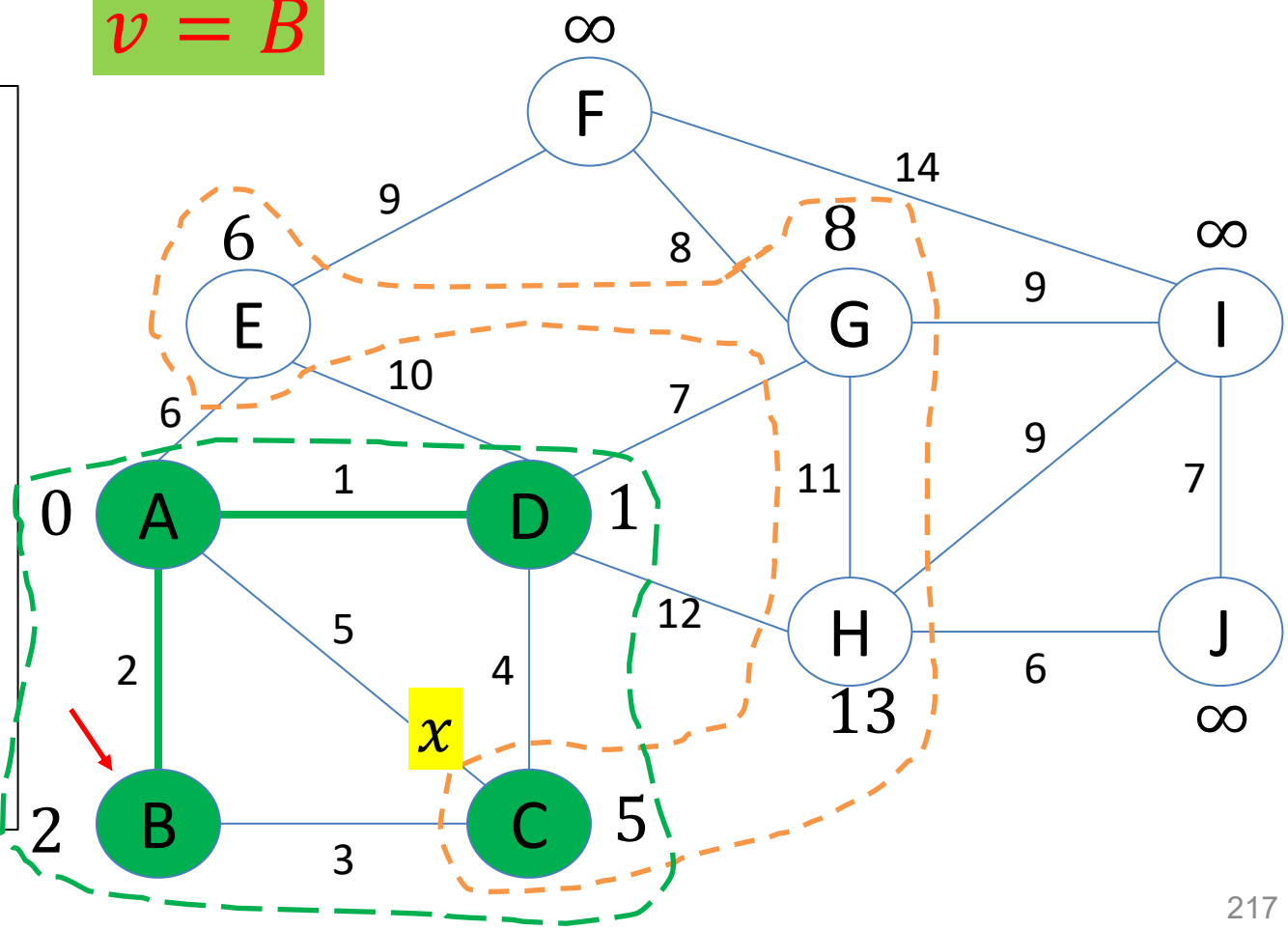
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C\}$
$F = \{C, H, G, E\}$
$E(T) = \{(A, D), (A, B)\}$

$v = B$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

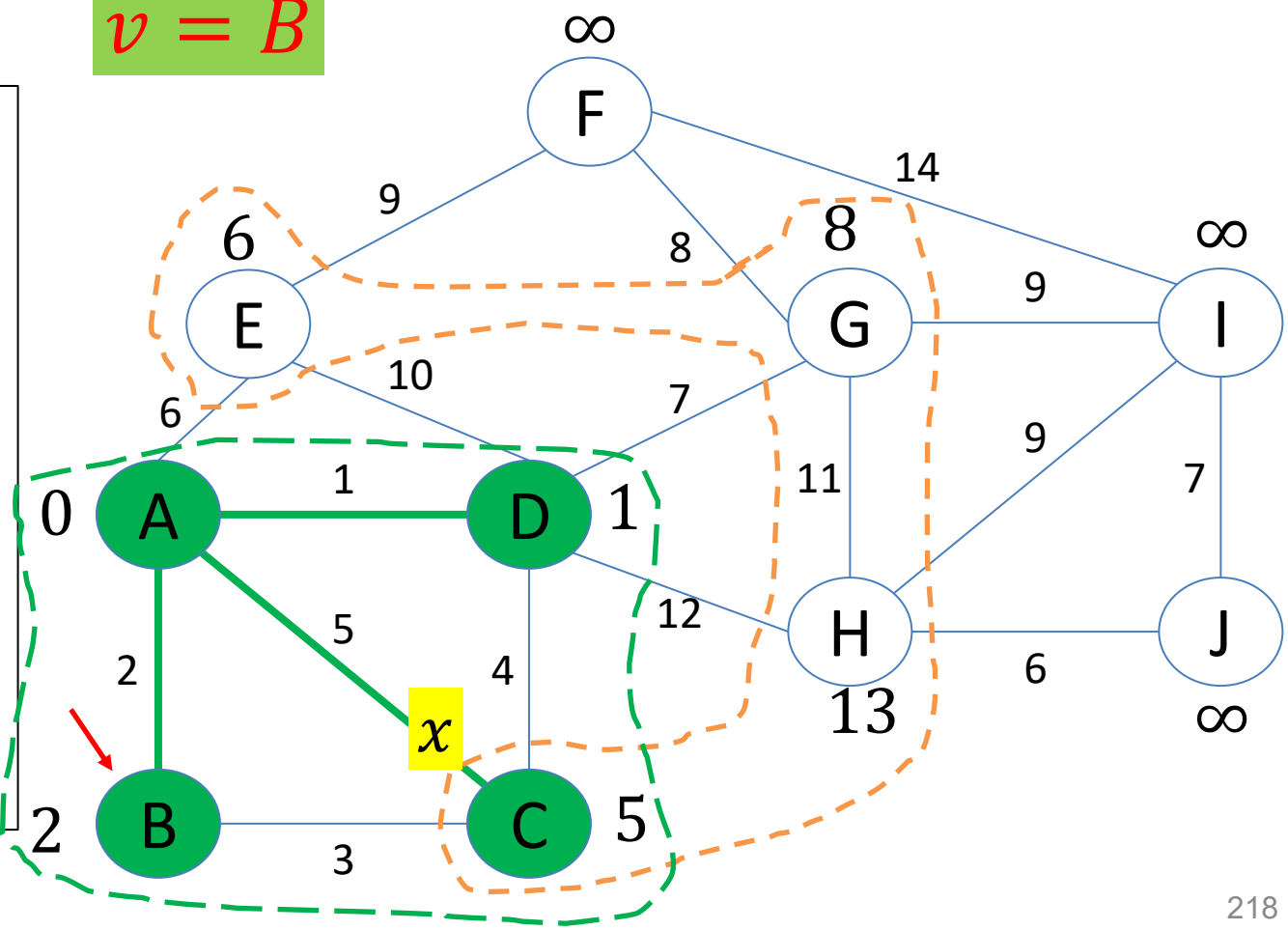
$F = \{C, H, G, E\}$

$E(T) = \{(A, D), (A, B), (A, C)\}$

$v = B$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

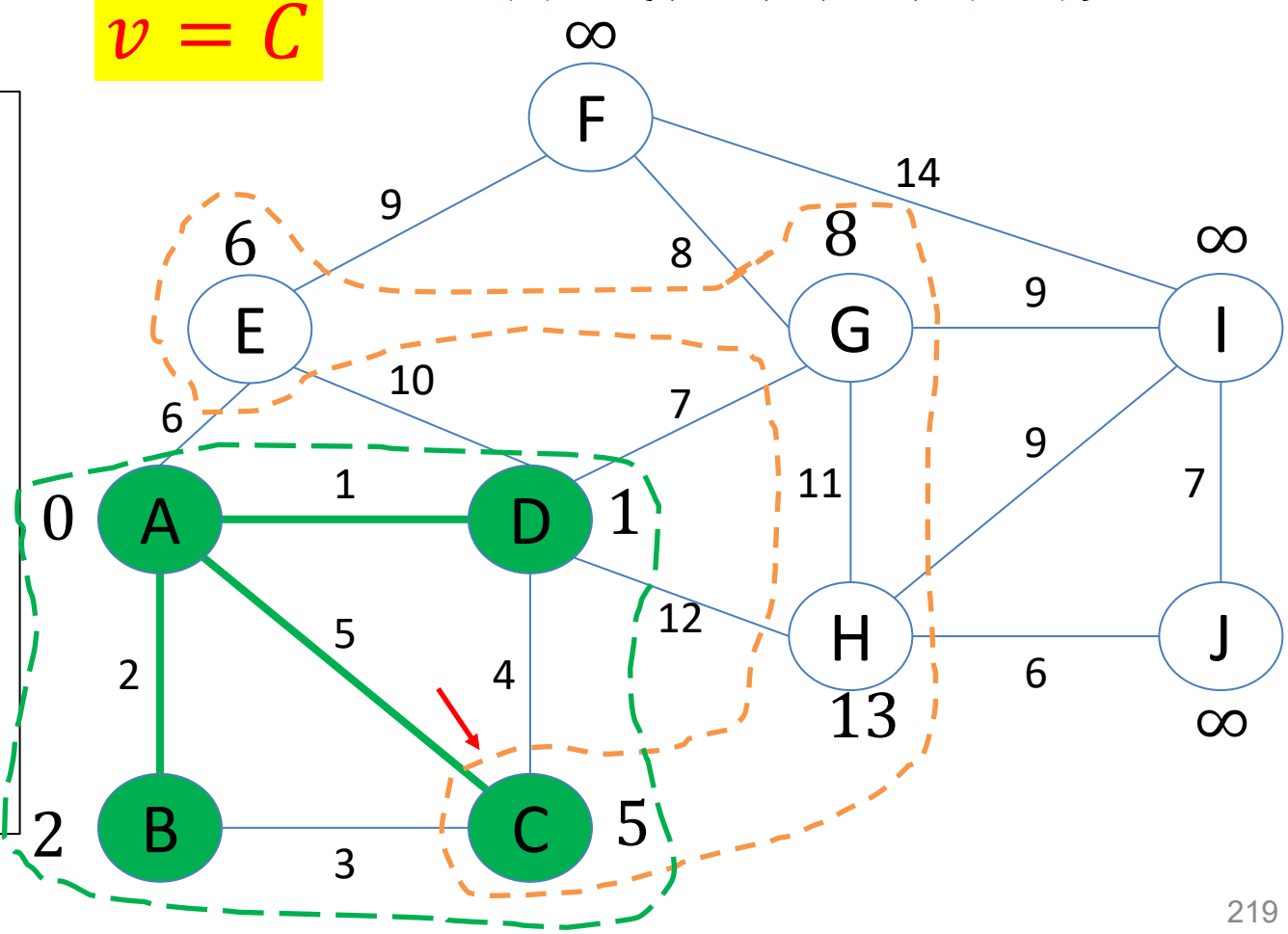
$F = \{C, H, G, E\}$

$E(T) = \{(A, D), (A, B), (A, C)\}$

$v = C$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B\}$

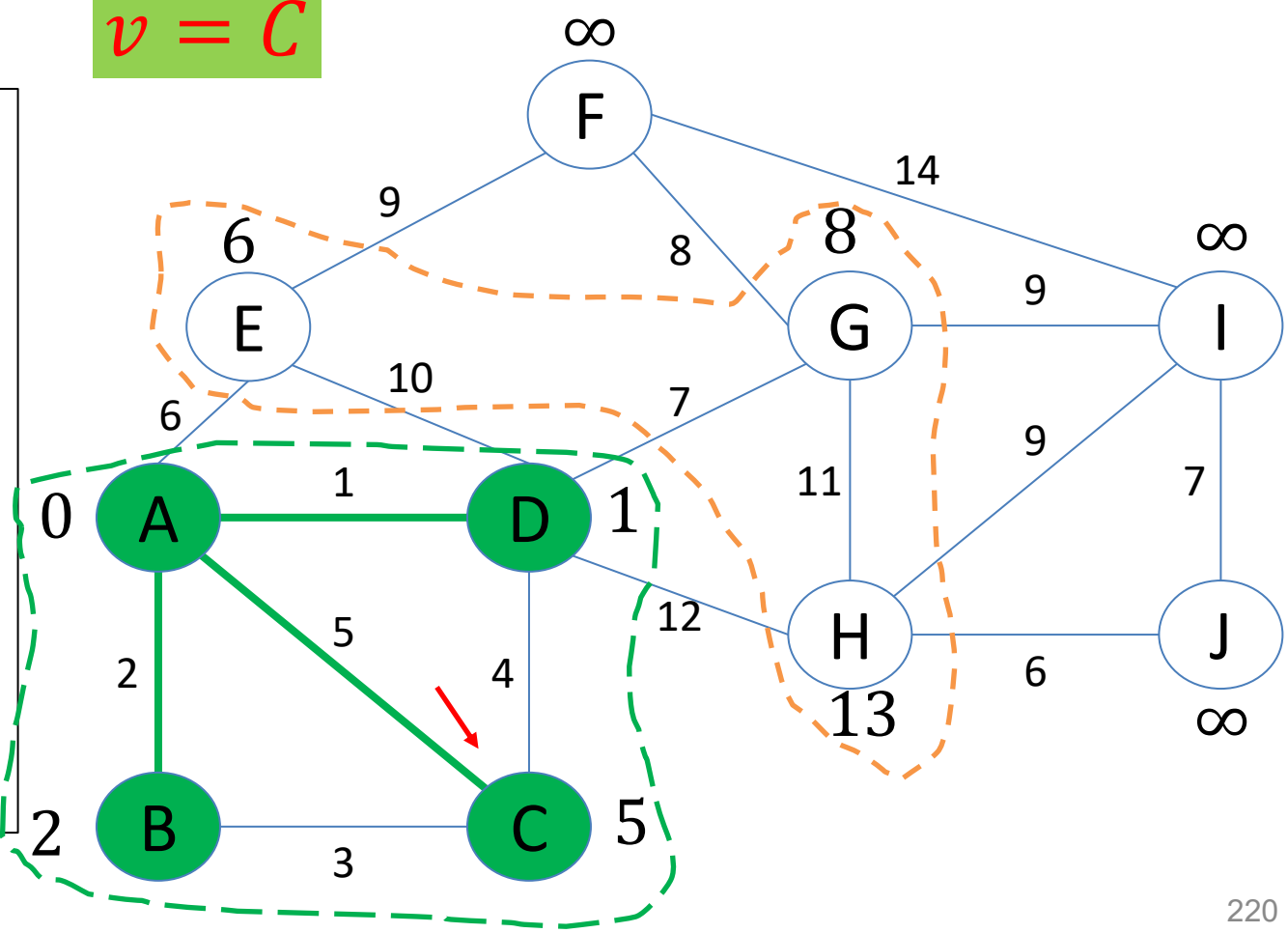
$F = \{H, G, E\}$

$E(T) = \{(A, D), (A, B), (A, C)\}$

$v = C$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C\}$

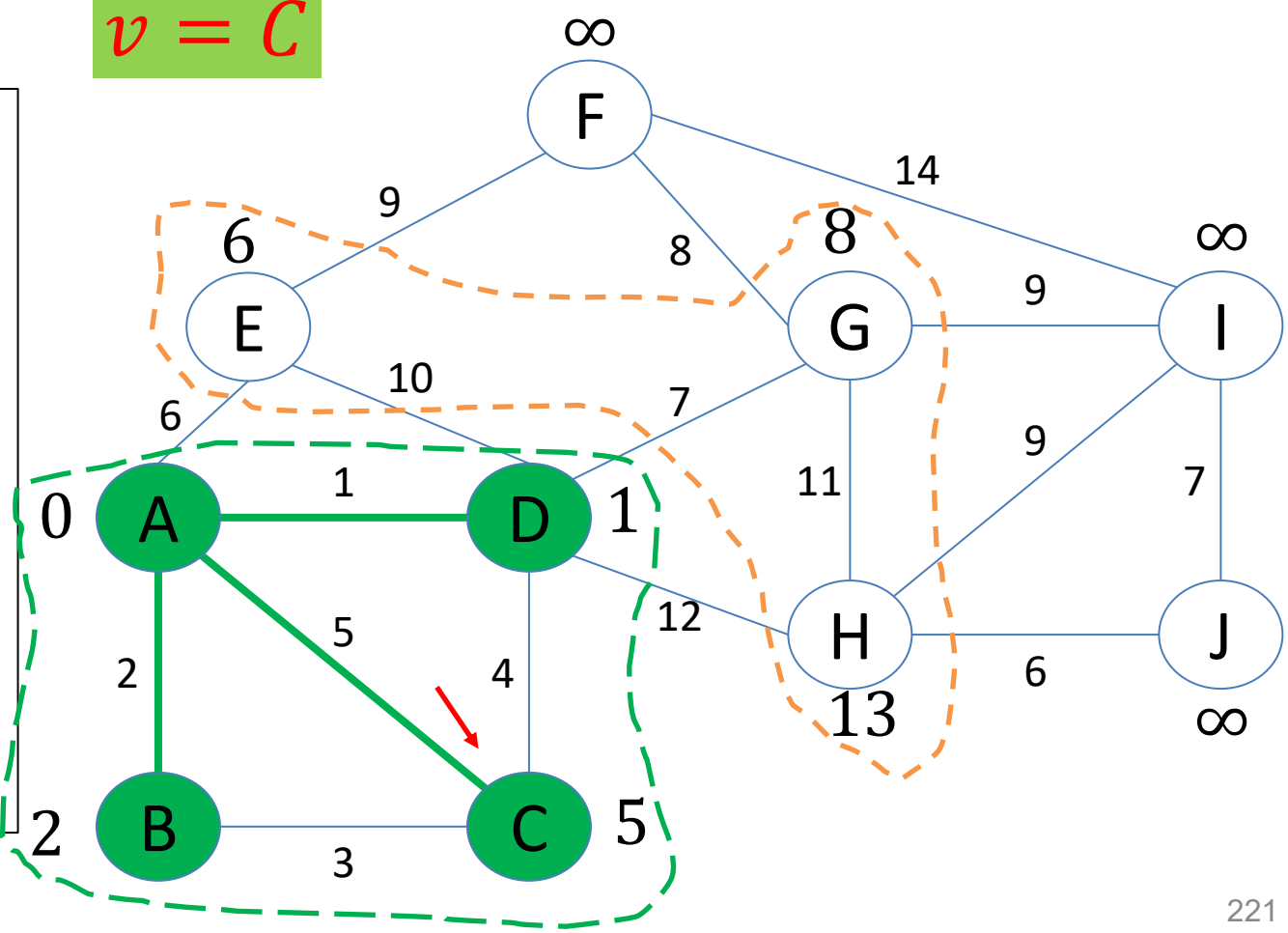
$F = \{H, G, E\}$

$E(T) = \{(A, D), (A, B), (A, C)\}$

$v = C$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \emptyset$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C\}$

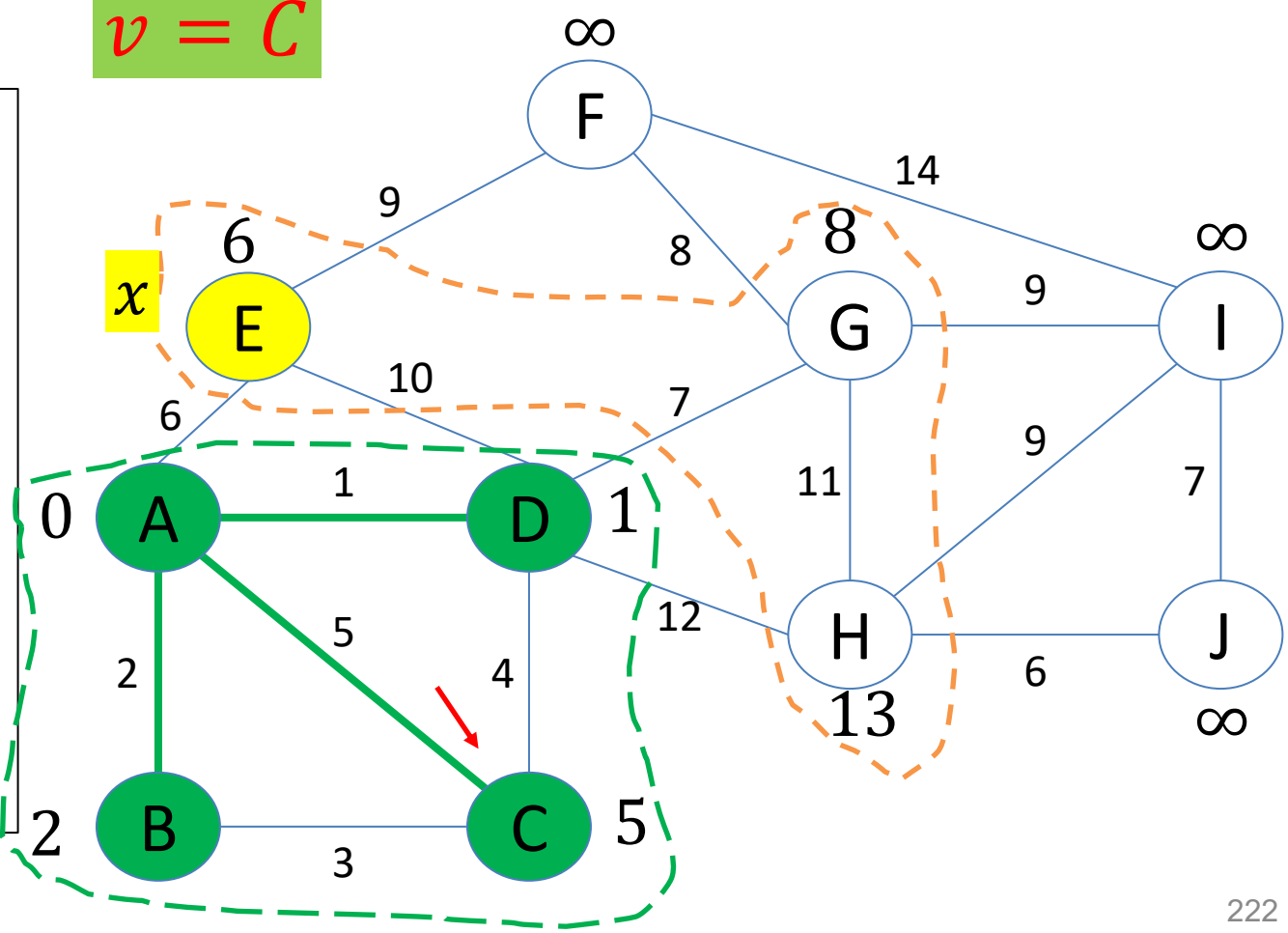
$F = \{H, G, E\}$

$E(T) = \{(A, D), (A, B), (A, C)\}$

$v = C$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

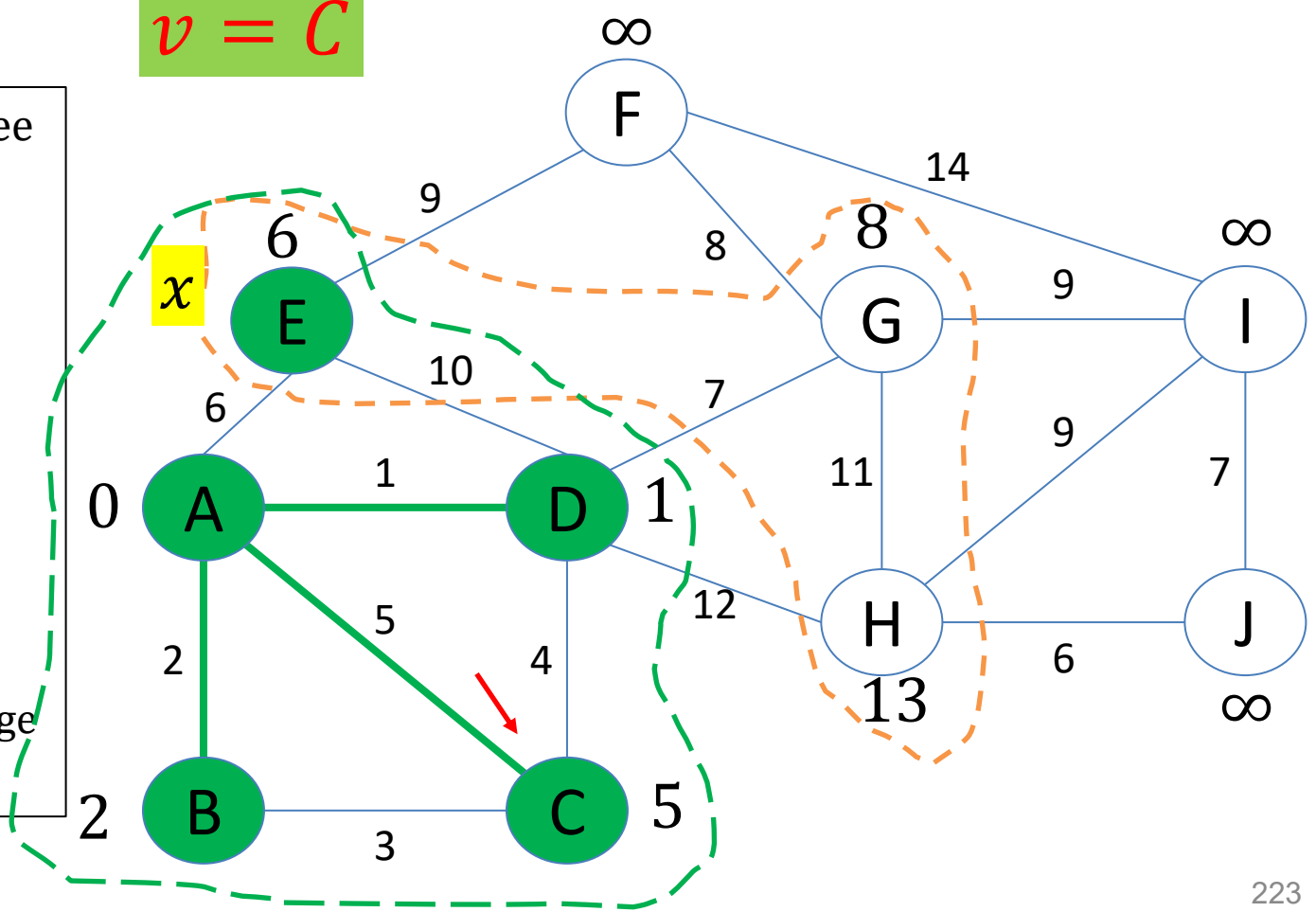
A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E\}$
$F = \{H, G, E\}$
$E(T) = \{(A, D), (A, B), (A, C)\}$

$v = C$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E\}$

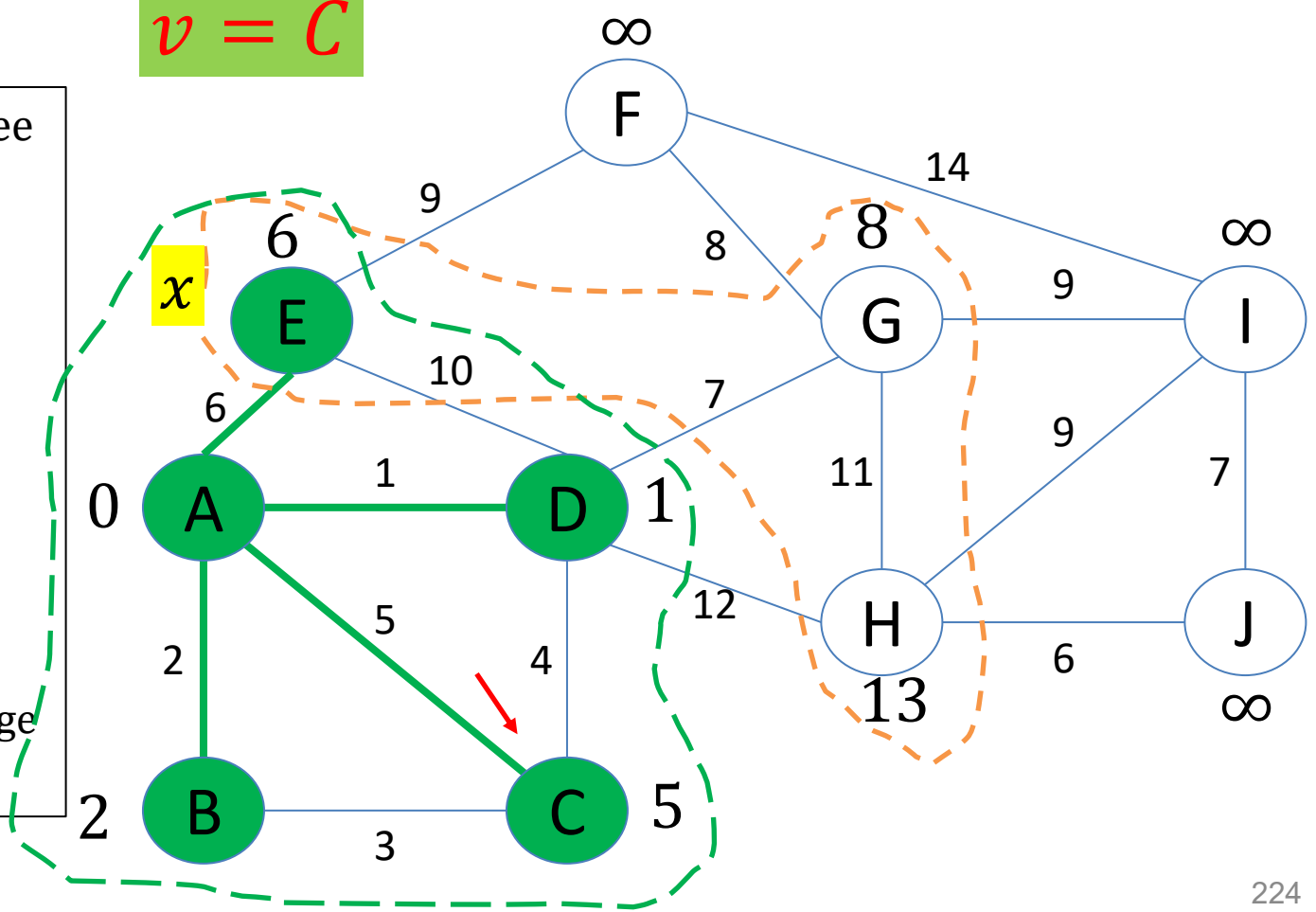
$F = \{H, G, E\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)\}$

$v = C$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E\}$

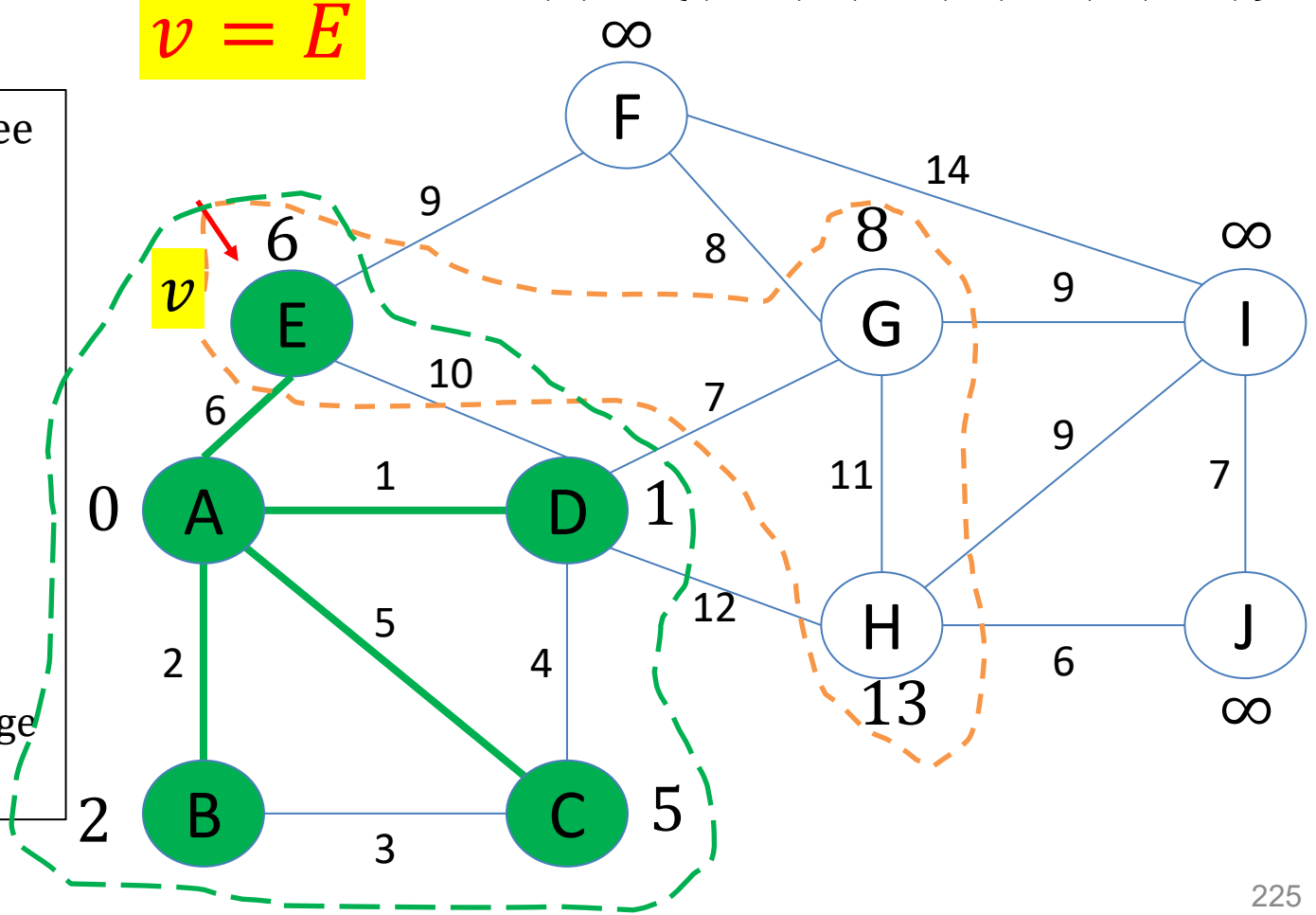
$F = \{H, G, E\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)\}$

$v = E$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E\}$

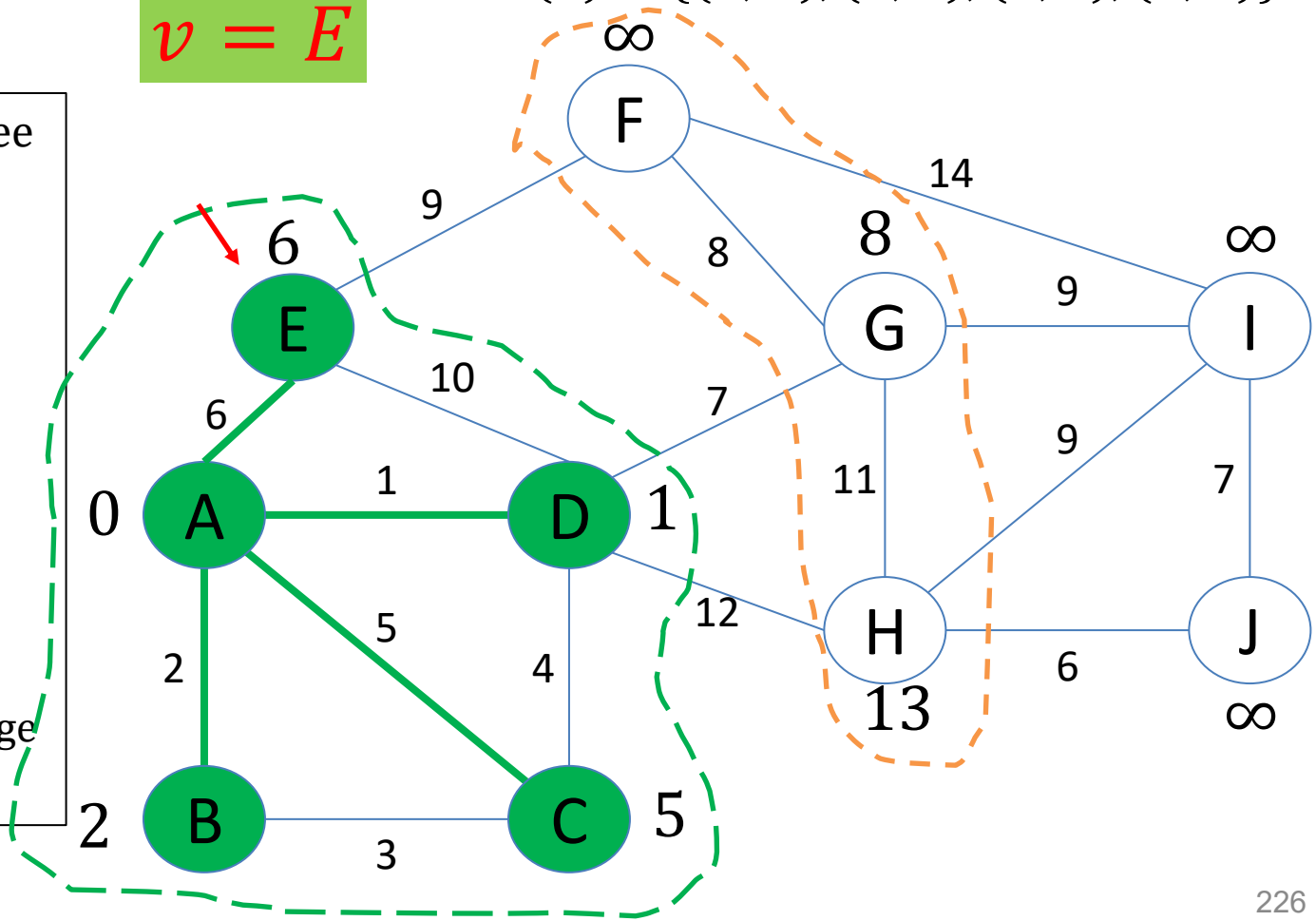
$F = \{H, G, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)\}$

$v = E$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E\}$

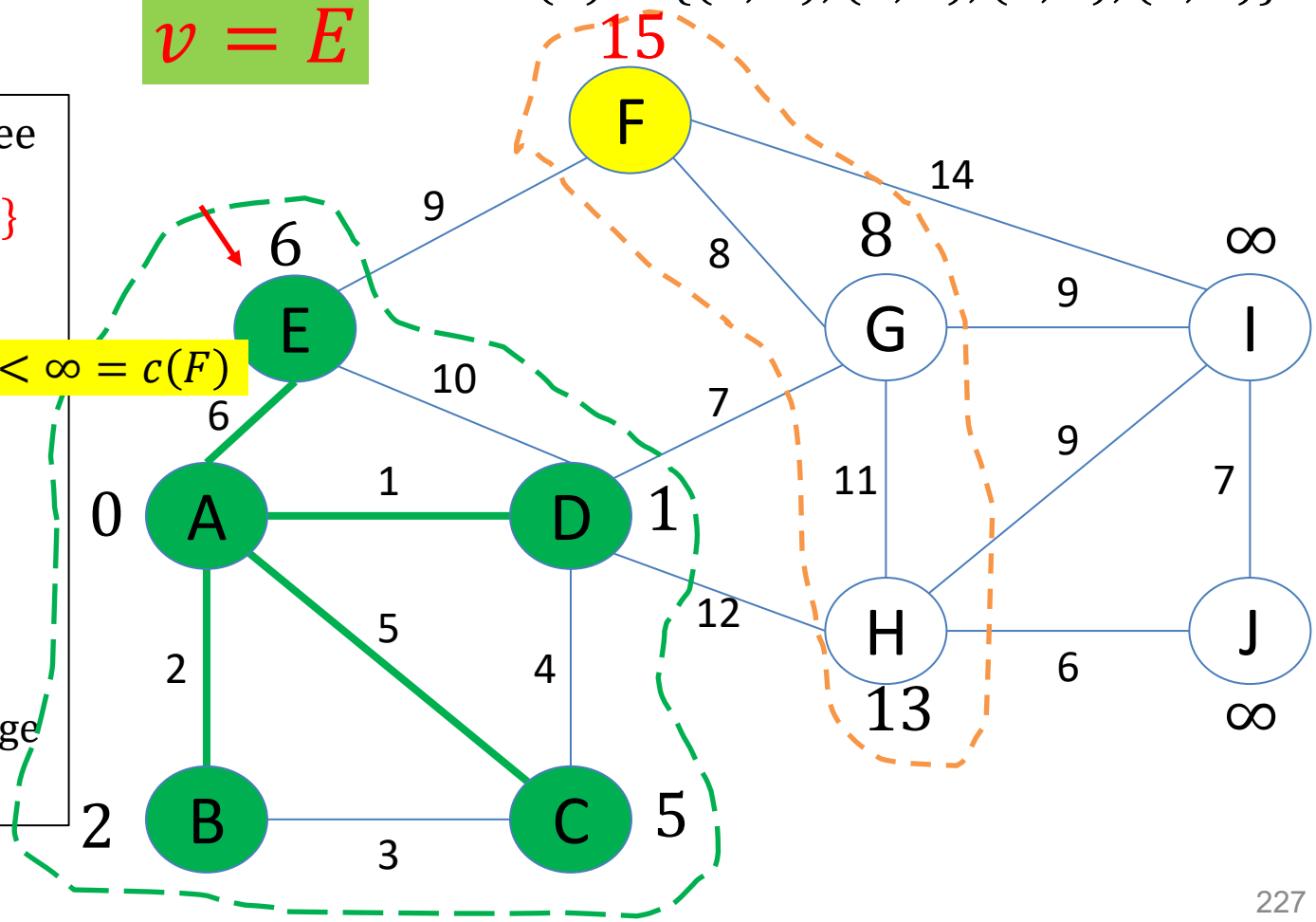
$F = \{H, G, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)\}$

$v = E$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{F\}$
2. for each u in Adj(v) \ V(T): $u = F$
3. if c(v) + w(v, u) < c(u): $c(E) + w(F, E) = 15 < \infty = c(F)$
4. c(u) := c(v) + w(v, u) $c(F) = 15$
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E\}$

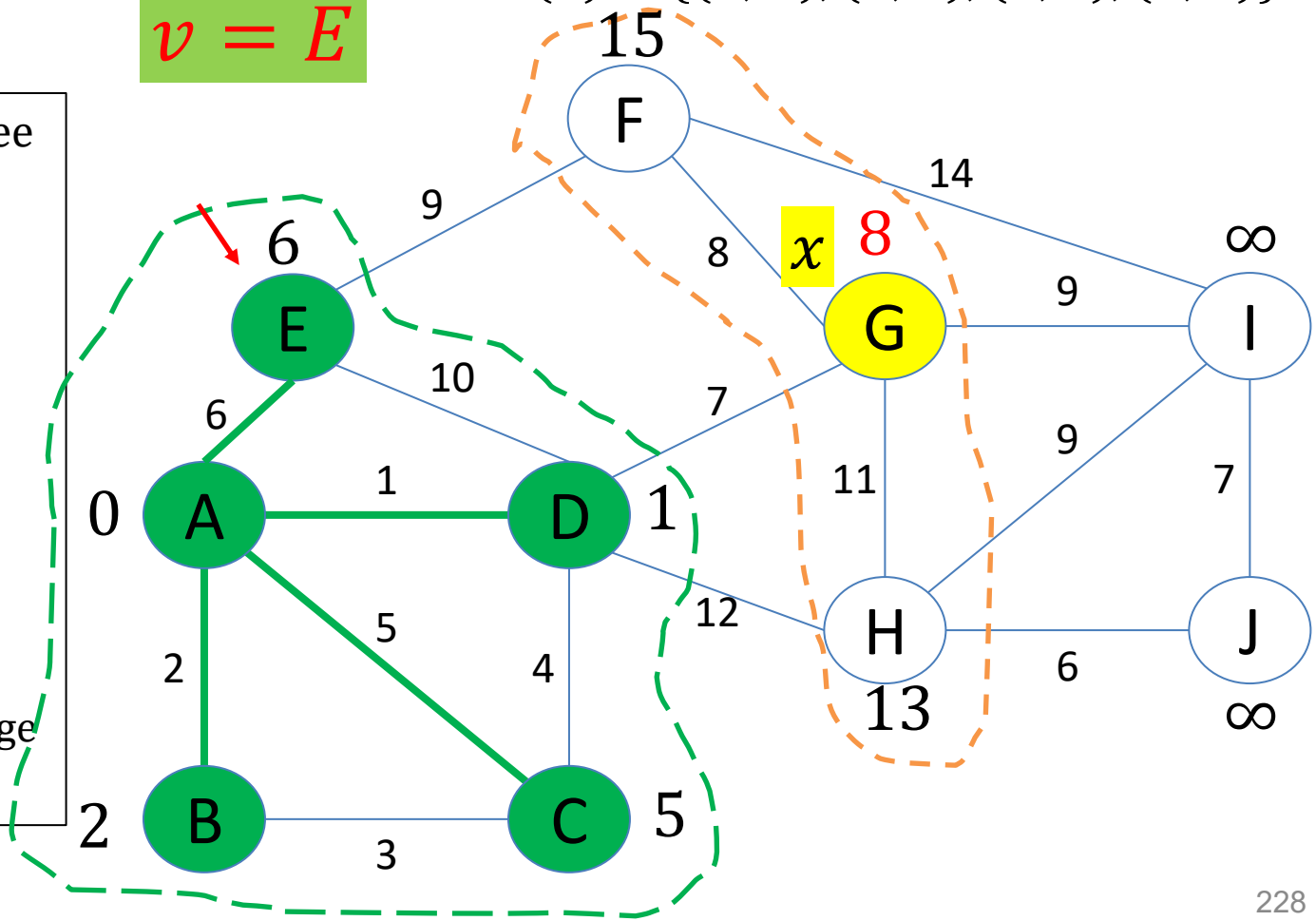
$F = \{H, G, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)\}$

$v = E$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E, G\}$

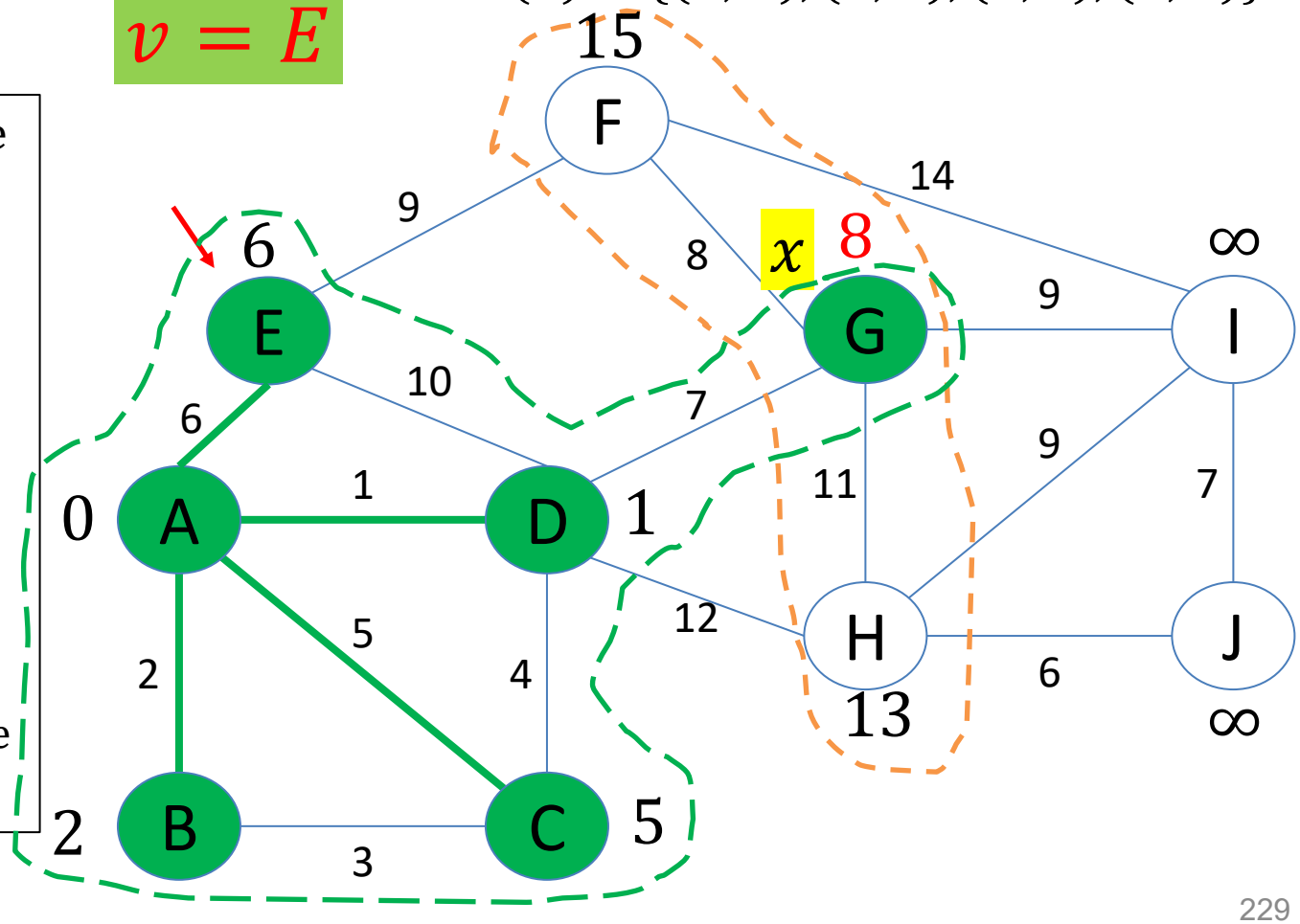
$F = \{H, G, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)\}$

$v = E$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$v = E$

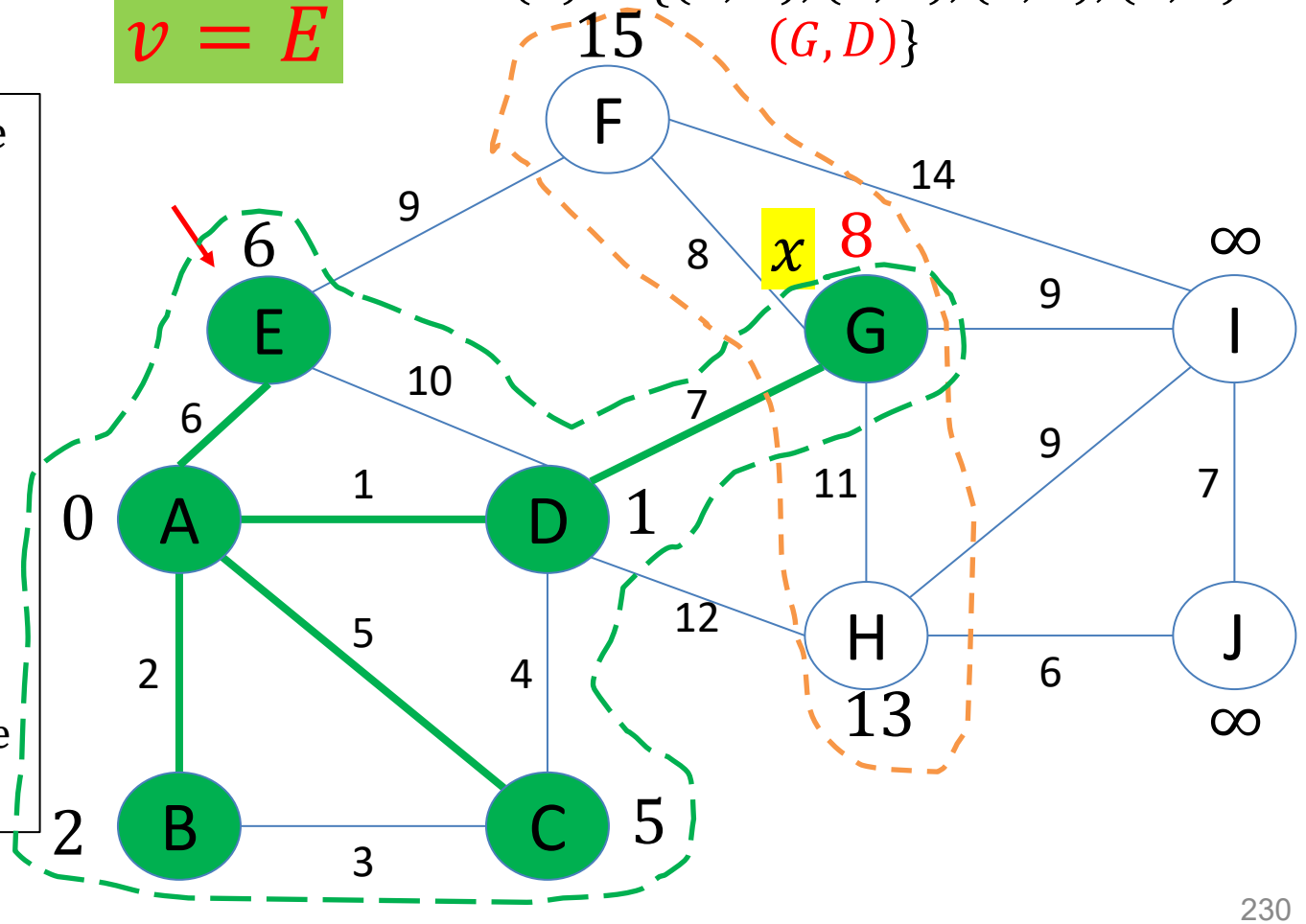
$V(T) = \{A, D, B, C, E, G\}$

$F = \{H, G, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$V(T) = \{A, D, B, C, E, G\}$

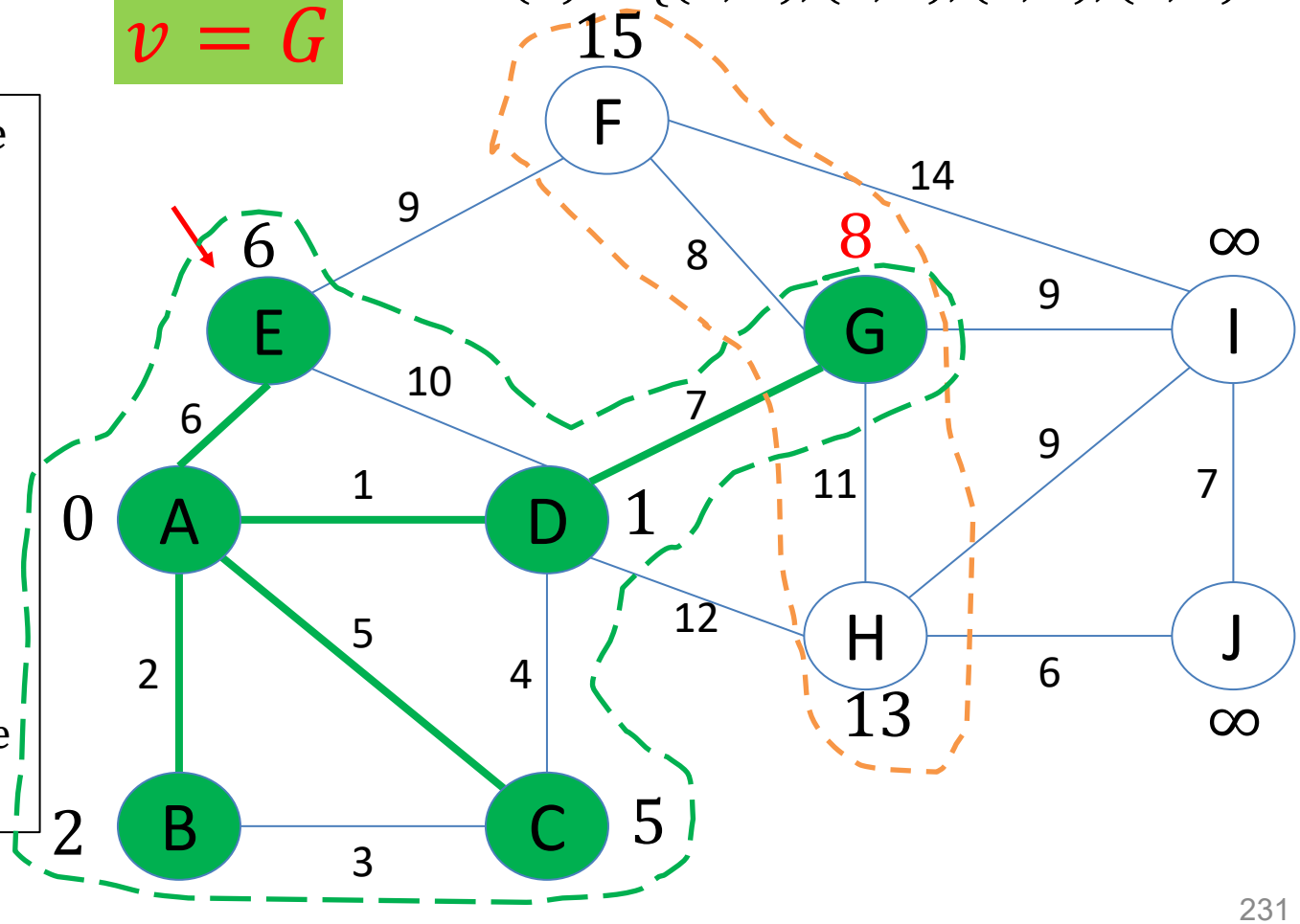
$F = \{H, G, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$

$v = G$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$v = G$

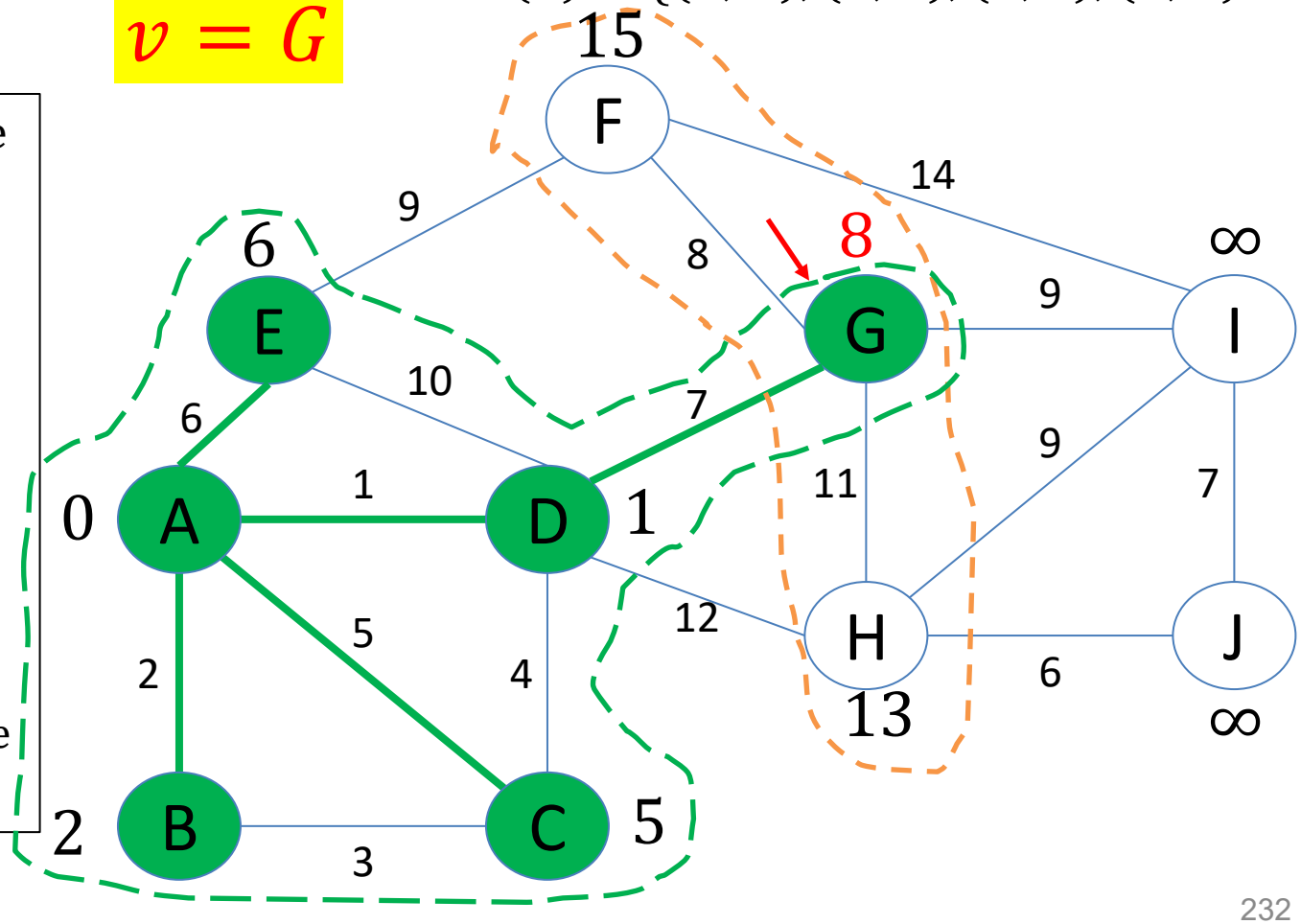
$V(T) = \{A, D, B, C, E, G\}$

$F = \{H, G, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$v = G$

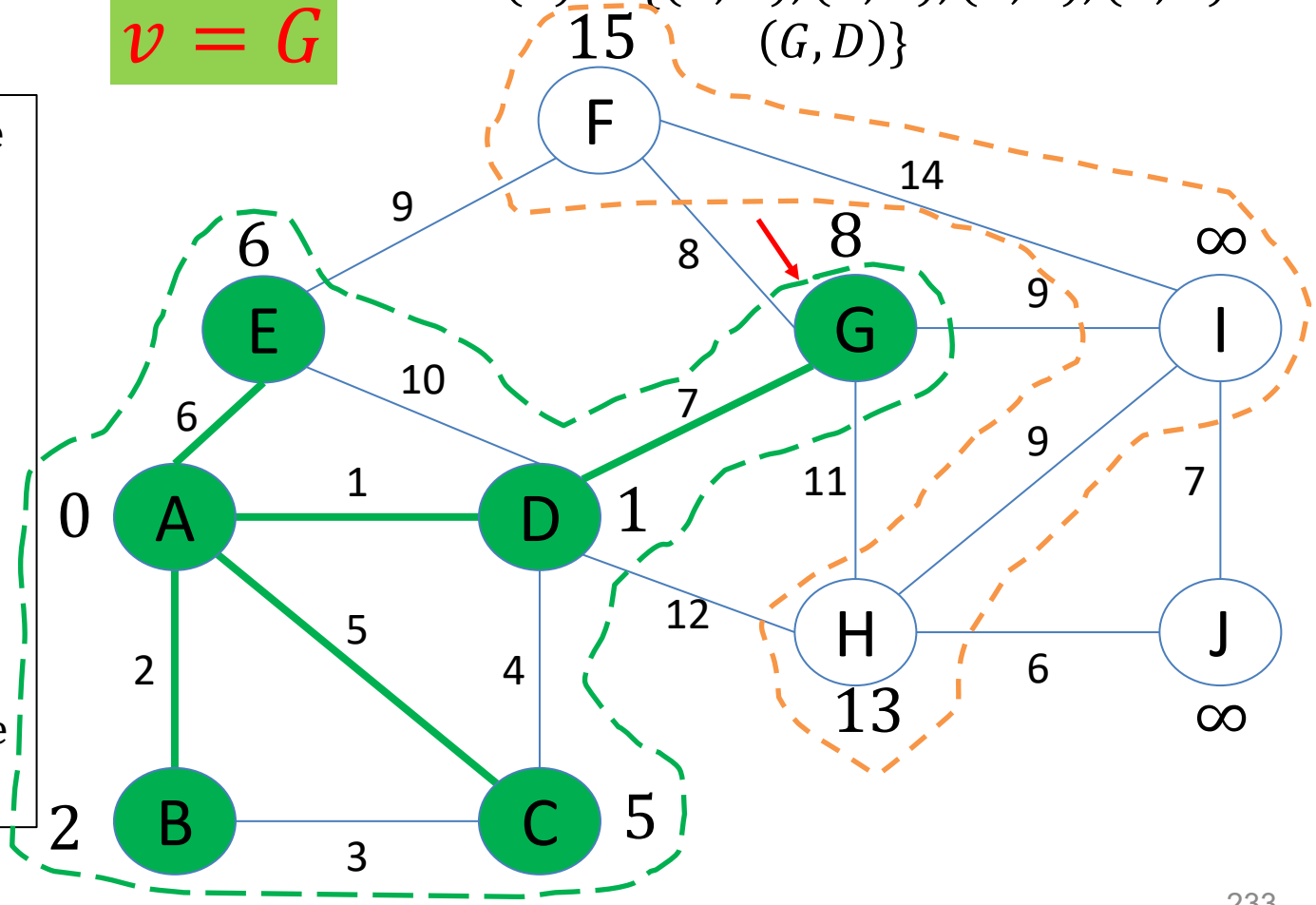
$V(T) = \{A, D, B, C, E, G\}$

$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	(15, E)	(8, D)	$(13, D)$	$(\infty, -)$	$(\infty, -)$

$v = G$

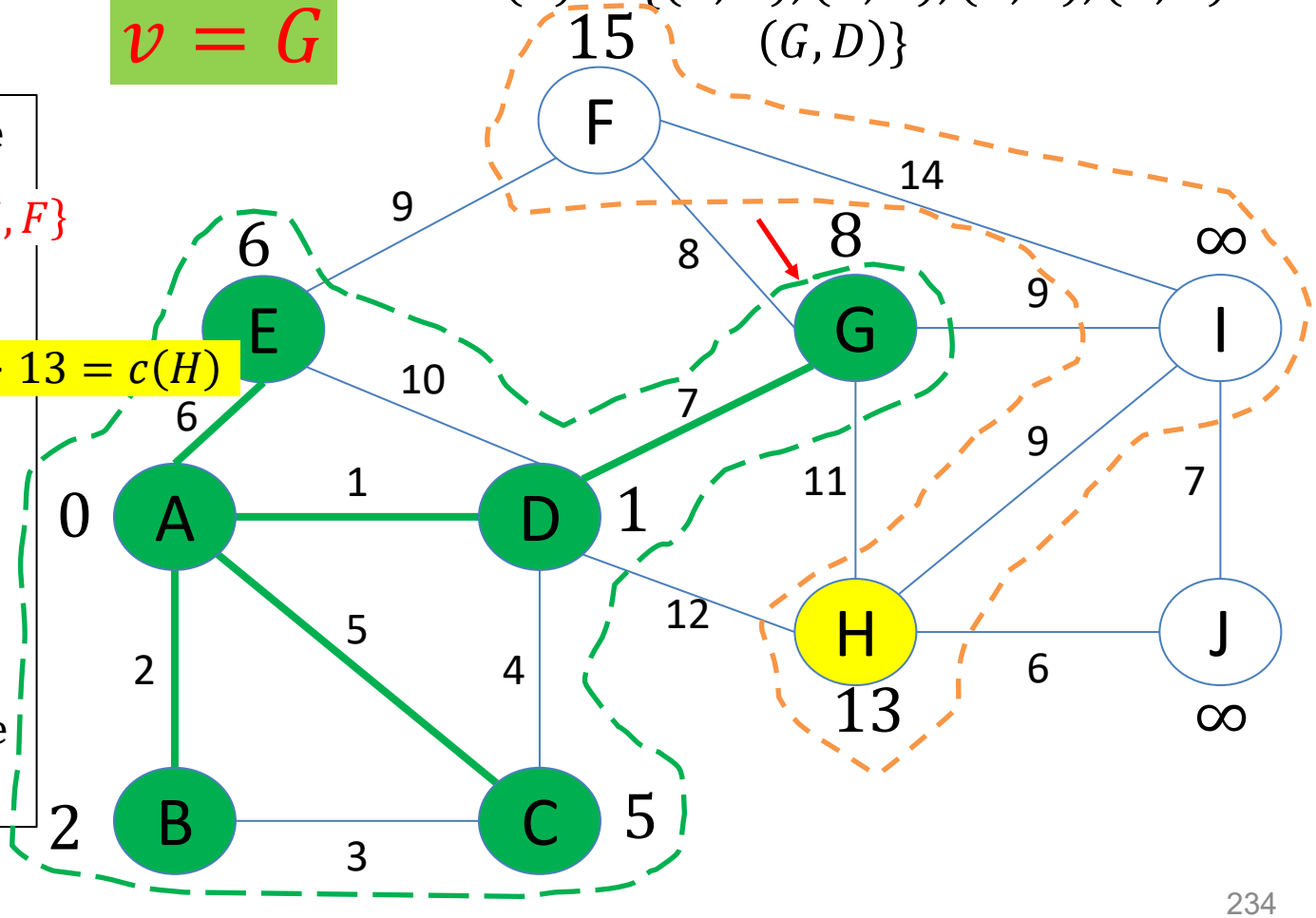
$V(T) = \{A, D, B, C, E, G\}$

$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$ $\text{Adj}(v) \setminus V(T) = \{H, I, F\}$
2. for each u in Adj(v) \ V(T): $u = H$
3. if c(v) + w(v, u) < c(u): $c(G) + w(H, G) = 19 > 13 = c(H)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(\infty, -)$

$v = G$

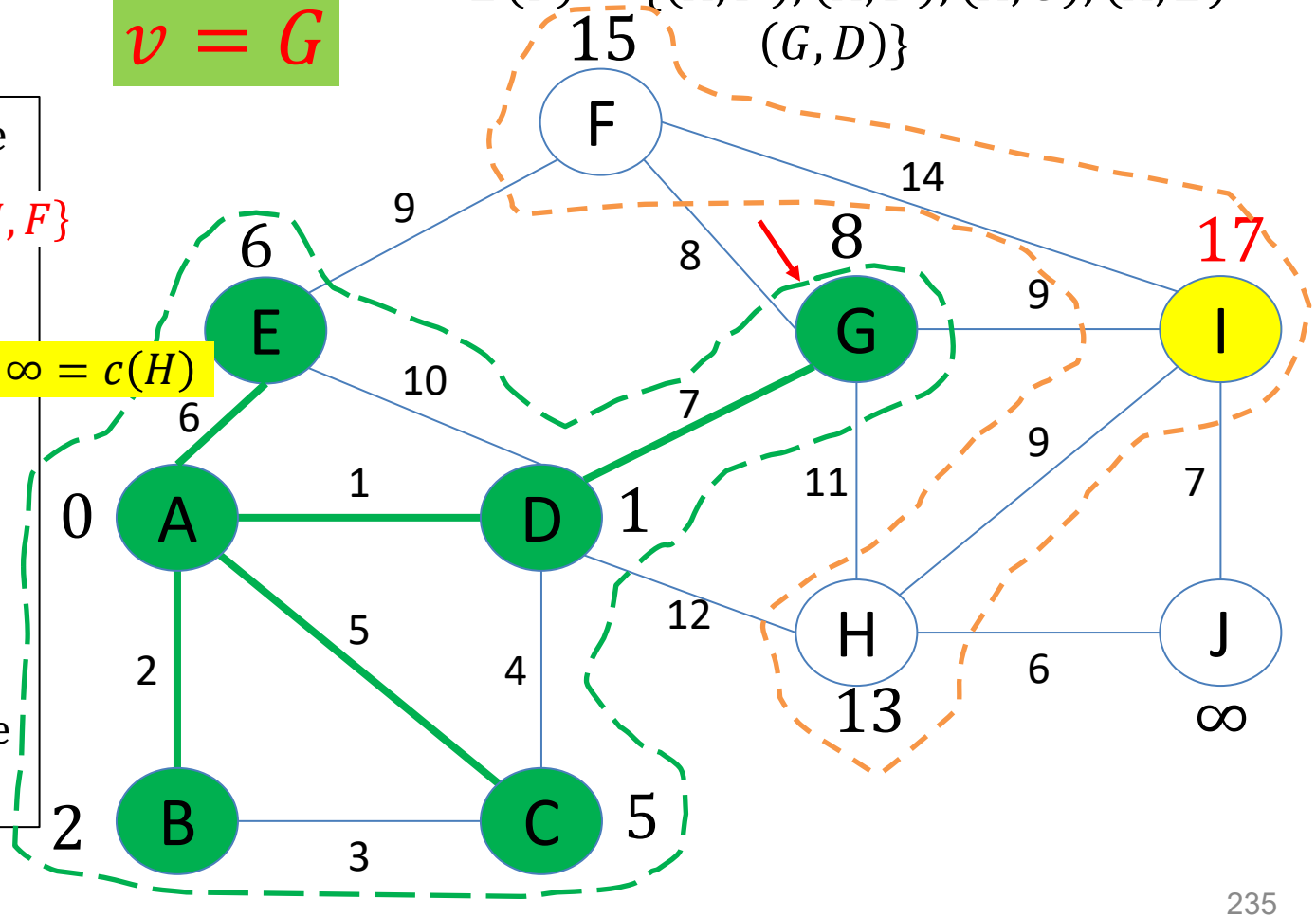
$V(T) = \{A, D, B, C, E, G\}$

$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{H, I, F\}$
2. for each u in Adj(v) \ V(T): $u = I$
3. if c(v) + w(v, u) < c(u): $c(G) + w(I, G) = 17 < \infty = c(H)$
4. c(u) := c(v) + w(v, u) $c(F) = 17$
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(\infty, -)$

$V(T) = \{A, D, B, C, E, G\}$

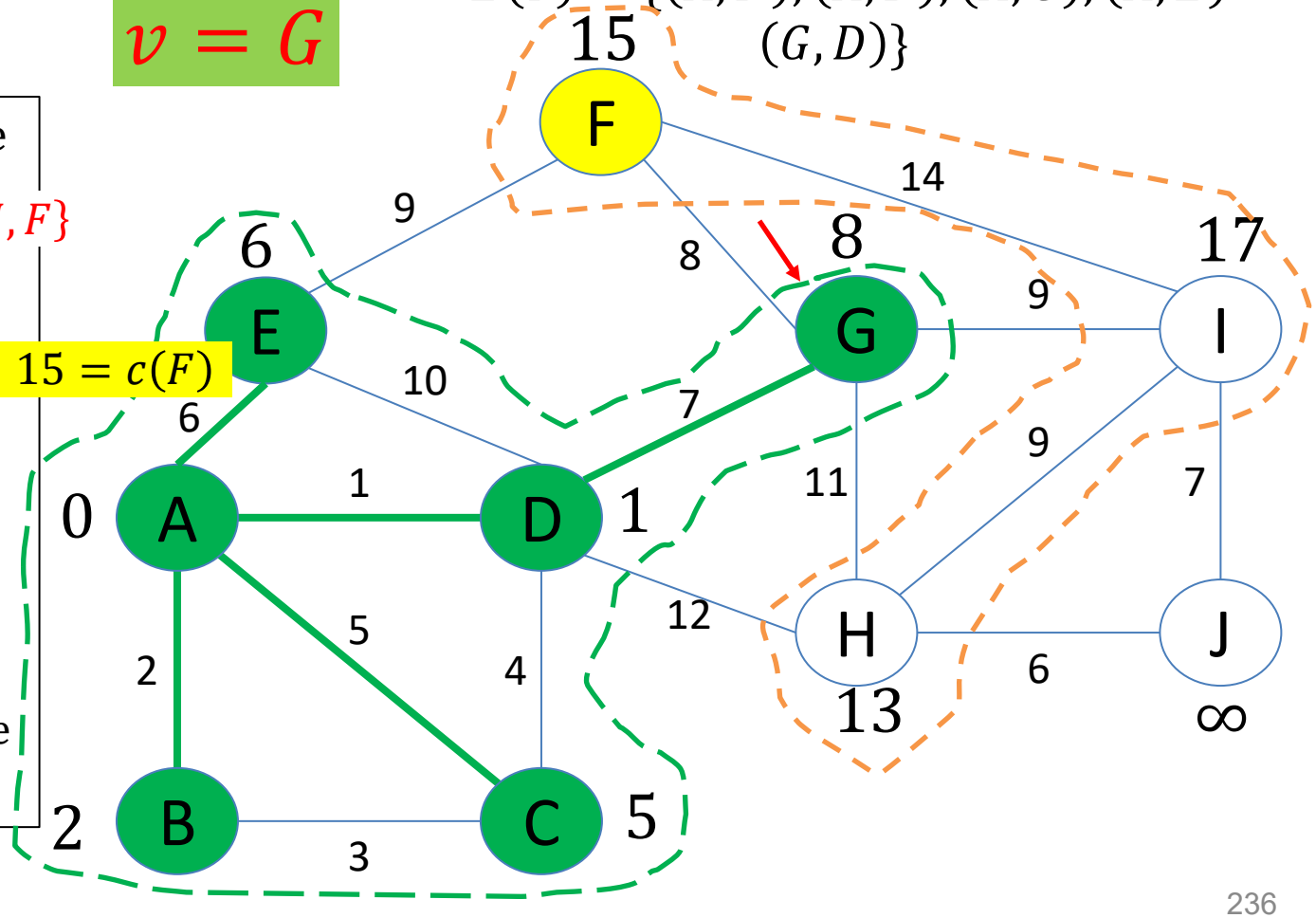
$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D)\}$

$v = G$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{H, I, F\}$
2. for each u in Adj(v) \ V(T): $u = F$
3. if c(v) + w(v, u) < c(u): $c(G) + w(F, G) = 16 > 15 = c(F)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	(15, E)	(8, D)	$(13, D)$	(17, G)	$(\infty, -)$

$v = G$

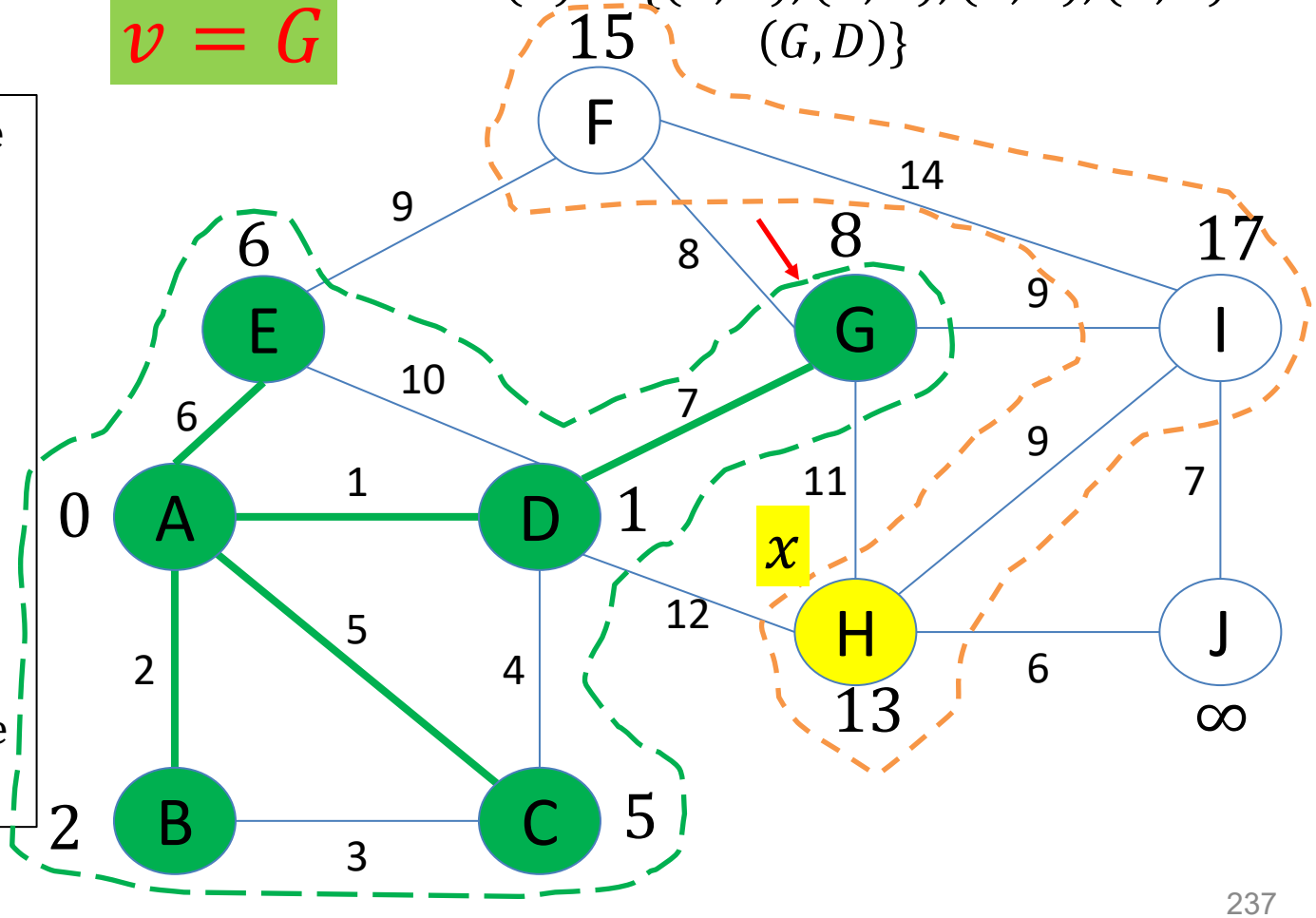
$V(T) = \{A, D, B, C, E, G\}$

$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	(15, E)	(8, D)	$(13, D)$	(17, G)	$(\infty, -)$

$v = G$

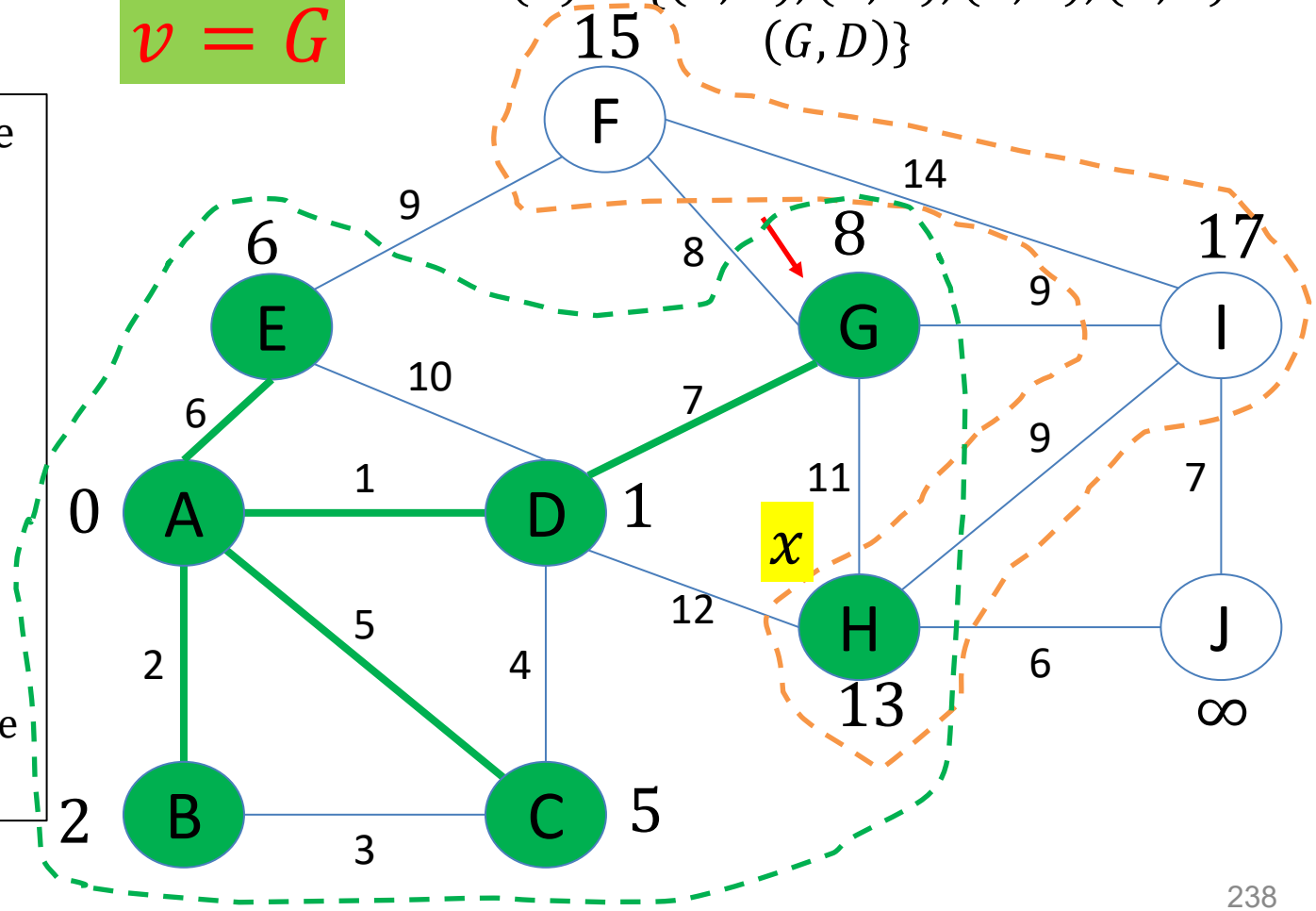
$V(T) = \{A, D, B, C, E, G, H\}$

$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	(15, E)	(8, D)	$(13, D)$	(17, G)	$(\infty, -)$

$v = G$

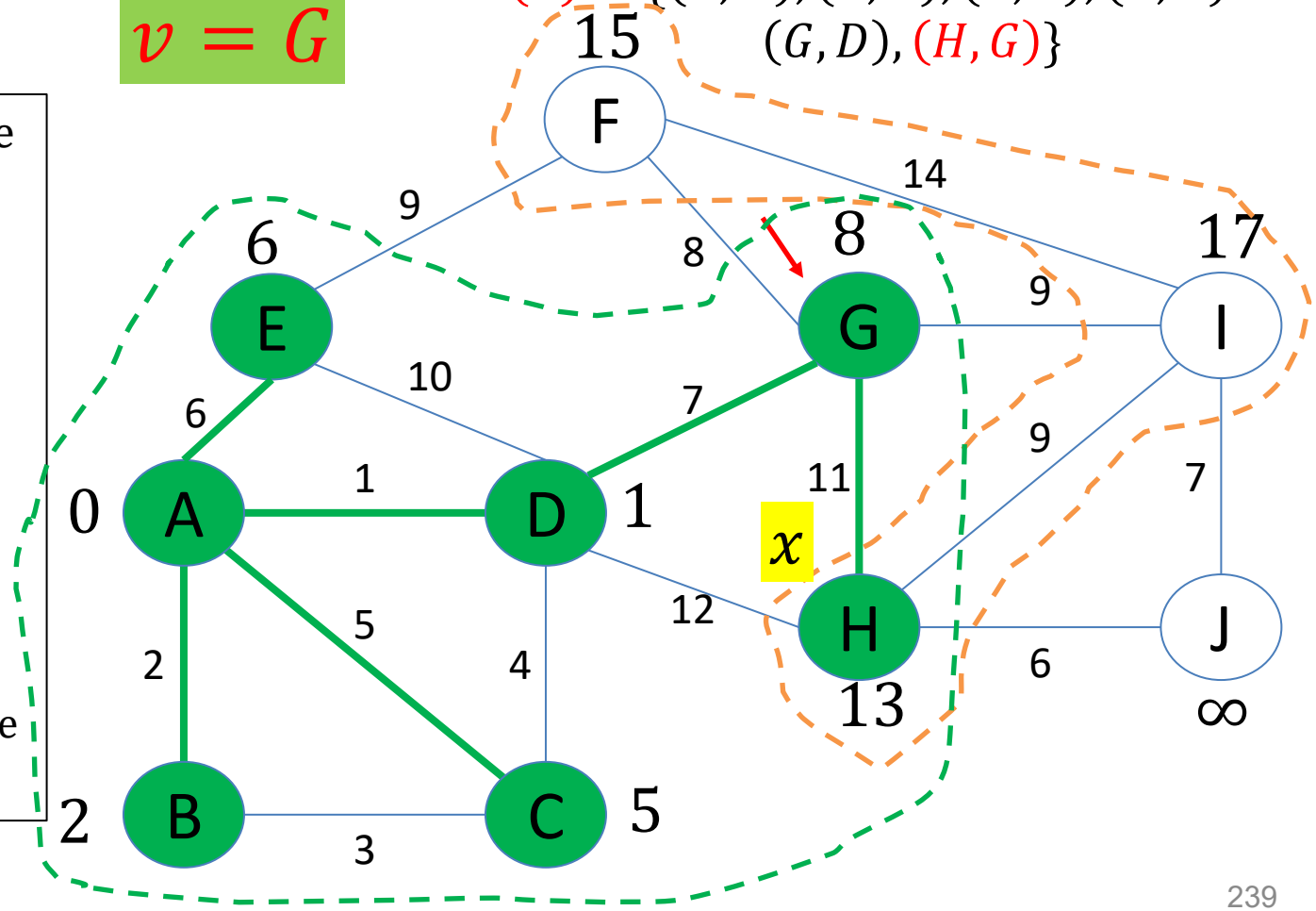
$V(T) = \{A, D, B, C, E, G, H\}$

$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D), (H, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(\infty, -)$

$v = H$

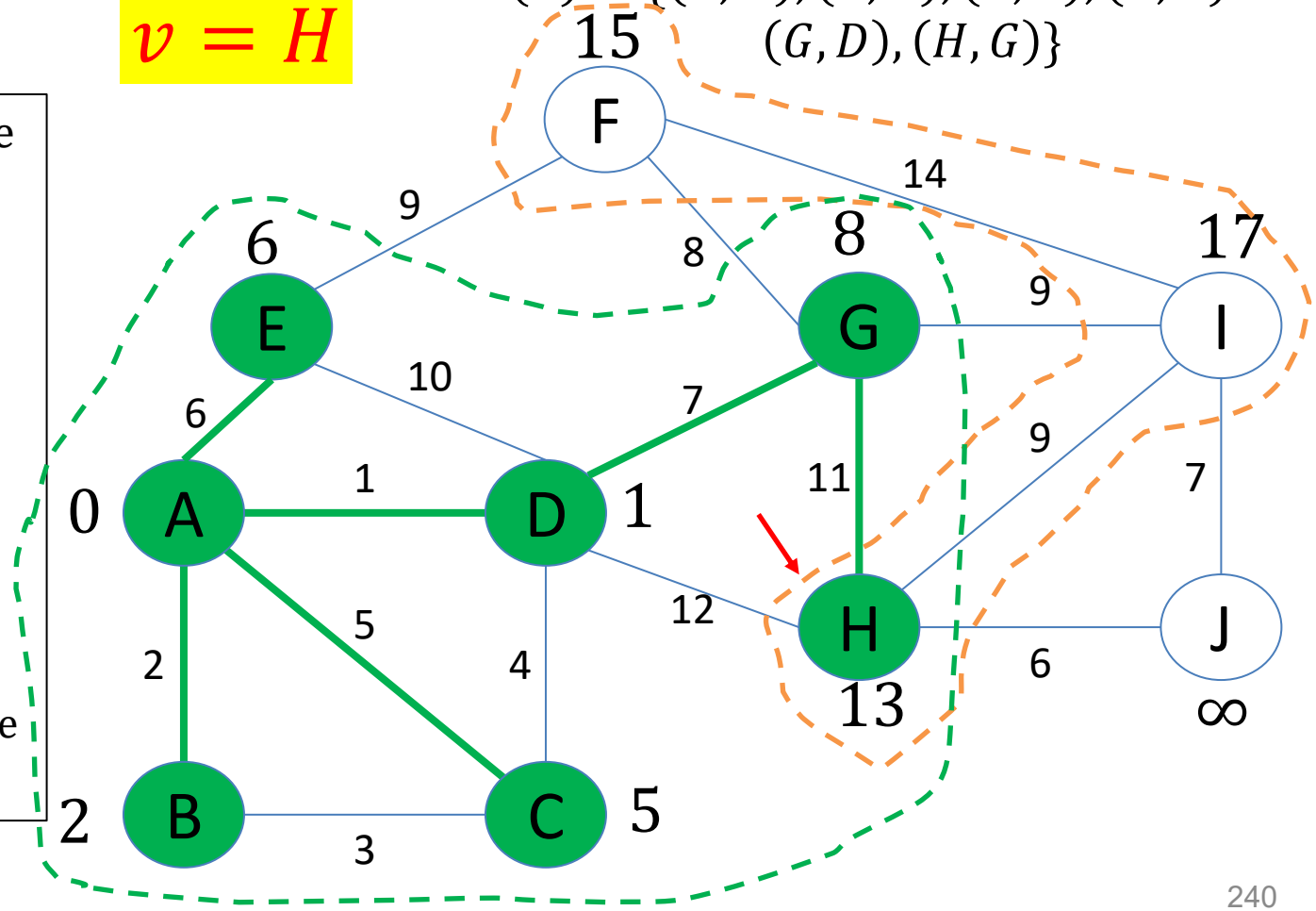
$V(T) = \{A, D, B, C, E, G, H\}$

$F = \{H, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(\infty, -)$

$v = H$

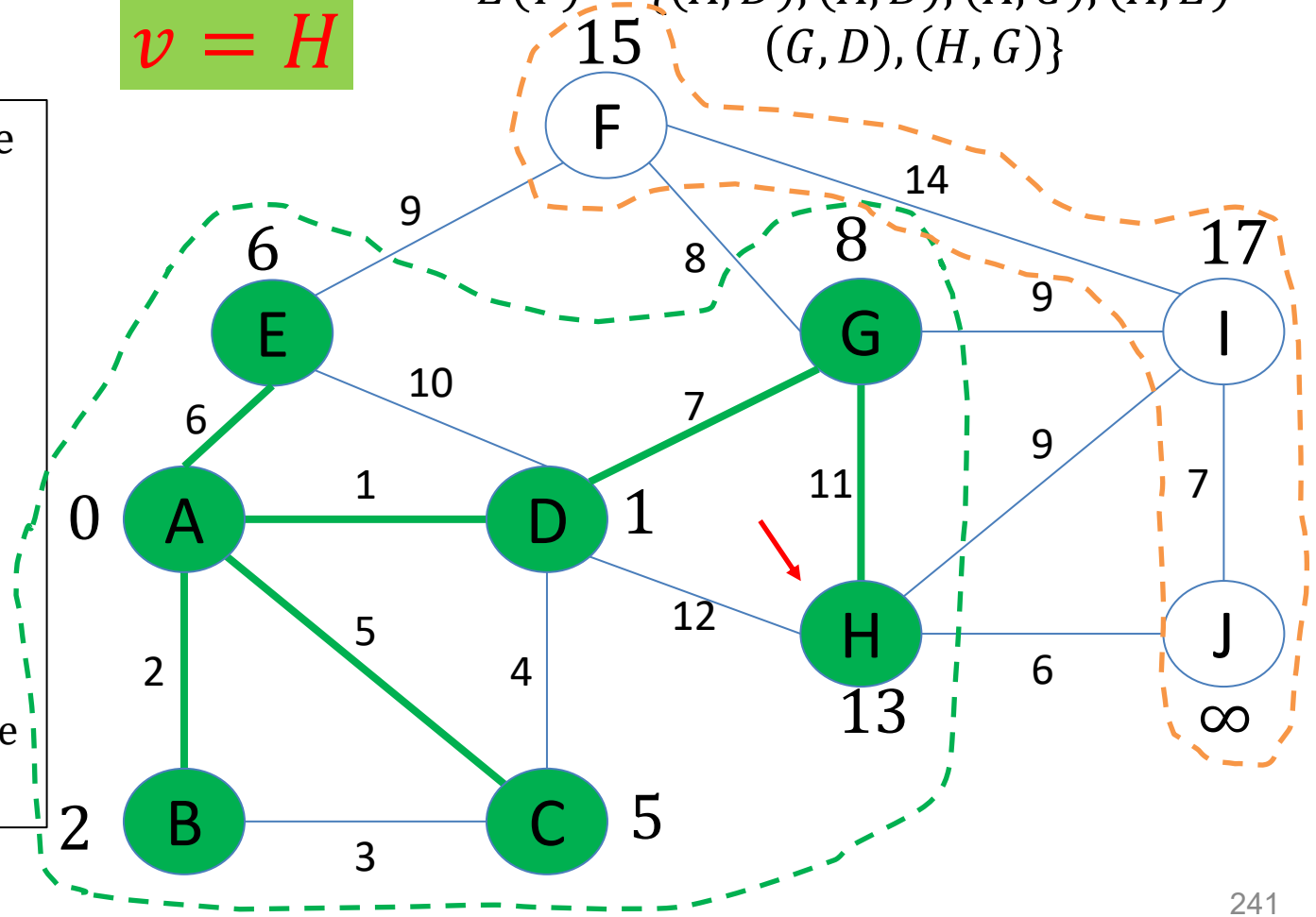
$V(T) = \{A, D, B, C, E, G, H\}$

$F = \{J, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = H$

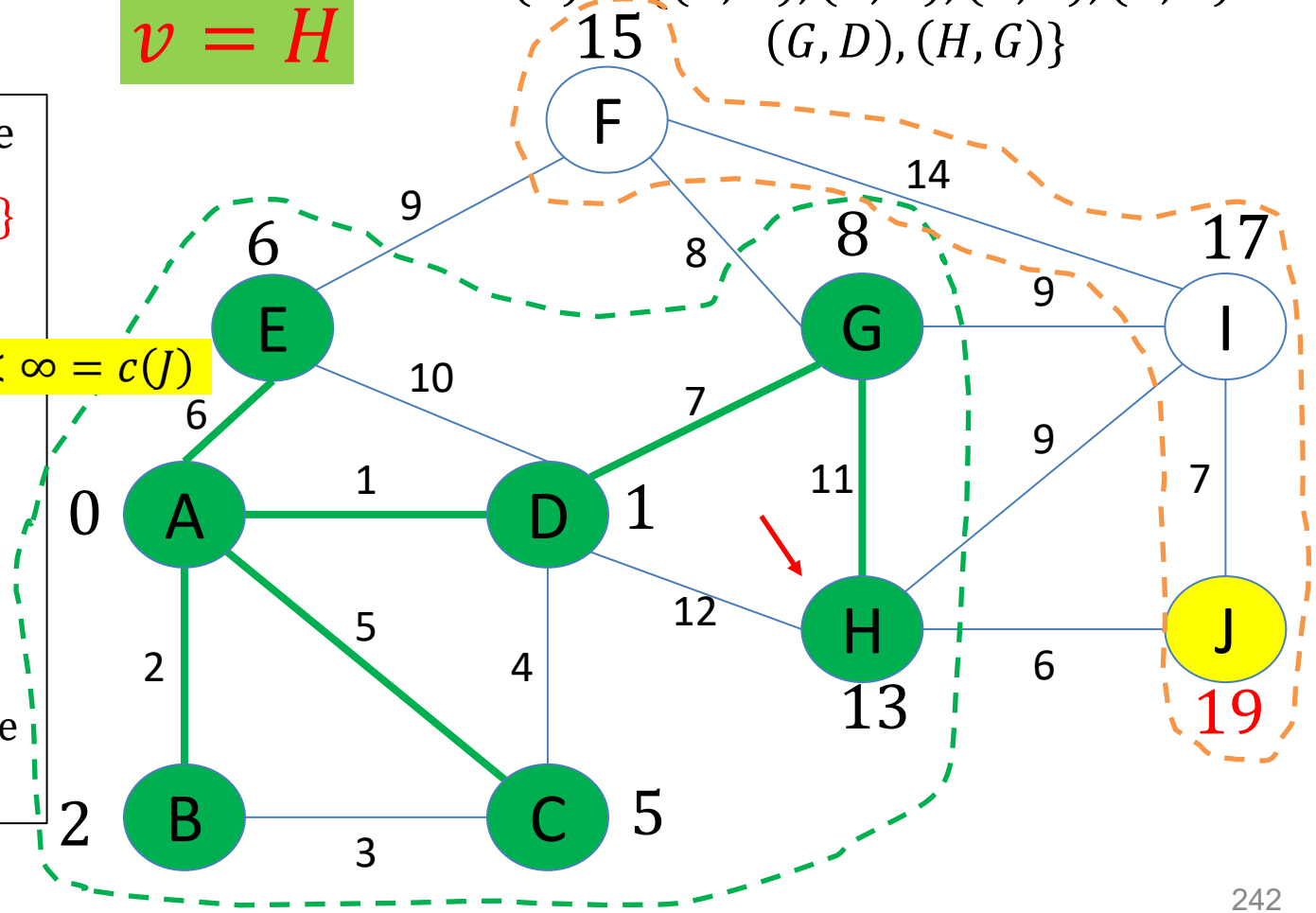
$V(T) = \{A, D, B, C, E, G, H\}$

$F = \{J, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D), (H, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{I, J\}$
2. for each u in Adj(v) \ V(T): $u = J$
3. if c(v) + w(v, u) < c(u): $c(H) + w(J, H) = 19 < \infty = c(J)$
4. c(u) := c(v) + w(v, u) $c(J) = 19$
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = H$

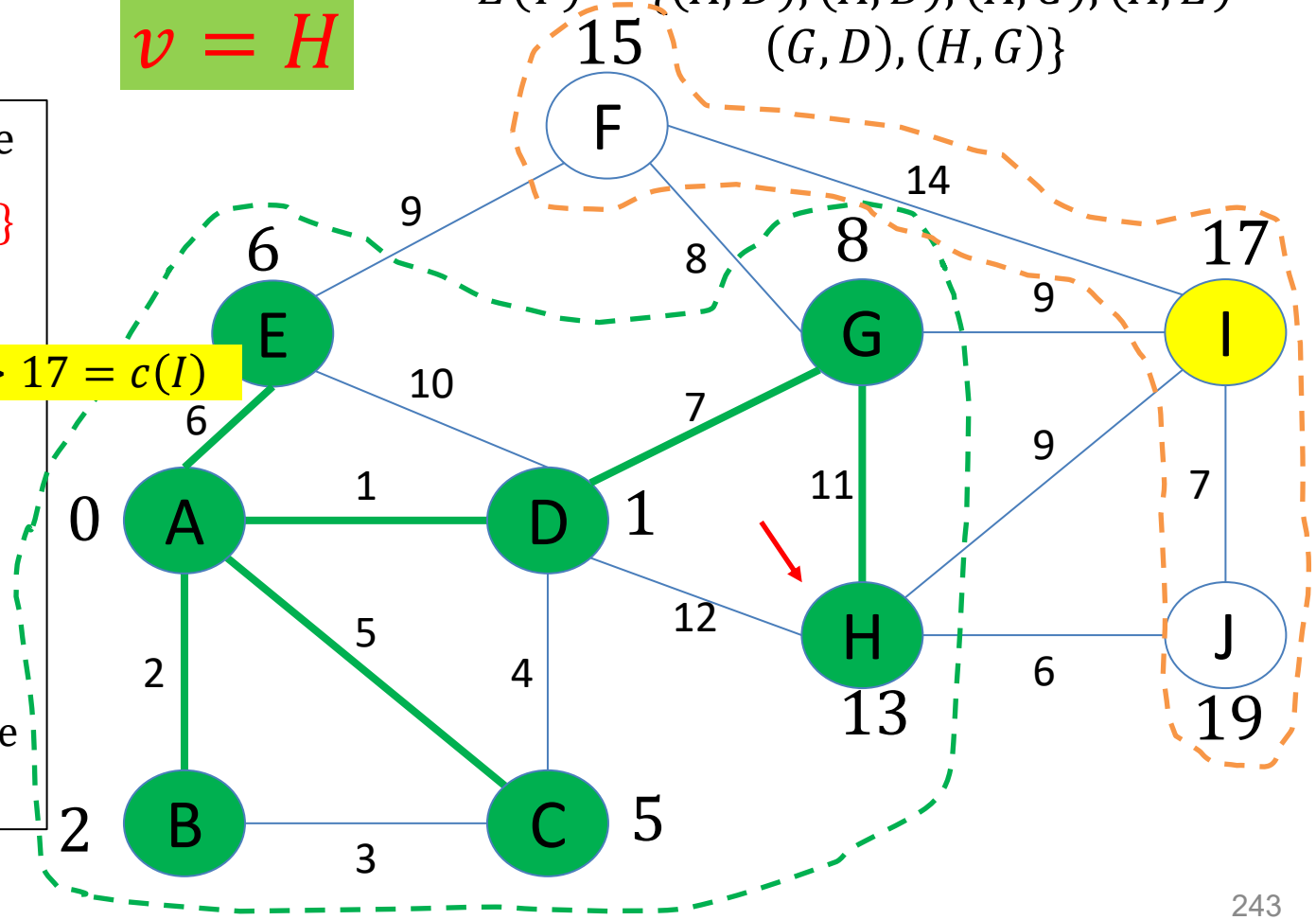
$V(T) = \{A, D, B, C, E, G, H\}$

$F = \{J, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \backslash \mathrm{V}(T) = \{I, J\}$
2. for each u in Adj(v) \ V(T): $u = I$
3. if c(v) + w(v, u) < c(u): $c(H) + w(I, H) = 22 > 17 = c(I)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = H$

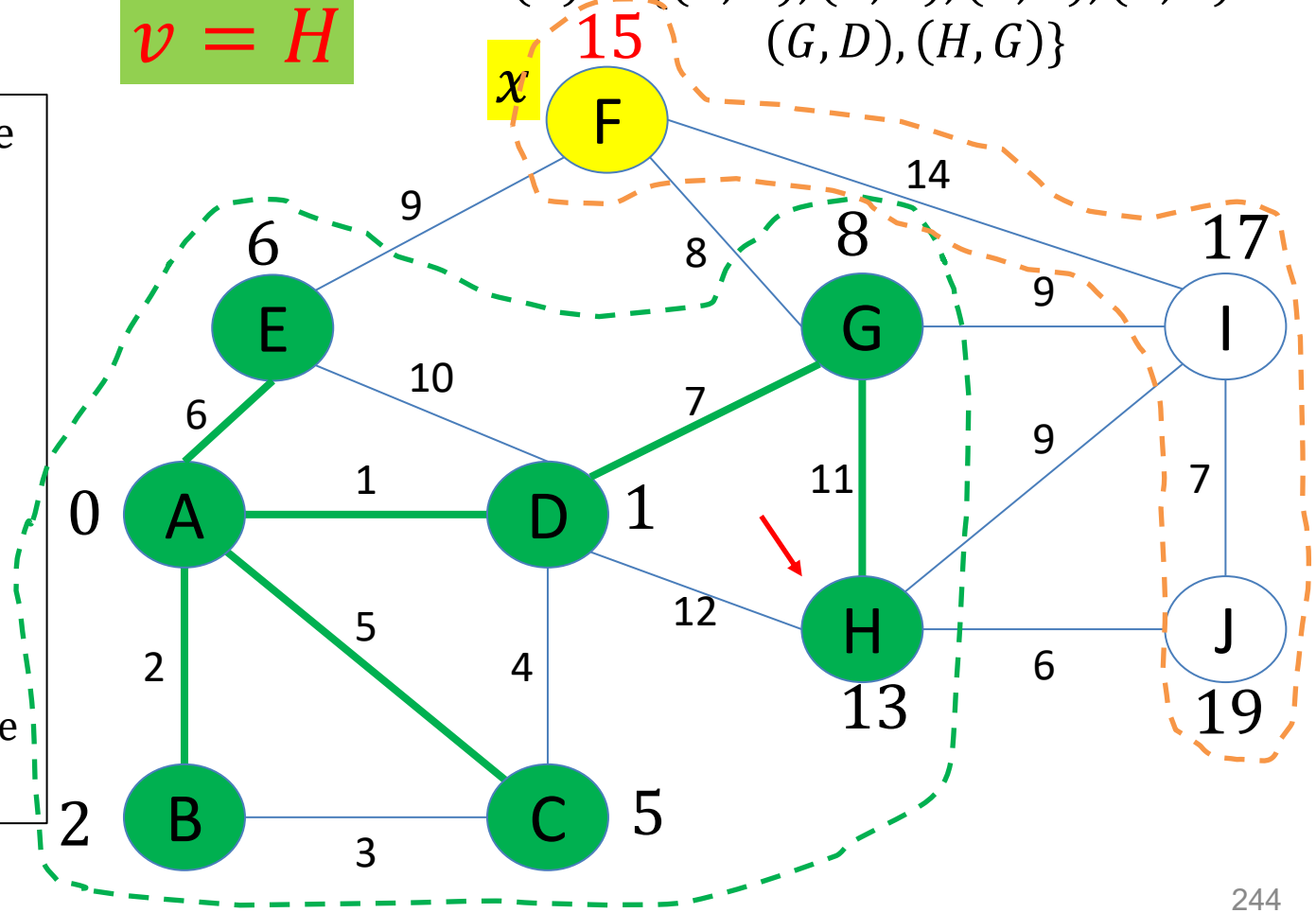
$V(T) = \{A, D, B, C, E, G, H\}$

$F = \{J, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = H$

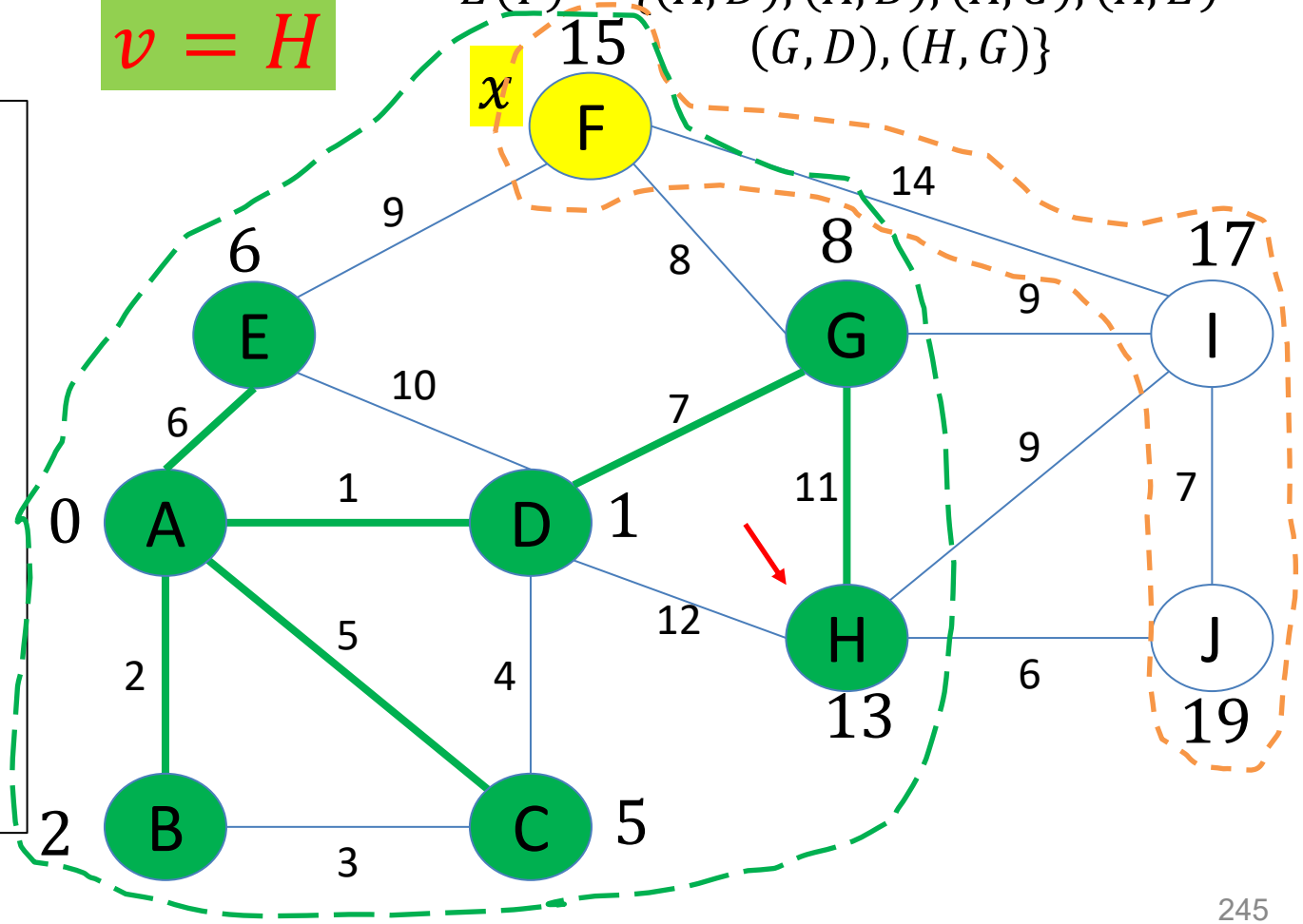
$V(T) = \{A, D, B, C, E, G, H, F\}$

$F = \{J, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = H$

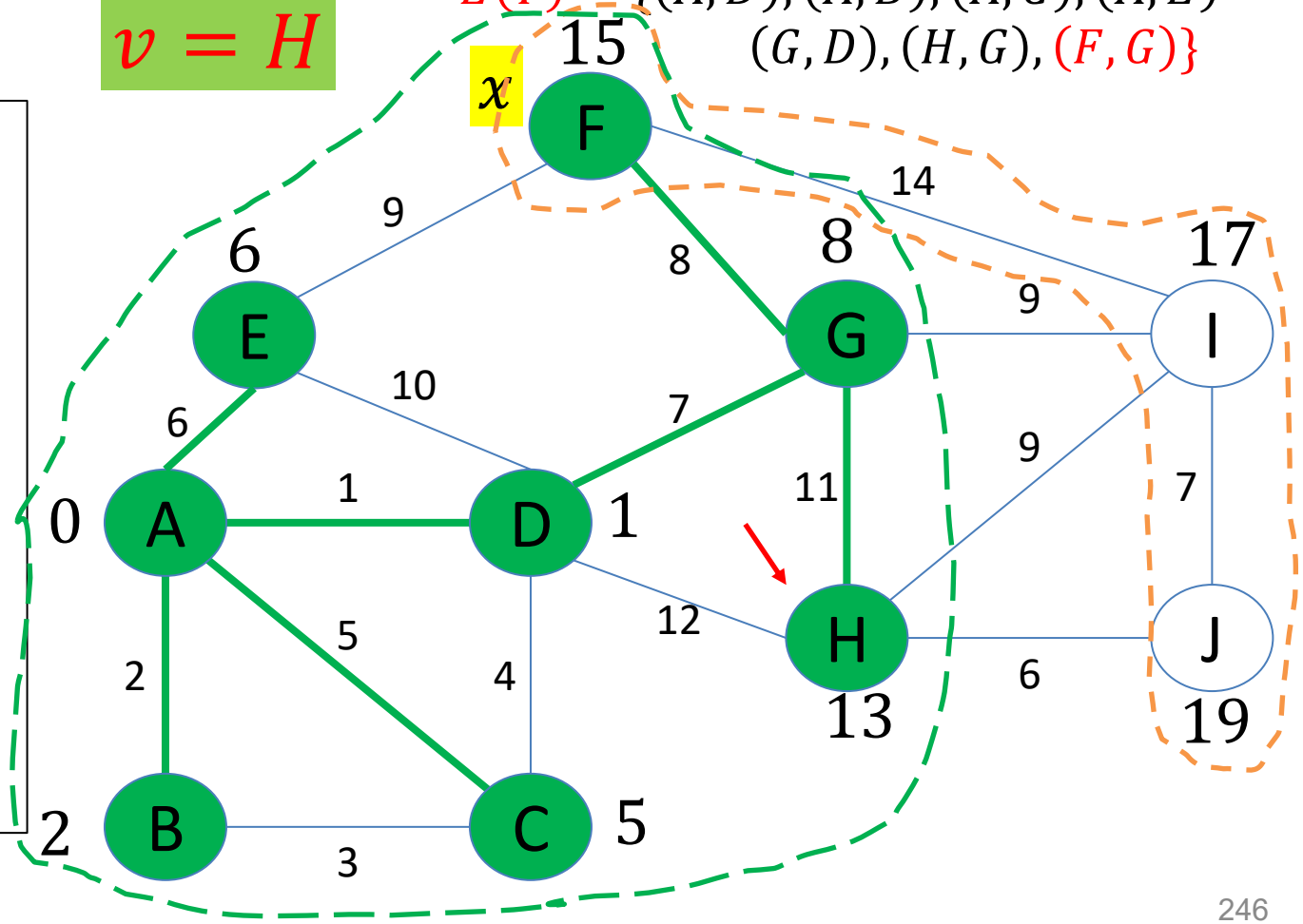
$V(T) = \{A, D, B, C, E, G, H, F\}$

$F = \{J, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = F$

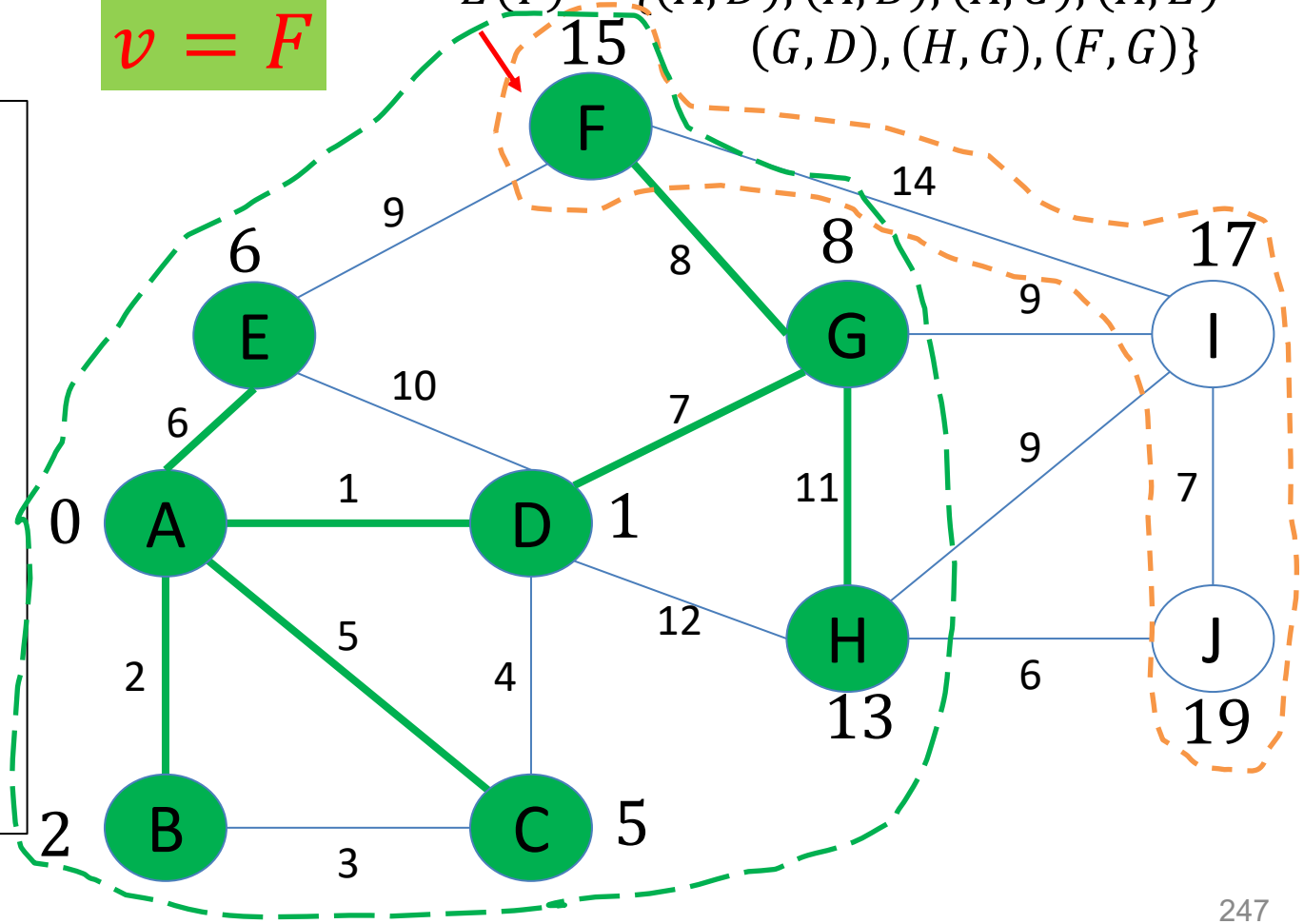
$V(T) = \{A, D, B, C, E, G, H, F\}$

$F = \{J, I, F\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$V(T) = \{A, D, B, C, E, G, H, F\}$

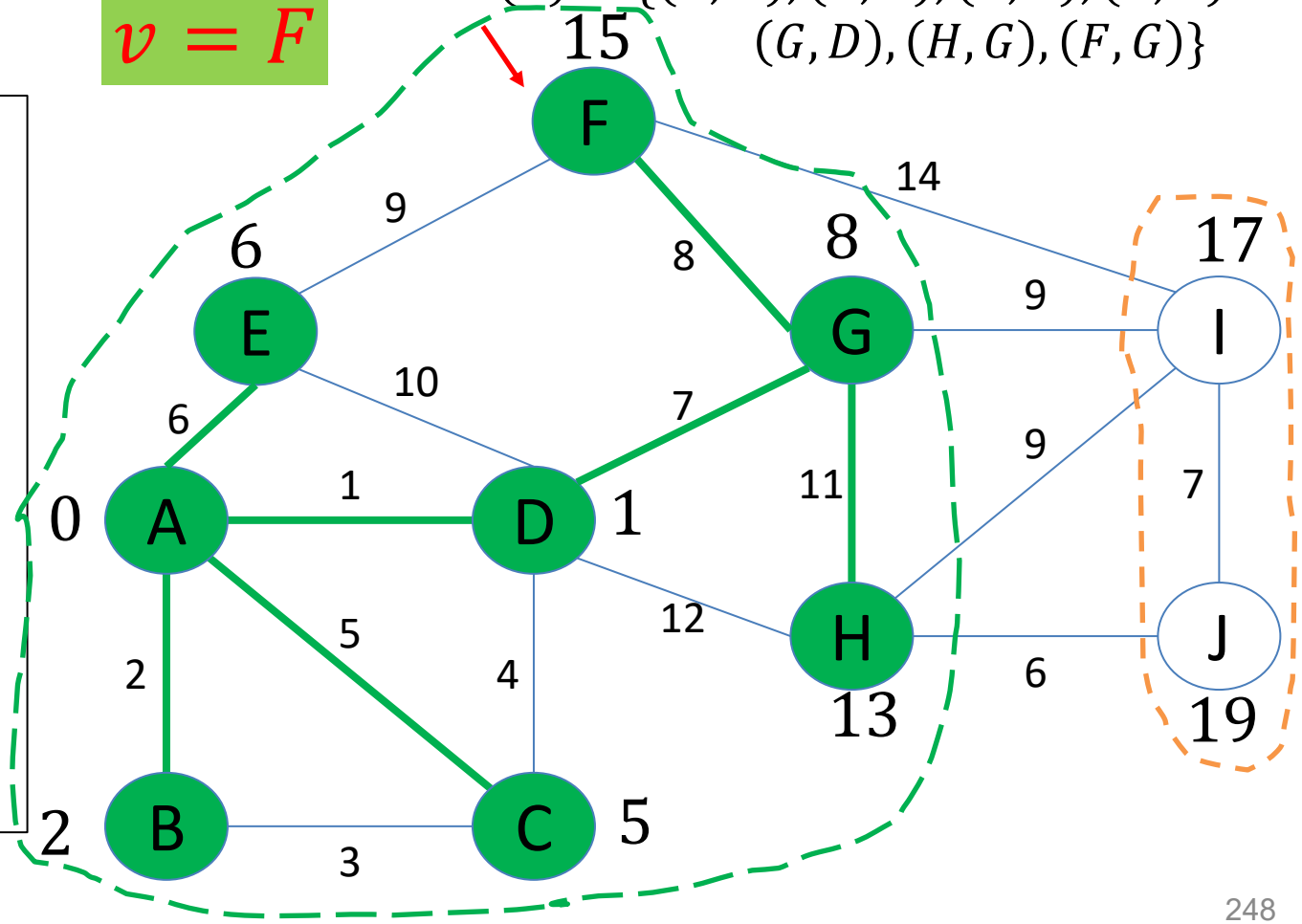
$F = \{J, I\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G)\}$

$v = F$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$V(T) = \{A, D, B, C, E, G, H, F\}$

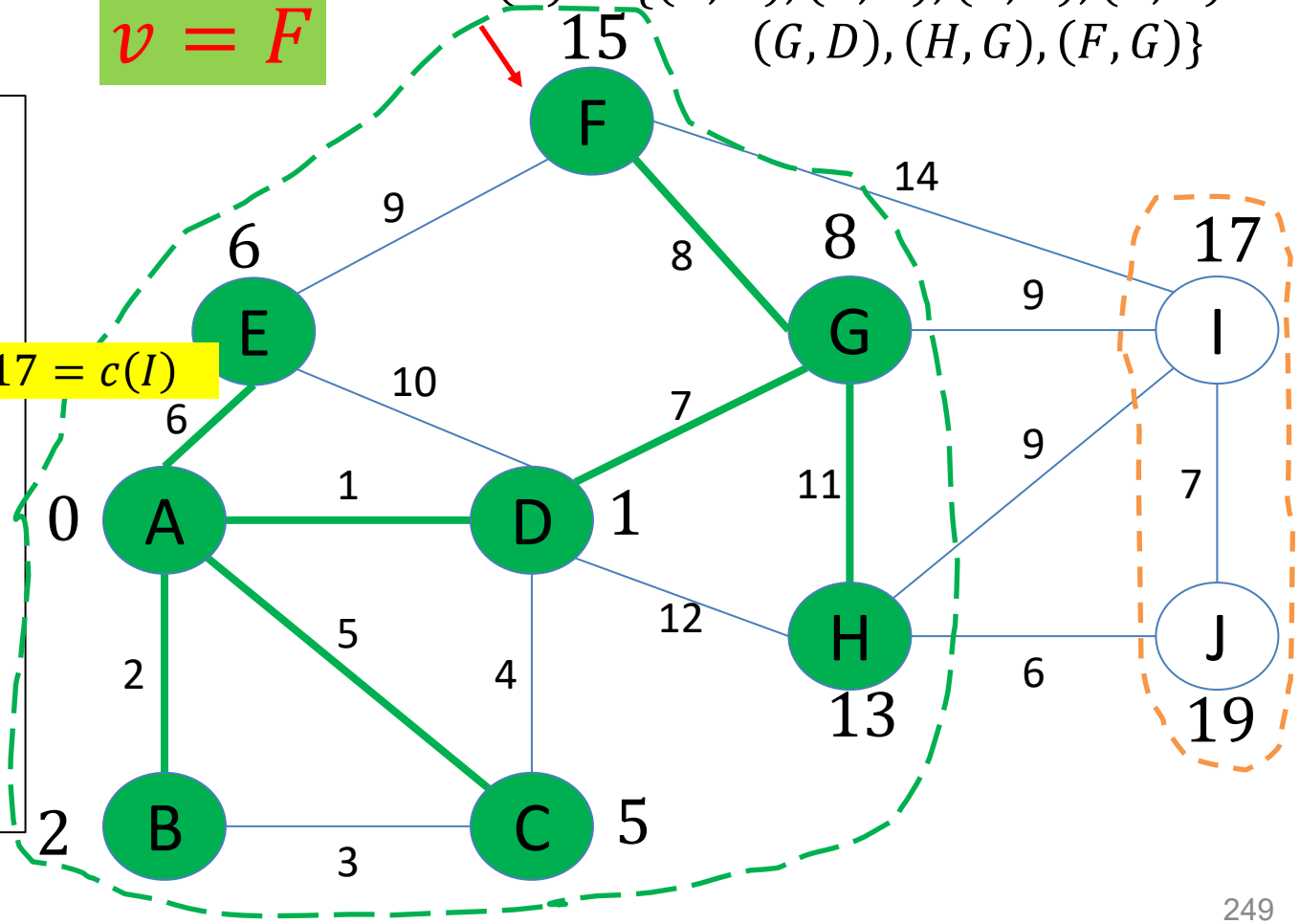
$F = \{J, I\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G)\}$

$v = F$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{I\}$
2. for each u in Adj(v) \ V(T): $u = I$
3. if c(v) + w(v, u) < c(u): $c(F) + w(I, F) = 29 > 17 = c(I)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$V(T) = \{A, D, B, C, E, G, H, F\}$

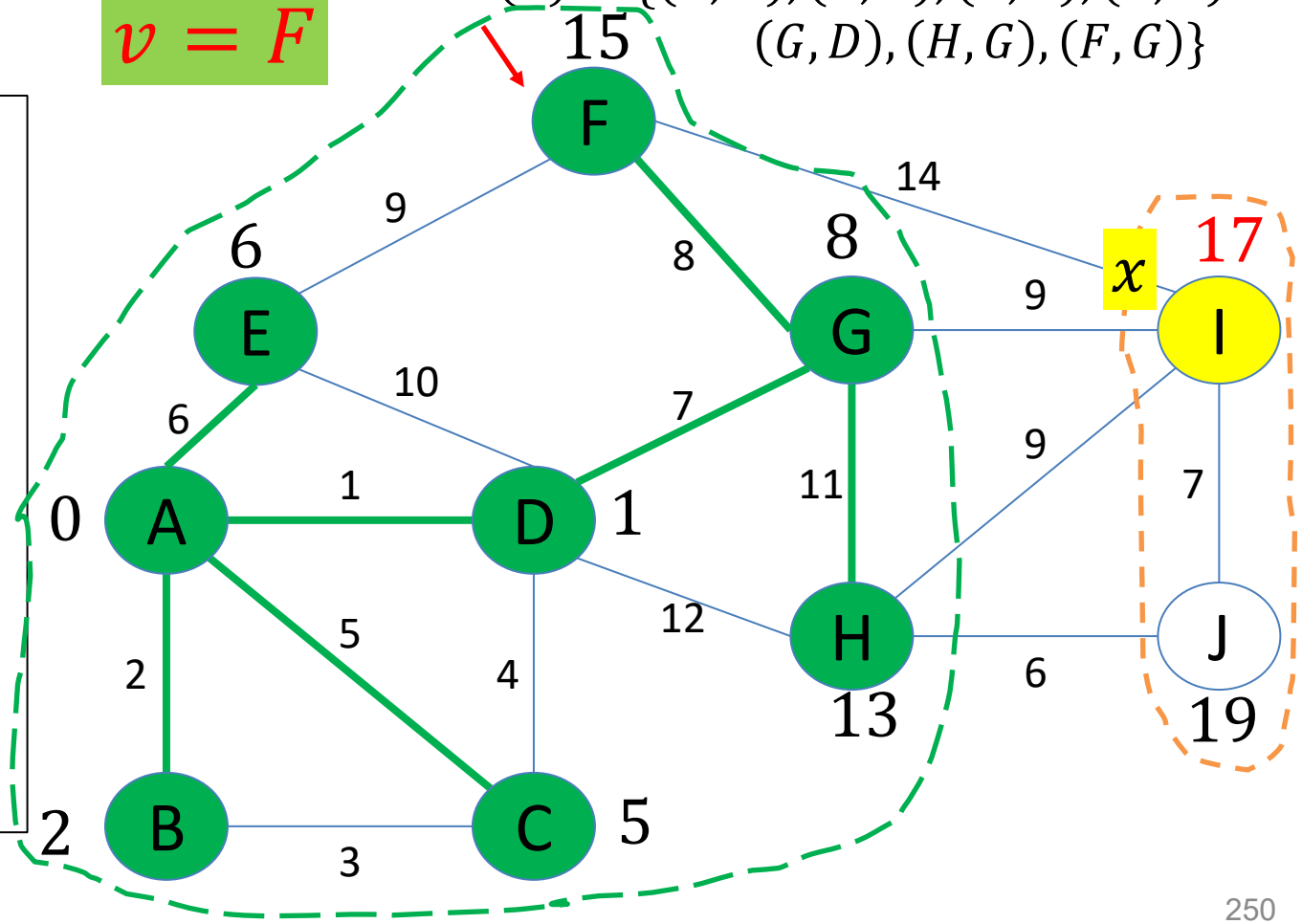
$F = \{J, I\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G)\}$

$v = F$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$V(T) = \{A, D, B, C, E, G, H, F, I\}$

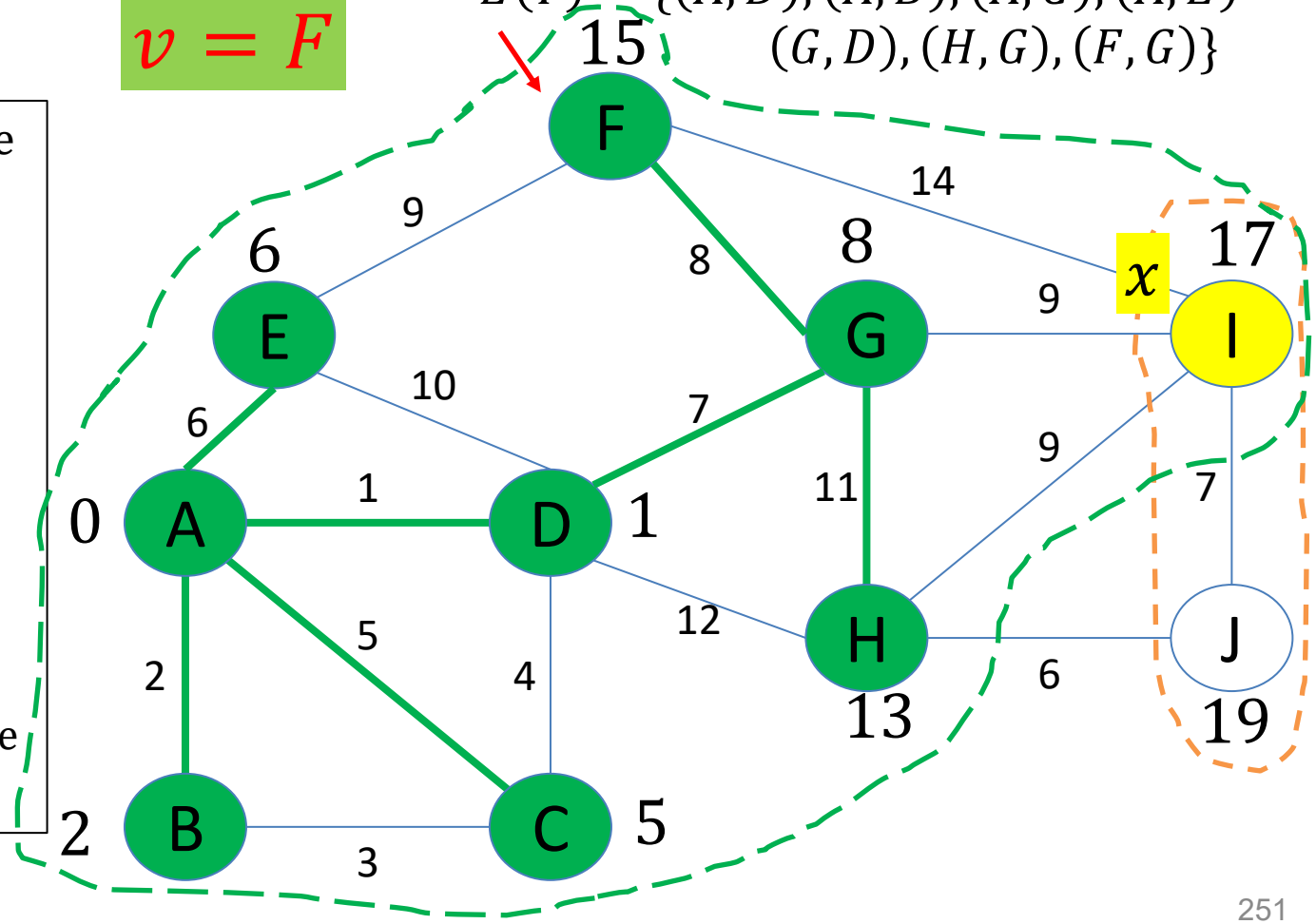
$F = \{J, I\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G)\}$

$v = F$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = F$

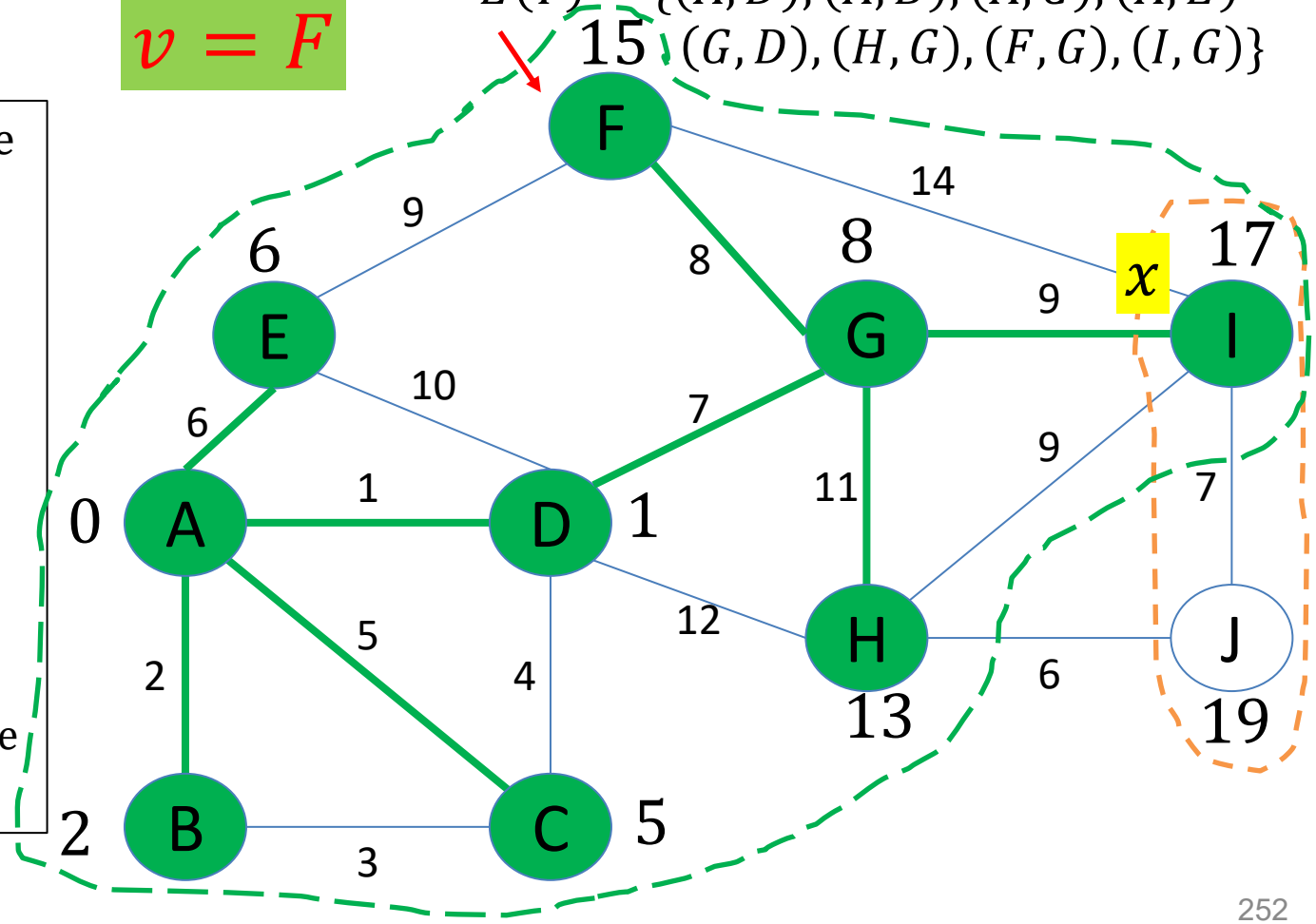
$V(T) = \{A, D, B, C, E, G, H, F, I\}$

$F = \{J, I\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G), (I, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = I$

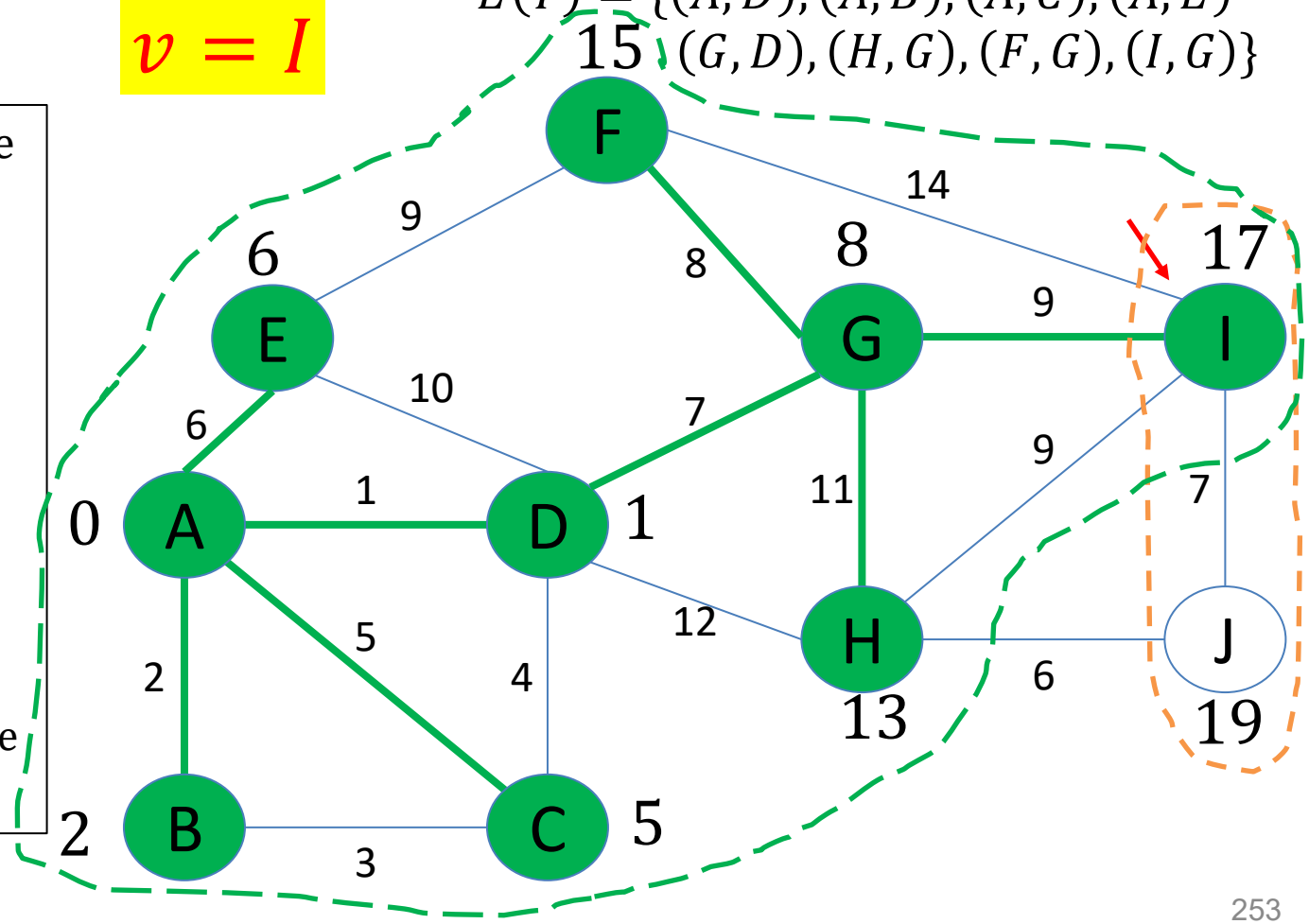
$V(T) = \{A, D, B, C, E, G, H, F, I\}$

$F = \{J, I\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D), (H, G), (F, G), (I, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = I$

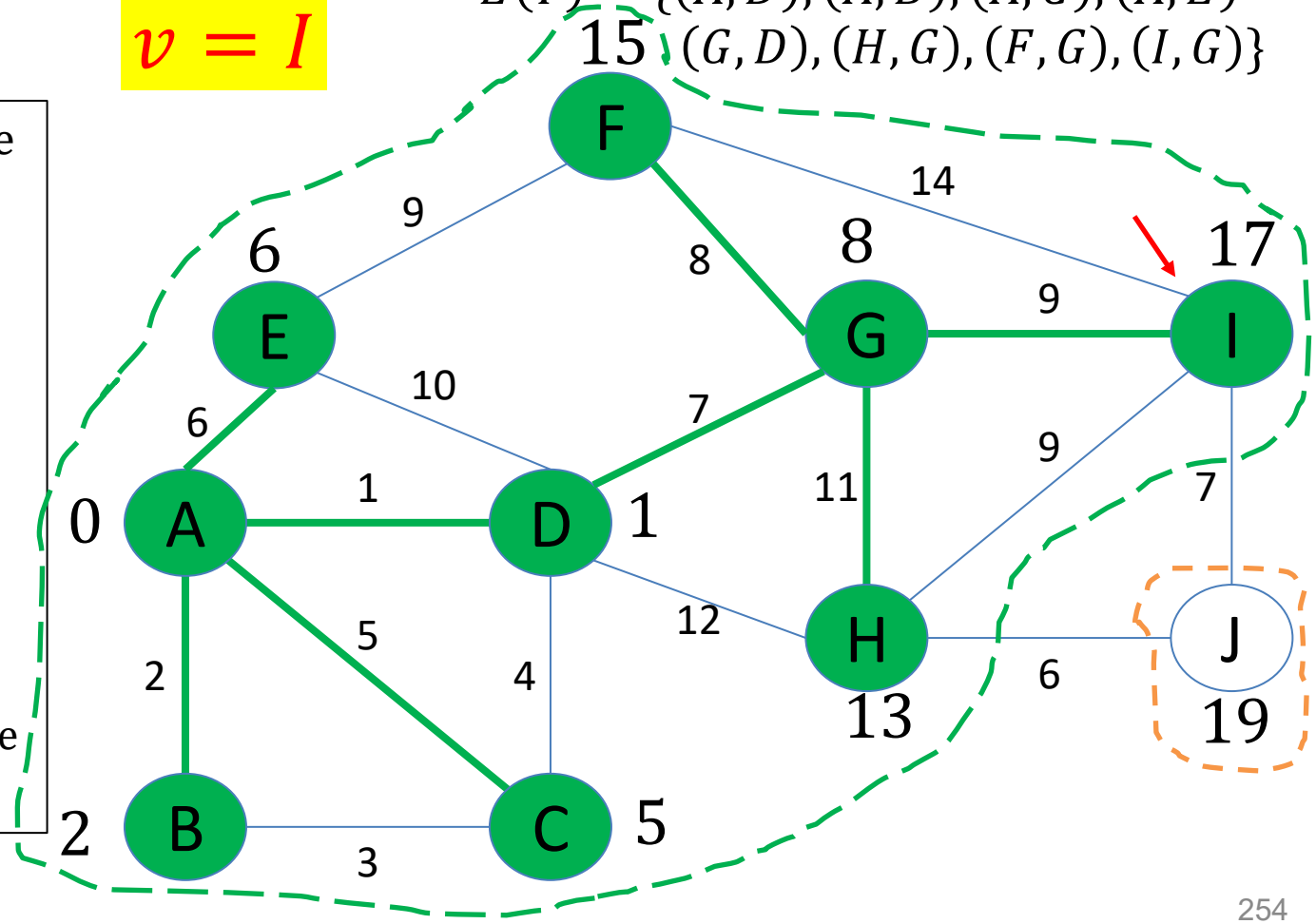
$V(T) = \{A, D, B, C, E, G, H, F, I\}$

$F = \{J\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G), (I, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = I$

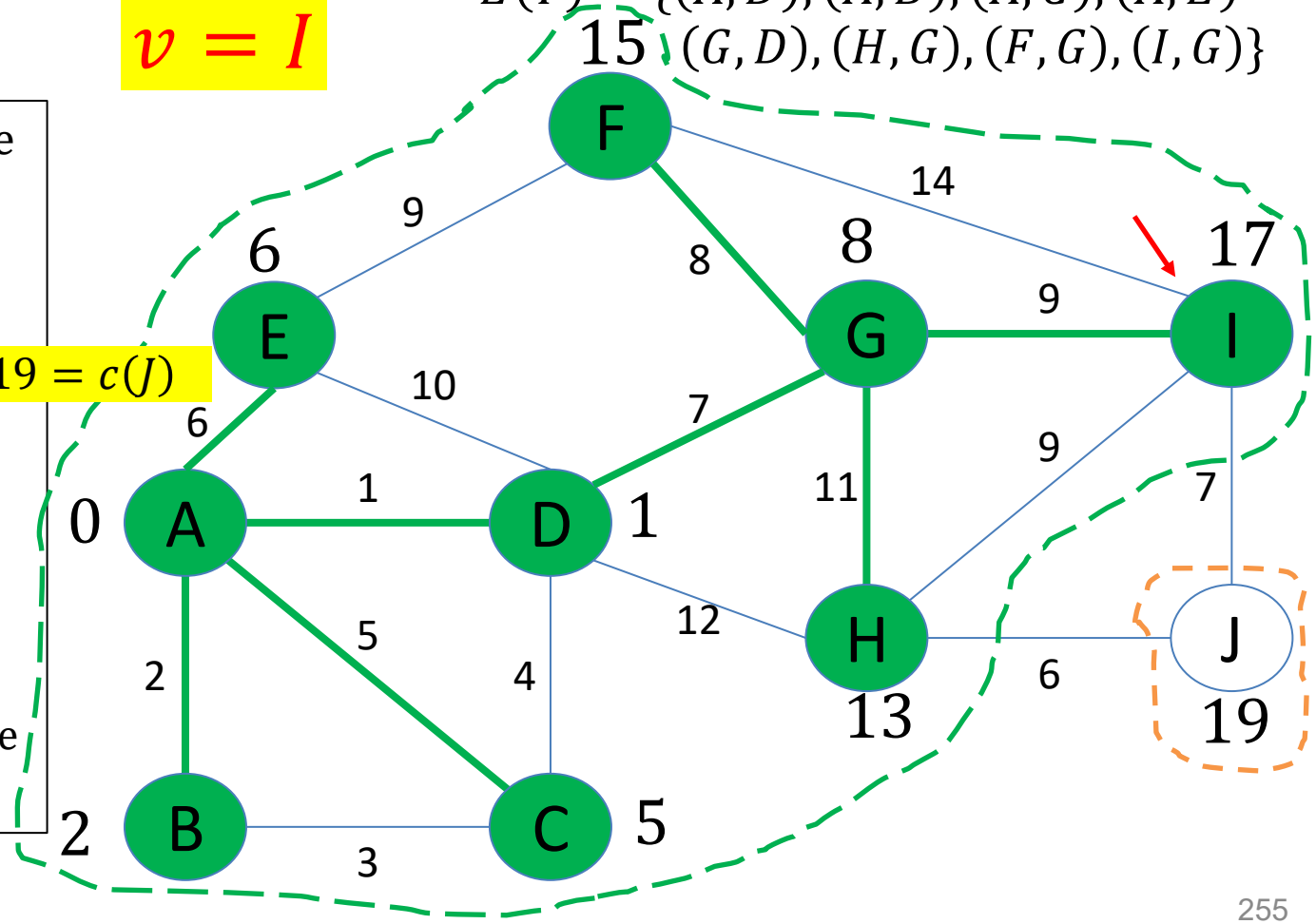
$V(T) = \{A, D, B, C, E, G, H, F, I\}$

$F = \{J\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G), (I, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$ $\mathrm{Adj}(v) \setminus \mathrm{V}(T) = \{J\}$
2. for each u in Adj(v) \ V(T): $u = J$
3. if c(v) + w(v, u) < c(u): $c(I) + w(J, I) = 24 > 19 = c(J)$
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = I$

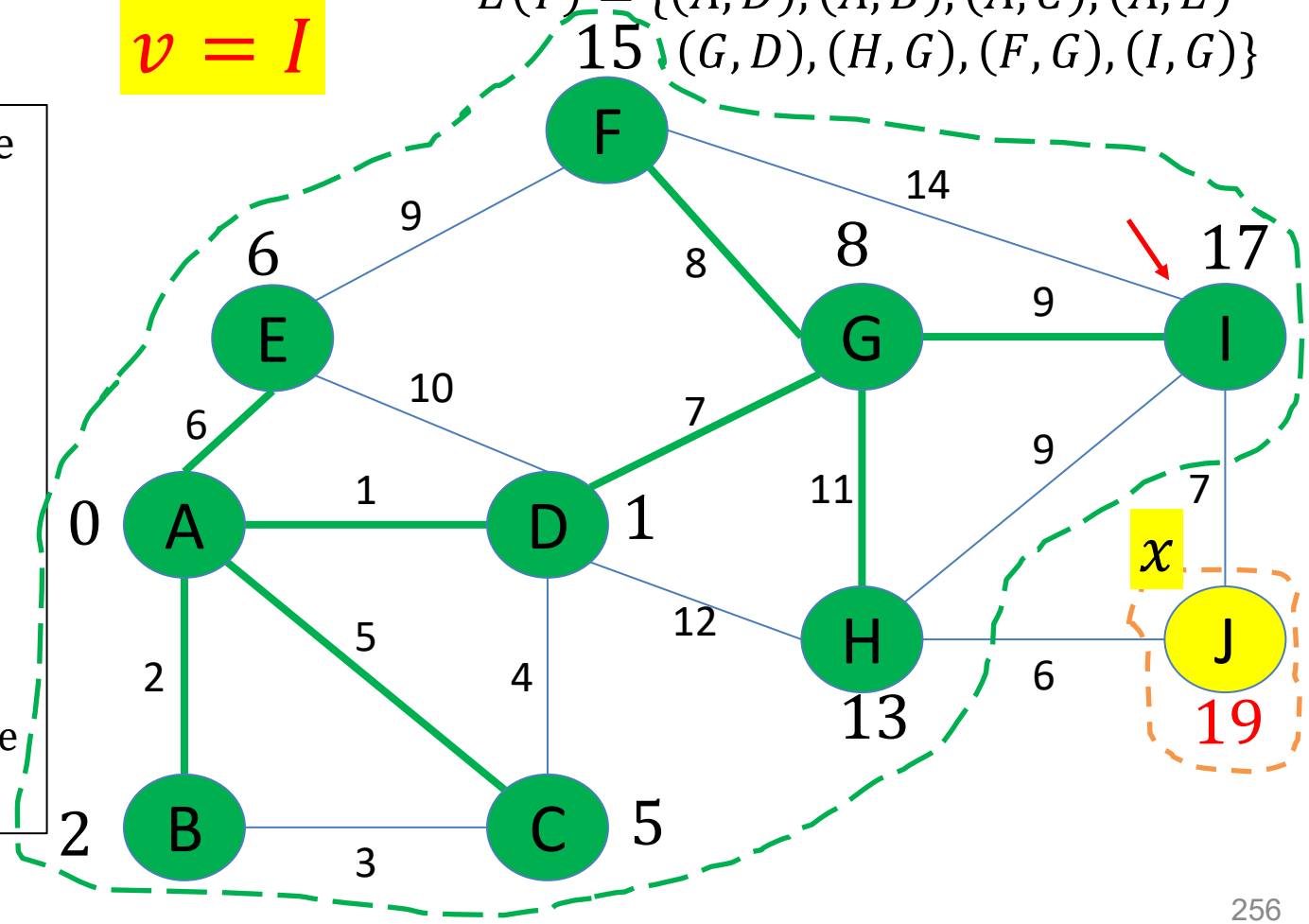
$V(T) = \{A, D, B, C, E, G, H, F, I\}$

$F = \{J\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G), (I, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$v = I$

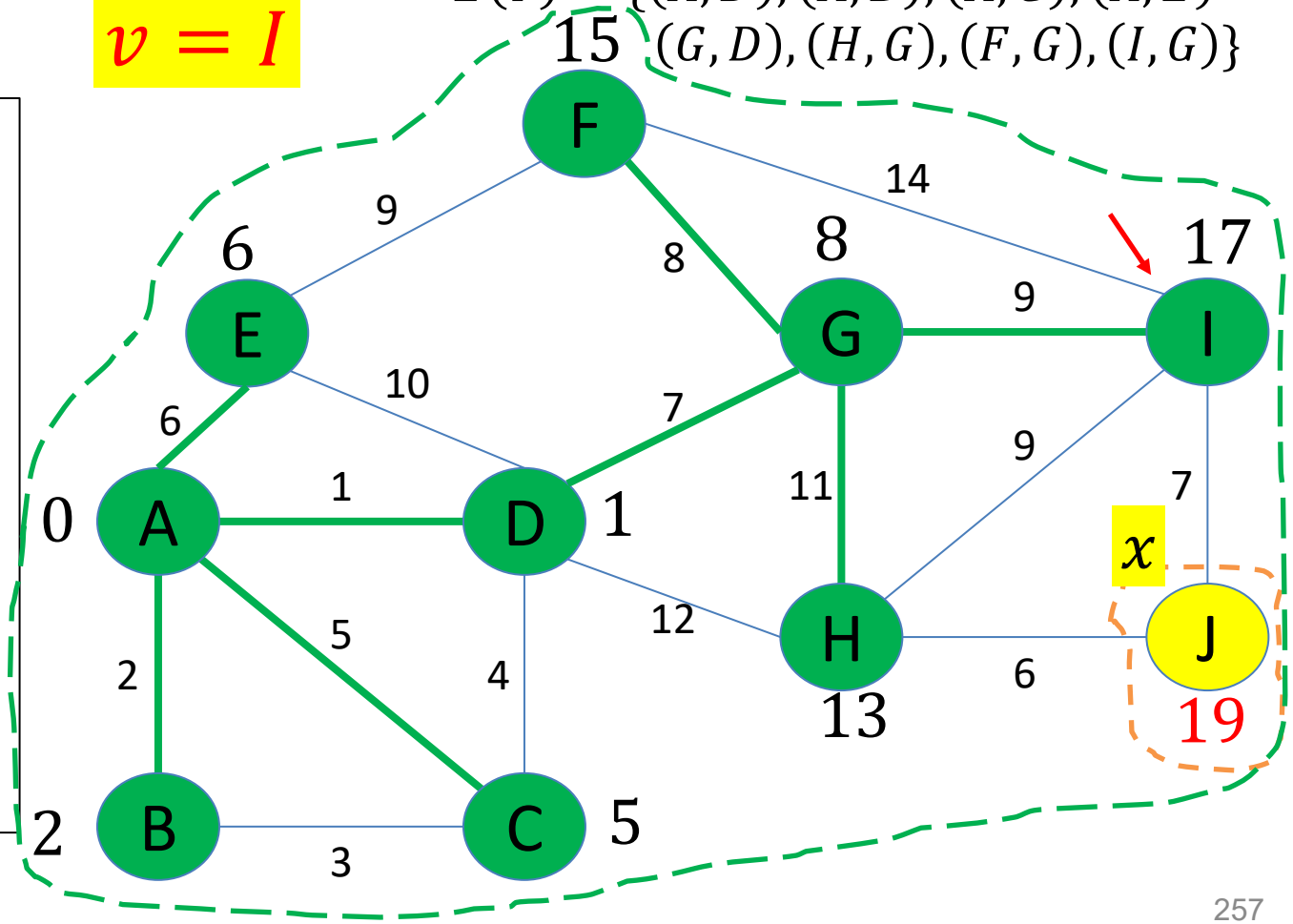
$V(T) = \{A, D, B, C, E, G, H, F, I, J\}$

$F = \{J\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G), (I, G)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$V(T) = \{A, D, B, C, E, G, H, F, I, J\}$

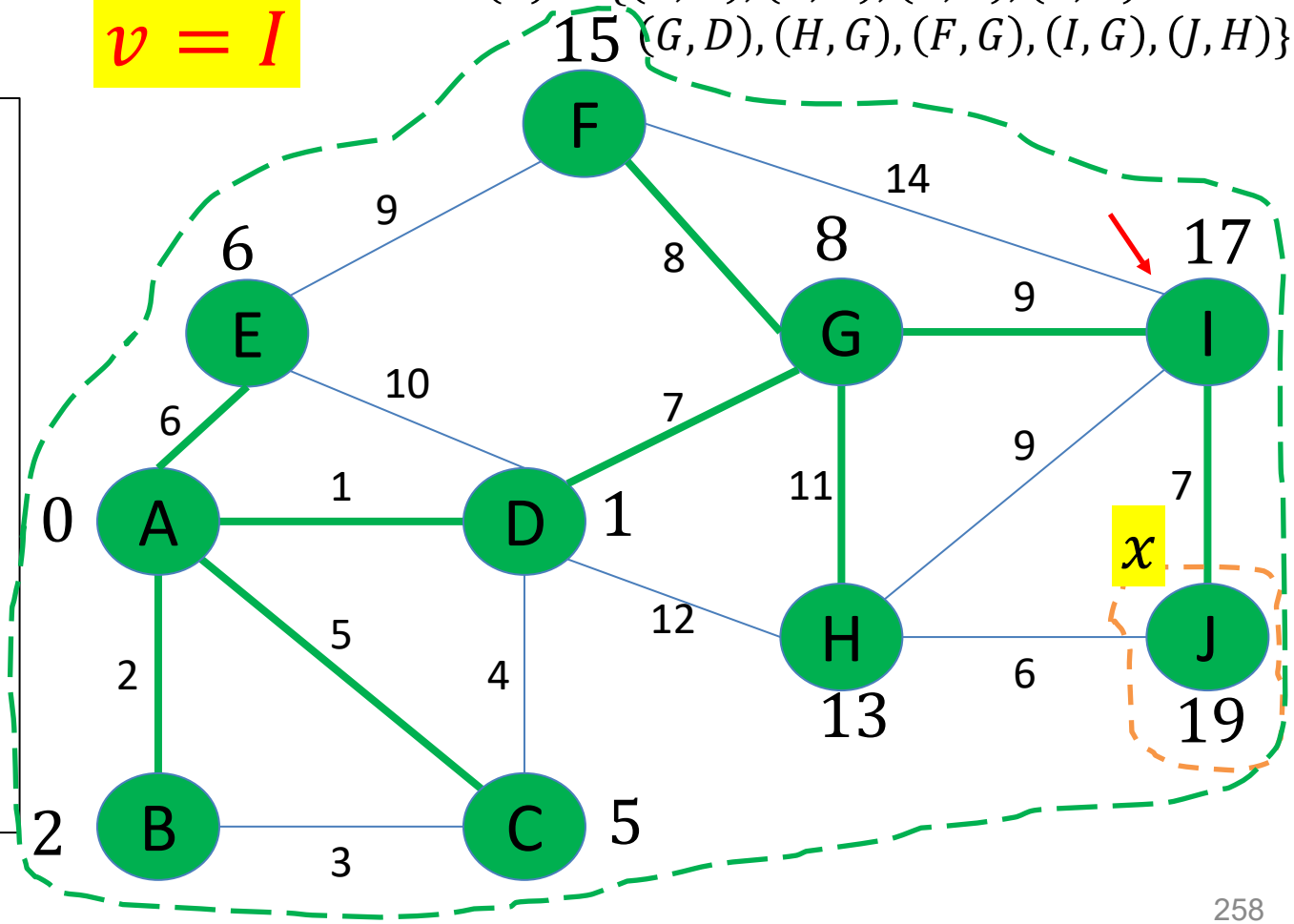
$F = \{J\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D), (H, G), (F, G), (I, G), (J, H)\}$

$v = I$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$V(T) = \{A, D, B, C, E, G, H, F, I, J\}$

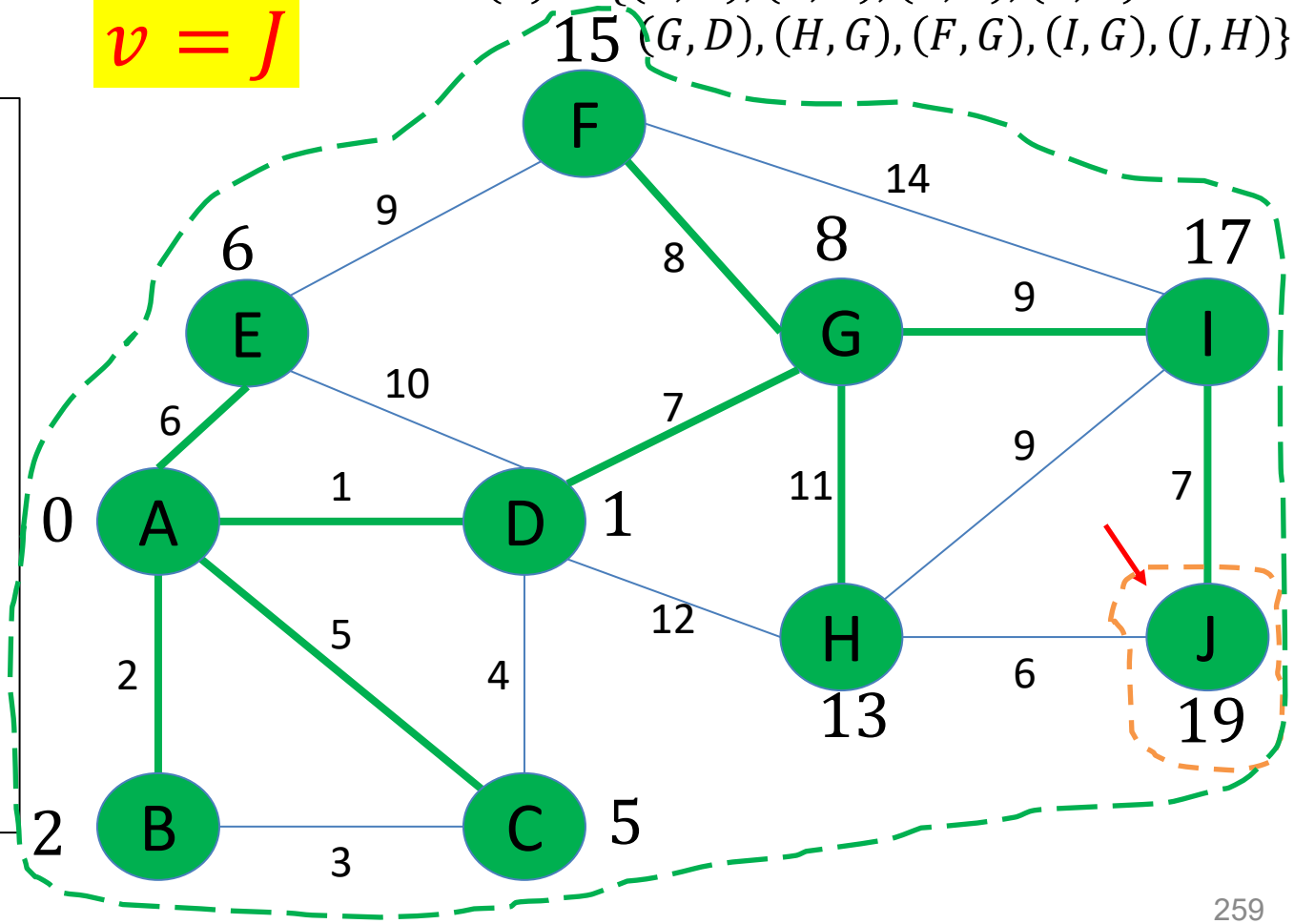
$F = \{J\}$

$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D), (H, G), (F, G), (I, G), (J, H)\}$

$v = J$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



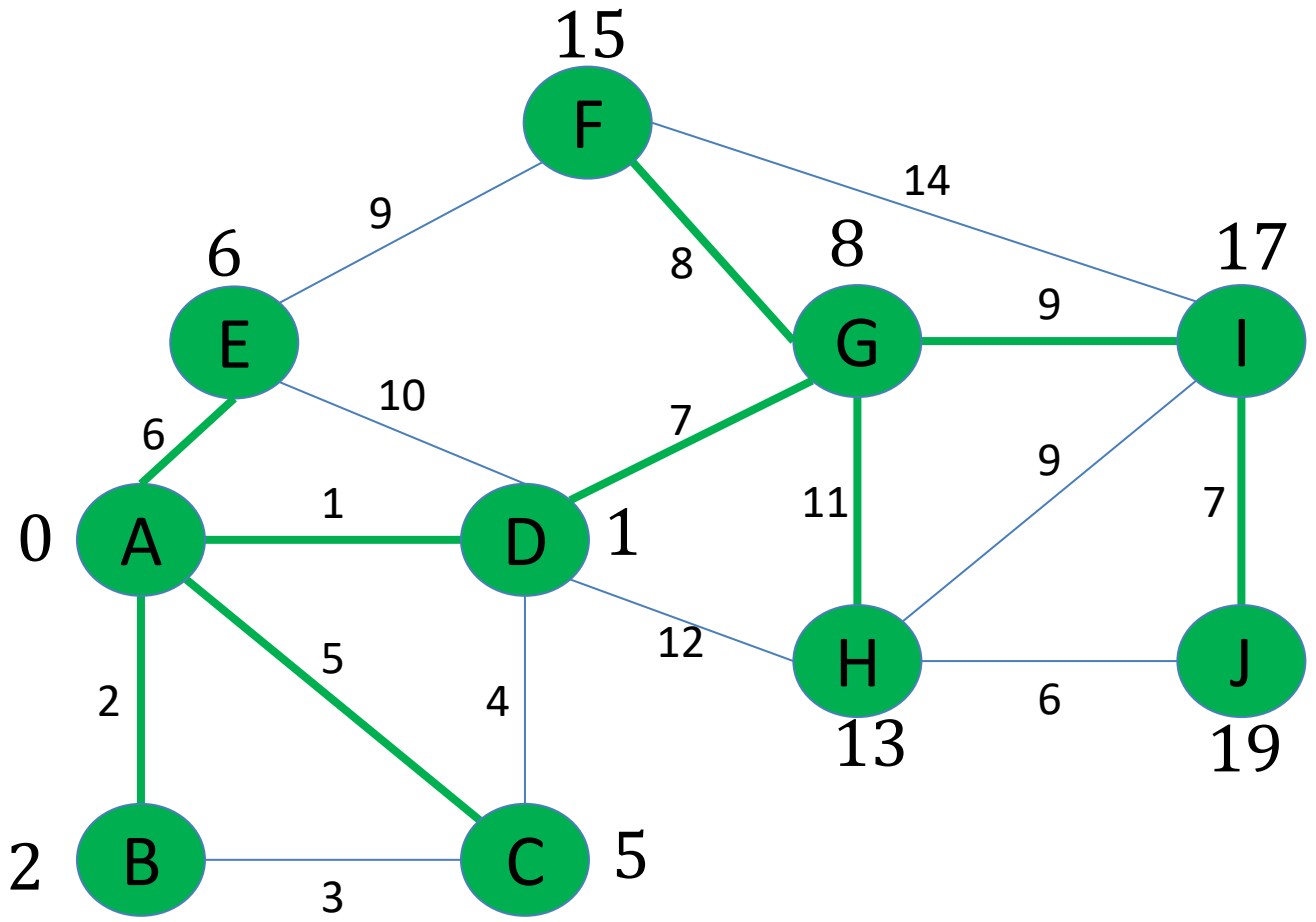
Example (1/11)

A	B	C	D	E	F	G	H	I	J
$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

$V(T) = \{A, D, B, C, E, G, H, F, I, J\}$
$E(T) = \{(A, D), (A, B), (A, C), (A, E)$
$(G, D), (H, G), (F, G), (I, G), (J, H)\}$

while $t \notin V(T)$: ▸ repeat until destination is in the tree

1. $F := \left(\cup_{k \in V(T)} \mathrm{Adj}(k)\right) \setminus V(T)$
2. for each u in Adj(v) \ V(T):
3. if c(v) + w(v, u) < c(u):
4. c(u) := c(v) + w(v, u)
5. previous(u) := v
6. x := $\underset{p \in F}{\mathrm{argmin}}\{c(p)\}$ ▸ fringe vertex with min label
7. V(T) := V(T) ∪ {x}
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge
9. v := x ▸ x becomes the new "current" vertex



Example (68/68): Summary

$$V(T) = \{A, D, B, C, E, G, H, F, I, J\}$$
$$E(T) = \{(A, D), (A, B), (A, C), (A, E), (G, D), (H, G), (F, G), (I, G), (J, H)\}$$

Step	A	B	C	D	E	F	G	H	I	J
0	$(0, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$
1	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$
2	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$
3	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$
4	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(\infty, -)$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$
5	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(\infty, -)$	$(\infty, -)$
6	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(\infty, -)$
7	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$
8	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$
9	$(0, A)$	$(2, A)$	$(5, A)$	$(1, A)$	$(6, A)$	$(15, \mathrm{E})$	$(8, \mathrm{D})$	$(13, D)$	$(17, \mathrm{G})$	$(19, \mathrm{H})$

Complexity

while $t \notin V(T)$: ▸ repeat until destination is in the tree	$O(\lvert V\rvert)$
1. $F \coloneqq \left(\cup_{k \in V(T)} \text{Adj}(k)\right) \setminus V(T)$	$O(\lvert V\rvert)$
2. for each u in Adj(v) \ V(T):	$O(\lvert V\rvert)$
3. if c(v) + w(v, u) < c(u):	
4. c(u) := c(v) + w(v, u)	
5. previous(u) := v	
6. x := $\underset{p \in F}{\text{argmin}}\{c(p)\}$ ▸ fringe vertex with min label	$O(\lvert V\rvert)$
7. V(T) := V(T) ∪ {x}	$O(1)$
8. E(T) := E(T)∪{{previous(x), x} } ▸ add the chosen edge	$O(1)$
9. v := x ▸ x becomes the new "current" vertex	$O(1)$

Total cost: $O(\lvert V\rvert^2)$

Q & A